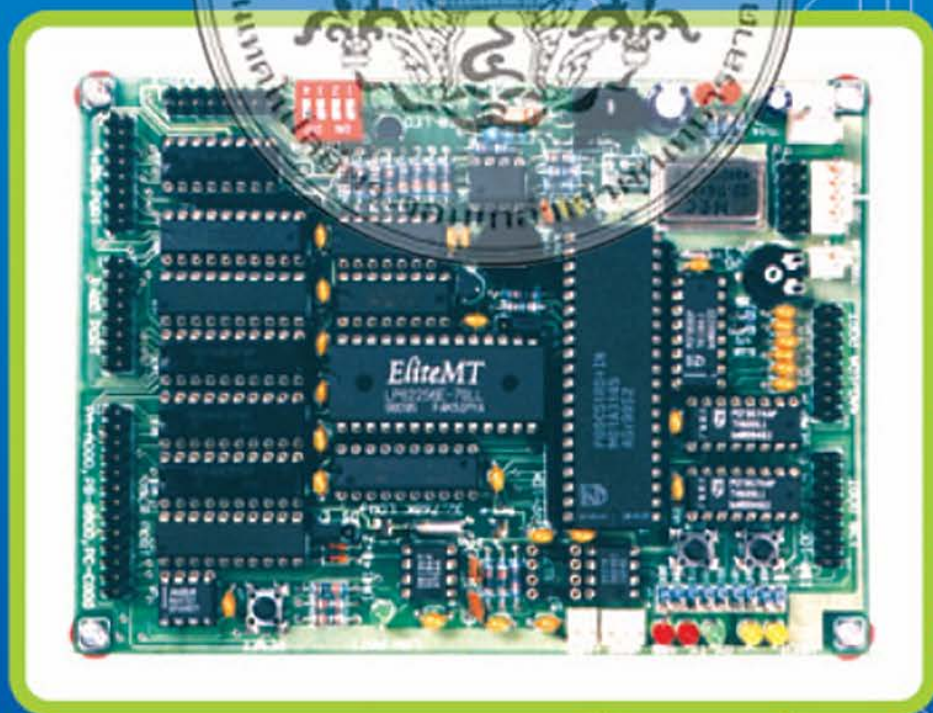


ภาษาแอสเซมบลี สำหรับ

MCS-51



โดย... รศ.ธีรวัฒน์ ประกอบผล



สำนักพิมพ์ ส.ส.ท.

สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

ภาษาแอสเซมบลี สำหรับ MCS-51

โดย

รศ.ธีรวัฒน์ ประกอบพล



สำนักพิมพ์ ส.ส.ท.
สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)



ภาษาแอสเซมบลี สำหรับ MCS-51

โดย รศ.ธีรวัฒน์ ประกอบพล

พิมพ์ครั้งที่	1	กันยายน	2546	จำนวนพิมพ์	2,000	เล่ม
พิมพ์ครั้งที่	2	พฤษภาคม	2547	จำนวนพิมพ์	1,000	เล่ม
พิมพ์ครั้งที่	3	พฤษภาคม	2548	จำนวนพิมพ์	1,000	เล่ม (ปรับปรุงครั้งที่ 1)

สงวนลิขสิทธิ์ตามพระราชบัญญัติลิขสิทธิ์ พ.ศ. 2537 โดย สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

ห้ามลอกเลียนไม่ว่าส่วนใดส่วนหนึ่งของหนังสือเล่มนี้ ไม่ว่าในรูปแบบใด ๆ

นอกจากจะได้รับอนุญาตเป็นลายลักษณ์อักษร

ข้อมูลทางบรรณานุกรมของหอสมุดแห่งชาติ

ธีรวัฒน์ ประกอบพล.

ภาษาแอสเซมบลี สำหรับ MCS-51. - - กรุงเทพฯ : สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น), 2546.
268 หน้า.

1. การเขียนโปรแกรมคอมพิวเตอร์. 2. ภาษาแอสเซมบลี (ภาษาคอมพิวเตอร์)

I. ชื่อเรื่อง.

005.13

ISBN 974-9569-90-3

จัดพิมพ์โดย



5-7 ซอยสุขุมวิท 29 ถนนสุขุมวิท แขวงคลองเตยเหนือ เขตวัฒนา กรุงเทพฯ 10110

โทร. 0-2258-0320 (6 เลขหมายอัตโนมัติ), 0-2259-9160 (10 เลขหมายอัตโนมัติ)

<http://www.tpabookcentre.com>

ออกแบบปกและรูปเล่ม : งานออกแบบสิ่งพิมพ์ ส่วนตำราฯ

จัดจำหน่ายโดย

บริษัท ดวงกลมสมัย จำกัด 15/234 ซอยเสือใหญ่อุทิศ ถนนรัชดาภิเษก แขวงจันทระเกษม เขตจตุจักร กรุงเทพฯ 10900

โทร. 0-2541-7375, 0-2930-6215 โทรสาร 0-2541-7377, 0-2930-7733

E-mail : dktoday@inet.co.th

พิมพ์ที่ : ห้างหุ้นส่วนจำกัด ที. เอส. บี. โปรดักส์ โทร. 0-2536-6085 โทรสาร 0-2536-6087

“ถ้ามีข้อผิดพลาดเนื่องจากการพิมพ์ให้นำมาแลกเปลี่ยนได้ที่สมาคมฯ”

โทร. 0-2258-0320-5, 0-2259-9160-9 ต่อ 1560, 1570

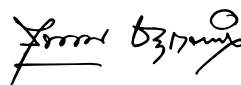
คำนำนายกสมาคม

สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น) ก่อตั้งขึ้นเมื่อปี พ.ศ. 2516 ด้วยความร่วมมือร่วมใจกันของกลุ่มบุคคลที่ได้ผ่านการศึกษาในระดับต่าง ๆ จากประเทศญี่ปุ่น โดยได้รับความร่วมมือทางด้านทุนทรัพย์บางส่วนจาก The Japan-Thailand Economic Cooperation Society (JTECS) เพื่อใช้ช่วยในการดำเนินงานกิจกรรมของสมาคมฯ ซึ่งประกอบด้วย ฝ่ายสำนักพิมพ์ ฝ่ายภาษาและวัฒนธรรม ฝ่ายการศึกษาและฝึกอบรม ฝ่ายบริการสอบเทียบและวิเคราะห์สิ่งแวดล้อม ฝ่ายวินิจัยและให้คำปรึกษา สถานประกอบการ และฝ่ายเทคโนโลยีสารสนเทศ ซึ่งแต่ละฝ่ายจัดดำเนินการโดยสมาชิกของสมาคมฯ ทำหน้าที่เป็นคณะที่ปรึกษารับผิดชอบบริหารงานให้เป็นไปตามเป้าหมาย

สำหรับฝ่ายสำนักพิมพ์ฯ โดยส่วนตำราสนับสนุนเทคนิคอุตสาหกรรม มีนโยบายพื้นฐานคือการส่งเสริมและเร่งรัดให้มีการจัดพิมพ์หนังสือตำราทางเทคโนโลยีทุกประเภท รวมถึงหนังสือทางด้านการบริหารจัดการธุรกิจ อุตสาหกรรม ทั้งที่เป็นงานแปลโดยตรง งานแปลเรียบเรียง งานถอดความ งานรวบรวม งานแต่ง และงานสำรวจวิจัยทางด้านอุตสาหกรรม โดยที่สมาคมฯ มีความเห็นว่า หนังสือตำราภาษาไทย โดยเฉพาะในระดับอาชีวศึกษาแขนงวิชาเทคโนโลยีต่าง ๆ ยังมีอยู่ในปริมาณจำกัดไม่พอเพียง ถ้าส่งเสริมให้มีหนังสือเช่นนี้เพิ่มขึ้นย่อมมีส่วนช่วยยกระดับมาตรฐานการศึกษาทางเทคโนโลยีให้สูงขึ้นและแพร่หลายขึ้นโดยปริยาย อีกทั้งยังช่วยสร้างสรรค์ปัญญา ความคิดริเริ่ม และความรู้ความเข้าใจอันถูกต้อง ซึ่งจะเป็นการปูรากฐานสำคัญในการพัฒนาอุตสาหกรรมของประเทศไทย เป็นประโยชน์แก่สังคมอุตสาหกรรมโดยรวม

ปัจจุบัน สมาคมฯ ยังคงมีเจตนารมณ์อันแน่วแน่ที่จะขยายงานทุก ๆ ฝ่ายต่อไปอย่างไม่หยุดยั้ง และหวังเป็นอย่างยิ่งว่าด้วยเจตนานับประสิทธิ์ของสมาคมฯ ในการดำเนินกิจการที่มีใ้การแสวงหาผลกำไร หากมุ่งมั่นที่จะให้นักศึกษาและประชาชนได้มีโอกาสซื้อหาหนังสือตำราในราคาย่อมเยาเช่นนี้ คงเกิดประโยชน์แก่สังคมส่วนรวมทั้งในทางตรงและทางอ้อมและหากสถาบันการศึกษาใดต้องการใช้ 'วนหนึ่งส่วนใดเพื่อใช้ประกอบการศึกษา ทางสมาคมฯ ก็ได้ขัดข้อง แต่ใคร่ขอให้ทำเรื่องขออนุญาตต่อทางสมาคมฯ ก่อน

อนึ่ง สมาคมฯ ใคร่ขอแสดงความขอบคุณเป็นอย่างยิ่งต่อผู้เขียนและคณะผู้จัดทำที่ได้พากเพียรจนทำให้ตำราชุดนี้สำเร็จขึ้นมาได้ไว้ ณ ที่นี้ด้วย



(นายสุพงศ์ ชุตสาหกิจ)

นายกสมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

คำแถลงของสำนักพิมพ์ ส.ส.ท.

สำนักพิมพ์ ส.ส.ท. โดยส่วนตำราสนับสนุนเทคนิคอุตสาหกรรม วิวัฒนาการมาจากโครงการสนับสนุนเทคนิคอุตสาหกรรม ซึ่งแต่เดิมใช้ชื่อว่า โครงการตำรา ซึ่งจัดตั้งขึ้นเมื่อปี พ.ศ. 2516 พร้อม ๆ กับการก่อตั้งสมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น) โดยมุ่งหวังที่จะให้มีตำราทางด้านวิทยาการต่าง ๆ ทั้งในระดับอาชีวศึกษา และประชาชนทั่วไป เพื่อเพิ่มพูนความรู้ทางด้านช่างที่สามารถนำไปใช้ปฏิบัติงานได้

ในระยะแรกนั้น ตำราที่ผลิตโดยโครงการตำรา ส่วนใหญ่จะเป็นหนังสือแปลจากต้นฉบับภาษาญี่ปุ่น ต่อมาจึงได้ขยายขอบข่ายของการจัดพิมพ์ครอบคลุมไปถึงงานแปลและเรียบเรียงจากต้นฉบับภาษาอื่น งานเรียบเรียง-เขียนตำราจากประสบการณ์ของผู้ชำนาญในแต่ละสาขา ซึ่งส่วนใหญ่จะเป็นอาจารย์จากสถาบันการศึกษา

ต่อมาในระยะ 4 - 5 ปีหลังจากการก่อตั้งสมาคมฯ โครงการตำราได้วิวัฒนาการเป็นโครงการสนับสนุนเทคนิคอุตสาหกรรม และเป็นส่วนตำราสนับสนุนเทคนิคอุตสาหกรรมในปี พ.ศ. 2539 พร้อม ๆ กับการขยายขอบข่ายหนังสือที่จัดพิมพ์เพิ่มเติม ได้แก่ หนังสือทางด้านการบริหารจัดการธุรกิจ การบริหารจัดการคุณภาพ และอื่น ๆ ซึ่งสอดคล้องกับวัตถุประสงค์การดำเนินงาน

โดยที่หนังสือที่จัดพิมพ์โดยสมาคมฯ ได้รับการต้อนรับเป็นอย่างดีจากนักศึกษาและประชาชนทั่วไป ทางสมาคมฯ จึงใคร่ขอเชิญชวนให้ผู้เชี่ยวชาญในวงการอุตสาหกรรมและสถาบันการศึกษาได้ช่วยกันเขียน-เรียบเรียงหนังสือทางด้านเทคนิคอุตสาหกรรมและอื่น ๆ ให้แพร่หลายยิ่งขึ้น โดยสมาคมฯ ยินดีให้การสนับสนุนในด้านการจัดพิมพ์

สำนักพิมพ์ ส.ส.ท. ขอขอบคุณท่านผู้เขียน-เรียบเรียงและเจ้าหน้าที่ของสมาคมฯ ทุกท่านที่ให้ความร่วมมือเป็นอย่างดีในการจัดพิมพ์หนังสือเล่มนี้ และหวังว่าหนังสือเล่มนี้จะมีส่วนช่วยในการพัฒนาเศรษฐกิจและอุตสาหกรรมของประเทศ และหากท่านผู้อ่านมีข้อชี้แนะประการใด ขอได้โปรดแจ้งให้ทางสำนักพิมพ์ทราบด้วย จักเป็นพระคุณยิ่ง



สำนักพิมพ์ ส.ส.ท.
สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

“บุญเราไม่เคยสร้างไว้เลย จะมีใครที่ไหนมาช่วยเจ้า”

“ลูกเอ๋ย ก่อนจะเที่ยวไปขอบารมีหลวงพ่อก็คดี เจ้าจะต้องมีทุนของตัวเอง คือบารมีของตนลงทุนไปก่อน เมื่อบารมีของเจ้าไม่พอจึงค่อยขอยืมบารมีคนอื่นมาช่วย มิฉะนั้นเจ้าจะเอาตัวไม่รอดเพราะหนี้สินในบุญบารมีที่เที่ยวไปขอยืมมาจนพันตัว”

คำเทศนาตอนหนึ่งของเจ้าประคุณสมเด็จพระพุฒาจารย์
(โต พรหมรังสี)

ไมโครคอนโทรลเลอร์ที่ได้รับการกล่าวขวัญถึงและได้รับความนิยมอย่างมากคือ ไมโครคอนโทรลเลอร์ตระกูล MCS-51 ซึ่งในปัจจุบันได้มีผู้ผลิตออกมาจำหน่ายกันหลายเบอร์ แต่ทุกเบอร์จะมีส่วนหลัก ๆ คล้าย ๆ กัน การทำงานพื้นฐานเหมือนกัน นั่นคือสามารถใช้คำสั่งได้เหมือนกันทุกประการ สำหรับรายละเอียดที่แตกต่างกันของชิปแต่ละเบอร์ได้แก่ จำนวนคำสั่งที่เพิ่มมากขึ้น เพื่อให้สามารถเขียนโปรแกรมได้สะดวกขึ้น ไมโครคอนโทรลเลอร์ตระกูลนี้ถือว่าเป็นชิพคอมพิวเตอร์รูปแบบหนึ่งที่ถูกนำไปใช้งานในรูปแบบ Embedded ซึ่งเป็นระบบฝังตัวอยู่ภายในสินค้าทางอิเล็กทรอนิกส์ต่าง ๆ ทำให้การควบคุมต่าง ๆ ซึ่งแต่เดิมจะเน้นที่การพัฒนาทางด้าน Hardware ได้เปลี่ยนมาเป็นการพัฒนาทางด้าน Software หรือการเขียนโปรแกรมแทน

การเขียนโปรแกรมในวงการไมโครคอนโทรลเลอร์มักจะคุ้นเคยกับการพัฒนาด้วยภาษา Assembly แม้ว่าในปัจจุบันจะมีการใช้ภาษาระดับสูงเช่นภาษาซีในการพัฒนาโปรแกรมก็ตาม แต่การเรียนรู้ด้วยภาษา Assembly นั้นจะทำให้เข้าใจโครงสร้างการทำงานของไมโครคอนโทรลเลอร์ได้เป็นอย่างดี

หนังสือเล่มนี้จะอธิบายหลักการเขียนโปรแกรมภาษา Assembly สำหรับใช้กับ MCS-51 โดยจะมีตัวอย่างการเชื่อมต่อกับอุปกรณ์ภายนอก และตัวอย่างโปรแกรมต่าง ๆ ซึ่งผู้ศึกษาสามารถนำไปประยุกต์ใช้ในงานต่าง ๆ ได้ โดยในท้ายเล่มได้ยกตัวอย่างการใช้งานกับชิพเบอร์ P89C51RD2 ซึ่งเป็นชิพที่มีความเร็วสูงและสามารถเขียนโปรแกรมเข้าไปในตัวชิพได้โดยตรง ตัวอย่างโปรแกรมในหนังสือเล่มนี้ผู้ศึกษาสามารถดาวน์โหลดได้จาก www.kmitl.ac.th/~kpteeraw

และท้ายที่สุดนี้ผู้เขียนขอขอบคุณส่วนตำราฯ และสำนักพิมพ์ สมาคมส่งเสริมเทคโนโลยี(ไทย-ญี่ปุ่น) ที่ได้สนับสนุนการจัดทำหนังสือเล่มนี้จนสำเร็จได้ด้วยดี

รองศาสตราจารย์ธีรวัฒน์ ประกอบผล

บทที่ 1 ไมโครคอนโทรลเลอร์ MCS-51	1
1.1 ภาพรวมของ MCS-51	1
1.2 โครงสร้างหน่วยความจำของ MCS-51	4
1.3 ความเร็วในการทำงานของ MCS-51	6
1.4 การพัฒนางานไมโครคอนโทรลเลอร์	6
บทที่ 2 การเขียนโปรแกรมด้วยภาษาแอสเซมบลี	9
2.1 Addressing Modes	10
2.2 ภายใน MCS-51	15
2.3 การเขียนโปรแกรมภาษาแอสเซมบลีเบื้องต้น	17
2.4 ประเภทของข้อมูลและคำสั่งเทียม	21
2.5 รีจิสเตอร์แบงก์และสแตก	22
2.6 แฟลชของ MCS-51 และรีจิสเตอร์ PSW	24
2.7 หน่วยความจำสแตกของ MCS-51	28
บทที่ 3 คำสั่งควบคุม	31
3.1 คำสั่งทำซ้ำและคำสั่งกระโดด	31
3.2 การเรียกโปรแกรมย่อย	37
3.3 การเขียนโปรแกรมหน่วงเวลา	42
บทที่ 4 การโปรแกรมพอร์ตอินพุตเอาต์พุต	47
4.1 การจัดขาของ MCS-51	47
บทที่ 5 คำสั่งทางคณิตศาสตร์และการโปรแกรม	55
5.1 การบวกลบเลขแบบไม่คิดเครื่องหมาย	55
5.2 การคูณและการหารแบบไม่คิดเครื่องหมาย	64
5.3 ตัวเลขแบบคิดเครื่องหมาย	65

บทที่ 6	คำสั่งทางลอจิกและการโปรแกรม	69
6.1	คำสั่งทางลอจิกและคำสั่งเปรียบเทียบ	69
6.2	คำสั่งหมุนและสลับข้อมูล	78
6.3	การเขียนโปรแกรมประยุกต์ BCD และ ASCII	82
บทที่ 7	คำสั่งกระทำต่อบิตและการโปรแกรม	87
7.1	การโปรแกรมแบบหนึ่งบิต	87
7.2	การกระทำกับบิตตัวทด	94
บทที่ 8	การโปรแกรมตัวนับตัวจับเวลา	99
8.1	การโปรแกรมเป็นตัวจับเวลา	99
8.2	การโปรแกรมเป็นตัวนับ	113
บทที่ 9	การสื่อสารข้อมูลแบบอนุกรม	119
9.1	พื้นฐานการสื่อสารอนุกรม	119
9.2	การต่อ MCS-51 กับ RS-232	124
9.3	การโปรแกรมการสื่อสารแบบอนุกรมของ 8051	125
บทที่ 10	การโปรแกรมอินเทอร์รัปต์	137
10.1	การอินเทอร์รัปต์ของ MCS-51	137
10.2	การโปรแกรมไทม์เมอร์อินเทอร์รัปต์	141
10.3	การโปรแกรมอินเทอร์รัปต์ภายนอก	144
10.4	การโปรแกรมการอินเทอร์รัปต์พอร์ตอนุกรม	147
บทที่ 11	การขยายพอร์ต	153
11.1	การถอดรหัสตำแหน่งพอร์ต	153
11.2	ไอซี 8255 และการโปรแกรม	155
11.3	ระบบบัสแบบ I ² C และการเชื่อมต่อ	164

บทที่ 12 การทดลองกับหน่วยแสดงผล	169
12.1 การเชื่อมต่อกับไดโอดเปล่งแสง	169
12.2 การแสดงผลทาง LED แบบ 7 ส่วน	172
12.3 การแสดงผลทาง LCD	180
 บทที่ 13 การทดลองกับพอร์ตแบบ I²C	 191
13.1 การใช้งานไอซี PCF8574	191
 บทที่ 14 การทดลองสร้างฐานเวลา	 203
14.1 ไอซีฐานเวลาจริง DS1307	203
14.2 การเชื่อมต่อ DS1307 กับไมโครคอนโทรลเลอร์ MCS-51	206
 บทที่ 15 เครื่องมือการทดลองไมโครคอนโทรลเลอร์	 221
15.1 การพัฒนาด้วยซิงเกิลบอร์ด	222
15.2 การพัฒนาด้วย Embedded Board	223
15.3 หน่วยความจำของ 89C51RD2 และการโปรแกรม	225
15.4 การเขียนโปรแกรมส่งข้อมูลทางพอร์ตอนุกรม	
15.5 การขยายพอร์ตและการเขียนโปรแกรม	
15.6 การสแกนหน่วยแสดงผลด้วยชิป MAX7219	

บทที่

1

ไมโครคอนโทรลเลอร์ MCS-51

ในปัจจุบันเทคโนโลยีระบบฝังตัว (embedded system) ได้เข้ามามีบทบาทอย่างมาก อุปกรณ์ไฟฟ้าอิเล็กทรอนิกส์ต่าง ๆ ได้มีการเพิ่มหน่วยประมวลผลเข้าไปภายในตัวมัน ในขณะที่เทคโนโลยีของหน่วยประมวลผลได้มีการพัฒนาอยู่ตลอดเวลา โดยทั่วไปแล้ววงจรควบคุมการทำงานของอุปกรณ์อิเล็กทรอนิกส์ต่าง ๆ อาจอยู่ในรูปของวงจรดิจิทัลทั่ว ๆ ไปก็ได้ แต่ในปัจจุบันจะพบว่าระบบฝังตัวจะใช้ไมโครคอนโทรลเลอร์มากขึ้น เนื่องจากมีความยืดหยุ่นสูง ทำให้การควบคุมการทำงานจะเน้นไปที่การพัฒนาทางซอฟต์แวร์ (software) แทนการพัฒนาทางด้านฮาร์ดแวร์ (hardware) เช่นในอดีต ส่วนใหญ่ผู้พัฒนาเทคโนโลยีระบบฝังตัวจะให้ความสำคัญกับไมโครโปรเซสเซอร์ หรือไมโครคอนโทรลเลอร์เป็นหลัก การแบ่งระดับความซับซ้อนของระบบฝังตัวสามารถแบ่งได้หลายระดับ ระบบฝังตัวที่มีความซับซ้อนมาก ๆ จะใช้ไมโครโปรเซสเซอร์ที่มีความสามารถสูงเป็นหน่วยประมวลผลกลาง เช่นตระกูล x86 ของบริษัทอินเทล สำหรับระบบฝังตัวขนาดเล็กจะใช้ไมโครคอนโทรลเลอร์ที่มีการประมวลผลแบบ 4 บิต หรือ 8 บิต ไมโครคอนโทรลเลอร์ที่เหมาะสมกับการศึกษาในเบื้องต้นตัวหนึ่งก็คือไมโครคอนโทรลเลอร์ตระกูล MCS-51



1.1 ภาพรวมของ MCS-51

ในปี ค.ศ. 1981 บริษัทอินเทลได้เปิดตัวไมโครคอนโทรลเลอร์แบบ 8 บิต โดยให้ชื่อว่า 8051 โดยไมโครคอนโทรลเลอร์ตัวนี้จะมีหน่วยความจำประเภท RAM ขนาด 128 ไบต์ หน่วยความจำประเภท ROM ขนาด 4 กิโลไบต์ มีพอร์ตนานขนาด 8 บิต จำนวน 4 พอร์ต มีไทมเมอร์ 2 ตัว และพอร์ตอนุกรม 1 พอร์ต



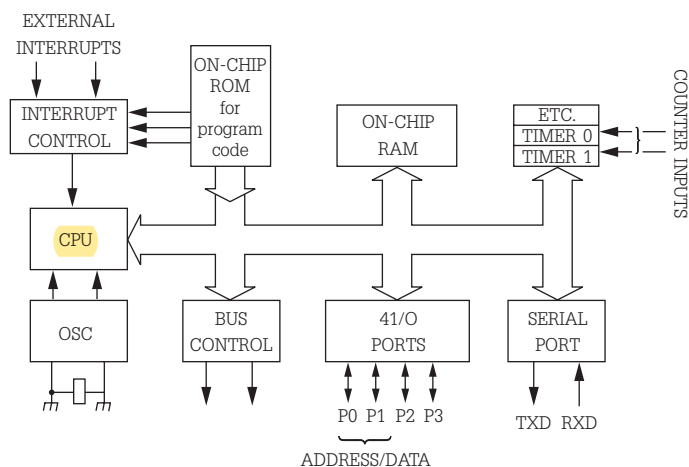
โดยทั้งหมดจะรวมอยู่ในชิปเพียงชิปเดียว ไคอะแกรมภายในตัวมันแสดงได้ดังรูปที่ 1.1 และเนื่องจากหน่วยความจำโปรแกรมเป็นแบบ ROM ที่อยู่ในตัวมัน ดังนั้นการโปรแกรมการทำงานต้องโปรแกรมมาจากโรงงานที่ผลิตชิปนี้ออกมาโดยตรง นอกจากนี้บริษัทอื่นก็ยังได้ผลิตไมโครคอนโทรลเลอร์ออกมาอีกหลายเบอร์ที่มีโครงสร้างภายในใกล้เคียงกับ 8051 โดยเรียกรวม ๆ ว่า ตระกูล MCS-51 หรือตระกูล 51 โดยการมีการปรับปรุงโครงสร้างภายใน และเพิ่มองค์ประกอบต่าง ๆ เข้าไปอีก และยังมีบริษัทอื่น ๆ ผลิตตามออกมาด้วย แต่จะใช้เบอร์ที่เรียกแตกต่างกันออกไป

8052

ไมโครคอนโทรลเลอร์เบอร์ 8052 จะมีลักษณะต่าง ๆ ที่เป็นมาตรฐานเหมือนกับ 8051 แต่จะเพิ่มหน่วยความจำ RAM ภายในเข้าไปอีก 128 ไบต์ และเพิ่มไทมเมอร์อีกหนึ่งตัว ทำให้ 8052 มีหน่วยความจำ RAM ภายในทั้งหมด 256 ไบต์ และมีไทมเมอร์ 3 ตัว นอกจากนี้ยังมีหน่วยความจำ ROM ภายในเพิ่มเป็น 8 กิโลไบต์ ดังตารางที่ 1.1

8031

ไมโครคอนโทรลเลอร์เบอร์ 8031 จะคล้ายกับ 8051 แต่จะไม่มีหน่วยความจำ ROM ภายในชิป โดยโปรแกรมควบคุมการทำงานจะอยู่ในหน่วยความจำภายนอกเท่านั้น



รูปที่ 1.1 ไคอะแกรมภายในของ 8051



ตารางที่ 1.1

ส่วนประกอบ	8051	052	8031
หน่วยความจำ ROM ภายใน	4k	8k	0
หน่วยความจำ RAM ภายใน (byte)	128	256	128
Timers	2	3	2
I/O (บิต)	32	32	32
Serial port	1	1	1
Interrupt source	6	8	6

8751

จัดเป็นไมโครคอนโทรลเลอร์ตระกูล MCS-51 ตัวหนึ่ง แต่จะมีหน่วยความจำโปรแกรมแบบ UV-EPROM ขนาด 4k byte อยู่ภายในชิป โดยผู้พัฒนาสามารถโปรแกรมลงไปในชิปได้โดยใช้เครื่องโปรแกรม และสามารถแก้ไขโปรแกรมได้โดยการลบโปรแกรมเก่าออกโดยใช้แสง UV

AT89C51

เป็นไมโครคอนโทรลเลอร์ตระกูล MCS-51 ที่ผลิตโดย Atmel Corporation ตัวนี้จะมีหน่วยความจำภายในเป็นแบบ flash memory ทำให้ใช้งานสะดวกกว่า 8751 มาก เนื่องจาก AT89C51 สามารถโปรแกรมและลบโปรแกรมได้ด้วยกระแสไฟฟ้าบนชิปที่ต่ออยู่ในวงจรได้เลย

P89C51RD2

เป็นไมโครคอนโทรลเลอร์ตระกูล MCS-51 ของ Philips Semiconductor โครงสร้างภายในจะคล้ายกับ MCS-51 มาตรฐาน แต่จะมีส่วนเพิ่มเติมขึ้นมาเช่น มีหน่วยความจำภายในเป็นแบบแฟลช และมีการบรรจุโปรแกรมบูตรอม (Boot ROM) เข้าไปภายในตัวมัน ทำให้สามารถเขียนหรือลบโปรแกรมได้โดยตรงโดยไม่ต้องถอดชิปออกจากวงจร ที่เรียกว่าการโปรแกรมแบบ ISP (In-System Programming)

ในปัจจุบันมีผู้ผลิตไมโครคอนโทรลเลอร์ตระกูล MCS-51 นี้ออกมาหลายบริษัท และมีเบอร์ที่ใช้เรียกแตกต่างกันไป ขึ้นอยู่กับโครงสร้างภายในและเทคโนโลยีที่ใช้ในการสร้างชิป โดยสามารถสรุปได้ดังนี้

803x : ไม่มีหน่วยความจำโปรแกรมภายใน เช่น 8031 , 8032 เป็นต้น

805x : มีหน่วยความจำโปรแกรมประเภท ROM ภายใน

8xxx : ใช้เทคโนโลยีแบบ NMOS

8xCxx : ใช้เทคโนโลยีแบบ CMOS

87xx : มีหน่วยความจำโปรแกรมภายในเป็นแบบ EPROM

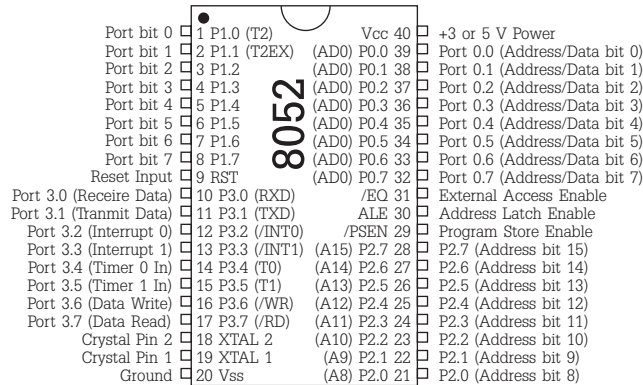
89xx : มีหน่วยความจำโปรแกรมภายในเป็นแบบ flash EPROM



8xx1 : มีหน่วยความจำโปรแกรมภายใน 4 กิโลไบต์ และมี RAM ภายใน 128 ไบต์

8xx2 : มีหน่วยความจำโปรแกรมภายใน 8 กิโลไบต์ และมี RAM ภายใน 256 ไบต์

อย่างไรก็ตามแม้ว่าไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะมีอยู่หลายเบอร์ หลายรุ่น แต่ลักษณะการจัดขาของตัวชิปไอซีอาจแตกต่างกันไปบ้าง บ้างเบอร์มี 20 ขา บ้างเบอร์มี 40 ขา โดยผู้สนใจสามารถหาศึกษาได้จากคู่มือโดยตรง โดยตำราเล่มนี้จะกล่าวถึงเทคนิคการเขียนโปรแกรม และการประยุกต์ใช้ และจะใช้ MCS-51 ตัวมาตรฐานเป็นหลักในการอธิบายต่าง ๆ สำหรับลักษณะการจัดขาของไมโครคอนโทรลเลอร์เบอร์ 8052 แสดงได้ดังรูปที่ 1.2



รูปที่ 1.2 ลักษณะการจัดขาของ 8052



1.2 โครงสร้างหน่วยความจำของ MCS-51

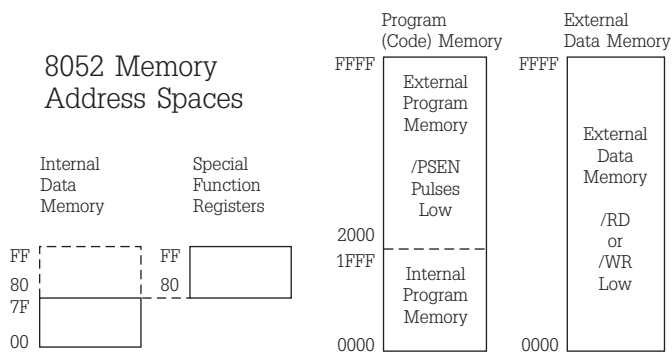
ไมโครคอนโทรลเลอร์ตระกูล MCS-51 ได้ออกแบบการจัดหน่วยความจำเป็นสองส่วน คือหน่วยความจำโปรแกรม (program memory) และหน่วยความจำข้อมูล (data memory) โดยทั้งสองส่วนนี้จะมีแอดเดรสแยกออกจากกัน ดังรูปที่ 1.3 สำหรับ MCS-51 บางเบอร์จะมีหน่วยความจำโปรแกรมอยู่ในชิป บางเบอร์ต้องต่อเพิ่มภายนอกชิป จากรูปที่ 1.3 จะเห็นว่าแอดเดรสของหน่วยความจำจะมีแอดเดรสตำแหน่งตรงกัน แต่เมื่อ MCS-51 ต้องการติดต่อกับหน่วยความจำในส่วนใดจะใช้คำสั่งสัญญาณต่างกัน อย่างเช่นถ้า MCS-51 ต้องการติดต่อกับหน่วยความจำภายนอก ไชเคิลการทำงานจะเป็นดังนี้

- ถ้าอ่านข้อมูลจากหน่วยความจำโปรแกรมภายนอก ขา PSEN จะเป็น Low
- ถ้าอ่านข้อมูลจากหน่วยความจำข้อมูลภายนอก ขา RD เป็น Low
- ถ้าเขียนข้อมูลลงหน่วยความจำข้อมูลภายนอก ขา WR เป็น Low



หน่วยความจำโปรแกรม

ใน MCS-51 จะมีหน่วยความจำโปรแกรมได้สูงสุด 64k byte หลังจากตัวมันถูกรีเซ็ตจะเริ่มทำงานที่แอดเดรส 0000 ของหน่วยความจำโปรแกรม การอ่านข้อมูลจากหน่วยความจำโปรแกรมนี้จะต้องใช้คำสั่ง MOVC นอกจากนี้ในหน่วยความจำโปรแกรมยังแบ่งแอดเดรสบางส่วนสำหรับเก็บโปรแกรมตอบสนองอินเทอร์รัปต์ เช่นแอดเดรส 0003H เป็นตำแหน่งของโปรแกรมตอบสนองการอินเทอร์รัปต์จากภายนอก (INT0) เมื่อมีสัญญาณอินเทอร์รัปต์เข้ามาทางขานี้ โปรแกรมจะกระโดดไปทำงานยังตำแหน่งนี้ทันที



รูปที่ 1.3 การจัดแอดเดรสหน่วยความจำ

หน่วยความจำข้อมูลภายนอก

หน่วยความจำข้อมูลภายนอกของ MCS-51 สามารถมีได้ 64k Byte เมื่อ MCS-51 ต้องการติดต่อกับหน่วยความจำส่วนนี้จะต้องใช้คำสั่ง MOVX และใช้รีจิสเตอร์ DPTR, R0 หรือ R1 ในการอ้างตำแหน่งหน่วยความจำ ในการขยายพอร์ตเพิ่มให้กับ MCS-51 จะต้องใช้ตำแหน่งในส่วนนี้เป็นตำแหน่งของพอร์ตด้วย

หน่วยความจำข้อมูลภายใน

หน่วยความจำข้อมูลภายในของ MCS-51 จะมีจำนวน 256 ไบต์ โดย 128 ไบต์แรกที่มีแอดเดรสอยู่ในช่วง 00H - 7FH จะเป็นส่วนของ RAM ที่ใช้งานได้ทั่วไป รีจิสเตอร์แบ่งคี่จำนวน 4 แบนด์ หน่วยความจำที่สามารถเข้าถึงข้อมูลระดับบิตได้ ส่วนอีก 128 ไบต์หลังเริ่มต้นตั้งแต่แอดเดรส 80H เป็นต้นไปจะเป็นส่วนของรีจิสเตอร์ฟังก์ชันพิเศษ หรือ SFR (Special Function Register) และรีจิสเตอร์ที่ใช้งานทั่วไปเช่น รีจิสเตอร์ ACC, B, PSW, SP และ DPTR เป็นต้น การอ่านหรือเขียนข้อมูลกับหน่วยความจำในส่วนนี้จะใช้คำสั่ง MOV



1.3 ความเร็วในการทำงานของ MCS-51

การให้ไมโครคอนโทรลเลอร์ทำงานนั้นจะต้องโปรแกรมให้กับตัวมันก่อน การวัดความเร็วในการทำคำสั่งของโปรแกรมจะดูการรอบสัญญาณนาฬิกา หรือที่เรียกว่าแมกซ์ซินไซเคิล ซึ่งในตารางคำสั่งของไมโครคอนโทรลเลอร์แต่ละตัวจะมีข้อมูลบอกไว้ว่าการทำคำสั่งแต่ละคำสั่งจะใช้สัญญาณนาฬิกาที่แมกซ์ซินไซเคิล สำหรับไมโครคอนโทรลเลอร์ MCS-51 เบอร์ที่เป็นมาตรฐานนั้น 1 แมกซ์ซินไซเคิลจะใช้สัญญาณนาฬิกา 12 ลูก ดังนั้นถ้า MCS-51 ทำงานที่สัญญาณนาฬิกา 12 MHz แล้ว การทำงาน 1 ซิเคิลจะมีค่าเท่ากับ 1 ไมโครวินาที หรือมีความเร็วในการทำงาน 1 MHz ถ้าหากต้องการให้ MCS-51 ตัวนั้นทำงานได้เร็วขึ้นจะต้องเพิ่มสัญญาณนาฬิกาให้กับมัน สำหรับ MCS-51 บางเบอร์จะมีความเร็วมากขึ้นกว่าปกติ 2 เท่า เช่นเบอร์ P89C51RD2 เนื่องจากหนึ่งแมกซ์ซินไซเคิลของมันจะใช้สัญญาณนาฬิกาเพียง 6 ลูก



1.4 การพัฒนางานไมโครคอนโทรลเลอร์

การเรียนรู้ไมโครคอนโทรลเลอร์นั้นส่วนใหญ่จะอยู่ในรูปของบอร์ดฝังตัว (embedded board) จากนั้นก็พัฒนาโปรแกรมสำหรับใช้งานลงไป โดยบอร์ดที่ใช้ศึกษาอาจอยู่ในรูปของซิงเกิลบอร์ด (single board) ซึ่งมีความสะดวกในการพัฒนาโปรแกรมและแก้ไขโปรแกรมหรุมทั้งมีโปรแกรมย่อยในรูปของ System Call ให้เรียกใช้ด้วย บอร์ดประเภทนี้จะมีอุปกรณ์แสดงผล และคีย์บอร์ดต่ออยู่ด้วย ซึ่งเหมาะสำหรับผู้เริ่มต้นศึกษาทางด้านไมโครคอนโทรลเลอร์ การโปรแกรมสามารถทำได้โดยป้อนออปโค้ดของโปรแกรมในรูปของเลขฐานสิบหกลงไป และยังสามารถเขียนโปรแกรมบนคอมพิวเตอร์ PC จากนั้นใช้โปรแกรมแปลภาษาให้เป็นออปโค้ดที่เรียกว่าการทำแอสเซมเบลอร์ (assembler) และโหลดโปรแกรมที่ได้ลงบนบอร์ดเพื่อใช้งานต่อไป

สำหรับการพัฒนาด้วยอีพรอมอีมูเลเตอร์ (EPROM emulator) ก็เป็นอีกวิธีหนึ่งที่ใช้กันมานาน โดยที่บอร์ดไมโครคอนโทรลเลอร์จะอยู่ในรูปของ embedded board ซึ่งมีราคาถูกกว่าแบบซิงเกิลบอร์ด โดยจะเขียนโปรแกรมบนคอมพิวเตอร์ PC และนำรหัสโปรแกรมที่ได้โปรแกรมลงบนอีพรอมอีมูเลเตอร์ แทนที่จะโปรแกรมลงหน่วยความจำ ROM จริง ๆ วิธีนี้จะช่วยให้พัฒนาโปรแกรมได้ง่าย เนื่องจากว่าการแก้ไขโปรแกรมไม่ต้องใช้ตัวล้างหน่วยความจำ ถ้าหากโปรแกรมที่พัฒนาเป็นโปรแกรมที่ใช้ได้จริงไม่มีการแก้ไขส่วนใดอีกแล้วก็นำรหัสโปรแกรมที่ได้โปรแกรมลงหน่วยความจำ ROM จริง ๆ เพื่อใช้งานต่อไป

ในปัจจุบันได้มีการพัฒนาระบบไมโครคอนโทรลเลอร์ด้วยเครื่องโปรแกรมแบบ single chip โดยใช้ไมโครคอนโทรลเลอร์ที่มีหน่วยความจำภายในเป็นแบบแฟลช (flash memory) และโปรแกรมข้อมูลลงไปภายในตัวชิปเลย โดยชิปประเภทนี้สามารถโปรแกรมและลบโปรแกรมได้มากกว่า 1,000 ครั้ง นอกจากนี้ยังได้มีการพัฒนาไมโครคอนโทรลเลอร์ในรูปแบบของ ISP (In-System Programming) โดยไม่ต้องใช้เครื่องโปรแกรมลงบนชิป แต่จะโหลดโปรแกรมเข้าไปทางพอร์ตอนุกรมของชิปเลย แต่ราคาของชิปจะมีราคาสูง



ไมโครคอนโทรลเลอร์ตระกูล MCS-51 ที่สามารถพัฒนาแบบ ISP ได้มีหลายเบอร์ เช่น AT89S8252 ของ Atmel, PIC16F84 ของ Microchip และตัวที่กำลังได้รับความนิยมในขณะนี้คือ P89C51RD+ , 89C51RD2 ของบริษัท Philips



บทที่

2

การเขียนโปรแกรมด้วย ภาษาแอสเซมบลี

การเขียนโปรแกรมภาษาแอสเซมบลีให้กับไมโครคอนโทรลเลอร์นั้นจะทำได้โดยการนำรหัสนิมิก (mnemonics) ของชุดคำสั่งมาต่อเรียงกัน ไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีชุดคำสั่งทั้งหมด 111 คำสั่ง โดยแบ่งออกได้ดังนี้

1. ชุดคำสั่งทางคณิตศาสตร์ 24 คำสั่ง โดยมีรหัสนิมิกเป็น ADD, ADDC, SUBB, INC, DEC, MUL, DIV, DA
2. ชุดคำสั่งทางลอจิก 25 คำสั่ง โดยมีรหัสนิมิกเป็น ANL, ORL, XRL, CLR, CPL, RL, RLC, RR, RRC, SWAP
3. ชุดคำสั่งการโอนย้ายข้อมูล 28 คำสั่ง โดยมีรหัสนิมิกเป็น MOV, MOVX, MOVC, PUSH, POP, XCH, XCHD
4. ชุดคำสั่งดำเนินการระดับบิต 17 คำสั่ง โดยมีรหัสนิมิกเป็น CLR, SETB, CPL, ANL, ORL, MOV, JC, JNC, JB, JNB, JBC
5. ชุดคำสั่งควบคุมการทำงานของโปรแกรม 17 คำสั่ง โดยมีรหัสนิมิกเป็น ACALL, LCALL, RET, RETI, AJMP, LJMP, SJMP, JMP, JZ, JNZ, CJNE, DJNZ, NOP

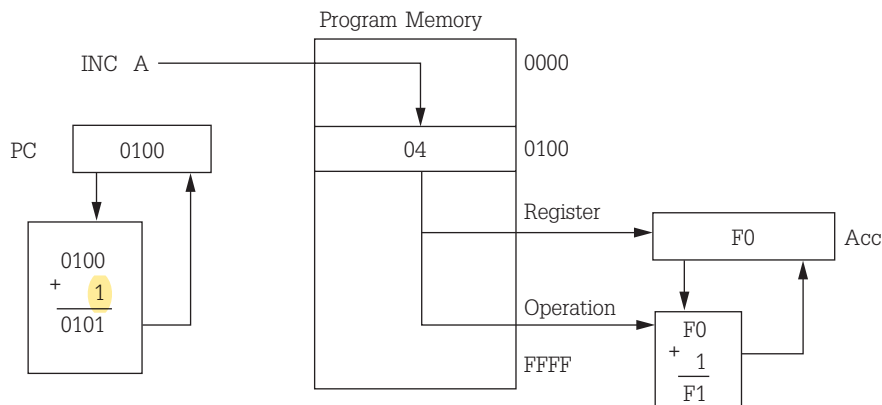


2.1 Addressing Modes

การที่ CPU จะนำข้อมูลเข้ามาประมวลผลนั้นสามารถทำได้หลายวิธี วิธีการนำข้อมูลที่อยู่ในตำแหน่งต่าง ๆ เข้ามาเรียกว่าโหมดการอ้างแอดเดรส (addressing modes) สำหรับไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะมีโหมดการอ้างแอดเดรสอยู่ 8 โหมด ได้แก่ Register addressing mode, Direct addressing mode, Immediate addressing mode, Indirect addressing mode, Based-indexed indirect addressing mode, Relative addressing mode, Extended addressing mode และ Implied addressing mode

2.1.1 Register Addressing Mode

เป็นการอ้างแอดเดรสโดยใช้รีจิสเตอร์ ข้อมูลที่นำมาประมวลผลจะเป็นข้อมูลที่ถูกเก็บอยู่ในรีจิสเตอร์โดยตรง โดย MCS-51 ได้ออกแบบมาให้ประมวลผลกับรีจิสเตอร์แอกคิวมูเลเตอร์ รีจิสเตอร์ B รีจิสเตอร์ DPTR และรีจิสเตอร์ R0 ถึง R7 ที่เก็บอยู่ในแบงค์ที่กำลังทำงานอยู่ สำหรับรูปที่ 2.1 เป็นตัวอย่างการทำคำสั่ง INC A ซึ่งจะใช้การอ้างแอดเดรสแบบ Register addressing mode



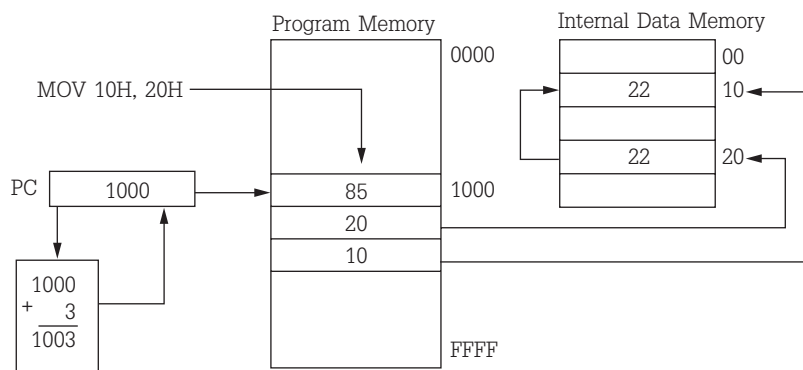
รูปที่ 2.1 การอ้างแอดเดรสแบบ Register addressing mode

ในรูปที่ 2.1 เป็นคำสั่งเพิ่มค่าในรีจิสเตอร์ A (`INC A`) โดยมีรหัสคำสั่งภาษาเครื่องขนาดหนึ่งไบต์เป็น `04H` จากรูปจะเห็นว่าถ้าคำสั่งนี้อยู่ที่แอดเดรส `0100` และในรีจิสเตอร์ A มีค่า `F0H` อยู่ เมื่อ MCS-51 ทำคำสั่งนี้มันจะเพิ่มค่าใน PC ขึ้นหนึ่งค่าเพื่อชี้คำสั่งถัดไป จากนั้นจะเพิ่มค่าในรีจิสเตอร์ A ขึ้นหนึ่งค่าเป็น `F1H` และเก็บไว้ในรีจิสเตอร์ A ตามเดิม



2.1.2 Direct Addressing Mode

เป็นโหมดการอ้างแอดเดรสที่ติดต่อกับข้อมูลที่อยู่ในหน่วยความจำโดยตรง ตัวอย่างเช่นการทำคำสั่ง MOV 10H,20H จะเป็นการอ่านไบต์ข้อมูลที่อยู่ในตำแหน่งแอดเดรส 20H มาเก็บในตำแหน่งแอดเดรส 10H ดังรูปที่ 2.2

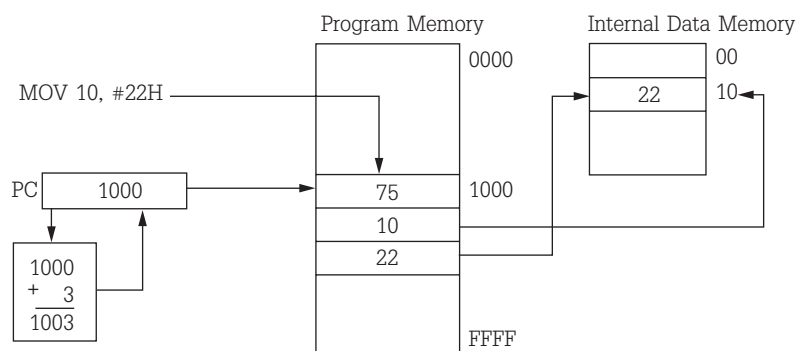


รูปที่ 2.2 การอ้างแอดเดรสแบบ Direct addressing mode

รหัสคำสั่งของ MOV 10H,20H จะมีขนาด 3 ไบต์ โดยไบต์แรกเป็นออปโค้ดคือ 85H อีกสองไบต์ต่อมาจะเป็นแอดเดรสข้อมูลต้นทางและแอดเดรสข้อมูลปลายทาง ในที่นี้คือ 20H และ 10H ถ้าหากคำสั่งนี้อยู่ในตำแหน่ง 1000 เมื่อ MCS-51 ทำคำสั่งนี้ รีจิสเตอร์ PC จะชี้ไปที่ตำแหน่ง 1003H ซึ่งเป็นแอดเดรสของคำสั่งถัดไป

2.1.3 Immediate Addressing Mode

การอ้างแอดเดรสแบบนี้ข้อมูลที่นำมาประมวลผลจะอยู่ในรหัสคำสั่งโดยตรง การเขียนรหัสนี้โมนิกจะใช้เครื่องหมาย # (pound sign) นำหน้าค่าของข้อมูล เช่น MOV 10H,#22H จะเป็นการนำค่าข้อมูล 22H มาเก็บในแอดเดรส 10H พิจารณารูปที่ 2.3



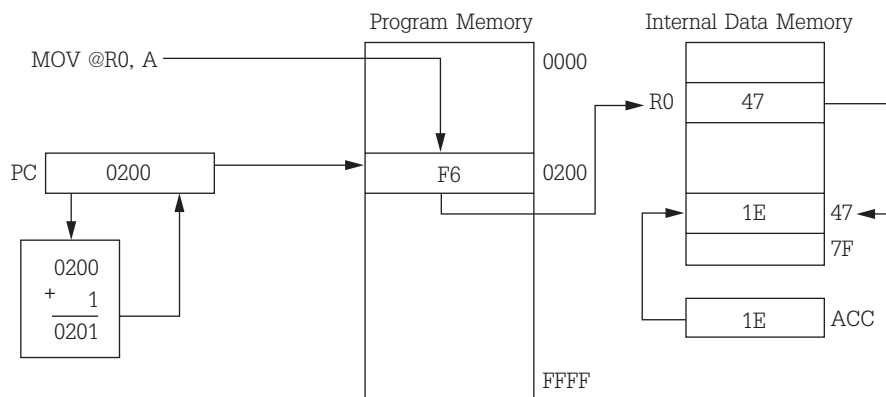
รูปที่ 2.3 การอ้างแอดเดรสแบบ immediate addressing mode



จากรูปที่ 2.3 จะพบว่าคำสั่ง MOV 10H,#22H เป็นคำสั่งที่มีขนาด 3 ไบต์ โดยไบต์แรกเป็น ออปโค้ดมีค่าเป็น 75 ไบต์ต่อมาเป็นตำแหน่งของข้อมูลปลายทาง ไบต์สุดท้ายเป็นค่าของข้อมูลที่จะนำมา ดำเนินการ หลังจาก MCS-51 ทำคำสั่งนี้ PC จะชี้ที่แอดเดรส 1003 ซึ่งเป็นแอดเดรสของคำสั่งถัดไป

2.1.4 Indirect Addressing Mode

ในโหมดนี้จะเป็นการอ้างแอดเดรสโดยอ้อม โดยกำหนดให้ใช้รีจิสเตอร์ R0, R1 หรือ DPTR เก็บค่า ตำแหน่งของหน่วยความจำที่ต้องการดำเนินการ โดยถ้าหากต้องการติดต่อกับหน่วยความจำข้อมูลภายใน จะใช้รีจิสเตอร์ R0 หรือ R1 เป็นตัวชี้ตำแหน่งหน่วยความจำ ถ้าต้องการติดต่อกับหน่วยความจำข้อมูลภายนอก จะใช้รีจิสเตอร์ DPTR เป็นตัวชี้ ในการเขียนรหัสนี้โมดิกจะใช้สัญลักษณ์ @ เป็นตัวบอกว่ารีจิสเตอร์นี้เก็บค่า ตำแหน่งหน่วยความจำ ตัวอย่างเช่นการทำคำสั่ง MOV @R0,A การทำงานจะเป็นดังรูปที่ 2.4



รูปที่ 2.4 การอ้างแอดเดรสแบบ Indirect addressing mode

จากรูปที่ 2.4 จะเห็นว่าคำสั่ง MOV @R0 , A จะมีขนาด 1 ไบต์ โดยมีรหัสคำสั่งเป็น F6 หลังจาก ทำคำสั่ง PC จะชี้ไปที่แอดเดรส 0201 คำสั่งนี้จะเป็นการนำข้อมูลที่เก็บอยู่ในแอมคิวมูลเตอร์ไปเก็บใน แอดเดรสที่ชี้โดยรีจิสเตอร์ R0 ถ้าหาก R0 มีค่าเป็น 47H ก็จะเป็นการนำข้อมูลในรีจิสเตอร์ A ไปเก็บในหน่วย ความจำข้อมูลภายในตำแหน่ง 47H

สำหรับคำสั่งอื่น ๆ ที่ใช้การอ้างแอดเดรสในโหมดนี้ได้แก่

MOV A,@R1	นำข้อมูลจากแอดเดรสที่ชี้โดย R1 มาเก็บในรีจิสเตอร์ A
MOVBX @R0,A	นำข้อมูลจากรีจิสเตอร์ A ไปเก็บยังหน่วยความจำภายนอกที่ชี้โดย R0
MOVBX @DPTR,A	นำข้อมูลจากรีจิสเตอร์ A ไปเก็บยังหน่วยความจำภายนอกที่ชี้โดย DPTR



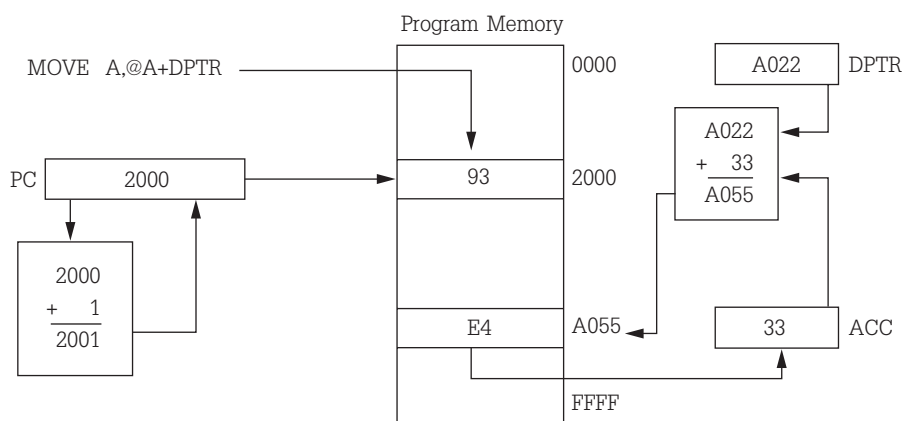
2.1.5 Based-Indexed Indirect Addressing Mode

การอ้างแอดเดรสในโหมดนี้จะใช้รีจิสเตอร์สองตัวในการอ้างแอดเดรส โดยจะอ่านข้อมูลออกมาในลักษณะของการเปิดตาราง (look-up tables) ซึ่งจะมีคำสั่งอยู่สองรูปแบบคือ

```
MOVC A,@A+DPTR
```

```
MOVC A,@A + PC
```

จากคำสั่งจะเห็นว่าเป็นการอ่านข้อมูลจากหน่วยความจำโปรแกรมมาเก็บในรีจิสเตอร์ A โดยใช้รีจิสเตอร์ DPTR หรือ PC เป็นตัวชี้ตำแหน่งเริ่มต้นของตารางจากนั้นนำค่าใน A เป็นตัวระบุตำแหน่งของข้อมูลที่อยู่ในตารางนั้น ดังแสดงในรูปที่ 2.5



รูปที่ 2.5 การอ้างแอดเดรสแบบ Based-indexed indirect addressing mode

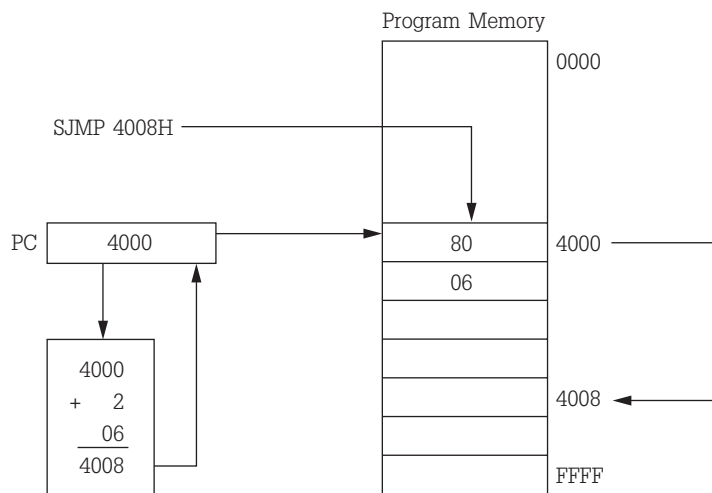
จากรูปที่ 2.5 จะเห็นว่าการทำคำสั่ง `MOVC A,@A+DPTR` นั้นจะนำข้อมูลที่รีจิสเตอร์ A และ DPTR ซ้ำเข้ามาเก็บในรีจิสเตอร์ A โดยให้ DPTR ชี้ที่ตำแหน่งเริ่มต้นคือแอดเดรส A022H และถ้าใน ACC มีค่าเป็น 33H จะเป็นการนำข้อมูลจากแอดเดรส A055H มาเก็บในรีจิสเตอร์ A

2.1.6 Relative Addressing Mode

การอ้างแอดเดรสประเภทนี้จะใช้กับคำสั่งควบคุมการทำงานของโปรแกรม เช่นคำสั่งกระโดด คำสั่งเรียกโปรแกรมย่อย โดยจะเป็นการระบุตำแหน่งของหน่วยความจำที่อยู่ด้านหน้า หรือด้านหลังของตำแหน่งที่รีจิสเตอร์ PC ชี้อยู่ โดยนำค่าในรีจิสเตอร์ PC บวกกับค่าตัวเลขแบบ 2's complement พิจารณาตัวอย่างในรูปที่ 2.6 ถ้าหากที่แอดเดรส 4000 เก็บคำสั่ง `SJMP 4008` เพื่อให้โปรแกรมกระโดดไปยังแอดเดรส 4008 โดยคำสั่งนี้จะมีรหัสคำสั่งขนาด 2 ไบต์ ไบต์แรกเป็นออปโค้ดมีค่าเป็น 80H เพื่อบอกว่าเป็นคำสั่งกระโดด



ส่วนไบต์ต่อไปเป็นการอ้างแอดเดรสแบบสัมพัทธ์ (relative) มีค่าเป็น 06H เมื่อ MCS-51 ทำงานจะนำค่า 06H นี้ไปบวกกับค่าที่ PC ซ้ำอยู่ในขณะนั้น ซึ่งจะได้เป็น $4000H + 2 + 06H = 4008H$ ซึ่งจะเป็นคำสั่งต่อไปที่ MCS-51 จะกระโดดไปทำงาน



รูปที่ 2.6 การอ้างแอดเดรสแบบ Relative addressing mode

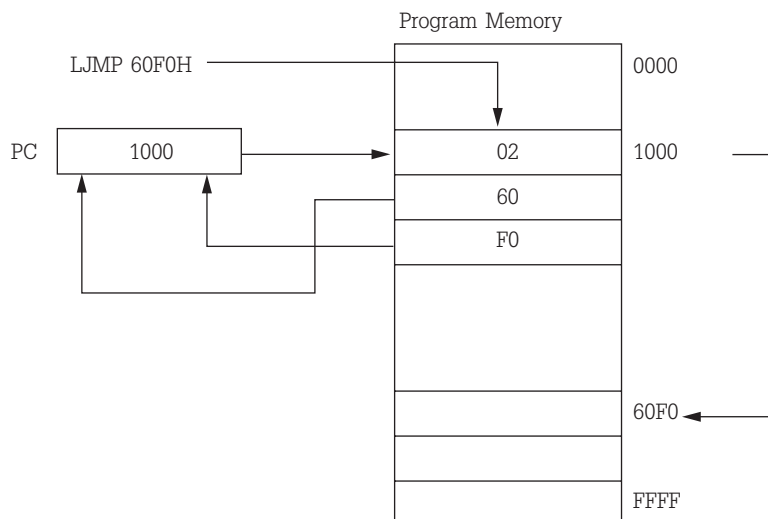
2.1.7 Extended Addressing Mode

ในโหมดนี้จะเป็นการเข้าถึงข้อมูลที่อยู่ในแอดเดรสในหน่วยความจำทั้งหมดที่ MCS-51 สามารถมีได้ตามที่ทราบมาแล้วว่า MCS-51 สามารถมีหน่วยความจำข้อมูลได้ 64 กิโลไบต์ และมีหน่วยความจำโปรแกรมได้ 64 กิโลไบต์ ในรูปที่ 2.7 แสดงการทำคำสั่งกระโดดไปยังหน่วยความจำแอดเดรส 60F0H โดยคำสั่งนี้จะมีรหัสคำสั่งจำนวน 3 ไบต์ โดยไบต์แรกจะเป็นอ็อปโค้ด อีกสองไบต์ต่อมาจะเป็นแอดเดรสไบต์สูงและแอดเดรสไบต์ต่ำ

2.1.8 Implied Addressing Mode

ในโหมดนี้จะไม่มีการอ้างตำแหน่งของข้อมูล หมายถึงคำสั่งที่ไม่มีโอเปอเรนด์ (operands) เช่นคำสั่ง

NOP เป็นต้น



รูปที่ 2.7 การอ้างแอดเดรสแบบ Extended addressing mode



2.2 ภายใน MCS-51

หัวข้อนี้จะแสดงการใช้รีจิสเตอร์ภายใน MCS-51 โดยจะพิจารณาจากคำสั่งพื้นฐานสองคำสั่งคือคำสั่ง MOV และ ADD ภายใน MCS-51 รีจิสเตอร์ภายในจะมีขนาด 8 บิต ส่วนรีจิสเตอร์ที่ใช้ในการอ้างตำแหน่งจะมีขนาด 16 บิต โดยรีจิสเตอร์ที่ใช้งานทั่วไปได้แก่ A (แอกคิวมูเลเตอร์), B, R0, R1, R2, R3, R4, R5, R6, R7, DPTR (data pointer) และ PC (program counter) โดยรีจิสเตอร์ทุกตัวจะเป็นแบบ 8 บิต ยกเว้น DPTR และ PC ที่เป็นแบบ 16 บิต ตัวแอกคิวมูเลเตอร์หรือรีจิสเตอร์ A จะใช้ได้กับทุกคำสั่งทางคณิตศาสตร์และลอจิก

คำสั่ง MOV

คำสั่งนี้จะใช้สำเนาข้อมูลจากตำแหน่งต้นทางไปยังตำแหน่งปลายทาง รูปแบบของคำสั่งเป็นดังนี้

MOV destination,source

คำสั่ง MOV นี้จะใช้ในการโอนย้ายข้อมูลภายใน CPU เท่านั้น ตัวอย่างเช่น MOV A,R0 จะเป็นการย้ายข้อมูลที่เก็บใน R0 ไปเก็บในรีจิสเตอร์ A หลังจากทำคำสั่งแล้วค่าใน A จะเท่ากับค่าใน R0 โอเปอเรชั่นต้นทางของคำสั่งนี้สามารถเป็นตำแหน่งของหน่วยความจำภายในโดยตรงก็ได้เช่น MOV A,50H หมายความว่าให้คัดลอกค่าที่เก็บในหน่วยความจำภายในตำแหน่ง 50H มาเก็บในรีจิสเตอร์ A หรือจะใส่เป็นข้อมูลโดยตรงก็ได้ โดยเครื่องหมาย # นำหน้าข้อมูลนั้น ตัวอย่างการใช้คำสั่ง MOV มีดังนี้



```
MOV A,#40H      ; นำค่า 40H ใส่ในรีจิสเตอร์ A
MOV R0,A        ; นำค่าในรีจิสเตอร์ A เก็บใน R0 (ดังนั้น R0 = A = 40H)
MOV R1,A        ; นำค่าใน A เก็บใน R1 (ดังนั้น R1 = A = R0 = 40H)
MOV R2,A        ; นำค่าใน A เก็บใน R2 (ดังนั้น A = R0 = R1 = R2 = 40H)
MOV R3,#45H     ; นำค่า 45 เก็บใน R3 (ดังนั้น R3 = 45H)
MOV A,R3        ; นำค่าใน R3 เก็บใน A (ดังนั้น A = R3 = 45H)
```

ในการเขียนคำสั่งให้ไมโครคอนโทรลเลอร์ MCS-51 ทำงานมีข้อควรระวังดังนี้

1. การใส่ค่าให้กับรีจิสเตอร์ A,B หรือ R0-R7 ถ้าเป็นการป้อนข้อมูลโดยตรงจะต้องใส่เครื่องหมาย # นำหน้าข้อมูลนั้น ดังตัวอย่างต่อไปนี้

```
MOV A,#23H      ; นำค่า 23H ใส่ในรีจิสเตอร์ A (A = 23H)
MOV R0,#12H     ; นำค่า 12H มาใส่ในรีจิสเตอร์ R0
MOV R1,#1FH     ; นำค่า 1FH มาใส่ในรีจิสเตอร์ R1
MOV R2,#2BH     ; นำค่า 2BH มาเก็บในรีจิสเตอร์ R2
MOV B,#3CH      ; นำค่า 3CH มาเก็บในรีจิสเตอร์ B
MOV R7,#9DH     ; นำค่า 9DH มาเก็บในรีจิสเตอร์ R7
MOV R5,#0F9H    ; นำค่า F9H มาเก็บในรีจิสเตอร์ R5
MOV R6,#12      ; นำค่า 12 แลวนสิบ หรือ 0CH มาเก็บในรีจิสเตอร์ R6
```

จะสังเกตเห็นว่าในคำสั่ง MOV R5,#0F9H จะต้องใส่เลข 0 ตามหลัง # เนื่องจากเลขฐานสิบหกค่าแรกเป็น F ถ้าเขียนเป็น MOV R5,#F9H จะเกิดข้อผิดพลาด

2. ถ้าหากใส่ค่า 0 ถึง F ลงในรีจิสเตอร์ 8 บิต สามารถเขียนเป็นเลขหลักเดียว ไม่ต้องมี 0 นำหน้าก็ได้ เช่น MOV A,#5 จะทำให้ค่าใน A = 05 ข้อมูลในรูปแบบของเลขฐานสองในรีจิสเตอร์ A จะเป็น 00000101
3. ถ้าหากนำค่าข้อมูลที่โตกว่าที่รีจิสเตอร์สามารถจะเก็บได้ไปเก็บในรีจิสเตอร์ จะทำให้เกิดข้อผิดพลาด ตัวอย่างเช่น

```
MOV A,#7F2H     ; ค่า 7F2H > 8 บิต (FFH)
MOV R2,#456     ; ค่า 456 > 255 (FFH)
```

4. ในการป้อนข้อมูลเข้าในรีจิสเตอร์ ถ้าหากไม่มีเครื่องหมาย # จะหมายถึงเป็นการนำข้อมูลจากหน่วยความจำเช่น MOV A,17H หมายความว่านำข้อมูลที่เก็บอยู่ในหน่วยความจำตำแหน่ง 17H มาเก็บใน A ซึ่งค่าจะไม่เท่ากับ MOV A,#17H

คำสั่ง ADD

คำสั่งนี้ใช้สำหรับการบวกเลข มีรูปแบบดังนี้

```
ADD A,source    ; บวกค่าจากโอเปอเรนด์ต้นทางกับค่าในแอมคิวมูลเตอร์และ
                  ; เก็บผลลัพธ์ในแอมคิวมูลเตอร์
```



คำสั่ง ADD นี้ CPU จะบวกข้อมูลจากตำแหน่งต้นทางกับรีจิสเตอร์ A และเก็บผลลัพธ์ในรีจิสเตอร์ A ถ้าหากต้องการบวกค่า 25H กับค่า 34H จะต้องย้ายข้อมูลไปเก็บในรีจิสเตอร์ก่อน จากนั้นใช้คำสั่งบวก ซึ่งสามารถเขียนคำสั่งได้ดังนี้

```
MOV  A,#25H      ; โหลดค่า 25H ไปเก็บใน A
MOV  R2,#34H     ; โหลดค่า 34H ไปเก็บในรีจิสเตอร์ R2
ADD  A,R2        ; บวกค่าใน R2 กับแอดคิวมูลเตอร์
                        ; (A = A + R2)
```

จากการทำคำสั่งผลลัพธ์ที่ได้ $A = 59H$ ($25H + 34H = 59H$) และค่าใน R2 จะเท่ากับ 34H โดยหลังจากการบวกค่าข้อมูลใน R2 จะไม่เปลี่ยนแปลง การบวกเลขดังกล่าวสามารถเขียนคำสั่งได้หลายแบบขึ้นกับจำนวนรีจิสเตอร์ที่จะใช้ ตัวอย่างเช่น

```
MOV  R5,#25H     ; โหลดค่า 25H ใส่ใน R5 (R5 = 25H)
MOV  R7,#34H     ; โหลดค่า 34H ใส่ใน R7 (R7 = 34H)
MOV  A,#0        ; โหลดค่า 0 ลงใน A (A = 0)
ADD  A,R5        ; บวกค่าใน A ด้วยค่าใน R5
                        ; A = A + R5
ADD  A,R7        ; บวกค่าใน A ด้วยค่าใน R7
                        ; A = A + R7
```

ในการเขียนคำสั่ง ADD นี้สามารถใช้โอเปอเรนด์ที่มีการอ้างแอดเดรสแบบ immediate ได้ เช่น การเขียนคำสั่งต่อไปนี้

```
MOV  A,#25H      ; A = 25H
ADD  A,#34H      ; บวกค่าใน A กับ 34H ผลลัพธ์เก็บใน A
```

ตำแหน่งปลายทางของคำสั่ง ADD ต้องเป็นรีจิสเตอร์ A เท่านั้น ถ้าเป็นคำสั่ง ADD R2,#12H จะทำงานไม่ได้



2.3 การเขียนโปรแกรมภาษาแอสเซมบลีเบื้องต้น

ไมโครคอนโทรลเลอร์หรือ CPU สามารถทำงานได้กับคำสั่งที่อยู่ในรูปของเลขฐานสองเท่านั้น โปรแกรมที่ประกอบด้วยเลขฐานสองคือลอจิก “0” และ “1” เรียกว่าเป็นภาษาเครื่อง (machine language) แต่การเขียนโปรแกรมในลักษณะนี้จะใช้ได้ดีกับการเขียนโปรแกรมสั้น ๆ ทุกวันนี้การเขียนชุดคำสั่งต่าง ๆ ให้กับ CPU จะใช้รหัสช่วยจำ (mnemonic) แทนเลขฐานสองแต่ละคำสั่ง เรียกว่าการเขียนโปรแกรมภาษา



แอสเซมบลี เนื่องจากสามารถเข้าใจโปรแกรมได้ง่าย เมื่อจะให้โปรแกรมทำงานจะต้องแปลภาษาแอสเซมบลีให้เป็นรหัสภาษาเครื่องอีกที ขั้นตอนนี้เรียกว่า แอสเซมเบลอร์ (assembler)

ภาษาแอสเซมบลีถือว่าเป็นภาษาระดับต่ำ (low-level language) เนื่องจากสามารถเข้าถึงโครงสร้างภายใน CPU ได้เป็นอย่างดี การเขียนโปรแกรมด้วยภาษานี้ ผู้เขียนโปรแกรมจะต้องเข้าใจส่วนประกอบภายใน CPU และรีจิสเตอร์ต่าง ๆ ของ CPU เป็นอย่างดี

การเขียนโปรแกรมให้กับไมโครคอนโทรลเลอร์ในปัจจุบันสามารถเขียนด้วยภาษาระดับสูงได้ เช่น ภาษา BASIC หรือภาษาซี แต่ถ้าจะให้ไมโครคอนโทรลเลอร์ทำงานจะต้องแปลภาษาเหล่านี้ให้เป็นรหัสภาษาเครื่องอีกทีหนึ่ง โดยใช้ตัวคอมไพเลอร์ (compiler) เป็นตัวแปรภาษา สำหรับในหัวข้อนี้จะกล่าวถึงรูปแบบทั่ว ๆ ไปในการเขียนโปรแกรมภาษาแอสเซมบลีให้กับไมโครคอนโทรลเลอร์ตระกูล MCS-51

โครงสร้างของภาษาแอสเซมบลี

การเขียนโปรแกรมภาษาแอสเซมบลีจะทำได้โดยนำรหัสนิโมนิคของแต่ละคำสั่งมาต่อเรียงกัน รหัสนิโมนิคแต่ละตัวอาจมีโอเปอเรนด์หรือไม่ก็ได้ และการเขียนโปรแกรมด้วยภาษาแอสเซมบลีนี้ สามารถนำคำสั่งเทียมมาใช้ได้ ซึ่งจะทำให้การเขียนโปรแกรมทำได้ง่ายขึ้น ตัวอย่างการเขียนโปรแกรมภาษาแอสเซมบลีแ ดงได้ดังโปรแกรมที่ 2.1

```
ORG 0000H           ; เริ่มต้นที่แอดเดรส 0000
MOV R5,#25H          ; โหลดค่า 25H ใส่ใน R5
MOV R7,#34H          ; โหลดค่า 34H ใส่ใน R7
MOV A,#0             ; โหลดค่า 0H ใส่ในรีจิสเตอร์ A
ADD A,R5             ; บวกค่า R5 กับ A ผลลัพธ์เก็บใน A
ADD A,R7             ; บวกค่า R7 กับ A ผลลัพธ์เก็บใน A
ADD A,#12H           ; บวกค่า A กับค่า 12H ผลลัพธ์เป็น
                    ; A = A + 12H
HERE: SJMP HERE      ; วนลูปซ้ำอยู่กับที่
END
```

โปรแกรมที่ 2.1 ตัวอย่างภาษาแอสเซมบลีอย่างง่าย

จากโปรแกรมที่ 2.1 จะเป็นตัวอย่างโปรแกรมภาษาแอสเซมบลีที่ใช้คำสั่ง MOV และ ADD ส่วนคำสั่ง ORG และ END เป็นคำสั่งเทียม (pseudo-instruction) บางครั้งเรียกว่าไดเรกทีฟ (directives) ซึ่งช่วยในการเขียนโปรแกรม คำสั่ง ORG 0000H เป็นการบอกว่าให้ออปโค้ดเริ่มต้นที่หน่วยความจำตำแหน่งที่ 0 ส่วน END บอกว่าเป็นการจบโปรแกรม



ในแต่ละบรรทัดของภาษาแอสเซมบลีจะแบ่งออกเป็น 4 ส่วนดังนี้

[label:] mnemonic [operands] [;comment]

ส่วนที่อยู่ในเครื่องหมาย bracket จะมีหรือไม่ก็ได้ แต่ละส่วนของโปรแกรมจะเป็นดังนี้

1. 'วนลาเบล (label) จะเป็นส่วนที่ไม่ถูกแปลเป็นรหัสคำสั่ง จะใช้แทนค่าแอดเดรสหน่วยความจำที่เก็บคำสั่งนั้น โดยจะตามด้วยเครื่องหมายโคลอน

2. 'วนที่สองอาจเป็นคำสั่งเทียม หรือเป็นคำสั่งภาษาแอสเซมบลีที่ CPU รู้จัก ประกอบด้วยคำสั่งนิโมนิก (mnemonic) และโอเปอเรนด์ (operand) เช่น

ADD A,B

MOV A,#67

จากตัวอย่างส่วนที่เขียนเป็น ADD และ MOV จะเป็นรหัสนิโมนิก ส่วน "A,B" และ "A,#67" จะเป็นโอเปอเรนด์ (operands)

3. 'วนคำอธิบาย (comment) จะใช้อธิบายต่าง ๆ จะต้องเขียนตามหลักเครื่องหมายเซมิโคลอน ";" ส่วนนี้จะไม่ถูกแปลเป็นรหัสคำสั่ง

4. จากโปรแกรมที่ 2.1 จะเห็นว่าส่วนลาเบล HERE จะตามด้วยเครื่องหมายโคลอน และนิโมนิกเป็นคำสั่ง SJMP (short jump instruction) ซึ่งจะทำให้โปรแกรมวนซ้ำกลับมาทำคำสั่งเดิม



คำถาม ?

1. คำสั่งเทียมที่ใช้กันมากได้แก่คำสั่งใด
2. ภาษาแอสเซมบลีเป็นภาษาระดับต่ำหรือภาษาระดับสูง
3. คำสั่งใดเป็นคำสั่งเทียม
 - ก. ADD A,R1 ข. MOV A,#12 ค. ORG 2000H ง. SJMP HERE
4. คำสั่งเทียมเรียกอีกชื่อหนึ่งว่าอะไร

ไฟล์ ASM และไฟล์ OBJ

การเขียนโปรแกรมภาษาแอสเซมบลีบนโปรแกรม DOS, EDIT หรือ WINDOW'S Notepad ที่เป็นไฟล์นามสกุล ASM นั้นจะเรียกว่าไฟล์ต้นฉบับ (source file) เมื่อใช้โปรแกรมแอสเซมเบลอร์แปลงให้เป็นรหัสภาษาเครื่องนั้นจะได้ไฟล์ที่มีนามสกุลเป็น obj (object) และในการแอสเซมเบลอร์โปรแกรมนั้นสามารถร่างไฟล์นามสกุล lst (list file) ออกมาได้ด้วย ไฟล์นามสกุล lst นี้จะเป็นไฟล์ที่แสดงค่า opcodes และ address ของโปรแกรมภาษาแอสเซมบลีออกมาหลังจากการแอสเซมเบลอร์โปรแกรม ซึ่งเราสามารถดูในโปรแกรม



dos edit หรือพิมพ์ออกมาทางเครื่องพิมพ์ได้ และสามารถใช้ไฟล์นี้ดูข้อผิดพลาดจากการเขียนโปรแกรมได้อีกด้วย จากตัวอย่างโปรแกรมที่ 2.1 ถ้าหากโปรแกรมไม่มีข้อผิดพลาดไฟล์นามสกุล .lst ที่ได้ออกมาจะเป็นดังนี้

	แอดเดรส	รหัสคำสั่งภาษาเครื่อง			
	↓	↓			
1.	0000		ORG	0000H	; เริ่มต้นโปรแกรม
2.	0000	7D25	MOV	R5,#25H	; โหลดค่า 25H ใส่ใน R5
3.	0002	7F34	MOV	R7,#34H	; โหลดค่า 34 ใส่ใน R7
4.	0004	7400	MOV	A,#0	; โหลดค่า 0 ใส่ในรีจิสเตอร์ A
5.	0006	2D	ADD	A,R5	; บวกค่า R5 กับ A ผลลัพธ์เก็บใน A
6.	0007	2F	ADD	A,R7	; บวกค่า R7 กับ A ผลลัพธ์เก็บใน A
7.	0008	2412	ADD	A,#12H	; บวกค่า A กับค่า 12H ผลลัพธ์เป็น ; A = A + 12H
8.	000A	80FE	HERE: SJMP	HERE	; วนลูปซ้ำอยู่กับที่
9.	000C		END		

จากโปรแกรมจะเห็นว่าแอดเดรสเริ่มต้นจะเป็น 0000 ซึ่งจะเก็บออปโค้ดที่มีค่าเป็น 7D เมื่อ MCS-51 ตีความหมายจะเป็นตัวบอกว่าให้ย้ายข้อมูลมาใส่ในรีจิสเตอร์ R5 ส่วนโอเปอเรนด์จะอยู่ที่แอดเดรส 0001H จะเก็บค่า 25H ดังนั้น คำสั่ง MOV R5,#25H จะมีรหัสภาษาเครื่องเป็น 7D25 โดย 7D เป็นออปโค้ดและ 25 เป็นโอเปอเรนด์ คำสั่งต่อมาจะมีขนาด 2 ไบต์จะมีรหัสภาษาเครื่องเป็น 7F34 โดยจะเก็บอยู่ในหน่วยความจำตำแหน่ง 0002 และ 0003 ตามลำดับ

การทำงานของโปรแกรม

จากโปรแกรมที่ 2.1 จะพบว่ารหัสโปรแกรมจะเริ่มต้นที่แอดเดรส 0000 ถ้าจะให้ MCS-51 ทำงานจะต้องนำรหัสโปรแกรมนี้ไปเก็บในหน่วยความจำ ROM ตั้งแต่แอดเดรส 0000 เป็นต้นไป ถ้าหากโปรแกรมเคาน์เตอร์ (PC) ซ้ำอยู่ที่แอดเดรสของหน่วยความจำ ROM ตำแหน่งใดหมายความว่า CPU จะต้องทำงานคำสั่งที่เก็บอยู่ในแอดเดรสนั้น เมื่อเริ่มจ่ายไฟให้ MCS-51 หรือ MCS-51 ถูกรีเซ็ต การทำงานจะเป็นขั้นตอนดังต่อไปนี้

1. รีจิสเตอร์โปรแกรมเคาน์เตอร์ PC จะมีค่าเป็น 0000 ดังนั้น CPU จะเริ่มอ่านรหัสคำสั่งที่แอดเดรสนี้ และได้อปโค้ดค่าแรกเป็น 7D เข้ามา จากนั้นค่า PC จะเพิ่มขึ้น 1 ค่า และ CPU จะตีความว่าจะต้องนำข้อมูลจากโอเปอเรนด์มาเก็บใน R5 โดยโอเปอเรนด์จะเป็นข้อมูลที่ PC ซ้ำอยู่ ดังนั้น จะนำค่า 25H มาเก็บใน R5 และ PC เพิ่มค่าอีกหนึ่งค่าเป็น 0002 ซึ่งจะเป็นการจบการทำงานคำสั่งแรก



2. MCS-51 ทำคำสั่งต่อไป โดยอ่านรหัสคำสั่งที่ PC ซ้ำอยู่ และจะได้แอดเดรสค่า 7F เข้ามา และจะนำค่า 34H มาเก็บใน R7 จากนั้น PC จะไปที่แอดเดรส 0004

3. ที่หน่วยความจำ ROM ตำแหน่ง 0004 เก็บรหัสของคำสั่ง MOV A,#0 เมื่อ CPU ทำคำสั่งนี้จบ รีจิสเตอร์ PC จะชี้ไปที่ตำแหน่ง 0006

4. ที่ตำแหน่ง 0006 เก็บรหัสของคำสั่ง ADD A,R5 โดยคำสั่งนี้มีขนาด 1 ไบต์ เมื่อ CPU ทำคำสั่งนี้จบ รีจิสเตอร์ PC จะชี้ไปที่ตำแหน่ง 0007

5. ที่ตำแหน่ง 0007 รหัสคำสั่งคือ 2F ซึ่งมีขนาด 1 ไบต์เช่นกัน ดังนั้นหลังจากทำคำสั่งนี้ PC จะมีค่าเป็น 0008 จากนั้นจะทำคำสั่งต่อไปจนจบโปรแกรม

จากขั้นตอนการทำงานของโปรแกรมที่กล่าวมาจะเห็นว่าถ้าหากโปรแกรมเคาน์เตอร์มีค่าใด CPU จะทำคำสั่งที่อยู่ในตำแหน่งนั้น ในไมโครโปรเซสเซอร์บางตัวจะเรียกโปรแกรมเคาน์เตอร์นี้ว่า ตัวชี้คำสั่ง (instruction pointer) เช่นใน CPU ตระกูล x86



2.4 ประเภทของข้อมูลและคำสั่งเทียม

การเขียนโปรแกรมด้วยภาษาแอสเซมบลีสามารถเขียนคำสั่งเทียม (pseudo instructions)แทรกเข้าไปในโปรแกรมได้ โดยคำสั่งประเภทนี้จะไม่ถูกโปรแกรมแอสเซมเบลอร์แปลความหมายออกเป็นคำสั่งของไมโครคอนโทรลเลอร์ แต่จะช่วยให้การเขียนโปรแกรมทำได้ง่ายขึ้น คำสั่งเทียมที่พบบ่อยได้แก่

คำสั่งเทียม	ความหมาย
DB (define byte)	กำหนดให้ข้อมูลมีขนาด 1 ไบต์ ที่แอดเดรสของบรรทัดนั้น
ORG (origin)	กำหนดแอดเดรสเริ่มต้นของชุดคำสั่งในบรรทัดนั้น
EQU (equate)	กำหนดให้ลาเบลมีค่าเท่ากับโอเปอเรนด์ที่กำหนด
END	บอกจุดสิ้นสุดของโปรแกรม
BIT	กำหนดตำแหน่งบิตของข้อมูล

ในการเขียนโปรแกรมด้วยภาษาแอสเซมบลีนั้นจะมีข้อกำหนดบางประการเกี่ยวกับการกำหนดค่าให้กับลาเบล ตัวอักษร และสัญลักษณ์ต่าง ๆ ดังนี้

การตั้งชื่อ

การตั้งชื่อลาเบล เขียนคำสั่ง หรือประกาศตัวแปรต่าง ๆ นั้นการเขียนเป็นตัวอักษรภาษาอังกฤษเป็นตัวเขียนเล็ก หรือตัวใหญ่จะให้ผลออกมาเหมือนกัน



ค่าตัวเลข

ถ้าหากมีการกำหนดค่าโดยใส่เป็นตัวเลขตามปกติ จะหมายถึงเป็นเลขฐานสิบ (decimal number) ถ้าต้องการกำหนดเป็นเลขฐานอื่น ๆ จะต้องใส่ตัวอักษรกำกับดังต่อไปนี้

ตัวอักษร	ฐานตัวเลข	ตัวอย่าง
B	กำหนดเป็นเลขฐานสอง	10101110B
O	กำหนดเป็นเลขฐานแปด	107O
D	กำหนดเป็นเลขฐานสิบ	1235
H	กำหนดเป็นเลขฐานสิบหก	0E5H

ถ้าหากตัวเลขฐานสิบหกขึ้นต้นด้วยตัวอักษร จะต้องใส่เลขศูนย์นำหน้าเสมอ

การอ้างตำแหน่งของโปรแกรมเคาน์เตอร์

ถ้าหากต้องการกำหนดตำแหน่งปัจจุบันของโปรแกรมเคาน์เตอร์จะใช้ตัวอักษร \$ (Dollar sign) กำหนดในส่วนของโอเปอเรนด์คำสั่ง ตัวอย่างเช่นถ้าหากต้องการให้โปรแกรมทำลูปซ้ำ ๆ ไม่รู้จบจะเขียนคำสั่งเป็น

HERE : SJMP HERE

หรือสามารถเขียนอีกแบบหนึ่งได้เป็น

SJMP \$



2.5 รีจิสเตอร์แบงก์และสแตก

หน่วยความจำ RAM ภายในของ MCS-51 ที่อยู่ในตำแหน่ง 128 ไบต์แรก คือตั้งแต่แอดเดรส 00H ถึง 7FH จะถูกแบ่งออกเป็น 3 ส่วน ดังรูปที่ 2.8 โดยแต่ละส่วนมีความหมายดังนี้

1. ใน 32 ไบต์แรก ตั้งแต่แอดเดรส 00H ถึง 1FH จะถูกใช้เป็นส่วนของรีจิสเตอร์แบงก์และสแตก
2. อีก 16 ไบต์ต่อมา ตั้งแต่แอดเดรส 20H ถึง 2FH จะเป็นหน่วยความจำที่สามารถอ่านเขียนข้อมูลแบบบิตได้ และอ้างตำแหน่งเป็นตำแหน่งบิตได้
3. อีก 80 ไบต์ ตั้งแต่ตำแหน่ง 30H ถึง 7FH จะเป็นหน่วยความจำที่สามารถใช้เก็บข้อมูลทั่วไปได้



7F		หน่วยความจำใช้งานทั่วไป
30		
2F		
20		หน่วยความจำระดับบิต (Bit-Addressable RAM)
1F		
18		Register Bank 3
17		
10		Register Bank 2
0F		
08		Register Bank 1 (Stack)
07		
00		Register Bank 0

รูปที่ 2.8 ตำแหน่งหน่วยความจำของ RAM ภายใน 128 ไบต์แรก

รีจิสเตอร์แบบคี่ใน MCS-51

หน่วยความจำ RAM ภายใน 32 ไบต์แรก ถูกแบ่งออกเป็น 4 แบงก์ หน่วยความจำแต่ละไบต์ในแต่ละแบงก์จะถูกตั้งชื่อเป็นรีจิสเตอร์ 8 ตัว คือ R0 - R7 ดังนั้น RAM ภายในตำแหน่งแอดเดรส 0 ถึง 7 จะเป็นตำแหน่งแบงก์แรก (Bank 0) ซึ่งจะมีชื่อเรียกเป็นรีจิสเตอร์ R0 ถึง R7 ตามลำดับ สำหรับรีจิสเตอร์ R0 ถึง R7 ในแบงก์ต่อมาจะอยู่ที่ตำแหน่ง 08H ถึง 0FH โดยการเรียกชื่อรีจิสเตอร์แบงก์ทั้ง 4 แบงก์ที่ตำแหน่งต่าง ๆ แดงได้ดังรูปที่ 2.9

Bank 0		Bank 1		Bank 2		Bank 3	
7	R7	F	R7	17	R7	1F	R7
6	R6	E	R6	16	R6	1E	R6
5	R5	D	R5	15	R5	1D	R5
4	R4	C	R4	14	R4	1C	R4
3	R3	B	R3	13	R3	1B	R3
2	R2	A	R2	12	R2	1A	R2
1	R1	9	R1	11	R1	19	R1
0	R0	8	R0	10	R0	18	R0

รูปที่ 2.9 ชื่อรีจิสเตอร์แบงก์และตำแหน่งหน่วยความจำที่สอดคล้องกัน



ในการเขียนโปรแกรมประมวลผลกับรีจิสเตอร์ R0 - R7 โดยทั่วไปแล้วจะเป็นการติดต่อกับรีจิสเตอร์ในแบงก์ 0 เสมอ แต่ถ้าหากต้องการติดต่อกับรีจิสเตอร์ในแบงก์อื่น ๆ จะต้องกำหนดค่าในรีจิสเตอร์ PSW (Program status word) ซึ่งจะกล่าวถึงในหัวข้อต่อไป

ตัวอย่างที่ 2.1 จงหาค่าที่เก็บอยู่ในตำแหน่งหน่วยความจำ ถ้าหากมีการทำคำสั่งต่อไปนี้

```
MOV R0 , #99H      ; โหลดค่า 99H ใส่ใน R0
MOV R1 , #85H      ; โหลดค่า 85H ใส่ใน R1
MOV R2 , #3FH      ; โหลดค่า 3FH ใส่ใน R2
MOV R7 , #63H      ; โหลดค่า 63H ใส่ใน R7
MOV R5 , #12H      ; โหลดค่า 12H ใส่ใน R5
```

คำตอบ

หลังจากทำโปรแกรมแล้วค่าในหน่วยความจำ RAM ภายในจะเป็นดังนี้

ค่าใน RAM ตำแหน่ง 0 จะเก็บค่า 99H	ค่าใน RAM ตำแหน่ง 1 จะเก็บค่า 85H
ค่าใน RAM ตำแหน่ง 2 จะเก็บค่า 3FH	ค่าใน RAM ตำแหน่ง 7 จะเก็บค่า 63H
ค่าใน RAM ตำแหน่ง 5 จะเก็บค่า 12H\	



2.6 แฟล็กของ MCS-51 และรีจิสเตอร์ PSW

ข้อมูลในบิตแฟล็กจะเป็นตัวแสดงถึงผลจากการทำคำสั่งของ CPU และสถานะการทำงานต่าง ๆ อย่างเช่นถ้าหาก CPU ทำการบวกเลข และผลลัพธ์ที่ได้มีค่ามากกว่าที่รีจิสเตอร์สามารถจะเก็บได้ สถานะดังนี้จะถูกแสดงผลออกมาทางบิตแฟล็ก นอกจากนี้ยังสามารถใช้บิตแฟล็กกำหนดการทำงานให้กับ CPU ได้อีกด้วย

โปรแกรมสเตตัสเวิร์ด (PSW)

รีจิสเตอร์โปรแกรมสเตตัสเวิร์ด (Program Status Word : PSW) เป็นรีจิสเตอร์ขนาด 8 บิต ที่แต่ละบิตจะหมายถึงบิตแฟล็กที่บอกสถานะการทำงานต่าง ๆ รวมทั้งใช้กำหนดแบงก์ (bank) ของรีจิสเตอร์ที่ใช้งานด้วย ภายในรีจิสเตอร์จะประกอบด้วยบิตแฟล็กเงื่อนไข (conditional flags) จำนวนสี่บิต ได้แก่ แฟล็กตัวทด, แฟล็กช่วยทด, แฟล็กโอเวอร์โฟลว์ และแฟล็กพาริตี รวมทั้งบิตในการเลือกแบงก์อีกสองบิต ดังรูปที่ 2.10 รีจิสเตอร์ตัวนี้เป็นรีจิสเตอร์ที่สามารถเข้าถึงข้อมูลระดับบิตได้



CY	AC	F0	RS1	RS0	OV	-	P
----	----	----	-----	-----	----	---	---

บิต	ตำแหน่ง	ความหมาย
CY	PSW.7	แฟลกทาด (carry flag)
AC	PSW.6	แฟลกช่วยทาด (auxiliary carry flag)
F0	PSW.5	แฟลกสำหรับผู้กำหนดเองอย่างอิสระ
RS1	PSW.4	บิตเลือกแแบ่งค์ของรีจิสเตอร์บิต 1
RS0	PSW.3	บิตเลือกแแบ่งค์ของรีจิสเตอร์บิต 0
OV	PSW.2	แฟลกโอเวอร์โฟลว์ (overflow flag)
-	PSW.1	ไม่ใช้งาน
P	PSW.0	แฟลกพาริตี (parity flag)

รูปที่ 2.10 รีจิสเตอร์โปรแกรมสเตตัสเวิร์ด

รีจิสเตอร์ R0 - R7 ไม่ว่าจะเป็นรีจิสเตอร์ที่อยู่ในหน่วยความจำส่วนใด จะมีชื่อเดียวกันทั้งสิ้น แต่ถ้าหากอยู่ในหน่วยความจำแ่งค์ 0 รีจิสเตอร์ R0 - R7 จะอยู่ในแอดเดรส 00H - 07H จากรูปที่ 2.10 จะเห็นว่า PSW.3 และ PSW.4 จะเป็นตำแหน่งของ RS0 และ RS1 ถ้าหากต้องการใช้งานรีจิสเตอร์ในแ่งค์ใดจะต้องกำหนดค่าลงไปทั้งสองบิตนี้ โดยการกำหนดค่าแสดงได้ดังตารางต่อไปนี้

บิต RS1	บิต RS0	รีจิสเตอร์	ตำแหน่งหน่วยความจำ
0	0	แ่งค์ 0	00H - 07H
0	1	แ่งค์ 1	08H - 0FH
1	0	แ่งค์ 2	10H - 17H
1	1	แ่งค์ 3	18H - 1FH

โดยทั่วไปแล้วการพัฒนาทางด้านไมโครคอนโทรลเลอร์ MCS-51 มักจะใช้รีจิสเตอร์ R0 - R7 ในแ่งค์ที่ 0 เท่านั้น ส่วนที่หน่วยความจำแ่งค์อื่น ๆ มักจะถูกใช้งานเป็น RAM ตามปกติ สำหรับบิตแฟลกอีก 4 บิตในรีจิสเตอร์ PSW เป็นบิตที่ใช้บอกสถานะของข้อมูลหลังจากมีการทำคำสั่งต่าง ๆ โดยแต่ละบิตมีความหมายดังนี้

แฟลกทาด CY

เป็นบิต D7 ของรีจิสเตอร์ PSW โดยบิตนี้จะมีการเปลี่ยนแปลงหลังจากการบวกหรือลบข้อมูลแบบ 8 บิต แล้วมีตัวทาดเกิดขึ้น นอกจากนี้สามารถเซตให้เป็นลอจิก “1” หรือลอจิก “0” ได้โดยตรง โดยใช้คำสั่ง SETB C หรือ CLR C



แฟลกช่วยทด AC

หลังจากการทำคำสั่ง ADD หรือ SUB แล้วมีการทดจาก D3 ไป D4 จะทำให้บิตนี้ถูกเซต โดยทั่วไปแล้วบิตแฟลกบิตนี้จะใช้เมื่อมีการทำคำสั่งทางคณิตศาสตร์แบบ BCD

แฟลกพาริตี P

แฟลกนี้จะแสดงถึงจำนวนลอจิก “1” ที่อยู่ในรีจิสเตอร์ A ถ้าค่าในรีจิสเตอร์ A มีลอจิก “1” เป็นคู่บิต (even) บิตแฟลกพาริตีจะเป็นลอจิก “0” ถ้าค่าในรีจิสเตอร์ A มีลอจิก “1” เป็นคี่บิต (odd) บิตแฟลกพาริตีจะเป็นลอจิก “1”

แฟลกโอเวอร์โฟลว์ (OV)

บิตแฟลกนี้จะถูกเซตถ้าผลลัพธ์จากการดำเนินการกับตัวเลขแบบคิดเครื่องหมายมีค่าเกินกว่าที่รีจิสเตอร์จะเก็บได้ โดยทั่วไปแล้วแฟลกทดจะใช้ตรวจสอบความผิดพลาดจากการกระทำทางคณิตศาสตร์ของตัวเลขแบบไม่คิดเครื่องหมาย แต่แฟลกโอเวอร์โฟลว์จะใช้ตรวจสอบความผิดพลาดของการกระทำทางคณิตศาสตร์ของตัวเลขแบบคิดเครื่องหมาย

ตัวอย่างที่ 2.2 จงแสดงค่าของบิตแฟลก CY, AC และ P หลังจากการบวกค่า 38H กับ 2FH ด้วยคำสั่งต่อไปนี้

```
MOV A , #38H
```

```
ADD A , #2FH ; หลังจากการบวก A = 67H , CY = 0
```

วิธีทำ

```

38  00111000
+ 2F  00101111
-----
67  01100111

```

CY = 0 เพราะไม่มีการทดจากบิต D7

AC = 1 เพราะมีการทดจากบิต D3 ไปยัง D4

P = 1 เพราะค่าในรีจิสเตอร์ A มีค่าเป็น “1” เป็นจำนวนคี่บิต

ตัวอย่างที่ 2.3 จงแสดงค่าของบิตแฟลก CY , AC และ P หลังจากการบวกค่า 9CH กับ 64H ด้วยคำสั่งต่อไปนี้

```
MOV A , #9CH
```

```
ADD A , #64H ; หลังจากการบวก A = 00 และ CY = 1
```

**วิธีทำ**

```

      9C      10011100
+     64      01100100
-----
     100     1 00000000
           ↑
        ตัวทด

```

CY = 1 เพราะมีการทดออกจากบิต D7
 AC = 1 เพราะมีการทดจากบิต D3 ไปยังบิต D4
 P = 0 เพราะมีลอจิก “1” ในรีจิสเตอร์ A เป็นคู่บิต (เป็นศูนย์บิต)

ตัวอย่างที่ 2.4 จงแสดงค่าของบิตแฟลก CY, AC และ P หลังจากการบวกค่า 88H กับ 93H ด้วยคำสั่งต่อไปนี้

```

MOV  A , #88H
ADD  A , #93H      ; หลังจากการบวก A = 1BH , CY = 1

```

วิธีทำ

```

      88      10001000
+     93      10010011
-----
     11B     1 00011011

```

CY = 1 เพราะมีการทดออกจากบิต D7
 AC = 0 เพราะไม่มีการทดจากบิต D3 ไปบิต D4
 P = 0 เพราะค่าในรีจิสเตอร์ A มีลอจิก “1” เป็นจำนวนคู่บิต

ตัวอย่างที่ 2.5 จงหาค่าที่เก็บใน RAM ตำแหน่งต่าง ๆ หลังจากทำชุดคำสั่งต่อไปนี้

```

SETB PSW.4      ; เลือกแบงค์ 2
MOV  R0 , #99H   ; โหลดค่า 99H เก็บใน R0
MOV  R1 , #85H   ; โหลดค่า 85H เก็บใน R1
MOV  R2 , #3FH   ; โหลดค่า 3FH เก็บใน R2
MOV  R7 , #63H   ; โหลดค่า 63H เก็บใน R7
MOV  R5 , #12H   ; โหลดค่า 12H เก็บใน R5

```



วิธีทำ หลังจากรีเซ็ต MCS-51 จะทำให้ PSW.3 = 0 และ PSW.4 = 0 จากชุดคำสั่ง ถ้าหากมีการทำคำสั่ง SETB PSW.4 จะทำให้ RS1 = 1 และ RS0 = 0 ดังนั้นจะเป็นการเลือกรีจิสเตอร์แบงค์ 2 โดย R0 - R7 จะใช้หน่วยความจำ RAM ภายในตำแหน่ง 10H - 17H ดังนี้

RAM ตำแหน่ง 10H เก็บค่า 99H	RAM ตำแหน่ง 11H เก็บค่า 85H
RAM ตำแหน่ง 12H เก็บค่า 3FH	RAM ตำแหน่ง 17H เก็บค่า 63H
RAM ตำแหน่ง 15H เก็บค่า 12	



คำถาม ?

1. รีจิสเตอร์แฟลกใน 8051 มีชื่อเรียกว่าอะไร
2. บิตใดของรีจิสเตอร์ PSW ที่ไม่ถูกใช้งาน
3. จงหาค่าบิต CY และ AC หลังจากทำคำสั่งต่อไปนี้

```
MOV A, #0FFH
```

```
ADD A, #01
```

4. จงหาค่าบิต CY และ AC หลังจากทำคำสั่งต่อไปนี้

```
MOV A, #0C2H
```

```
ADD A, #3DH
```



2.7 หน่วยความจำสแต็กของ MCS-51

สแต็ก (stack) เป็นหน่วยความจำ RAM จำนวนหนึ่งที่ CPU จองขึ้นมาสำหรับเก็บข้อมูลและจำตำแหน่งของโปรแกรม ใน MCS-51 จะใช้รีจิสเตอร์ SP (stack pointer) เป็นตัวชี้ตำแหน่งหน่วยความจำที่เป็นสแต็ก ซึ่งจะเป็นตัวบอกว่าแอดเดรสของ RAM ตำแหน่งใดจะเป็นตำแหน่งที่จะนำข้อมูลเข้าไปเก็บ เมื่อมีการเก็บข้อมูลลงสแต็กแล้วค่าของ SP จะเพิ่มขึ้นหนึ่งค่า เพื่อชี้ตำแหน่งถัดไป และเมื่อมีการอ่านข้อมูลออกจากสแต็กค่าของ SP จะลดลงหนึ่งค่า

การโอนย้ายข้อมูลกับสแต็กจะใช้คำสั่ง PUSH และ POP โดยคำสั่ง PUSH จะเป็นการนำข้อมูลจากโอเปอเรนด์ที่ตามมาเก็บยังสแต็ก โดย SP จะเพิ่มค่าขึ้นหนึ่งค่าก่อนจึงเก็บข้อมูล สำหรับคำสั่ง POP จะเป็นการอ่านข้อมูลออกจากหน่วยความจำสแต็ก หลังจากอ่านข้อมูลออกมาแล้ว SP จะลดลงหนึ่งค่า รูปแบบของคำสั่ง PUSH และ POP มีลักษณะดังนี้



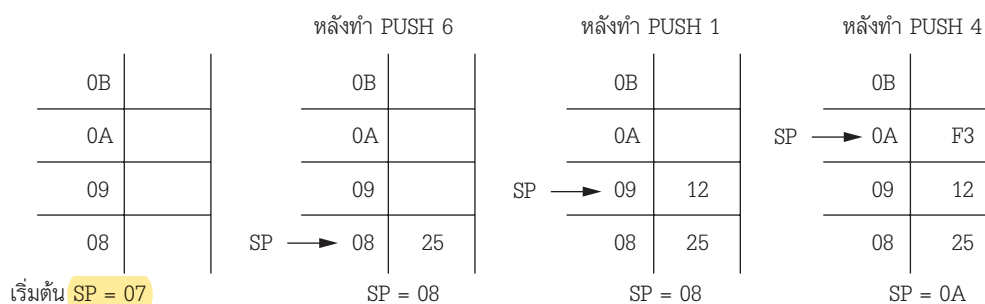
รูปแบบ	การทำงาน
PUSH direct	เพิ่มค่า SP และนำค่าในหน่วยความจำแอดเดรส direct เก็บยังหน่วยความจำที่ SP ชี้อยู่
POP direct	อ่านค่าจากตำแหน่งที่ SP ชี้อยู่ มาเก็บในหน่วยความจำ แอดเดรส direct และลดค่า SP ลงหนึ่งค่า

เมื่อ MCS-51 เริ่มต้นการทำงานรีจิสเตอร์ SP จะมีค่าเป็น 07H หมายความว่าหน่วยความจำสแต็กจะเริ่มต้นที่แอดเดรสนี้ ซึ่งจะพบว่าในตำแหน่งนี้เป็นแอดเดรสของรีจิสเตอร์ R0 - R7 ในแบงค์ 1 ดังนั้นถ้าหากต้องการใช้รีจิสเตอร์ในแบงค์นี้จะต้องกำหนดให้ SP ชี้อย่างตำแหน่งอื่นเสียก่อน แต่ไม่ควรให้หน่วยความจำสแต็กอยู่ในช่วงแอดเดรส 20H - 2FH เนื่องจากเป็นตำแหน่งของหน่วยความจำที่สามารถเข้าถึงข้อมูลในระดับบิตได้ โดยทั่วไปแล้วมักจะกำหนดให้สแต็กอยู่ในช่วงแอดเดรส 30H - 7FH ของ RAM ภายใน โดยใช้คำสั่ง MOV SP, #XX เมื่อ XX เป็นแอดเดรสที่กำหนด

ตัวอย่างที่ 2.6 จงแสดงสแต็กและตัวชี้สแต็ก เมื่อมีการทำชุดคำสั่งต่อไปนี้

```
MOV    R6 , #25H
MOV    R1 , #12H
MOV    R4 , #0F3H
PUSH   6
PUSH   1
PUSH   4
```

วิธีทำ



ตัวอย่างที่ 2.7 จงแสดงค่าของรีจิสเตอร์ SP หลังจากการทำคำสั่งต่อไปนี้

```
POP    3    ; POP ค่าในสแต็กมาเก็บใน R3
POP    5    ; POP ค่าในสแต็กมาเก็บใน R5
POP    2    ; POP ค่าในสแต็กมาเก็บใน R2
```



วิธีทำ

		หลังทำ POP 3		หลังทำ POP 5		หลังทำ POP 2	
SP →	0B	54	0B		0B		0B
	0A	F9	0A	F9	0A		0A
	09	76	09	76	09	76	09
	08	6C	08	6C	08	6C	08
เริ่มต้น SP = 0B		SP = 0A		SP = 09		SP = 08	

ตัวอย่างที่ 2.8 จงแสดงค่าของรีจิสเตอร์ SP และค่าในสแตก เมื่อมีการทำคำสั่งต่อไปนี้

```

MOV    SP , #5FH          ; กำหนดให้ SP ชี้ที่ RAM ตำแหน่ง 5FH
                                ; ดังนั้นสแตกแอดเดรสแรกคือตำแหน่ง 60H

MOV    R2 , #25H
MOV    R1 , #12H
MOV    R4 , #0F3H
PUSH   2
PUSH   1
PUSH   4
  
```

วิธีทำ

		หลังทำ PUSH 2		หลังทำ PUSH 1		หลังทำ PUSH 4	
	63		63		63		63
	62		62		62		62
	61		61		61	12	61
	60		60	25	60	25	60
เริ่มต้น SP = 5F		SP = 60		SP = 61		SP = 62	



3

บทที่

คำสั่งควบคุม

ในบทนี้จะกล่าวถึงชุดคำสั่งควบคุมการทำงานของโปรแกรม ซึ่งจะทำให้ MCS-51 กระโดดไปทำงานยังตำแหน่งต่าง ๆ ได้โดยจะกล่าวถึงคำสั่งทำซ้ำ ที่ทำให้ระบบทำงานซ้ำตามรอบที่กำหนด คำสั่งกระโดดซึ่งจะทำให้โปรแกรมกระโดดไปยังแอดเดรสที่ต้องการได้ และคำสั่งเรียกโปรแกรมย่อย



3.1 คำสั่งทำซ้ำและคำสั่งกระโดด

การให้ MCS-51 ทำงานใด ๆ ซ้ำ ๆ สามารถทำได้หลายวิธี แต่จะเรียกรวมกันว่าการทำลูป (loop) ในการทำซ้ำแต่ละครั้งจะมีการตรวจสอบเงื่อนไขด้วยว่าจะต้องทำงานนั้นซ้ำหรือไม่ สำหรับการทำคำสั่งกระโดดจะมีสองประเภทใหญ่ ๆ คือการกระโดดแน่ ๆ ถ้าโปรแกรมทำงานมาถึงคำสั่งนั้น และจะกระโดดถ้าเงื่อนไขเป็นไปตามที่กำหนด

ลูปของ 8051

การเขียนโปรแกรมให้กับไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะใช้คำสั่ง DJNZ reg,label เมื่อ MCS-51 ทำคำสั่งนี้แต่ละครั้งจะลดค่าในรีจิสเตอร์ reg ลงหนึ่งค่า และคำสั่งนี้จะทำให้โปรแกรมกระโดดไปยังแอดเดรส label ถ้าค่าในรีจิสเตอร์ reg ยังไม่เป็นศูนย์ แต่ถ้าค่าในรีจิสเตอร์ reg เป็นศูนย์ โปรแกรมจะทำคำสั่งต่อจากคำสั่ง DJNZ นี้



ตัวอย่างที่ 3.1 จงเขียนโปรแกรมให้ทำงานดังต่อไปนี้

- ก. เคลียร์ค่าในแอมคิวมูลเตอร์
- ข. บวกค่า 3 เข้ากับแอมคิวมูลเตอร์จำนวน 10 ครั้ง

วิธีทำ

	MOV	A,#0	; A = 0 , เคลียร์ค่าในรีจิสเตอร์ A
	MOV	R2,#10D	; โหลดค่าตัวนับ R2 = 10
AGAIN:	ADD	A,#03	; บวกค่า 3 เข้ากับ ACC
	DJNZ	R2,AGAIN	; ทำซ้ำจนกว่า R2 = 0
	MOV	R5,A	; เก็บผลลัพธ์ลงใน R5

จากตัวอย่างโปรแกรมที่ 3.1 จะใช้รีจิสเตอร์ R2 เป็นตัวนับ การนับครั้งแรกจะมีค่าเป็น 10 ต่อมาเมื่อพบกับคำสั่ง DJNZ จะทำการลดค่า R2 ลงหนึ่งค่า และทดสอบค่า ถ้าค่าใน R2 ไม่เท่ากับ 0 โปรแกรมจะกระโดดไปยังแอตเตอเรส AGAIN แต่ถ้าค่าใน R2 เท่ากับ 0 โปรแกรมจะเก็บค่าคำตอบในรีจิสเตอร์ A ลงในรีจิสเตอร์ R5 ด้วยคำสั่ง MOV R5,A และในการทำลูปแต่ละครั้งนั้นจะทำการบวกค่าในรีจิสเตอร์ A ด้วยค่า 3

คำสั่ง DJNZ นี้ค่ารีจิสเตอร์ที่ใช้เป็นตัวนับและลดค่าจะเป็นรีจิสเตอร์ R0-R7 และยังใช้ค่าใน RAM ภายในเป็นตัวนับได้อีกด้วย

ลูปซ้อนลูป

ในการทำซ้ำแต่ละครั้งนั้นสามารถทำซ้ำได้ทั้งหมด 256 ครั้ง ซึ่งเป็นค่าที่มากที่สุดที่จะเก็บในรีจิสเตอร์ได้ ถ้าหากต้องการทำซ้ำมากกว่านี้สามารถทำได้โดยการทำลูปซ้อนลูป หรือที่เรียกกันว่า nested loop ดังแสดงในตัวอย่างโปรแกรมที่ 3.2

ตัวอย่างที่ 3.2 จงเขียนโปรแกรมโหลดค่าในรีจิสเตอร์ A ด้วย 55H และทำการคอมพลีเมนต์ค่าในรีจิสเตอร์ A จำนวน 700 ครั้ง

วิธีทำ เนื่องจากจะต้องทำซ้ำเป็นจำนวน 700 ครั้ง ซึ่งมากกว่า 255 ดังนั้นจะต้องทำลูปซ้อนลูป โดยใช้รีจิสเตอร์สองตัวเป็นตัวนับ ในที่นี้จะใช้รีจิสเตอร์ R2 และ R3

	MOV	A,#55H	; A = 55H
	MOV	R3,#10D	; R3 = 10 เป็นการนับจำนวนลูปนอก
NEXT:	MOV	R2,#70D	; R2 = 70 เป็นการนับจำนวนลูปใน
AGAIN:	CPL	A	; คอมพลีเมนต์ค่าในรีจิสเตอร์ A
	DJNZ	R2,AGAIN	; ทำซ้ำจำนวน 70 ครั้ง
	DJNZ	R3,NEXT	; ทำซ้ำจำนวน 10 ครั้ง



การกระโดดแบบมีเงื่อนไข

คำสั่งกระโดดแบบมีเงื่อนไขของ MCS-51 สรุปได้ดังตารางที่ 3.1 ซึ่งมีอยู่หลายคำสั่ง เช่น JZ คำสั่งนี้จะทำการกระโดดถ้าค่าใน A เป็น 0 และคำสั่ง JC จะกระโดดถ้าแฟลกตัวทดถูกเซต เป็นต้น คำสั่งกระโดดแบบมีเงื่อนไขมีตัวอย่างดังนี้

ตารางที่ 3.1 คำสั่งกระโดดแบบมีเงื่อนไข

คำสั่ง	การทำงาน
JZ	กระโดดถ้า A เท่ากับ 0
JNZ	กระโดดถ้า A ไม่เท่ากับ 0
DJNZ	ลดค่าและกระโดดถ้า A ไม่เท่ากับ 0
CJNE A,byte	กระโดดถ้า A ไม่เท่ากับ byte
CJNE reg,#data	กระโดดถ้า reg ไม่เท่ากับ #data
JC	กระโดดถ้า CY เท่ากับ 1
JNC	กระโดดถ้า CY เท่ากับ 0
JB	กระโดดถ้าบิตที่กำหนดเท่ากับ 1
JNB	กระโดดถ้าบิตที่กำหนดเท่ากับ 0
JBC	กระโดดถ้าบิตที่กำหนดเท่ากับ 1 และเคลียร์บิต

JZ (กระโดดถ้า A = 0)

คำสั่งนี้จะตรวจสอบค่าในรีจิสเตอร์ A ถ้าหากมีค่าเป็น 0 จะกระโดดไปยังแอดเดรสที่กำหนด ดังตัวอย่างการเขียนคำสั่งต่อไปนี้

```
MOV    A,R0          ; A = R0
JZ     OVER          ; กระโดดไป OVER ถ้า A = 0
MOV    A,R1          ; A = R1
JZ     OVER          ; กระโดดไป OVER ถ้า A = 0
.....
```

OVER:

จากโปรแกรมจะเห็นว่าถ้าค่าในรีจิสเตอร์ R0 หรือ R1 เป็นศูนย์ โปรแกรมจะกระโดดไปยังแอดเดรสที่มีลาเบล OVER แต่เนื่องจากคำสั่ง JZ จะตรวจสอบค่าในรีจิสเตอร์ A เท่านั้น จึงต้องย้ายค่าจากรีจิสเตอร์ R0 และ R1 ไปเก็บในรีจิสเตอร์ A เสียก่อน ถ้าหากต้องการเขียนโปรแกรมให้ตรวจสอบค่าในรีจิสเตอร์ A และให้กระโดดถ้าค่าใน A ไม่เป็น 0 ต้องใช้คำสั่ง JNZ ดังตัวอย่างต่อไปนี้



ตัวอย่างที่ 3.3 จงเขียนโปรแกรมทดสอบค่าใน R5 ถ้าหากมีค่าเป็นศูนย์ให้ใส่ค่า 55H ลงไป

วิธีทำ

```

MOV    A,R5                ; ใส่ค่า R5 ลงใน A
JNZ    NEXT                ; กระโดดถ้าค่าใน A ไม่เป็นศูนย์
MOV    R5,#55H              ; ใส่ค่า 55H ลงใน A
NEXT:  .....

```

JNC (กระโดดถ้าไม่มีตัวทศ หรือ CY = 0)

คำสั่งนี้จะตรวจสอบค่าบิตแฟลกตัวทศที่อยู่ในรีจิสเตอร์ PSW ถ้าไม่มีการทศจะกระโดดไปยังแอดเดรสที่กำหนด โดยมีรูปแบบคำสั่งเป็น JNC label ซึ่งจะทดสอบแฟลกตัวทศเช่นกัน แต่จะกระโดดไปยังแอดเดรส label เมื่อแฟลกตัวทศมีค่าเป็น “0”

สำหรับคำสั่ง JB (jump if bit is high) และ JNB (jump if bit is low) จะเป็นคำสั่งที่ทดสอบบิต และจะกระโดดไปยังแอดเดรสที่กำหนด ถ้าบิตนั้นเป็น “1” หรือ “0” ตามลำดับ

ตัวอย่างที่ 3.4 จงเขียนโปรแกรมบวกค่า 79H , F5H และ E2H และใส่ค่าผลบวกไบต์ต่ำลงใน R0 และไบต์สูงลงใน R5

วิธีทำ

```

MOV    A,#0                ; เคลียร์ค่าใน A
MOV    R5,A                ; เคลียร์ค่าใน R5
ADD    A,#79H               ; A = 0 + 79H = 79H
JNC    N_1                 ; ถ้าไม่มีตัวทศให้กระโดดไปบวกเลขต่อไป
INC    R5                  ; ถ้าตัวทศเป็น 1 ให้เพิ่มค่า R5
N_1:   ADD    A,#0F5H        ; A = 79 + F5 = 6E และแฟลกตัวทศเป็น 1
JNC    N_2                 ; กระโดดถ้าตัวทศเป็น 0
INC    R5                  ; ถ้าตัวทศเท่ากับ 1 ให้เพิ่มค่า R5 (R5 = 1)
N_2:   ADD    A,#0E2H        ; A = 6E + E2 = 50 และตัวทศเป็น 1
JNC    OVER               ; กระโดดถ้าตัวทศเป็น 0
INC    R5                  ; ถ้าตัวทศเป็น “1” เพิ่มค่าใน R5
OVER:  MOV    R0,A           ; ค่าใน R0 = 50H และ R5 = 02

```

ในการทำคำสั่งกระโดดแบบมีเงื่อนไขทุกคำสั่งจะเป็นการกระโดดแบบ short jump ซึ่งจะกระโดดถอยหลังได้ -128 ไบต์ และกระโดดไปข้างหน้าได้ +127 ไบต์ โดยจะนำค่าในโอเปอเรนด์ของคำสั่งไปบวกกับค่าโปรแกรมเคาน์เตอร์ (PC)



คำสั่งกระโดดแบบไม่มีเงื่อนไข

การกระโดดแบบไม่มีเงื่อนไข (unconditional jump) จะเป็นคำสั่งที่ทำให้โปรแกรมกระโดดไปยังตำแหน่งที่กำหนดแน่ ๆ ใน MCS-51 มีคำสั่งกระโดดแบบไม่มีเงื่อนไขสองคำสั่งคือคำสั่ง LJMP (long jump) และ SJMP (short jump) โดยทั้งสองคำสั่งจะต่างกันที่ระยะทางกระโดดของโปรแกรม

LJMP (long jump)

คำสั่ง LJMP เป็นคำสั่งที่มีขนาด 3 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด อีกสองไบต์ต่อมาจะแทนตำแหน่งแอดเดรสขนาด 16 บิต ซึ่งสามารถกระโดดไปยังหน่วยความจำใด ๆ ได้ในช่วง 0000 ถึง FFFFH

แม้ว่า MCS-51 สามารถอ้างตำแหน่งหน่วยความจำภายนอกได้ 64 กิโลไบต์ หรือมีโปรแกรมเคาน์เตอร์ขนาด 16 บิต แต่ MCS-51 ที่มี ROM อยู่บนชิปก็ไม่มีทั้ง 64 กิโลไบต์อยู่ภายในทั้งหมด MCS-51 บางรุ่นมีหน่วยความจำภายในชิปสูงสุดเพียง 4 กิโลไบต์ ดังนั้นการใช้คำสั่งกระโดดที่กระโดดได้ในช่วง 0000 ถึง FFFF ก็ไม่มีความจำเป็น จึงมีคำสั่งกระโดดอีกคำสั่งหนึ่งที่กระโดดได้ในช่วงสั้น ๆ คือ SJMP (short jump) ซึ่งเป็นคำสั่งที่มีขนาด 2 ไบต์ ทำให้การกระโดดในช่วงสั้น ๆ สามารถใช้คำสั่งกระโดดที่มีขนาดของรหัสคำสั่งน้อยลงได้ ดังนั้นคำสั่งกระโดดของ MCS-51 อาจมองได้ว่ามีสองคำสั่งคือ คำสั่งขนาด 2 ไบต์ SJMP และคำสั่งแบบ 3 ไบต์ LJMP

SJMP (short jump)

คำสั่งนี้มีขนาด 2 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด ไบต์ต่อมาจะเป็นแอดเดรสที่จะกระโดดโดยอ้างแอดเดรสแบบสัมพัทธ์ (relative address) ซึ่งจะอ้างตำแหน่งได้ในช่วง 00 ถึง FFH หรือกระโดดไปข้างหน้าได้ -128 ไบต์ และกระโดดไปข้างหน้าได้ +127 ไบต์ โดยไมโครคอนโทรลเลอร์จะนำค่าออปโค้ดซึ่งอ้างแอดเดรสแบบสัมพัทธ์นี้ไปบวกกับค่าในโปรแกรมเคาน์เตอร์ และให้เป็นค่าใหม่กับโปรแกรมเคาน์เตอร์

จากคำสั่งกระโดดทั้งหมดที่กล่าวมาจะพบว่ามีกลุ่มคำสั่งที่จะกระโดดไปยังแอดเดรสระยะสั้น ๆ ได้แก่คำสั่ง SJMP และคำสั่งกระโดดแบบมีเงื่อนไขทุกคำสั่งเช่น JNC, JZ และ DJNZ เป็นต้น คำสั่งเหล่านี้จะมีขนาด 2 ไบต์ และมีการอ้างตำแหน่งที่จะกระโดดไปแบบสัมพัทธ์ ซึ่งถ้านำค่าแอดเดรสแบบสัมพัทธ์นี้ไปบวกกับค่าใน PC ก็จะได้ตำแหน่งใหม่ที่จะกระโดดไป การคำนวณค่าแอดเดรสแบบสัมพัทธ์นี้จะคำนวณได้ดังตัวอย่างที่ 3.5

ตัวอย่างที่ 3.5 จากโปรแกรมต่อไปนี้จะแสดงวิธีการคำนวณค่าแอดเดรสของคำสั่งกระโดด

Line	PC	Opcode	Mnemonic	Operand
01	0000		ORG	0000
02	0000	7800	MOV	R0,#0
03	0002	7455	MOV	A,#55H



04	0004	6003		JZ	NEXT
05	0006	08		INC	R0
06	0007	04	AGAIN:	INC	A
07	0008	04		INC	A
08	0009	2477	NEXT:	ADD	A,#77H
09	000B	5005		JNC	OVER
10	000D	E4		CLR	A
11	000E	F8		MOV	R0,A
12	000F	F9		MOV	R1,A
13	0010	FA		MOV	R2,A
14	0011	FB		MOV	R3,A
15	0012	2B	OVER:	ADD	A,R3
16	0013	50F2		JNC	AGAIN
17	0015	80FE	HERE:	SJMP	HERE
18	0017			END	

วิธีทำ

จากคำสั่งกระโดดในโปรแกรมจะเป็นคำสั่งกระโดดแบบมีเงื่อนไข ซึ่งจะอ้างแอดเดรสแบบสัมพันธ์ โดยค่านี้นับจากตำแหน่งที่จะกระโดดไปนั้นอยู่ห่างจากตำแหน่งที่ PC อยู่ไปข้างหน้าหรือถอยหลังไปเป็นจำนวนกี่ไบต์ จากโปรแกรมในบรรทัดที่ 4 จะเป็นคำสั่ง JZ NEXT ซึ่งจะให้กระโดดไปยังแอดเดรส 0009 คำสั่งนี้เก็บอยู่ในแอดเดรส 0004 และ 0005 มีออปโค้ดเป็น 60 และ 03 เป็นค่าแอดเดรสแบบสัมพันธ์ เมื่อโปรแกรมทำงานมาถึงคำสั่งนี้ค่า PC จะอยู่ที่คำสั่งถัดไปคือคำสั่งในแอดเดรส 0006 ดังนั้นค่าโอเปอเรนด์ 03 เมื่อนำมาบวกกับค่า PC จะทำให้โปรแกรมกระโดดไปยังแอดเดรส 0009 ส่วนคำสั่ง JNC OVER ในบรรทัดที่ 9 จะอยู่ที่แอดเดรส 000B และจะเห็นว่าค่าโอเปอเรนด์เป็น 05 เมื่อคอมพิวเตอร์ทำคำสั่งนี้ค่า PC จะอยู่ที่แอดเดรส 000D ซึ่งอยู่ที่คำสั่ง CLR A ดังนั้นโปรแกรมจะกระโดดไปยังแอดเดรส 000D + 5 คือแอดเดรส 0012H

สำหรับคำสั่ง JNC AGAIN ในแอดเดรส 0013H มีค่าออปโค้ดเป็น 50 และค่าแอดเดรสแบบสัมพันธ์เป็น F2H เมื่อ MCS-51 ทำงานมาถึงคำสั่งนี้ PC จะชี้ไปยังแอดเดรสคำสั่งถัดไปคือคำสั่งในแอดเดรส 0015H ดังนั้นแอดเดรสที่จะกระโดดไปคือ $15H + F2H = 07$ ซึ่งเป็นตำแหน่งของคำสั่ง INC A ในเวลาेल AGAIN จะเห็นว่าการกระโดดของคำสั่งนี้จะเป็นการกระโดดกลับไปข้างหลังจำนวน 13 ไบต์ ค่าแอดเดรสแบบสัมพันธ์นี้จะต้องเป็น -13 ซึ่งเท่ากับ F2 (เลข 2's complement) สำหรับคำสั่ง SJMP HERE คำสั่งนี้จะมีขนาด 2 ไบต์ และจะต้องกระโดดกลับหลังเป็นระยะ 2 ไบต์ มายังตำแหน่งคำสั่งเดิม ดังนั้นค่า -2 จะเป็น FEH ซึ่งเท่ากับค่าโอเปอเรนด์ในโปรแกรม ดังนั้นตำแหน่งที่จะกระโดดไปจะเป็น $17H + FEH = 15H$ นั่นเอง

**คำถาม ?**

1. รหัสนิโมนิก DJNZ มีค่ารหัสภาษาเครื่องอะไร
2. จริงหรือไม่ที่คำสั่ง DJNZ R5, BACK จะลดค่าในรีจิสเตอร์ก่อนที่จะทำการกระโดด
3. คำสั่ง JZ NEXT จะตรวจสอบค่าในรีจิสเตอร์ใดว่ามีค่าเป็นศูนย์หรือไม่
4. คำสั่ง JZ NEXT มีขนาดกี่ไบต์
5. คำสั่ง LJMP มีขนาดกี่ไบต์

**3.2 การเรียกโปรแกรมย่อย**

การเขียนโปรแกรมประเภทโปรแกรมย่อยหรือ Subroutine สามารถถูกเรียกขึ้นมาใช้งานได้ด้วยคำสั่ง CALL ใน MCS-51 จะมีอยู่สองคำสั่งคือ LCALL (long call) และ ACALL (absolute call) โดยทั้งสองคำสั่งต่างกันตรงขนาดของคำสั่ง และระยะห่างของโปรแกรมย่อยที่จะเรียกใช้งาน

LCALL (long call)

คำสั่งนี้จะมีขนาด 3 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด อีกสองไบต์จะเป็นแอดเดรสที่จะกระโดดไปทำงานโปรแกรมย่อย ซึ่งสามารถเรียกโปรแกรมย่อยได้ในระยะ 64 กิโลไบต์ เมื่อโปรแกรมย่อยถูกเรียกใช้ MCS-51 จะนำค่าแอดเดรสของคำสั่งถัดไปหรือค่า PC เก็บลงหน่วยความจำสแต็กโดยอัตโนมัติ และโหลดค่าแอดเดรสของโปรแกรมย่อยให้กับรีจิสเตอร์ PC และเมื่อทำโปรแกรมย่อยไปจนพบคำสั่ง RET (return) MCS-51 จะกลับไปทำคำสั่งของโปรแกรมที่อยู่ต่จากคำสั่งเรียกโปรแกรมย่อย โดยคืนค่าที่เก็บอยู่ในสแต็กให้กับรีจิสเตอร์ PC ตามเดิม

ตัวอย่างที่ 3.6 จงเขียนโปรแกรมส่งข้อมูลออกทางพอร์ต 1 ให้ทุกบิตมีค่ากลับไปมา โดยส่งค่า 55H และ AAH สลับกัน โดยใช้คำสั่ง LCALL เรียกโปรแกรมหน่วงเวลาระหว่างการส่งข้อมูลแต่ละค่า

วิธีทำ

```

ORG      0
BACK:    MOV    A, #55H      ; โหลด A ด้วยค่า 55H
          MOV    P1, A        ; ส่งค่า 55H ออกไปทางพอร์ต P1
          LCALL  DELAY        ; เรียกโปรแกรมย่อยหน่วงเวลา
          MOV    A, #0AAH     ; โหลด A ด้วยค่า 0AAH

```



```

MOV    P1,A           ; ส่งค่า AAH ออกไปทางพอร์ต P1
LCALL  DELAY
SJMP   BACK           ; วนลูปทำซ้ำ
;***** เริ่มต้นโปรแกรมย่อยหน่วงเวลา *****
ORG    300H           ; โปรแกรมย่อยเริ่มต้นที่แอดเดรส 300H
DELAY: MOV    R5,#0FFH ; ให้ R5 เท่ากับ 255 หรือ 0FFH
AGAIN: DJNZ   R5,AGAIN ; ทำซ้ำจน R5 เท่ากับ 0
RET                                ; จบโปรแกรมย่อย
END

```

จากตัวอย่างโปรแกรมที่ 3.6 จะเห็นว่ามีการสร้างโปรแกรมย่อยชื่อ DELAY โดยท้ายของโปรแกรมย่อยจะจบด้วยคำสั่ง RET เมื่อ MCS-51 ทำคำสั่งเรียกโปรแกรมย่อย LCALL DELAY ตัว MCS-51 จะเก็บค่าของโปรแกรมเคาน์เตอร์ซึ่งเป็นค่าแอดเดรสของคำสั่ง MOV A,#0AAH ลงหน่วยความจำสแตค และโปรแกรมจะกระโดดไปทำคำสั่งของโปรแกรมย่อยที่แอดเดรส 300H ในโปรแกรมย่อยเริ่มแรกจะกำหนดค่าให้รีจิสเตอร์ R5 เท่ากับ 255 (R5 = 0FFH) และจะวนลูปจำนวน 256 ครั้ง เมื่อ R5 เป็นศูนย์จะทำคำสั่ง RET ซึ่งจะอ่านค่าในหน่วยความจำสแตคคืนให้โปรแกรมเคาน์เตอร์เพื่อทำคำสั่งที่ต่อจากคำสั่ง CALL

ในตัวอย่างที่ 3.6 จะเห็นว่าถ้ามีการเรียกใช้โปรแกรมย่อย ไมโครคอนโทรลเลอร์จะต้องมีการใช้งานหน่วยความจำสแตค ต่อไปจะแสดงการทำงานของหน่วยความจำสแตค จากตัวอย่างที่ 3.6 ถ้าหากแสดงค่าแอดเดรสและรหัสคำสั่งจะแสดงได้ดังนี้

001	0000		ORG	0
002	0000	7455	BACK:	MOV A,#55H
003	0002	F590		MOV P1,A
004	0004	120300		LCALL DELAY
005	0007	74AA		MOV A,#0AAH
006	0009	F590		MOV P1,A
007	000B	120300		LCALL DELAY
008	000E	80F0		SJMP BACK
009	0010			



010	0010	***** เริ่มต้นโปรแกรมย่อยหน่วงเวลา *****						
011	0300		ORG	300H		0A		
012	0300		DELAY:			09	00	
013	0300	7DFF		MOV	R5,#0FFH	08	07	
014	0302	DDFE	AGAIN:	DJNZ	R5,AGAIN	07		
015	0304	22		RET				
016	0305			END		SP = 09		

ถ้าหากตัวชี้สแต็ก (stack pointer) อยู่ที่แอดเดรส 07 เมื่อ MCS-51 ทำคำสั่ง LCALL แอดเดรสของคำสั่งถัดไปคือคำสั่ง MOV A,#0AAH ซึ่งมีค่าเท่ากับ 0007H จะถูกเก็บลงหน่วยความจำสแต็ก โดยเก็บไบต์ต่ำลงไปก่อนแล้วตามด้วยไบต์สูง โดยตัวชี้สแต็กจะเพิ่มตัวเองก่อนแล้วจึงเขียนข้อมูลลงไป ซึ่งจะทำให้แอดเดรส 08 มีค่าเป็น 07 และแอดเดรส 09 มีค่าเป็น 00 หลังจากทำการเรียกโปรแกรมย่อยแล้วจะทำให้ตัวชี้สแต็กอยู่ที่แอดเดรส 09 และเมื่อ MCS-51 ทำโปรแกรมย่อยจนพบคำสั่ง RET ไมโครคอนโทรลเลอร์จะอ่านค่าจากหน่วยความจำสแต็กคืนให้กับโปรแกรมเคาน์เตอร์ โดยจะอ่านไบต์สูงออกมาก่อน และลดค่าตัวชี้สแต็กและอ่านไบต์ต่ำออกมา ทำให้ MCS-51 ทำคำสั่งต่อจากคำสั่ง LCALL ได้ และหลังจากจบการทำโปรแกรมย่อยแล้วตัวชี้สแต็กจะอยู่ที่แอดเดรส 07 ตามเดิม

ในหน่วยความจำสแต็กเราสามารถนำมาประยุกต์ใช้เก็บข้อมูลชั่วคราวได้ การเขียนข้อมูลลงไปจะใช้คำสั่ง PUSH ส่วนการอ่านข้อมูลออกมาจะใช้คำสั่ง POP ดังตัวอย่างที่ 3.7 แต่การใช้งานนั้นข้อมูลที่ถูกเขียนลงไปก่อนจะถูกอ่านออกมาทีหลัง

ตัวอย่างที่ 3.7 จงวิเคราะห์การทำงานหน่วยความจำสแต็ก ของการทำโปรแกรมต่อไปนี้

01	0000		ORG	0		
02	0000	7455	BACK:	MOV	A,#55H	; โหลดรีจิสเตอร์ A ด้วยค่า 55H
03	0002	F590		MOV	P1,A	; ส่งค่า 55H ออกไปทางพอร์ต P1
04	0004	7C99		MOV	R4,#99H	
05	0006	7D67		MOV	R5,#67H	
06	0008	120300		LCALL	DELAY	; เรียกโปรแกรมย่อยหน่วงเวลา
07	000B	74AA		MOV	A,#0AAH	; โหลดรีจิสเตอร์ A ด้วยค่า AAH
08	000D	F590		MOV	P1,A	; ส่งค่า AA ออกไปทางพอร์ต P1
09	000F	120300		LCALL	DELAY	; เรียกโปรแกรมย่อยหน่วงเวลา
10	0012	80EC		SJMP	BACK	; ทำซ้ำ



```

11 0014          ;***** โปรแกรมย่อยหน่วงเวลา *****
12 0300          ORG      300H
13 0300  C004  DELAY:  PUSH  4          ; PUSH R4
14 0302  C005          PUSH  5          ; PUSH R5
15 0304  7CFF          MOV    R4,#0FFH
16 0306  7DFF  NEXT:  MOV    R5,#0FFH
17 0308  DDFE  AGAIN: DJNZ    R5,AGAIN
18 030A  DCFA          DJNZ    R4,NEXT
19 030C  D005          POP     5          ; POP R5
20 030E  D004          POP     4          ; POP R4
21 0310  22          RET              ; กลับสู่โปรแกรมหลัก
22 0311          END              ; จบโปรแกรม

```

วิธีทำ

จากโปรแกรมถ้าหากตัวชี้สแตกอยู่ที่แอดเดรส 07H เมื่อมีการทำคำสั่ง LCALL ในบรรทัดที่ 06 ตัว MCS-51 จะเก็บแอดเดรสของคำสั่ง MOV A,#0AAH ลงสแตกโดยจะเพิ่มค่าตัวเองมาอยู่ที่แอดเดรส 08H แล้วเก็บไบต์ต่ำลงไป จากนั้นเพิ่มค่าตัวเองอีกหนึ่งแล้วเก็บไบต์สูง เมื่อ MCS-51 ไปทำโปรแกรมย่อยจะมีการนำค่าใน R4 และ R5 เก็บลงสแตกด้วยคำสั่ง PUSH และก่อนจบโปรแกรมย่อยจะอ่านในสแตกออกมาด้วยคำสั่ง POP จากโปรแกรมจะเห็นว่าในการเขียนโปรแกรมย่อยจะต้องเก็บค่าของรีจิสเตอร์ต่าง ๆ ที่ใช้ในโปรแกรมย่อยลงสแตกเสียก่อน เพื่อให้รีจิสเตอร์ต่าง ๆ ที่ใช้ในโปรแกรมหลักไม่เปลี่ยนแปลงเมื่อทำโปรแกรมย่อย

หลังทำ LCALL			หลังทำ PUSH 4			หลังทำ PUSH 5		
0B			0B			0B	67	R5
0A			0A	99	R4	0A	99	R4
09	00	PCH	09	00	PCH	09	00	PCH
08	0B	PCL	08	0B	PCL	08	0B	PCL

โดยทั่วไปแล้วการเขียนโปรแกรมในลักษณะของโปรแกรมย่อยจะมีโครงสร้างของโปรแกรกดังนี้

```

ORG      0
MAIN:    LCALL  SUBR_1      ; เรียกใช้โปรแกรมย่อยชื่อ SUBR_1
         LCALL  SUBR_2      ; เรียกใช้โปรแกรมย่อยชื่อ SUBR_2
         LCALL  SUBR_3      ; เรียกใช้โปรแกรมย่อยชื่อ SUBR_3

```



```

HERE:      SJMP  HERE
;*****   จบโปรแกรมหลัก   *****
SUBR_1  :   .....
           .....
           RET                      ; คำสั่ง RET จบโปรแกรมย่อย
;*****   จบโปรแกรมย่อย 1   *****
SUBR_2:   .....
           .....
           RET
;*****   จบโปรแกรมย่อย 2   *****
SUBR_3:   .....
           .....
           RET
;*****   จบโปรแกรมย่อย 3   *****
           END                      ; จบโปรแกรม

```

จากตัวอย่างโปรแกรมที่ผ่านมาจะเป็นการสร้างโปรแกรมย่อยขึ้นมาสามโปรแกรมย่อย นอกจากนี้ในโปรแกรมย่อยยังสามารถเรียกโปรแกรมย่อยได้อีกด้วย

ACALL (absolute call)

คำสั่งเรียกโปรแกรมย่อย ACALL เป็นคำสั่งที่มีขนาด 2 ไบต์ โดยใช้ข้อมูล 11 บิตในคำสั่งเป็นค่าแอดเดรส ทำให้สามารถเรียกโปรแกรมย่อยได้ในช่วง 2 กิโลไบต์ การทำงานกับสแตคของคำสั่ง ACALL และ LCALL นี้จะไม่แตกต่างกัน แต่การใช้งานจะต่างกันตรงที่คำสั่ง LCALL เรียกโปรแกรมย่อยได้ในช่วง 64 กิโลไบต์ ส่วนคำสั่ง ACALL เรียกโปรแกรมย่อยได้ในช่วง 2 กิโลไบต์



คำถาม ?

1. รหัสนิโมนิก LCALL และ ACALL ต่างกันอย่างไร
2. จริงหรือไม่การใช้คำสั่ง LCALL สามารถเรียกโปรแกรมย่อยได้ในช่วง 64k ตลอดหน่วยความจำ
3. จะเกิดอะไรขึ้นหลังจากการทำคำสั่ง RET
4. คำสั่ง LCALL เป็นคำสั่งที่มีรหัสกี่ไบต์



3.3 การเขียนโปรแกรมหน่วงเวลา

จากตัวอย่างที่ผ่านมาได้ทดลองใช้โปรแกรมหน่วงเวลาโดยเขียนเป็นโปรแกรมย่อยชื่อ **DELAY** มาแล้ว โดยโปรแกรมย่อยในลักษณะนี้สามารถใช้หน่วงเวลาการทำงานของโปรแกรมหลักให้ช้าลงได้ ในหัวข้อนี้จะแสดงการคำนวณเวลาที่เสียไปในการทำโปรแกรมหน่วงเวลาต่าง ๆ

ไซเคิลคำสั่ง

ในการทำคำสั่งของ CPU จะต้องใช้สัญญาณนาฬิกาควบคุมการทำงาน จำนวนของสัญญาณนาฬิกาที่ใช้จะถูกอ้างเป็นไซเคิลคำสั่งหรือแมชชีนไซเคิล (machine cycle) คำสั่งบางคำสั่งมีจำนวนแมชชีนไซเคิลไม่เท่ากัน ซึ่งจะทำให้เวลาการทำงานของคำสั่งบางคำสั่งไม่เท่ากันด้วย โดยทั่วไปแล้วคำสั่งแต่ละคำสั่งจะบอกเวลาเป็นจำนวนแมชชีนไซเคิล ไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะทำงานที่ความถี่สัญญาณนาฬิกาตั้งแต่ 4 MHz ถึง 30 MHz และหนึ่งแมชชีนไซเคิลจะใช้สัญญาณนาฬิกาจำนวน 12 ลูก

ตัวอย่างที่ 3.8 ถ้าหากระบบ MCS-51 ทำงานที่ความถี่สัญญาณนาฬิกาแตกต่างกันดังต่อไปนี้ จงหาค่าเวลาในแต่ละแมชชีนไซเคิล

ก. 11.0592 MHz ข. 16 MHz ค. 20 MHz

วิธีทำ

ก. $11.0592 / 12 = 921.6 \text{ kHz}$; แมชชีนไซเคิลคือ $1 / 921.6 \text{ kHz} = 1.085 \text{ us}$

ข. $16 \text{ MHz} / 12 = 1.333 \text{ MHz}$; แมชชีนไซเคิลคือ $1 / 1.333 \text{ MHz} = 0.75 \text{ us}$

ค. $20 \text{ MHz} / 12 = 1.66 \text{ MHz}$; แมชชีนไซเคิลคือ $1 / 1.66 \text{ MHz} = 0.60 \text{ us}$

ตัวอย่างที่ 3.9 ระบบ MCS-51 ทำงานที่ความถี่ 11.0592 MHz อยากทราบว่าการทำงานคำสั่งต่อไปนี้ใช้เวลาเท่าใด

ก. MOV R3,#55 ข. DEC R3 ค. DJNZ R2,target
 ง. LJMP จ. SJMP ฉ. NOP
 ช. MUL AB

วิธีทำ หนึ่งแมชชีนไซเคิลที่ความถี่ 11.0592 MHz จะเท่ากับ 1.085 ไมโครวินาที จากตารางคำสั่งของ MCS-51 จะคำนวณเวลาของแต่ละคำสั่งได้ดังนี้



คำสั่ง	แมชชีนไซเคิล	เวลา
MOV R3,#55	1	$1 \times 1.085 \text{ us} = 1.085 \text{ us}$
DEC R3	1	$1 \times 1.085 \text{ us} = 1.085 \text{ us}$
DJNZ R2,target	2	$2 \times 1.085 \text{ us} = 2.17 \text{ us}$
LJMP	2	$2 \times 1.085 \text{ us} = 2.17 \text{ us}$
SJMP	2	$2 \times 1.085 \text{ us} = 2.17 \text{ us}$
NOP	1	$1 \times 1.085 \text{ us} = 1.085 \text{ us}$
MUL AB	4	$4 \times 1.085 \text{ us} = 4.34 \text{ us}$

การคำนวณเวลาหน่วง

การเขียนโปรแกรมหน่วงเวลาเพื่อให้ไมโครคอนโทรลเลอร์เสียเวลาไปจะทำได้สองวิธีหลัก ๆ ดังนี้
คือ วิธีแรกจะใช้ตัวนับตัวจับเวลา อีกวิธีหนึ่งจะใช้การวนลูปทำซ้ำดังตัวอย่างที่ 3.11

ตัวอย่างที่ 3.10 จงคำนวณเวลาหน่วงจากการทำโปรแกรมต่อไปนี้ ถ้าหากระบบทำงานที่ความถี่
11.0592 MHz

```

                                MOV        A,#55H
AGAIN:                        MOV        P1,A
                                ACALL      DELAY
                                CPL        A
                                SJMP      AGAIN
;..... Time delay
DELAY:                        MOV        R3,#200
HERE:                        DJNZ       R3,HERE
                                RET

```

วิธีทำ ในโปรแกรมหน่วงเวลาถ้าหากพิจารณาจำนวนแมชชีนไซเคิลจะคำนวณได้ดังนี้

		แมชชีนไซเคิล
DELAY:	MOV R3,#200	1
HERE:	DJNZ R3,HERE	2
	RET	1

ดังนั้นค่าเวลาหน่วงจะเท่ากับ $[(200 \times 2) + 1 + 1] \times 1.085 \text{ us} = 436.17 \text{ us}$



การเขียนโปรแกรมหน่วยเวลาโดยใช้การวนลูปนี้ค่าเวลานอกจากจะขึ้นกับจำนวนครั้งของการวนลูปแล้วยังขึ้นกับจำนวนคำสั่งของการวนลูปด้วย จากตัวอย่างที่ผ่านมาจะเห็นว่าค่าในรีจิสเตอร์ R3 จะมีค่าสูงสุดได้ 255 ทำให้ทำซ้ำได้ 255 ครั้ง และภายในลูปเราสามารถให้ CPU เสียเวลาไป 1 แมกซีนไซเคิลได้โดยใช้คำสั่ง NOP ดังตัวอย่างต่อไป

ตัวอย่างที่ 3.11 จงหาเวลาหน่วยจากการทำโปรแกรมน้อยต่อไปนี้ ถ้าหากระบบทำงานที่ความถี่ 11.0592 MHz

			แมกซีนไซเคิล
DELAY:	MOV	R3,#250	1
HERE:	NOP		1
	NOP		1
	NOP		1
	NOP		1
	DJNZ	R3,HERE	2
	RET		1

วิธีทำ จากโปรแกรมหน่วยเวลาในลูป HERE จะใช้เวลาทั้งหมดเท่ากับ $[250 (1 + 1 + 1 + 1 + 2)] \times 1.085 \text{ us} = 1500 \times 1.085 \text{ us} = 1627.5 \text{ us}$ ถ้ารวมกับคำสั่งภายนอกลูปอีกสองคำสั่งจะใช้เวลาทั้งหมด $1627.5 \text{ us} + 2 \times 1.085 \text{ us} = 1629.67 \text{ us}$

การหน่วยเวลาแบบทำลูปสามารถเขียนโปรแกรมแบบลูปซ้อนลูปได้ ที่เรียกว่า nested loop ดังตัวอย่างโปรแกรมที่ 3.12

ตัวอย่างที่ 3.12 ถ้าหาเวลาหนึ่งแมกซีนไซเคิลเท่ากับ 1.085 us จงหาเวลาหน่วยจากโปรแกรมต่อไปนี้

			แมกซีนไซเคิล
DELAY:			
	MOV	R2,#200	1
AGAIN:	MOV	R3,#250	1
HERE:	NOP		1
	NOP		1
	DJNZ	R3,HERE	2
	DJNZ	R2,AGAIN	2
	RET		1



จากโปรแกรมจะพบว่ามีลูป HERE เป็นลูปในซึ่งจะใช้รีจิสเตอร์ R3 เป็นตัวนับจำนวนลูป และลูปนอกคือลูป AGAIN จะใช้รีจิสเตอร์ R2 เป็นตัวนับ สำหรับลูป HERE จะใช้เวลาทั้งหมด $(4 \times 250) \times 1.085 \text{ us} = 1085 \text{ us}$ ส่วนลูป AGAIN จะให้ทำลูป HERE จำนวน 200 ครั้ง ดังนั้นเวลาที่เสียไปทั้งหมดจะเท่ากับ $200 \times 1085 \text{ us} = 217000 \text{ us}$ และในลูป AGAIN ยังต้องทำอีกสองคำสั่งคือ `MOV R3,#250` และ `DJNZ R2,AGAIN` ซึ่งต้องเสียเวลาไปอีก $3 \times 200 \times 1.085 \text{ us} = 651 \text{ us}$ ดังนั้นเวลาที่เสียไปทั้งหมดจะเท่ากับ $217000 + 651 = 217651 \text{ us} = 217.651 \text{ มิลลิวินาที}$



คำถาม ?

1. จริงหรือไม่ไมโครคอนโทรลเลอร์เบอร์ 8051 นั้นหนึ่งแมกซีนไซเคิลใช้สัญญาณนาฬิกา 12 ลูป
2. คำสั่งของ MCS-51 ที่ใช้แมกซีนไซเคิลน้อยที่สุดจะใช้กี่แมกซีนไซเคิล
3. ถ้าหาก MCS-51 ทำงานที่ความถี่ 12 MHz อยากทราบว่าโปรแกรมน้อย DELAY ต่อไปนี้จะหน่วงเวลาได้เท่าใด

DELAY:

MOV R3,#100

HERE: NOP

NOP

NOP

DJNZ R3,HERE

RET

บทที่

4

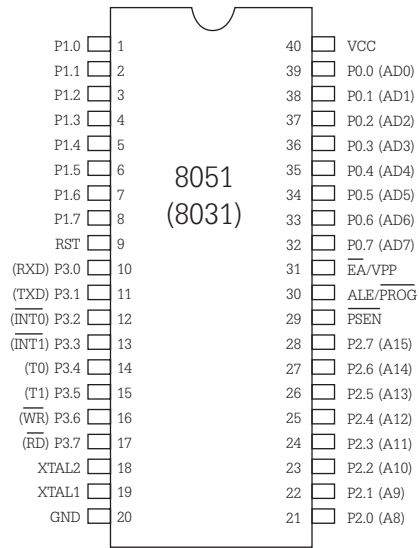
การโปรแกรมพอร์ต อินพุตเอาต์พุต

การใช้งานไมโครคอนโทรลเลอร์มักจะใช้ในการควบคุมอุปกรณ์หรือวงจรอิเล็กทรอนิกส์ภายนอก การที่ไมโครคอนโทรลเลอร์จะติดต่อกับอุปกรณ์ภายนอกได้จะต้องติดต่อผ่านทางพอร์ต ในบทนี้จะแสดงขาต่าง ๆ ของ MCS-51 เบอร์ที่เป็นมาตรฐานในการใช้งานทั่วไป และตัวอย่างการเขียนโปรแกรมติดต่อกับพอร์ตขนานที่อยู่ในตัวมัน



4.1 การจัดขาของ MCS-51

ไอซีไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีลักษณะตัวถังอยู่หลายรูปแบบ อย่างเช่นถ้าเป็นเบอร์ 8051, 8031, 8751, 89C51 จะมีแบบ 40 ขา และเป็นตัวถังแบบ DIP (Dual In-line Package) โดยจะประกอบด้วยขาแอดเดรส ขาข้อมูล ขาอินเทอร์รัปต์ และขาควบคุมต่าง ๆ เช่น I/O, RD, WR เป็นต้น



รูปที่ 4.1 การจัดขาของ 8051 (8031)

ในรูปที่ 4.1 จะเป็นแบบตัวถัง 40 ขา โดยเป็นขาของพอร์ตขนาน 4 พอร์ตคือ P0, P1, P2 และ P3 จำนวน 32 ขา ส่วนอีก 8 ขา เป็น V_{cc} , GND, XTAL1, XTAL2, RST, EA, PSEN และ ALE โดยแต่ละตัวมีหน้าที่ดังนี้

V_{cc}

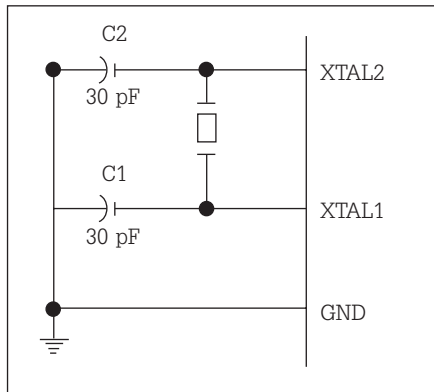
เป็นขาที่ 40 ใช้ต่อกับแหล่งกำเนิดแรงดัน +5V

GND

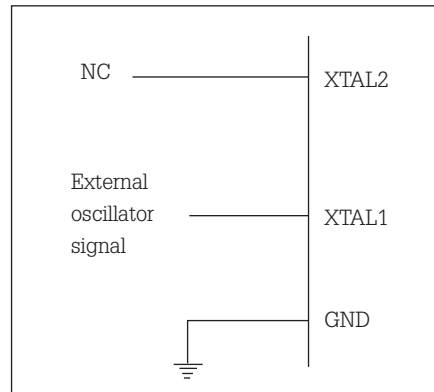
เป็นขาที่ 20 ใช้ต่อกับกราวด์

XTAL1 และ XTAL2

แม้ว่าตัว MCS-51 จะมีวงจรกำเนิดสัญญาณนาฬิกาอยู่ภายในชิป แต่ก็ต้องต่อคริสตัล (crystal) ภายนอกเพื่อควบคุมสัญญาณนาฬิกา โดยต่อเข้าทางขา XTAL1 และ XTAL2 ดังรูปที่ 4.2 (ก) โดยมีตัวเก็บประจุ 30 pF ต่ออยู่ด้วย แต่ถ้า MCS-51 ต้องการสัญญาณนาฬิกาจากภายนอกก็สามารถทำได้ โดยต่อเข้าทางขา XTAL1 ดังรูปที่ 4.2 (ข)



(ก) การต่อ XTAL กับ 8051



(ข) การต่อวงจรกำเนิดสัญญาณโดยตรง

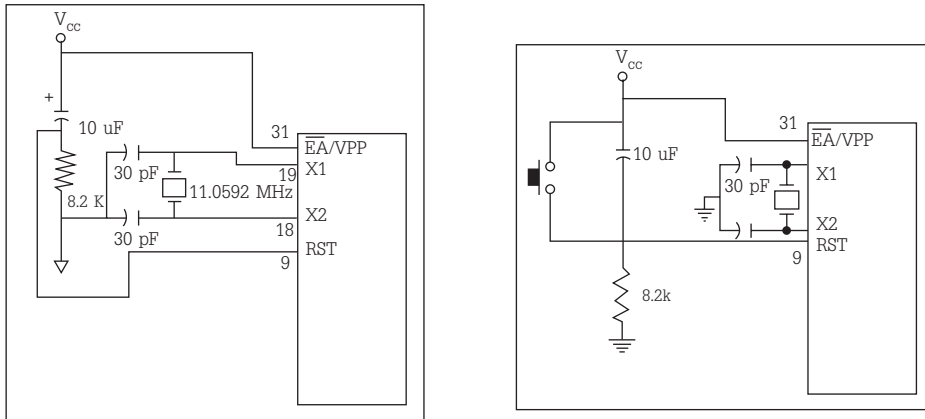
รูปที่ 4.2

RST

เป็นขารีเซ็ต (reset) เมื่อต้องการรีเซ็ต MCS-51 จะต้องทำให้ขาเปลี่ยนจากลอจิก “0” เป็นลอจิก “1” หลังจาก MCS-51 ถูกรีเซ็ต ค่าในรีจิสเตอร์ต่าง ๆ จะเป็นดังนี้

รีจิสเตอร์	ค่าเริ่มต้น
PC	0000
ACC	0000
B	0000
PSW	0000
SP	0007
DPTR	0000

เมื่อ PC มีค่าเป็น 0 จะทำให้ CPU เริ่มทำคำสั่งแรกที่อยู่ในหน่วยความจำ ROM ที่แอดเดรส 0000 และจะทำชุดคำสั่งต่าง ๆ ไปตามโปรแกรม การรีเซ็ต MCS-51 จะมีสองแบบคือ การรีเซ็ตเมื่อเริ่มต้นจ่ายไฟให้กับวงจร เรียกว่า power-on reset อีกวิธีหนึ่งเป็นการรีเซ็ตเมื่อมีการจ่ายไฟให้กับวงจรอยู่แล้วแต่จะกดปุ่มรีเซ็ตของวงจร การต่อวงจรรีเซ็ตทั้งสองประเภททำได้ดังรูปที่ 4.3 สำหรับ MCS-51 เบอร์ที่เป็นมาตรฐานเช่น 8051 ตัวมันจะถูกรีเซ็ตได้นั้นจะต้องให้ขา RST เป็นลอจิก “1” อย่างน้อย 2 แมกซีนไซเคิล การหาช่วงเวลาในแต่ละแมกซีนไซเคิลทำได้ดังตัวอย่างที่ 4.1



รูปที่ 4.3 การต่อวงจรรีเซต

ตัวอย่างที่ 4.1 จงหาเวลาแมชชีนไซเคิลถ้าหาก XTAL ที่ต่ออยู่มีความถี่ดังนี้

ก. XTAL = 11.0592 MHz ข. XTAL = 16 MHz

วิธีทำ

ก. $11.0592 \text{ MHz} / 12 = 921.6 \text{ kHz}$
 แมชชีนไซเคิล $= 1 / 921.6 \text{ kHz} = 1.085 \text{ ไมโครวินาที}$

ข. $16 \text{ MHz} / 12 = 1.333 \text{ MHz}$
 แมชชีนไซเคิล $= 1 / 1.333 \text{ MHz} = 0.75 \text{ ไมโครวินาที}$

EA (External Access)

ตามที่ทราบมาแล้วว่า MCS-51 บางรุ่นมีหน่วยความจำโปรแกรมภายในชิปเช่น 8751, 89C51 เป็นต้น บางรุ่นต้องต่อหน่วยความจำโปรแกรมภายนอกชิป เช่น 8031, 8032 เป็นต้น ถ้าหากขา EA นี้ต่อกับลอจิก “1” หมายความว่าให้อ่านโปรแกรมจากหน่วยความจำภายในชิป แต่ถ้าเป็น MCS-51 ที่มีหน่วยความจำ ROM อยู่ภายนอก ขา EA นี้ต้องต่อลง GND ให้เป็นลอจิก “0” เสมอ เพื่อให้ทำโปรแกรมที่อยู่ใน ROM ภายนอกชิป

PSEN (Program Store Enable)

ขานี้จะแอกทีฟเป็นลอจิก “0” เมื่อ MCS-51 ต้องการอ่านโปรแกรมจากหน่วยความจำ ROM ภายนอก โดยทั่วไปแล้วมักจะนำขานี้ต่อกับขา OE ของหน่วยความจำ ROM

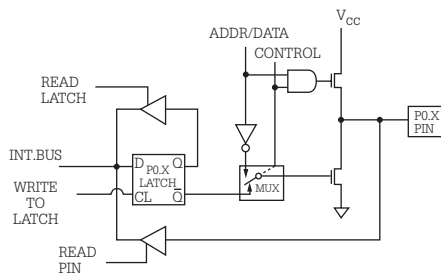
ALE (Address Latch Enable)

เป็นขาเอาต์พุตที่แอกทีฟลอจิก “1” โดย CPU จะส่งสัญญาณนี้ออกมาเอง เนื่องจากพอร์ต P0 นี้สามารถทำงานได้สองหน้าที่คือเป็นขาแอดเดรส และขาข้อมูล ตัว MCS-51 จะใช้สัญญาณทางขา ALE นี้ มัลติเพล็กซ์พอร์ต P0 ว่าขณะนั้นจะใช้งานเป็นแอดเดรส หรือใช้งานเป็นขาข้อมูล โดยทั่วไปแล้วในวงจรระบบไมโครคอนโทรลเลอร์ MCS-51 จะต่อขานี้กับขา G ของชิป 74LS373

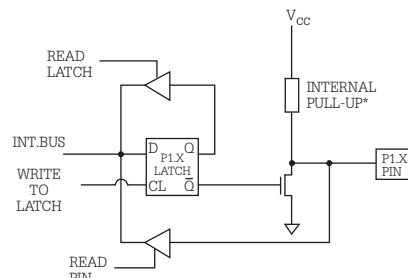


I/O Port และลักษณะการใช้งาน

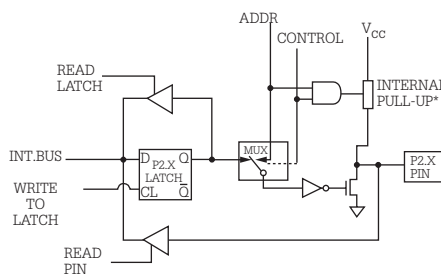
พอร์ตขนานของ MCS-51 มี 4 พอร์ตคือ พอร์ต 0 พอร์ต 1 พอร์ต 2 และพอร์ต 3 โดยทั้งหมดเป็นพอร์ตแบบ 8 บิต ซึ่งสามารถใช้เป็นพอร์ตอินพุตหรือเอาต์พุตได้ สำหรับพอร์ต 3 นั้นนอกจากจะใช้เป็นพอร์ตตามปกติแล้วยังใช้เป็นสัญญาณควบคุมต่าง ๆ อีกด้วย จากโครงสร้างภายในของ MCS-51 จะพบว่าแต่ละบิตของพอร์ต 1, 2 และ 3 จะมีตัวต้านทานที่มีค่าประมาณ 50k ต่อ pull-up อยู่ ทำให้สภาวะปกติแต่ละบิตจะมีลอจิกเป็น “1” และจะทำให้พอร์ตเหล่านี้สามารถขับอุปกรณ์ภายนอกได้ แต่สำหรับพอร์ต 0 จะไม่มีตัวต้านทานต่ออยู่ภายใน



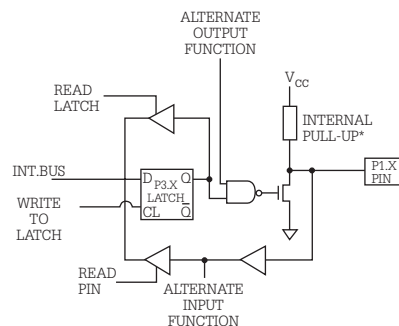
(ก) PORT 0 BIT



(ข) PORT 1 BIT



(ค) PORT 2 BIT



(ง) PORT 3 BIT

รูปที่ 4.4 โครงสร้างภายในของพอร์ตขนาน

พอร์ต 0

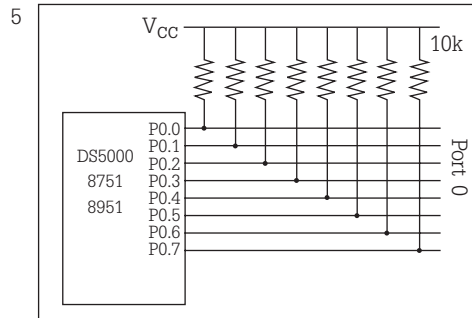
พอร์ตนี้อยู่ที่ขา 32 ถึง 39 นอกจากจะใช้งานเป็นพอร์ตอินพุตเอาต์พุตแล้ว ยังถูกใช้เป็นขาแอดเดรสและขาข้อมูลด้วย โดยแต่ละบิตจะมีชื่อเป็น AD0 - AD7 สำหรับการมัลติเพล็กซ์ระหว่างขาแอดเดรสกับขาข้อมูลจะต้องทำงานร่วมกับวงจรภายนอก และสัญญาณจากขา ALE จากโครงสร้างภายในของมันจะพบว่าแต่ละบิตของพอร์ตนี้จะเป็นวงจรแบบ open drain ถ้าชิปถูกสร้างแบบ MOS และเป็นแบบ open collector ถ้าชิปถูกสร้างแบบ TTL โดยจะไม่มีตัวต้านทานต่อ pull-up อยู่ภายใน โดยทั่วไปแล้วถ้าหากต้องการใช้พอร์ตนี้เป็นพอร์ตอินพุตเอาต์พุตจะต้องนำตัวต้านทานขนาด 10k มาต่อ pull-up ภายนอกดังรูปที่ 4.5 และถ้าหากต้องการเขียนชุดคำสั่งเพื่อส่งข้อมูลออกทางพอร์ต 0 ให้มีค่าลอจิกกลับไปกลับมาสามารถเขียนชุดคำสั่งได้ดังนี้



```

MOV    A , #55H
BACK:  MOV    P0 , A
        ACALL  DELAY
        CPL    A
        SJMP   BACK

```



รูปที่ 4.5 การใช้งานพอร์ต 0

สำหรับการใช้งานพอร์ต 0 เป็นพอร์ตอินพุตนั้นจะต้องเขียนลอจิก “1” ลงไปในแต่ละบิตก่อน เพื่อหยุดการทำงานของทรานซิสเตอร์ที่ขับเคลื่อนทางเอาต์พุต จึงจะอ่านข้อมูลเข้ามาทางอินพุตได้ ตัวอย่างการเขียนชุดคำสั่งต่อไปนี้จะเป็นการอ่านข้อมูลเข้ามาทางพอร์ต 0 และส่งข้อมูลที่อ่านได้ออกไปทางพอร์ต 1

```

MOV    A , #0FFH      ; กำหนดให้รีจิสเตอร์ A เป็นลอจิก “1” ทุกบิต
MOV    P0 , A          ; กำหนดให้พอร์ต P0 เป็นลอจิก “1” ทุกบิต
BACK:  MOV    A , P0    ; อ่านค่าจากพอร์ต P0 มาเก็บใน A
        MOV    P1 , A    ; ส่งค่าจาก A ออกไปทางพอร์ต P1
        SJMP   BACK      ; อ่านข้อมูลจากพอร์ต P0 ซ้ำ

```

พอร์ต P1, P2

สำหรับพอร์ต 1 และพอร์ต 2 โครงสร้างภายในจะมีตัวต้านทานต่อ pull-up อยู่ ทำให้สามารถใช้งานเป็นพอร์ตอินพุตเอาต์พุตได้ทันที นอกจากนี้พอร์ต P2 ยังถูกใช้งานเป็นขาแอดเดรสไบต์สูงด้วย (A8-A15) ถ้าต้องการใช้งานพอร์ตนี้เป็นพอร์ตเอาต์พุตสามารถทำได้โดยเขียนข้อมูลลงไปใน P1 หรือ P2 ได้ทันที แต่ถ้าต้องการใช้งานพอร์ตนี้เป็นพอร์ตอินพุตจะต้องส่งลอจิก “1” ออกไปในแต่ละบิตเสียก่อน ตัวอย่างเช่นถ้าหากต้องการอ่านข้อมูลจากพอร์ต P1 มาเก็บในรีจิสเตอร์ R7, R6 และ R5 สามารถเขียนชุดคำสั่งได้ดังนี้

```

MOV    A , #0FFH      ; ให้ A เป็นลอจิก “1” ทุกบิต
MOV    P1 , A          ; ให้ P1 เป็นอินพุต โดยกำหนดให้เป็น “1” ทุกบิต
MOV    A , P1          ; อ่านข้อมูลจาก P1 มาเก็บใน A
MOV    R7 , A          ; เก็บข้อมูลลง R7
ACALL  DELAY           ; หน่วงเวลา
MOV    A , P1          ; อ่านข้อมูลจาก P1 มาเก็บใน A

```



```

MOV    R6 , A           ; เก็บข้อมูลลง R6
ACALL  DELAY            ; หน่วงเวลา
MOV    A , P1           ; อ่านข้อมูลจาก P1 มาเก็บใน A
MOV    R5 , A           ; เก็บข้อมูลลง R5

```

พอร์ต P3

พอร์ต 3 นี้เป็นพอร์ตขนาดแบบ 8 บิตเช่นกัน สามารถใช้งานเป็นพอร์ตอินพุตเอาต์พุตเช่น P1 และ P2 ได้ นอกจากนี้พอร์ต P3 ในแต่ละบิตได้ออกแบบไว้ให้ใช้งานเป็นพอร์ตควบคุมอีกด้วย โดยหน้าที่ของแต่ละบิตแสดงได้ดังตารางที่ 4.1

ตารางที่ 4.1 พอร์ต P3 และการทำงานแต่ละบิต

ชื่อฟังก์ชัน	บิต	ขา	ลักษณะการใช้งาน
P3.0	RxD	10	ใช้เป็นขารับข้อมูลเข้าแบบอนุกรม
P3.1	TxD	11	ใช้เป็นขาส่งข้อมูลออกแบบอนุกรม
P3.2	INT0	12	ใช้เป็นขารับสัญญาณอินเทอร์รัปต์จากภายนอกหมายเลข 0
P3.3	INT1	13	ใช้เป็นขารับสัญญาณอินเทอร์รัปต์จากภายนอกหมายเลข 1
P3.4	T0	14	ใช้รับสัญญาณจากภายนอกของตัวนับหมายเลข 0
P3.5	T1	15	ใช้รับสัญญาณจากภายนอกของตัวนับหมายเลข 1
P3.6	WR	16	เป็นสัญญาณที่ใช้เขียนข้อมูลลงหน่วยความจำภายนอก
P3.7	RD	17	เป็นสัญญาณที่ใช้อ่านข้อมูลจากหน่วยความจำภายนอก

จากที่ศึกษาเรื่องพอร์ตขนานของ MCS-51 มาทั้งหมดจะพบว่า มีพอร์ตขนานให้เลือกใช้ถึง 4 พอร์ต แม้ว่าพอร์ต P1, P2 และ P3 จะมีตัวต้านทานต่อพูลัพอยู่ภายใน แต่เนื่องจากตัวต้านทานตัวนี้มีค่ามาก (ประมาณ 50k) ทำให้แต่ละบิตขับกระแสภายนอกได้ไม่มากนัก ในการใช้งานทั่วไปแล้วถ้าต้องการนำพอร์ตเหล่านี้ขับกระแสให้กับวงจรภายนอก มักจะต้องต่อไอซีที่เป็นบัฟเฟอร์เพิ่มเข้าไป หรือนำตัวต้านทานขนาด 1k ต่อขนานอยู่ภายนอก เพื่อให้สามารถขับกระแสได้มากขึ้น ถ้าหากต้องการใช้งานพอร์ตขนานเพียงพอร์ตเดียว ควรจะเลือกใช้พอร์ต P1 มากที่สุด เนื่องจากพอร์ตตัวนี้จะไม่ถูกใช้ในหน้าที่อื่น ๆ อีก

บทที่

5

คำสั่งทางคณิตศาสตร์ และการโปรแกรม

ในบทนี้จะกล่าวถึงคำสั่งทางคณิตศาสตร์ทั้งหมดของ MCS-51 ซึ่งมีทั้งแบบที่ทำกับตัวเลขแบบคิดเครื่องหมายและไม่คิดเครื่องหมาย โดยหัวข้อที่ 5.1 จะกล่าวถึงคำสั่งการบวกเลขแบบไม่คิดเครื่องหมาย รวมทั้งตัวเลขข้อมูลแบบ BCD (Binary Coded Decimal) ส่วนคำสั่งคูณและหารจะกล่าวถึงในหัวข้อที่ 5.2 'วนหัวข้อที่ 5.3 จะกล่าวถึงตัวเลขแบบคิดเครื่องหมาย



5.1 การบวกเลขแบบไม่คิดเครื่องหมาย

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะประมวลผลกับข้อมูลขนาด 8 บิต ถ้าหากนำข้อมูล 8 บิต มาเก็บตัวเลขแบบไม่คิดเครื่องหมายจะเก็บได้ค่าตั้งแต่ 00 ถึง FFH (0 ถึง 255)

การบวกแบบไม่คิดเครื่องหมาย

การบวกเลขของ MCS-51 จะกระทำกับรีจิสเตอร์แอดคิวมูเลเตอร์หรือรีจิสเตอร์ A โดยจะใช้คำสั่ง ADD ซึ่งมีรูปแบบดังนี้

ADD A,source ; A = A + Source



คำสั่ง ADD จะมีโอเปอเรนด์สองตัว โดยโอเปอเรนด์ปลายทางต้องเป็นรีจิสเตอร์ A เท่านั้น ส่วนโอเปอเรนด์ต้นทางจะเป็นรีจิสเตอร์ ข้อมูล หรือตำแหน่งหน่วยความจำก็ได้ เมื่อมีการทำคำสั่งแล้วจะมีผลต่อบิตแฟลก AC, CY หรือ P ในรีจิสเตอร์แฟลก ตัวอย่างการใช้คำสั่งแสดงได้ดังตัวอย่างที่ 5.1

ตัวอย่างที่ 5.1 จงแสดงการบวกเมื่อมีการทำคำสั่งต่อไปนี้

```
MOV  A,#0F5H          ; A = F5H
ADD  A,#0BH           ; A = F5 + 0B = 00
```

วิธีทำ	F5H	1111	0101
	+ 0BH	+ 0000	1011
	100H	0000	0000

หลังจากการบวกค่าในรีจิสเตอร์ A จะเป็น 00 ส่วนค่าของบิตแฟลกจะเป็นดังนี้

```
CY   = 1  มีการทดจากบิต D7
P    = 1  เพราะเป็น 0 ทุกบิต
AC   = 1  มีการทดจากบิต D3 ไปยัง D4
```

การบวกหลายไบต์

เราสามารถใส่คำสั่ง ADD บวกข้อมูลหลาย ๆ ไบต์ได้ แต่ผลรวมจะต้องไม่เกินค่าที่รีจิสเตอร์ 8 บิตจะเก็บได้ ซึ่งก็คือค่า FFH ในตัวอย่างที่ 5.2 เป็นตัวอย่างการบวกข้อมูลจำนวน 5 ไบต์ โดยใช้รีจิสเตอร์ R7 และแอดคิวมูลเตอร์ A เก็บผลลัพธ์

ตัวอย่างที่ 5.2 ถ้าหากมีข้อมูลเลขฐานสิบหกอยู่ใน RAM ตำแหน่ง 40 ถึง 44 จงเขียนโปรแกรมบวกค่าของข้อมูลดังกล่าวและให้ผลลัพธ์ไบต์ต่ำเก็บในรีจิสเตอร์ A ไบต์สูงเก็บในรีจิสเตอร์ R7

40H	=	[7D]
41H	=	[EB]
42H	=	[C5]
43H	=	[5B]
44H	=	[30]

**วิธีทำ**

	MOV	R0,#40H	; ใส่ค่าให้พอยน์เตอร์
	MOV	R2,#5	; ใส่ค่าให้ตัวนับ
	CLR	A	; A = 0
	MOV	R7,A	; เคลียร์ค่า R7
AGAIN:	ADD	A,@R0	; บวกค่าใน A กับค่าในตำแหน่ง R0
	JNC	NEXT	; ถ้า CY = 0 หมายความว่าไม่มีการทดจาก A
	INC	R7	; เพิ่มค่า R7
NEXT:	INC	R0	; เพิ่มค่าพอยน์เตอร์
	DJNZ	R2,AGAIN	; ทำซ้ำจนกว่า R2 จะเป็นศูนย์

จากตัวอย่างที่ 5.2 จะเห็นว่าจะใช้การวนลูปในการบวกเลข โดยมีลำดับการทำงานดังนี้

1. ลูปแรกจะบวกค่า 7DH กับรีจิสเตอร์ A จะได้ค่า CY = 0 และ R7 = 00 ส่วนตัวนับ R2 = 04
2. ลูปที่สองจะบวกค่า EBH กับรีจิสเตอร์ A ผลลัพธ์จะได้ A = 68H และ CY = 1 ซึ่งจะทำให้การเพิ่มค่าในรีจิสเตอร์ R7 ส่วนตัวนับ R2 = 03
3. ต่อไปบวกค่า C5H กับรีจิสเตอร์ A จะได้ A = 2DH และมีการทดอีกครั้ง ทำให้ค่าใน R7 เพิ่มอีกครั้งส่วนตัวนับ R2 = 02

เมื่อการทำงานไปถึงลูปสุดท้าย ผลลัพธ์จะเก็บในรีจิสเตอร์ A และ R7 โดยรีจิสเตอร์ A เก็บไบต์ต่ำ และ R7 เก็บไบต์สูง

คำสั่ง ADC กับ การบวกแบบ 16 บิต

เนื่องจากคำสั่งการบวกของ MCS-51 จะบวกเลขแบบ 8 บิต ถ้าหากเขียนคำสั่งสำหรับบวกข้อมูลขนาด 16 บิต หรือสองไบต์จะต้องบวกข้อมูลที่ละไบต์ โดยบวกไบต์ต่ำก่อนแล้วจึงบวกไบต์สูง โดยการบวกไบต์สูงจะต้องนำตัวทดจากการบวกไบต์ต่ำมารวมบวกด้วย เมื่อไบต์ต่ำบวกกันแล้วมีตัวทดเกิดขึ้น ตัวทอนั้นจะถูกเก็บในบิตแฟลกตัวทด สำหรับคำสั่งที่บวกโดยการนำตัวทอมารวมบวกด้วยคือคำสั่ง ADC (Add with carry) ตัวอย่างการบวกข้อมูลแบบ 16 บิต เช่นการบวกค่าของ 3CE7H กับ 3B8DH สามารถทำได้ดังนี้

$$\begin{array}{rcl}
 & 1 & \longleftarrow \text{ตัวทดจากไบต์ต่ำ} \\
 3C & E7 & \\
 + 3B & 8D & \\
 \hline
 78 & 74 &
 \end{array}$$

เมื่อบวกไบต์แรก ($E7 + 8D = 74$, CY = 1) จะมีตัวทดเกิดขึ้น และต้องนำไปบวกในไบต์ถัดไป ดังนั้นการบวกไบต์สูงจะเป็น $3C + 3B + 1 = 78$



ตัวอย่างที่ 5.3

จงเขียนโปรแกรมบวกข้อมูลขนาด 16 บิต ระหว่าง 3CE7H กับ 3B8DH โดยเก็บคำตอบในรีจิสเตอร์ R7 และ R6 โดย R6 เก็บไบต์ต่ำ ส่วน R7 เก็บไบต์สูง

วิธีทำ

```

CLR    C                ; ให้ค่า CY = 0
MOV    A,#0E7H          ; โหลดค่าไบต์ต่ำ A = E7H
ADD    A,#8DH           ; บวกค่าไบต์ต่ำได้ผลลัพธ์เป็น A = 74H และ CY = 1
MOV    R6,A             ; เก็บค่าไบต์ต่ำลงในรีจิสเตอร์ R6
MOV    A,#3CH           ; โหลดค่าไบต์สูง
ADDC   A,#3BH           ; บวกค่าไบต์สูงโดยบวกบิตตัวทดด้วย
                        ; 3B + 3C + 1 = 78
MOV    R7,A             ; เก็บค่าไบต์สูงของผลบวก
  
```

ระบบเลข BCD

เลข BCD (Binary Coded Decimal) เป็นการนำเลขฐานสองขนาด 4 บิต มาแทนตัวเลข 0 ถึง 9 ของเลขฐานสิบแต่ละหลัก ทำให้เราสามารถนำเลขฐานสิบมาเก็บในรูปแบบรหัสเลขฐานสองได้ง่าย โดยถ้ามีเลขฐานสิบค่าหนึ่งก็ให้แทนค่าแต่ละหลักด้วยเลขฐานสองขนาด 4 บิต ดังรูปที่ 5.1 โดยเลข BCD จะมีสองรูปแบบคือ 1. unpacked BCD และ 2. packed BCD

เลขฐานสิบ	เลข BCD	เลขฐานสิบ	เลข BCD
1	0001	15	0001 0101
2	0010	123	0001 0010 0011
8	1000	256	0010 0101 0110
10	0001 0000	702	0111 0000 0010
12	0001 0010	90	1001 0000

รูปที่ 5.1

Unpacked BCD

ตัวเลขแบบ unpacked BCD จะใช้เลขฐานสองจำนวน 8 บิต หรือหนึ่งไบต์แทนเลขฐานสิบแต่ละหลัก แต่จะให้ 4 บิตบนเป็น 0 ทั้งหมด และ 4 บิตล่างแทนเลข 0 ถึง 9 ตัวอย่างเช่นค่า 0000 1001 และ 0000 0101 จะเป็นเลข unpacked BCD ที่แทนค่า 9 และ 5 ตามลำดับ



Packed BCD

ตัวเลขแบบ packed BCD นี้เลขฐานสองหนึ่งไบต์จะแทนเลข BCD ได้สองหลัก คือ 4 บิตล่างหนึ่งหลัก และ 4 บิตบนอีกหนึ่งหลัก ตัวอย่างเช่น 0101 1001 จะเป็นเลข packed BCD ที่แทนค่า 59H ดังนั้นทำให้ข้อมูลขนาด 1 ไบต์สามารถเก็บข้อมูลในรหัส BCD ได้สองหลัก

การใช้คำสั่งบวกในการบวกเลขตามระบบ BCD นั้นจะต้องมีการปรับค่าหลังการบวกเพื่อให้คำตอบที่ได้อยู่ในรูปแบบของเลข BCD พิจารณาการบวกต่อไปนี้

เลขฐานสิบ	เลข BCD
17	0001 0111
+ 28	+ 0010 1000
45	0011 1111

ถ้าเขียนเป็นคำสั่งจะได้เป็น

```
MOV A,#17H
ADD A,#28H
```

จากตัวอย่างจะเห็นว่าถ้ามีการบวกเลขฐานสิบคำตอบที่ได้จะเป็น 45 แต่ถ้าเราแทนตัวเลขคำตอบนี้ด้วยรหัส BCD และให้ไมโครคอนโทรลเลอร์บวกกันผลลัพธ์ที่ได้จะเป็น 0011 1111B (3FH) ซึ่งจะไม่ใช่เลข BCD เพราะเลข BCD ต้องมีค่าอยู่ระหว่าง 0000 ถึง 1001 (หรือ 0 ถึง 9) ถ้าเป็นเลข BCD ที่ถูกต้องผลลัพธ์จะต้องเป็น 0100 0101 หรือ 45H ดังนั้นจะต้องเขียนโปรแกรมแก้ไขผลลัพธ์ที่ได้ โดยนำ 6 (0110) เข้าไปบวกกับเลข 4 บิตต่ำ จะได้เป็น 3FH + 06H = 45H ทำให้ผลลัพธ์ที่ออกมาเป็นเลข BCD ที่ถูกต้องต่อไปพิจารณาการบวกเลข BCD ระหว่าง 52H กับ 87H

52	52H
+ 87	+ 87H
139	D9

จะเห็นว่าผลลัพธ์จากการบวกเลข BCD จะได้ค่า D9 ซึ่งไม่เป็นเลข BCD จะต้องแก้ไขโดยการบวกค่า 6 เข้าไปในหลักสูง จะได้เป็น D9H + 60H = 139H ซึ่งจะทำให้ผลลัพธ์ที่ได้เป็นเลข BCD ที่ถูกต้อง เราจะเห็นว่าในการบวกเลข BCD ถ้าหากผลลัพธ์ที่ออกมาไม่เป็นเลข BCD เราจะต้องแก้ไขข้อมูลโดยนำค่า 6 ไปบวกกับเลขฐานสองในหลักที่ไม่เป็นเลข BCD ใน MCS-51 มีคำสั่งที่ใช้ปรับค่าหลังจากการบวกให้เป็นเลข BCD คือคำสั่ง DA A



คำสั่ง DA

MCS-51 มีคำสั่ง DA (decimal adjust for addition) เพื่อใช้ในการบวกเลขแบบ BCD โดยจะใช้ปรับค่าหลังจากการบวกให้ผลลัพธ์เป็นเลข BCD คำสั่งนี้จะมีโอเพอเรนด์ตัวเดียวคือ รีจิสเตอร์ A โดยคำสั่ง DA จะทำการบวกค่า 6 เข้าไปในสี่บิตต่ำถ้าหากค่าในสี่บิตต่ำมีค่ามากกว่า 9 ตัวอย่างการใช้คำสั่ง DA แสดงได้ดังนี้

MOV A,#47H	; ให้ A = 47 เป็นเลข BCD ค่าแรก
MOV B,#25H	; ให้ B = 25 เป็นเลข BCD ค่าที่สอง
ADD A,B	; บวกแบบไบนารี ได้ผลลัพธ์เป็น A = 6CH
DA A	; ปรับค่าหลังการบวกให้เป็นเลข BCD คือ A = 72H

จากตัวอย่างโปรแกรมผลลัพธ์จากการบวกจะเท่ากับ 72H ($47 + 25 = 72$) โดยคำสั่ง DA จะใช้หลังคำสั่งบวก และจะทำให้ผลลัพธ์ที่ได้ไม่มีหลักใดอยู่ระหว่าง A ถึง F โดยทั่วไปคำสั่ง DA นี้จะใช้หลังคำสั่ง ADD หรือ ADDC จะไม่ใช้หลังคำสั่ง INC โดยมีขั้นตอนดำเนินการของคำสั่งจะเป็นดังนี้

1. ถ้าสี่บิตต่ำมีค่ามากกว่า 9 หรือถ้า $AC = 1$ จะทำการบวก 4 บิตต่ำด้วย 0110
2. ถ้าสี่บิตบนมีค่ามากกว่า 9 หรือถ้า $CY = 1$ จะทำการบวก 4 บิตบนด้วย 0110

ตัวอย่างเช่นถ้าหากบวกเลข BCD ระหว่าง 29H กับ 18H ผลลัพธ์จะเป็น 41H โดยมีขั้นตอนดังนี้

HEX	BCD	
29	0010 1001	
+ 18	+ 0001 1000	
<u>41</u>	<u>0100 0001</u>	AC = 1
+ 6	+ 0110	
<u>47</u>	<u>0100 0111</u>	

จะเห็นว่าถ้า $AC = 1$ คำสั่ง DA A จะบวก 6 เข้าไปใน 4 บิตต่ำ



ตัวอย่างที่ 5.4 ถ้าหากมีเลข BCD จำนวน 5 ตัวอยู่ในหน่วยความจำ RAM ตั้งแต่ตำแหน่ง 40H เป็นต้นไป จงเขียนโปรแกรมบวกเลขแบบ BCD ทั้งหมด และเก็บผลลัพธ์เป็นเลข BCD

	44	37
	43	59
	42	65
	41	11
RO →	40	71

วิธีทำ	MOV	R0,#40H	; โหลดตัวชี้ไปยังแอดเดรส 40
	MOV	R2,#5	; โหลดตัวนับจำนวน 5 ตัว
	CLR	A	; เคลียร์ A = 0
	MOV	R7,A	; เคลียร์ค่า R7
AGAIN:	ADD	A,@R0	; บวกค่าใน A กับค่าที่ชี้โดยรีจิสเตอร์ R0
	DA	A	; ปรับค่าเป็นเลข BCD
	JNC	NEXT	; กระโดดไป NEXT ถ้ารีจิสเตอร์ A ไม่มีการทด
	INC	R7	; เก็บตัวทด
NEXT:	INC	R0	; เพิ่มค่าแอดเดรสตัวชี้
	DJNZ	R2,AGAIN	; ทำซ้ำจน R2 มีค่าเป็นศูนย์

คำตอบที่ได้จะเก็บผลลัพธ์ในรีจิสเตอร์ A และถ้ามีการทดจะเก็บตัวทดใน R7

การลบแบบไม่คิดเครื่องหมาย

โดยทั่วไปแล้วไมโครโปรเซสเซอร์จะมีคำสั่งลบเลขอยู่สองคำสั่งคือ SUB และ SUBB (subtract with borrow) แต่ใน MCS-51 มีคำสั่งลบคำสั่งเดียวคือ SUBB ซึ่งเป็นการลบที่นำแฟลกตัวทดมารวมลบด้วย แต่สามารถพิจารณาได้สองกรณีคือ กรณีที่ CY = 0 และกรณี CY = 1 โดยคำสั่ง SUBB มีรูปแบบดังนี้

SUBB A,source ; มีค่าเป็น A = A - source - CY

คำสั่ง SUBB เมื่อ CY = 0

การลบในไมโครโปรเซสเซอร์ยุคใหม่ ๆ จะใช้วิธีการทำทวิคอมพลีเมนต์ (2's complement) เมื่อ MCS-51 ทำคำสั่ง SUBB กับตัวเลขแบบไม่คิดเครื่องหมาย เมื่อ CY = 0 จะมีการทำงานดังนี้



1. ทำเลข 2's complement กับตัวเลขที่นำมาลบ (source operand)
2. บวกค่าที่ได้เข้ากับค่าในรีจิสเตอร์ A
3. กลับค่าบิตแฟลกตัวทด

โดยทั้งสามขั้นตอนนี้จะถูกทำโดย CPU ที่อยู่ภายใน MCS-51

ตัวอย่างที่ 5.5 จงแสดงขั้นตอนการทำคำสั่งต่อไปนี้

CLR	C	; เคลียร์ค่า CY = 0
MOV	A,#3FH	; โหลดค่า 3FH ใส่ใน A (A = 3FH)
MOV	R3,#23H	; โหลดค่า 23H ใส่ใน R3 (R3 = 23H)
SUBB	A,R3	; ทำ A - R3 ผลลัพธ์เก็บใน A

วิธีทำ

A =	3F	0011 1111	0011 1111	
R3 =	23	0010 0011	+ 1101 1101	(2's complement)
	1C		1 0001 1100	
			0	CF = 0 (ขั้นตอนที่ 3)

ผลลัพธ์ที่ได้จะพบว่ามีความเท่ากับ 1CH โดย CY = 0 , AC = 0 ซึ่งแฟลกตัวทอนี้จะเป็นตัวบอกว่าผลลัพธ์เป็นเลขบวกหรือเลขลบ

ถ้าหากหลังทำคำสั่ง SUBB ทำให้ CY = 0 หมายความว่า เป็นเลขบวก ถ้า CY = 1 หมายความว่า เป็นเลขลบ ถ้าหากคำตอบที่ได้เป็นเลขลบ และอยากทราบว่าเป็นค่าลบเท่าใดให้ทำ 2's complement กลับ ซึ่งอาจทำได้โดยการใช้คำสั่ง CPL ซึ่งเป็นการกลับค่าบิตทุกบิต จากนั้นตามด้วยคำสั่ง INC ก็จะได้เลข 2's complement ออกมา ดังตัวอย่างที่ 5.6

ตัวอย่างที่ 5.6 จงวิเคราะห์การทำโปรแกรมต่อไปนี้

CLR	C	
MOV	A,#4CH	; ใส่ค่า 4CH ลงในรีจิสเตอร์ A
SUBB	A,#6EH	; ลบค่าใน A ด้วย 6E
JNC	NEXT	; ถ้า CY = 0 ให้กระโดดไป NEXT
CPL	A	; ถ้า CY = 1 ทำ 1's complement
INC	A	; บวกด้วย 1 จะได้เลข 2's complement
NEXT:	MOV	R1,A ; เก็บค่าคำตอบ A ลงใน R1



วิธีทำ การทำคำสั่ง SUBB A,6EH จะมีขั้นตอนดังนี้

4C 0100 1100	0100 1100
- 6E 0110 1110	2's complement = 1001 0010
- 22	กลับค่า → 0 1101 1110

จะเห็นว่า CY = 1 หมายความว่า เป็นเลขลบ ให้ทำ 2's complement กลับ จะได้เป็น 0010 0010 ซึ่งก็คือ -22H นั่นเอง

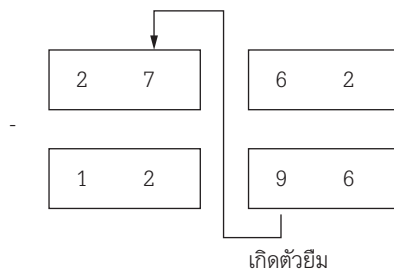
คำสั่ง SUBB เมื่อ CY = 1

เป็นการลบเลขที่นำค่าของการยืมมาดำเนินการด้วย ถ้าค่าของตัวลบมีค่ามากกว่าตัวตั้งจะต้องทำการยืมค่าจากบิตสูงถัดไป ดังตัวอย่างที่ 5.7

ตัวอย่างที่ 5.7 จงวิเคราะห์โปรแกรมต่อไปนี้

CLR C	; CY = 0
MOV A,#62H	; A = 62H
SUBB A,#96H	; 62H - 96H = CCH โดย CY = 1
MOV R7,A	; เก็บค่าผลลัพธ์
MOV A,#27H	; A = 27H
SUBB A,#12H	; 27H - 12H - 1 = 14H
MOV R6,A	; เก็บค่าผลลัพธ์

วิธีทำ จากโปรแกรมจะพบว่าหลังจากทำคำสั่ง SUBB คำสั่งแรกจะได้ผลลัพธ์เป็น A = 62H - 96H = CCH และ CY จะเป็น 1 เนื่องจากการยืมเกิดขึ้น เมื่อทำคำสั่ง SUBB ครั้งที่สองจะได้ A = 27H - 12H - 1 = 14H ดังนั้นโปรแกรมนี้อาจจะทำงานเป็น 2762H - 1296H = 14CCH





คำถาม ?

1. คำสั่ง ADD A,source ผลลัพธ์จะเก็บอยู่ที่ใด
2. คำสั่ง ADD R1,R2 จะทำงานได้หรือไม่
3. จงหาค่า A และแฟล็ก CY จากการทำคำสั่งต่อไปนี้

ղ. MOV A,#4FH

```
7. MOV    A,#9CH
```

```
ADD    A,#0B1H
```

```
ADD    A,#63H
```

4. จงแสดงการลบค่า 05H ออกจาก 43H



5.2 การคูณและการหารแบบไม่คิดเครื่องหมาย

คำสั่งคุณและคำสั่งหารของ MCS-51 จะใช้รีจิสเตอร์ A และ B เป็นโอเปอเรนด์เท่านั้น ไม่สามารถกระทำกับตัวเลขที่อยู่ในตำแหน่งอื่น ๆ ได้

การคูณ

ไมโครคอนโทรลเลอร์ MCS-51 จะทำการคูณแบบไบต์กับไบต์โดยใช้คำสั่งเป็น

MUL AB ; $A \times B$ ได้ผลลัพธ์เป็นข้อมูล 16 บิต

โดยการคูณนี้จะนำค่าในรีจิสเตอร์ A และรีจิสเตอร์ B มาคูณกัน ผลลัพธ์ที่ได้จะมีขนาด 16 บิต และจะเก็บในรีจิสเตอร์ A และ B โดยไบต์ต่ำเก็บใน A และไบต์สูงจะเก็บใน B ตัวอย่างเช่นถ้าหากคูณ 25H ด้วย 65H และเก็บผลลัพธ์ในรีจิสเตอร์ A และ B สามารถเขียนคำสั่งได้เป็น

MOV A,#25H ; โหลดค่า 25H เก็บใน A

MOV B,#65H : โหลดค่า 65H เก็บใน B

MUL AB ; 25H * 65H = 0E99H

: โดย B เก็บ 0EH และ A เก็บ 99H

ถ้าหากผลลัพธ์จากการคูณมีค่ามากกว่า 255 ซึ่งรีจิสเตอร์ตัวเดียวไม่สามารถเก็บค่าผลลัพธ์นี้ได้ จะทำให้แฟลกโอเวอร์โฟลว์เกิดการทดขึ้น ดังนั้นจะต้องอ่านค่าคำตอบไบต์สูงจากรีจิสเตอร์ B ออกมาด้วย และในการทำคำสั่งคูณนี้ค่าแฟลกตัวทดจะมีค่าเป็น “0” ตลอดการทำงาน



การหาร

ไมโครคอนโทรลเลอร์ MCS-51 จะทำการหารแบบไบนารีกับไบนารี โดยใช้คำสั่งที่มีรูปแบบดังนี้

DIV AB ; หารค่าใน A ด้วยค่าใน B

การหารจะใช้รีจิสเตอร์ A เป็นตัวตั้ง และใช้รีจิสเตอร์ B เป็นตัวหาร หลังจากการหารผลลัพธ์จะเก็บในรีจิสเตอร์ A และเศษจะเก็บใน B พิจารณาตัวอย่างต่อไปนี้

MOV A,#95 ; โหลด A ด้วย 95

MOV B,#10 ; โหลด B ด้วย 10

DIV AB ; A = 9 (เก็บผลลัพธ์)

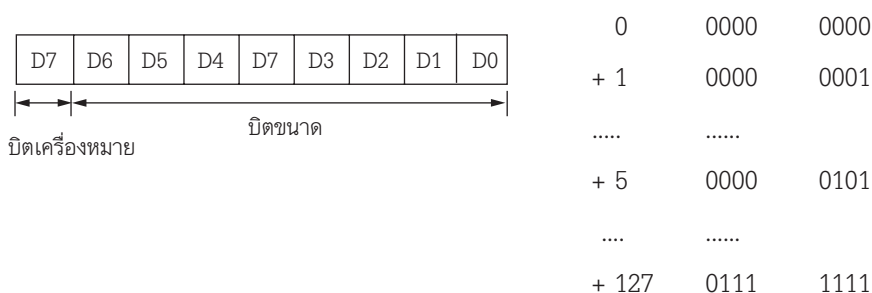
; B = 5 (เก็บเศษ)

ในการใช้คำสั่งหารนั้นจะต้องตรวจสอบก่อนว่าค่าในรีจิสเตอร์ B เป็น 0 หรือไม่ ถ้าหาก B เป็นค่า 0 จะทำให้ตัวหารเป็นศูนย์ ซึ่งจะทำให้แฟลกโอเวอร์โฟลว์ (OV) มีค่าเป็น “1”



5.3 ตัวเลขแบบคิดเครื่องหมาย

การเก็บเลขจำนวนเต็มในรูปแบบเลขฐานสองแบบคิดเครื่องหมายนั้นจะใช้ค่าบิตนัยสำคัญสูงสุด (MSB) เป็นบิตเครื่องหมาย เพื่อบอกว่าเลขจำนวนเต็มนั้นเป็นเลขบวกหรือเลขลบ ส่วนบิตนัยสำคัญต่ำลงมาจะเป็นบิตที่บอกขนาดของข้อมูล เช่นถ้าหากเก็บข้อมูลขนาดหนึ่งไบต์ จะใช้บิต 7 เป็นบิตเครื่องหมาย ส่วนบิต 0-6 เป็นบิตขนาดข้อมูล



รูปที่ 5.1 แสดงการเก็บข้อมูลแบบ 8 บิต



จากรูปที่ 5.1 จะพบว่าถ้าหากใช้รีจิสเตอร์ขนาด 8 บิต เก็บเลขจำนวนเต็ม จะได้เลขบวกที่มีค่าตั้งแต่ 0 ถึง +127

สำหรับการเก็บเลขจำนวนเต็มลบจะต้องให้บิตสูงสุดมีลอจิกเป็น “1” การแทนตัวเลขจำนวนเต็มแบบคิดเครื่องหมายนี้โดยทั่วไปจะใช้วิธีการทวอมพลีเมนต์ (2's complement) โดยการเปลี่ยนเลขลบให้เป็นเลขแบบทวอมพลีเมนต์จะมีวิธีการดังนี้

1. เขียนขนาดของตัวเลขแบบ 8 บิต (ไม่คิดเครื่องหมาย)
2. กลับค่าบิตทุกบิต
3. บวกค่าที่ได้ด้วย 1

ตัวอย่างที่ 5.8 จงแสดงการเก็บเลข -5

วิธีทำ สามารถทำได้ตามขั้นตอนต่อไปนี้

1. 0000 0101 ตัวเลข 8 บิต แทนค่า 5
2. 1111 1010 กลับค่าทุกบิต
3. 1111 1011 บวกด้วย 1 จะได้ผลลัพธ์เป็น OFBH

ดังนั้นค่า -5 จะถูกเก็บเป็น OFBH

ตัวอย่างที่ 5.9 จงแสดงการเก็บเลข -34H

วิธีทำ สามารถทำได้ตามขั้นตอนต่อไปนี้

1. 0011 0100 ตัวเลข 8 บิต แทนค่า 34H
2. 1100 10 11 กลับค่าทุกบิต
3. 1100 1100 บวกด้วย 1 จะได้ผลลัพธ์เป็น 0CCH

ดังนั้น -34H จะถูกเก็บเป็น 0CCH

ตัวอย่างที่ 5.10 จงแสดงการเก็บเลข -128

วิธีทำ สามารถทำได้ตามขั้นตอนต่อไปนี้

1. 1000 0000 ตัวเลข 8 บิต แทนค่า 128
2. 0111 1111 กลับค่าบิตทุกบิต
3. 1000 0000 บวกด้วย 1 จะได้เป็น 80H

ดังนั้น -128 จะ ถูกเก็บเป็น 80H

จากตัวอย่างที่ 5.10 จะพบว่ารีจิสเตอร์ขนาด 8 บิตนั้น สามารถเก็บเลขจำนวนเต็มลบได้ในช่วง -1 ถึง -128 การเก็บตัวเลขแบบคิดเครื่องหมายโดยใช้รีจิสเตอร์ขนาด 8 บิต สามารถสรุปได้ดังนี้



เลขฐานสิบ	เลขฐานสอง	เลขฐานสิบหก
-128	1000 0000	80
-127	1000 0001	81
-126	1000 0010	82
.....		
-2	1111 1110	FE
-1	1111 1111	FF
0	0000 0000	00
+1	0000 0001	01
+2	0000 0010	02
.....		
+127	0111 1111	7F

ปัญหาการโอเวอร์โฟลในตัวแทนแบบคิดเครื่องหมาย

การประมวลผลแบบคิดเครื่องหมายบางครั้งคำตอบที่ MCS-51 คิดมาจะได้ไม่ตรงกับคำตอบที่มนุษย์คิด ตัวอย่างเช่น ถ้าให้บวกตัวเลข +96 กับ +70 คำตอบที่ MCS-51 คิดได้คือ -90 โดยแฟลก OV จะเป็นลอจิก “1” แต่คำตอบที่มนุษย์คิดจะได้เป็น 166 ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 5.11 จงวิเคราะห์ผลลัพธ์จากการทำชุดคำสั่งต่อไปนี้

```
MOV A,#+96          ; A = 0110 0000 (A = 60H)
MOV R1,#+70          ; R1 = 0100 0110 (R1 = 46H)
ADD A,R1              ; A = 1010 0110
                     ; A = A6H = -90
```

วิธีทำ

```
+96      0110 0000
+ +70    0100 0110
-----
+166     1010 0110    และ OV = 1
```

ตัวอย่างที่ 5.12 ตัวอย่างนี้ จะทำให้แฟลก OV เป็นลอจิก “1”

```
MOV A,#-128          ; A = 1000 0000 (A = 80H)
MOV R4,#-2           ; R4 = 1111 1110 (R4 = FEH)
ADD A,R4              ; A = 0111 1110 (A = 7EH = +126 ไม่ถูกต้อง)
```

**วิธีทำ**

-128	1000 0000	
+ - 2	<u>1111 1110</u>	
-130	0111 1110	และ OV = 1

ใน CPU จะเก็บค่า +126 แต่จะพบว่า OV = 1

ตัวอย่างที่ 5.13 ตัวอย่างนี้ แฟล็ก OV จะเป็นลอจิก “0”

```
MOV A,#-2          ; A = 1111 1110 (A = FEH)
MOV R1,#-5          ; R1 = 1111 1011 (R1 = FBH)
ADD A,R1            ; A = 1111 1001 (A = F9H = -7 ถูกต้อง , OV = 0)
```

วิธีทำ

-2	1111 1110	
+ -5	1111 1011	
-7	<u>1111 1001</u>	และ OV = 0

ใน CPU เก็บค่า -7 ซึ่งเป็นค่าที่ถูกต้อง ขณะที่ OV = 0

ตัวอย่างที่ 5.14 ตัวอย่างนี้ แฟล็ก OV จะเป็นลอจิก “0”

```
MOV A,#+7           ; A = 0000 0111 (A = 07H)
MOV R1,#+18          ; R1 = 0001 0010 (R1 = 12H)
ADD A,R1             ; A = 0001 1001 (A = 19H = +25, ถูกต้อง , OV = 0)
```

วิธีทำ

7	0000 0111	
+ 18	0001 0010	
25	0001 1001	และ OV = 0

ใน CPU เก็บค่า +25 ซึ่งเป็นคำตอบที่ถูกต้อง

บทที่

6

คำสั่งทางลอจิกและ การโปรแกรม

ในบทนี้จะกล่าวถึงชุดคำสั่งทางลอจิกได้แก่คำสั่ง AND, OR และ XOR รวมทั้งคำสั่งกลับค่าข้อมูลด้วย ในหัวข้อที่ 6.2 จะกล่าวถึงคำสั่งการหมุนข้อมูล (rotate) และสลับข้อมูล (swap) ในหัวข้อที่ 6.3 จะพูดถึงการเขียนโปรแกรมเปลี่ยนรหัส BCD เป็นรหัส ASCII



6.1 คำสั่งทางลอจิกและคำสั่งเปรียบเทียบ

ไมโครคอนโทรลเลอร์สามารถประมวลผลแบบบิตต่อบิตได้ เช่น การดำเนินการแอนด์ (AND) การดำเนินการออร์ (OR) และการทำเอ็กซคลูซีฟออร์ (XOR) โดยลักษณะของคำสั่งต่าง ๆ จะเป็นดังนี้

การ AND

คำสั่งนี้ใช้ในการแอนด์ข้อมูลแบบบิตต่อบิตมีรูปแบบดังนี้

ANL Destination Source ; dest = dest AND Source

คำสั่งนี้จะนำค่าโอเปอเรนด์สองตัวมา AND กัน โดยผลลัพธ์ที่ได้จะถูกเก็บในตำแหน่งปลายทาง (destination) สำหรับตำแหน่งต้นทาง (source) จะเป็นข้อมูลในรีจิสเตอร์ หน่วยความจำ หรือข้อมูลแบบอินมีเดียตก็ได้ คำสั่งนี้หลังการทำคำสั่งจะไม่มีผลต่อบิตแฟลกใด ๆ การแอนด์แบบบิตต่อบิต ข้อมูลที่ได้จะเป็นไปตามตารางความจริงต่อไปนี้



A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

ตัวอย่างเช่น

```
MOV A, #39 H           ; A = 39H
ANL A, #09 H           ; A = 39H AND ด้วย 09
39   0011   1001
09   0000   1001
-----
09   0000   1001
```

การใช้คำสั่ง ANL จะมีการอ้างแอดเดรสแบบต่าง ๆ ได้ 6 โหมด โดยจะมี 4 โหมด ใช้รีจิสเตอร์ A เป็นตำแหน่งปลายทาง ดังนี้

1. Immediate: ANL A, # data ; ANL A, #25H
2. Register: ANL A, # Rn ; ANL A, R3
3. Direct: ANL A, direct ; ANL A, 30H
4. Register-indirect: ANL A, @R0 ; ANL A, @R0

สำหรับการอ้างแอดเดรสอีกสองโหมด ตำแหน่งปลายทางจะเป็นหน่วยความจำภายในหรือรีจิสเตอร์พิเศษตัวใดตัวหนึ่ง ส่วนตำแหน่งต้นทางจะเป็นรีจิสเตอร์ A หรือข้อมูลแบบอิมมิตี

5. ANL direct, #data
6. ANL direct, A

ตัวอย่างที่ 6.1 จงแสดงผลลัพธ์จากการทำคำสั่งต่อไปนี้

```
MOV A, #35H           ; A = 35H
ANL A, #0FH           ; A = A AND 0FH
```

วิธีทำ 35H 00110101
 0FH 00001111
 05H 00000101

คำตอบที่ได้จะทำให้ค่าใน A เป็น 05



การ OR มีรูปแบบคำสั่งดังนี้

ORL destination , source ; dest = dest OR source

คำสั่งนี้จะนำค่าโอเปอเรนด์ต้นทางและปลายทางมารวมกันแบบบิตต่อบิต โดยโอเปอเรนด์ต้นทางอาจจะเป็นค่าในรีจิสเตอร์ หน่วยความจำ หรือข้อมูลแบบอิมมิตีเดียตก็ได้ คำสั่งนี้หวังการทำงานจะไม่มีผลต่อบิตแฟล็กใด ๆ สำหรับตารางความจริงของการ OR แสดงได้ดังนี้

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

คำสั่งนี้จะมีการอัปเดตแคร์รี่ได้ 6 โหมด โดยมี 4 โหมดใช้รีจิสเตอร์ A เป็นตำแหน่งปลายทางดังนี้

1. Immediate: ORL A, #data ; ORL A, #25H
2. Register: ORL A, #R_x ; ORL A, R₃
3. Direct: ORL A, Direct ; ORL A, 30H
4. Register-Indirect: ORL A, @R_n ; ORL A, @R₀

สำหรับอีกสองโหมด จะใช้หน่วยความจำภายในหรือรีจิสเตอร์ฟังก์ชันพิเศษเป็นตำแหน่งปลายทาง สำหรับตำแหน่งต้นทางจะใช้รีจิสเตอร์ A หรือข้อมูลแบบอิมมิตีเดียตดังนี้

5. ORL direct, # data
6. ORL direct, A

ตัวอย่างที่ 6.2 จงหาค่าข้อมูลจากการทำคำสั่งต่อไปนี้

MOV A, #39H ; A = 39H

ORL A, #09H ; A = 39H OR 09H

วิธีทำ

39H	0011	1001
09H	0000	1001
39	0011	1001

คำตอบที่ได้จะทำให้ค่าใน A เท่ากับ 39H



ตัวอย่างที่ 6.3 จงหาค่าข้อมูลจากการทำคำสั่งต่อไปนี้

MOV A, #32H ; A = 32H

MOV R4, #50H ; R4 = 50H

ORL A, R4 ; A = 72H

วิธีทำ 32H 0011 0010

50H 0101 0000

72 0111 0010

คำตอบที่ได้จะทำให้ค่าใน A เท่ากับ 72H

การ XOR มีรูปแบบคำสั่งดังนี้

XRL destination, source ; dest = dest XOR source

คำสั่งนี้จะนำโอเปอร์แลนด์ทั้งสองตัวมาทำการเอกคลูซิฟออร์กันแบบบิตต่อบิต โดยตารางความจริงของการทำเอกคลูซิฟออร์แสดงได้ดังนี้

A	B	A AND B
0	0	0
0	1	1
1	0	1
1	1	0

คำสั่ง XRL จะมีการอ้างแอดเดรสได้คล้ายกับคำสั่งที่ผ่านมามา ดังนี้

1. Immediate: XRL A, #data ; XRL A, #25H
2. Register: XRL A, Rn ; XRL A, R3
3. Direct: XRL A, direct ; XRL A, 30H
4. Register-Indirect: XRL A, @Rn ; XRL A, @R0
5. XRL direct, #data ; XRL 32H, #44H
6. XRL direct, A ; XRL 30H, A

ตัวอย่างที่ 6.4 จงหาค่าจากการกระทำคำสั่งต่อไปนี้

MOV A, #32H ; A = 32H

MOV R4, #50H ; R4 = 50H

XRL A, R4



วิธีทำ	32H	0011	0010
	50H	0101	0000
	62H	0110	0010

หลังจากทำคำสั่งค่าใน A จะเท่ากับ 62H

ตัวอย่างที่ 6.5 จงหาค่าจากการทำคำสั่งต่อไปนี้

```
MOV 32H, #67H
XRL 32H, #44H
MOV A, 32H
```

วิธีทำ	44H	0100	0100
	67H	0110	0111
	23H	0010	0011

ตัวอย่างที่ 6.6 จงหาค่าในรีจิสเตอร์ B หลังจากทำคำสั่งต่อไปนี้

```
MOV B, #44H
MOV A, #67H
XRL 0F0H,A
```

วิธีทำ

เนื่องจากรีจิสเตอร์ B จะอยู่ในแอดเดรส F0H ของหน่วยความจำ RAM ภายใน ดังนั้นหลังจากทำคำสั่งจะทำให้ค่าใน B เท่ากับ 23H

คำสั่ง XRL นี้จะมีโอเปอเรนด์เป็นรีจิสเตอร์บางตัวได้ เช่น XRL A,R1 จะเป็นการกระทำการเอกคลูซิฟออร์ระหว่างรีจิสเตอร์ A กับรีจิสเตอร์ R1 ถ้าหากผลลัพธ์ใน A มีค่าเป็น 0 เราสามารถใช้คำสั่ง JZ ตามหลังให้โปรแกรมกระโดดได้

ตัวอย่างที่ 6.7 ตัวอย่างนี้จะอ่านค่าจากพอร์ต P1 และทดสอบว่าค่าเท่ากับ 45H หรือไม่ ถ้าเท่าจะส่งค่า 99H ออกไปหาพอร์ต P2

วิธีทำ	MOV P2, #00	; เคลียร์ค่า P2
	MOV P1, #0FFH	; ทำให้ P1 เป็นพอร์ตอินพุต
	MOV R3, #45H	; R3 = 45H
	MOV A, P1	; อ่านค่าจากพอร์ต P1
	XRL A,R3	; ถ้าหาก A เท่ากับ 45 ผลลัพธ์ที่ได้จะเท่ากับ 00H
	JNZ EXIT	; กระโดดถ้าค่าใน A ไม่เท่ากับศูนย์
	MOV P2, #99H	

EXIT:



เราสามารถนำคำสั่ง XRL นี้มาประยุกต์ในการกลับค่าบิตข้อมูลไปมา (toggle bit) ของโอเปอเรนด์ได้ ตัวอย่างเช่นถ้าต้องการกลับค่าบิต 2 ของรีจิสเตอร์ A ไปมาเราสามารถเขียนคำสั่งได้ดังนี้

```
XRL  A, #04H ; EX-OR      A กับค่า 0000 0101
```

คำสั่ง CPL A

คำสั่งนี้จะใช้ทำคอมพลีเมนต์ (complement) ค่าในรีจิสเตอร์ A โดยกลับค่าบิตที่เป็น 0 ให้เป็น 1 บิตที่เป็น 1 ให้เป็น 0 หรือที่เรียกอีกอย่างหนึ่งว่าการทำวันคอมพลีเมนต์ (1's complement)

```
MOV  A, #55H
```

```
CPL  A
```

หลังจากทำคำสั่งใน A จะเท่ากับ AAH เนื่องจากค่า 55H (0101 0101) เมื่อถูกทำคอมพลีเมนต์แล้วจะได้เป็น AAH (1010 1010)

สำหรับการทำเลขทวูคอมพลีเมนต์ (2's complement) เราสามารถทำได้โดยการบวก 1 เข้ากับตัวเลขแบบวันคอมพลีเมนต์ ดังเช่นตัวอย่างที่ 6.8

ตัวอย่างที่ 6.8 แสดงการทำทวูคอมพลีเมนต์ของค่า 85H

```
MOV  A, #85H      ; 85H = 1000 0101
CPL  A            ; 1's  = 0111 1010
ADD  A, #1        ; 2's  = _____ + 1
                        0111 1011  = 7BH
```

หลังจากทำคำสั่งค่าใน A จะเท่ากับ 7BH ซึ่งเป็นเลขทวูคอมพลีเมนต์ของ 85H

คำสั่งเปรียบเทียบ

ใน MCS-51 มีคำสั่งที่ใช้ในการเปรียบเทียบข้อมูลโดยมีรูปแบบคำสั่งดังนี้

```
CJNE destination, source, relative address
```

คำสั่ง CJNE นี้ย่อมาจาก Compare and jump if not equal คำสั่งนี้จะมีโอเปอเรนด์สองตัว เมื่อ MCS-51 ทำงานคำสั่งนี้จะนำโอเปอเรนด์ทั้งสองมาเปรียบเทียบกัน ถ้าค่าไม่เท่ากันจะกระโดดไปยังแอดเดรสแบบสัมพัทธ์ (relative address) โดยการเปรียบเทียบจะทำโดยนำค่าโอเปอเรนด์ ดันทางไปลบออกจากโอเปอเรนด์ปลายทาง และหลังทำคำสั่งค่าบิตแฟลก CY จะมีการเปลี่ยนแปลง ตัวอย่างเช่น CJNE, A,#67H, NEXT ถ้าค่าในรีจิสเตอร์ A ไม่เท่ากับ 67H ตัว MCS-51 จะกระโดดไปยังแอดเดรส NEXT

ในคำสั่ง CJNE ค่าโอเปอเรนด์ดันทางจะต้องเป็นรีจิสเตอร์ A หรือ Rn เท่านั้น ส่วนโอเปอเรนด์ดันทางจะเป็นรีจิสเตอร์ ค่าในหน่วยความจำ หรือการอ้างแอดเดรสแบบอิมมีเดียก็ได้ หลังจากทำคำสั่งค่าในบิตแฟลก CY จะเปลี่ยนไปดังตารางต่อไปนี้ ส่วนค่าโอเปอเรนด์จะไม่มีการเปลี่ยนแปลง



Compare	Carry Flag
destination > source	CY = 0
destination < source	CY = 1

พิจารณาการทำคำสั่งต่อไปนี้

```

CJNE R5, #80, NOT_EQUAL    ; ตรวจสอบว่า R5 เท่ากับ 80 หรือไม่
                           ; R5 = 80
NOT_EQUAL: JNC NEXT        ; กระโดดถ้า R5 > 80
                           ;
NEXT:                      ; R5 < 80

```

จากชุดคำสั่งด้านบนจะเปรียบเทียบว่าค่าใน R5 เท่ากับ 80 หรือไม่ ถ้าเท่าจะทำคำสั่งต่อมา ถ้าไม่เท่าจะกระโดดไปยังแอดเดรส NOT_EQUAL และในแอดเดรส NOT_EQUAL จะใช้คำสั่ง JNC ซึ่งถ้าบิตแฟล็ก CY เป็น 0 หมายความว่า R5 มากกว่า 80 ถึงกระโดดไปยังแอดเดรส NEXT ถ้า R5 น้อยกว่า 80 จะทำคำสั่งต่อจาก JNC นี้

ตัวอย่างที่ 6.9 จากชุดคำสั่งที่กำหนดให้จงตอบคำถามต่อไปนี้

- โปรแกรมจะกระโดดไปยัง NEXT หรือไม่
- หลังจากการทำคำสั่ง CJNE ในรีจิสเตอร์ A มีค่าอะไร

```

MOV     A, #55H
CJNE    A, #99H, NEXT
.....
NEXT:   .....

```

วิธีทำ

- จะกระโดดไปยังแอดเดรส NEXT เนื่องจากค่า 55H และ 99H ไม่เท่ากัน
- ค่าใน A เท่ากับ 55H เพราะหลังทำคำสั่งค่าในโอเปอเรนด์ไม่เปลี่ยนแปลง

ตัวอย่างที่ 6.10 จงเขียนคำสั่งตรวจสอบค่าใน A เท่ากับ 99H หรือไม่ ถ้าเท่าให้ R1 = FFH ถ้าไม่เท่าให้ R1 = 0

**วิธีทำ**

```

MOV R1, #0           ; เคลียร์ค่า R1
CJNE A, #99H, NEXT   ; ถ้า A ไม่เท่า 99 จะกระโดดไปยัง NEXT
MOV R1, #0FFH        ; ถ้า A เท่ากับ 99 ใส่ค่า FF ใน A
NEXT: .....
OVER: .....

```

ตัวอย่างที่ 6.11 ถ้าหากพอร์ต P1 ถูกใช้เป็นพอร์ตอินพุต ต่ออยู่กับตัววัดอุณหภูมิ จงเขียนโปรแกรมอ่านค่าอุณหภูมิ และทดสอบว่าเท่ากับ 75 หรือไม่ ถ้าน้อยกว่า 75 ให้เก็บค่าอุณหภูมิใน R1 แต่ถ้ามากกว่าให้เก็บใน R2 ตามเงื่อนไขดังนี้

```

T = 75      ; A = 75
T < 75      ; R1 = T
T > 75      ; R2 = T

```

```

วิธีทำ  MOV P1, #0FFH      ; โปรแกรมให้ P1 เป็นพอร์ตอินพุต
          MOV A, P1          ; อ่านค่าอุณหภูมิจาก P1
          CJNE A, #75, OVER   ; เปรียบเทียบค่า A เท่ากับ 75 หรือไม่
          SJMP EXIT          ; กระโดด ถ้า A เท่ากับ 75
OVER:     JNC NEXT           ; ถ้า CY = 0 หมายความว่า A > 75
          MOV R1, A          ; A < 75 เก็บค่าใน R1
          SJMP EXIT          ; จบโปรแกรม
NEXT:     MOV R2, A          ; A > 75 เก็บค่าใน R2
EXIT:     .....

```

ตัวอย่างที่ 6.12 จงเขียนโปรแกรมให้ MCS-51 คอยตรวจสอบค่าของพอร์ต P1 ถ้าหากค่าของ P1 เท่ากับ 63H ให้ทำคำสั่งต่อไป

```

วิธีทำ  MOV P1, #0FFH      ; โปรแกรมให้ P1 เป็นพอร์ตอินพุต
HERE:     MOV A, P1          ; อ่านค่าจาก P1 มาเก็บใน A
          CJNE A, #63H, HERE ; ทำซ้ำถ้า A ไม่เท่ากับ 63 H

```



ตัวอย่างที่ 6.13 ถ้าหากหน่วยความจำ RAM ภายในตั้งแต่ตำแหน่ง 40H ถึง 44H มีข้อมูลต่าง ๆ เก็บอยู่ จงเขียนโปรแกรมค้นหาว่าข้อมูลในตำแหน่งใดมีค่าเท่ากับ 65 โดยนำค่าตำแหน่งนั้นเก็บในรีจิสเตอร์ R4

วิธีทำ สมมติว่าข้อมูลในหน่วยความจำมีค่าดังนี้

[40H] = 76 [41H] = 79 [42H] = 69 [43H] = 65 [44H] = (62)

สามารถเขียนโปรแกรมได้ดังนี้

```

MOV   R4,#0           ; R4 = 0
MOV   R0, #40H         ; โหลดค่าให้ตัวชี้
MOV   R2, #05          ; โหลดค่าตัวนับ
MOV   A, #65           ; ให้ A = 65 เป็นข้อมูลที่จะค้นหา
BACK: CJNE A,@R0,NEXT  ; ถ้าค่าใน RAM ไม่เท่ากับ 65 ให้กระโดด
                        ; ไป NEXT
MOV   R4,R0            ; ค่าใน RAM เป็น 65 เก็บค่าแอดเดรส
SJMP  EXIT             ; จบโปรแกรม
NEXT: INC R0           ; เพิ่มค่าพอร์เตอร์
      DJNZ R2,BACK     ; ทำซ้ำจนกว่าตัวนับเท่ากับ 0
EXIT: .....

```



คำถาม ?

1. จงหาค่าในรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

- ก. MOV A, #37H
 ANL A, #0CAH
- ข. MOV A, #37H
 ORL A, #0CAH
- ค. MOV A, #37H
 XRL A, #0CAH

2. ถ้าหากต้องการให้บิต 3 บิต 4 ของแอมคิวมูลเตอร์มีค่าเป็น 0 จะทำอย่างไรโดยใช้คำสั่ง ANL

3. ถ้านำคำสั่ง XRL กระทำกับข้อมูลในรีจิสเตอร์ของตัวเอง จะได้ค่าใด

4. จากคำสั่งต่อไปนี้ค่าใน R4 ควรเป็นอย่างไร โปรแกรมจึงไม่กระโดด

CJNE R4, #53,OVER

5. จงหาค่าในรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

```

CLR   A
ORL   A, #99H
CPL   A

```



6.2 คำสั่งหมุนและสลับข้อมูล

ในไมโครคอนโทรลเลอร์ MCS-51 การหมุนบิตต่าง ๆ ในไบต์ของข้อมูลจะทำได้โดยใช้คำสั่งต่อไปนี้ RL, RR, RLC และ RRC โดยสามารถหมุนข้อมูลในแอมคิวมเลเตอร์ไปทางซ้ายหรือทางขวาได้ สำหรับลักษณะของการหมุนจะมีสองประเภทคือ การหมุนบิตในรีจิสเตอร์ A และการหมุนบิตผ่านตัวทด

การหมุนบิตไปทางซ้ายและขวา

การหมุนบิตข้อมูลแบบนี้จะใช้รีจิสเตอร์ A เป็นโอเปอเรนด์ ถ้าเป็นการหมุนไปทางขวาจะใช้คำสั่งดังนี้

RR A ; หมุนค่าใน A ไปทางขวา

ข้อมูลจากบิตสูงจะถูกส่งไปยังบิตต่ำ ข้อมูลจากบิตต่ำสุด (บิต Do) จะถูกดันออกมาและกลับไปยังบิตสูงสุด ตัวอย่างของการหมุนเป็นดังนี้

```
MOV A, #36H      ; A = 0011 0110
RR A             ; A = 0001 1011
RR A             ; A = 1000 1101
RR A             ; A = 1100 0110
RR A             ; A = 0110 0011
```

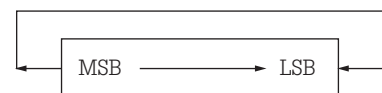


สำหรับการหมุนไปทางซ้ายคำสั่งจะเป็นรูปแบบดังนี้

RL A ; หมุนค่าใน A ไปทางซ้าย

ข้อมูล 8 บิต ในรีจิสเตอร์ A จะถูกหมุนโดยบิตต่ำ จะถูกเลื่อนไปยังบิตสูง และบิตสูงสุดจะถูกส่งกลับมายังบิตต่ำสุด ดังตัวอย่างต่อไปนี้

```
MOV A, # 72H      ; A = 0111 0010
RL A             ; A = 1110 0100
RL A             ; A = 1100 1001
```



คำสั่งหมุนทั้ง RR และ RL หลังจากทำคำสั่งจะไม่มีผลต่อบิตแฟลกใด ๆ

การหมุนผ่านตัวทด

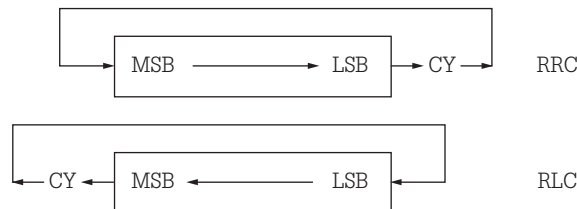
คำสั่งหมุนอีกประเภทหนึ่งของ MCS-51 จะนำข้อมูลบิตตัวทดมาช่วยหมุนด้วยเรียกว่าการหมุนผ่านบิตตัวทด หรือเรียกอีกอย่างหนึ่งว่าการหมุนแบบ 9 บิต ซึ่งมีทั้งหมุนทางซ้ายและหมุนทางขวาแต่ละคำสั่งมีรูปแบบดังนี้



RRC A ; หมุนค่าใน A ไปทางขวาผ่านบิตตัวทด

RLC A ; หมุนค่าใน A ไปทางซ้ายผ่านบิตตัวทด

การหมุนไปทางขวาผ่านบิตตัวทอนั้น ค่าในบิตต่ำสุดจะถูกส่งเข้าไปยังบิตแฟลกตัวทด ส่วนค่าในบิตแฟลกตัวทอดีมนจะถูกส่งไปยังบิตสูงสุดดังรูป



ลักษณะของการหมุนแสดงได้ดังชุดคำสั่งต่อไปนี้

CLR C ; ให้ CY = 0

MOV A, #26H ; A = 0010 0110

RRC A ; A = 0001 0011 CY = 0

RRC A ; A = 0000 1001 CY = 1

RRC A ; A = 1000 0100 CY = 1

สำหรับการหมุนไปทางซ้ายผ่านบิตตัวทด จะใช้คำสั่งรูปแบบดังนี้

RLC A ; หมุนค่าใน A ไปทางซ้ายผ่านบิตตัวทด

เมื่อ MCS-51 ทำคำสั่งนี้จะเลื่อนบิตต่ำไปยังบิตสูง และนำบิตสูงสุดเก็บในแฟลกตัวทด ส่วนค่าในแฟลกตัวทอดีมนจะถูกส่งกลับมายังบิตต่ำสุด ตัวอย่างดังชุดคำสั่งต่อไปนี้

SETB C ; เซตแฟลกตัวทให้เป็น 1

MOV A, #15H ; A = 0001 0101

RLC A ; A = 0010 1011 CY = 0

RLC A ; A = 0101 0110 CY = 0

RLC A ; A = 1010 1100 CY = 0

RLC A ; A = 0101 1000 CY = 1

คำสั่ง SWAP A

คำสั่งนี้จะกระทำกับแอมคิวมูลเตอร์เท่านั้น โดยจะสลับค่าระหว่าง 4 บิตบน กับ 4 บิตล่างการทำงาน
ของคำสั่งแสดงได้ดังต่อไปนี้



ก่อนทำคำสั่ง	D7 - D4	D3 - D0	ก่อนทำคำสั่ง	0111	0010
หลังทำคำสั่ง	D3 - D0	D7 - D4	หลังทำคำสั่ง	0010	0111

ตัวอย่างที่ 6.14 ก. จงยกตัวอย่างหาค่าของข้อมูลหลังจากทำคำสั่ง SWAP

ข. จงสร้างคำสั่ง SWAP จากคำสั่งหมุนข้อมูล

วิธีทำ ก. MOV A, #72H ; A = 72H
 SWAP A ; A = 27H
 ข. MOV A, #72H ; A = 0111 0010
 RL A ; A = 1110 0100
 RL A ; A = 1100 1001
 RL A ; A = 1001 0011
 RL A ; A = 0010 0111

ตัวอย่างที่ 6.15 จงเขียนโปรแกรมหาจำนวนบิตที่เป็น “1” ในไบต์ข้อมูล

วิธีทำ โปรแกรมลักษณะนี้สามารถทำได้โดยเลื่อนข้อมูลในไบต์ข้อมูลมายังบิตแฟลก แล้วทดสอบว่าบิตแฟลกเป็น “1” หรือไม่ ถ้าเป็นให้เพิ่มค่าตัวนับขึ้นหนึ่งค่า ดังโปรแกรมต่อไปนี้

```

MOV R1, #0 ; R1 เป็นตัวนับจำนวนบิตที่เป็น “1”
MOV R7, #8 ; ตัวนับจำนวนบิต
MOV A, #97H ; ข้อมูลตัวอย่างที่จะหาบิตที่เป็น “1”
AGAIN: RLC A ; หมุนค่าใน A ผ่านตัวทด
JNC NEXT ; ตรวจสอบค่าในแฟลกตัวทด
INC R1 ; ถ้าแฟลกตัวทดเป็น “1” ให้บวก 1 เข้ากับตัวนับ
NEXT: DJNZ R7, AGAIN ; ทำจนครบ 8 ครั้ง

```

คำสั่งการหมุนข้อมูลนี้ สามารถนำมาประยุกต์ใช้ในการเปลี่ยนข้อมูลแบบอนุกรมให้เป็นข้อมูลแบบขนานได้ ตัวอย่างเช่น ถ้าหากมีข้อมูลส่งเข้ามาทางบิต P1.3 เราสามารถนำค่าบิตนั้นมาเก็บในแฟลกตัวทดก่อน แล้วหมุนข้อมูลเข้ามาในรีจิสเตอร์ A ถ้าทำแบบนี้ 8 ครั้ง จะทำให้ข้อมูลที่เข้ามาแบบอนุกรมทางพอร์ต P1.3 ถูกเก็บเป็นข้อมูล 8 บิตในรีจิสเตอร์ A ได้ ดังการเขียนชุดคำสั่งต่อไปนี้



```

MOV A, #00H
MOV C, P1.3      ; อ่านข้อมูลบิตแรกที่เข้าทาง P1.3 ให้กับแฟล็กตัวทด
RLC A           ; เลื่อนข้อมูลบิตแรกเข้ามาใน A
MOV C, P1.3      ; อ่านข้อมูลบิตต่อไป
RLC A           ; เลื่อนข้อมูลเข้ามาใน A

```

**คำถาม ?**

1. หลังจากไมโครคอนโทรลเลอร์ ทำคำสั่งต่อไปนี้ค่าใน A จะมีค่าเท่าใด

```

MOV A, #25H
RR A
RR A
RR A
RR A

```

2. หลังจากไมโครคอนโทรลเลอร์ ทำคำสั่งต่อไปนี้ค่าใน A จะมีค่าเท่าใด

```

MOV A, #A2H
RL A
RL A
RL A
RL A

```

3. หลังจากไมโครคอนโทรลเลอร์ ทำคำสั่งต่อไปนี้ ค่าใน A จะมีค่าเท่าใด

```

CLR A
SETB C
RRC A
SETB C
RRC A

```

4. ถ้าเขียนคำสั่งเป็น RLC R1 ระบบจะทำงานอย่างไร

5. จงหาค่าใน A หลังจากทำคำสั่งต่อไปนี้

```

MOV A, #85H
SWAP A
ANL A, #0F0H

```



6.3 ถ้าเขียนโปรแกรมประยุกต์ BCD และ ASCII

ความสัมพันธ์ระหว่างตัวเลข รหัส ASCII และเลข BCD แสดงได้ดังนี้

Key	ASCII	Binary	BCD (Unpacked)
0	30	00110000	00000000
1	31	00110001	00000001
2	32	00110010	00000010
3	33	00110011	00000011
4	34	00110100	00000100
5	35	00110101	00000101
6	36	00110110	00000110
7	37	00110111	00000111
8	38	00111000	00001000
9	39	00111001	00001001

รหัส ASCII

โดยทั่วไปแล้วการติดต่อกับจอภาพ เครื่องพิมพ์ หรือแป้นพิมพ์ มักจะใช้รหัส ASCII ในการติดต่อ เช่น ถ้าหากเรากดคีย์บนแป้นพิมพ์ เราจะได้ค่าออกมาเป็นรหัส ASCII เช่น ถ้ากดคีย์ “0” ค่าที่ได้ออกมาคือ “011 0000” (30 H) ถ้ากดคีย์ “1” ค่าที่ได้ออกมาจะเป็น “011 0001” (31H) ดังตาราง ถ้าหากระบบคอมพิวเตอร์จะนำค่าจากคีย์ไปประมวลผลจะต้องแปลงค่ารหัส ASCII ที่ได้ให้เป็นเลข BCD จึงจะประมวลผลได้ง่าย

ถ้าหากระบบคอมพิวเตอร์ประมวลผลได้ข้อมูลออกมาเป็นเลข BCD และต้องการแสดงผลทางอุปกรณ์ต่าง ๆ จะต้องเปลี่ยนเลข BCD นั้นให้เป็นรหัส ASCII เช่นกัน

การแปลง Packed BCD เป็นรหัส ASCII

ในไมโครคอนโทรลเลอร์เบอร์ DS5000 T จะมีวงจรเวลาจริงอยู่ภายในที่เรียกกันว่า RTC (realtime clock) ในวงจร RTC นี้สามารถบอกเวลาบางวันได้โดยให้ค่าชั่วโมง นาที วินาที ออกมา และบอกวันออกมาเป็น ปี เดือน และวันของสัปดาห์ ค่าที่ได้ออกมาจะอยู่ในรูปของรหัส BCD และถ้าหากเราต้องการนำไปแสดงผลทาง LCD หรือ LED 7 ส่วน เราจะต้องแปลงค่ารหัส BCD นี้เป็นรหัส ASCII เสียก่อน



ตัวอย่างที่ 6.14 ถ้าหากมีข้อมูลรหัส **packed BCD** เก็บอยู่ในรีจิสเตอร์ A จงเขียนโปรแกรมเปลี่ยนเลข BCD ให้เป็นรหัส ASCII เก็บในรีจิสเตอร์ R2 และ R6

วิธีทำ

MOV A, #29H	; A = 29H, packed BCD
MOV R2,A	; ก๊อปปี้ตัวเลข BCD ลงใน R2
ANL A, #0FH	; เคลียร์ค่า 4 บิตบนจะได้ A = 09
ORL A, #30H	; เปลี่ยนเป็นรหัส ASCII A = 39H ('9')
MOV R6,A	; เก็บข้อมูล (R6 = 39H)
MOV A, R2	; A = 29H ข้อมูลเดิม
ANL A, #0F0H	; เคลียร์ค่าบิตล่าง (A = 20)
RR A	; หมุนไปทางขวา
RR A	; หมุนไปทางขวา
RR A	; หมุนไปทางขวา
RR A	; หมุนไปทางขวา (A = 02)
ORL A, #30H	; A = 32H รหัส ASCII ของ 2
MOV R2,A	; เก็บค่ารหัส ASCII ลงใน R2

จากตัวอย่างเราสามารถแทนคำสั่ง RR ลีคำสั่งด้วยคำสั่ง SAWP ได้

คำถาม

1. จงเปลี่ยนตัวเลขฐานสิบต่อไปนี้ให้เป็น packed BCD และ unpacked BCD
ก. 15 ข. 99
2. จงนำค่า 76 เขียนเป็นเลขขนานเลขฐานสิบหก และเลข BCD
3. ตัวเลข BCD ที่มีค่าเป็น 67H ถ้าเปลี่ยนเป็นรหัส ASCII จะได้เป็น H และ H
4. การเขียนคำสั่งต่อไปนี้สามารถเปลี่ยนเลข unpacked BCD ในรีจิสเตอร์ A ให้เป็นรหัส ASCII ได้หรือไม่

รูป ที่ผ่านมามีได้กล่าวถึงคำสั่งทางลอจิกของ MCS-51 ได้แก่การ AND, OR, XOR และ Complement ส่วนคำสั่งหมุนและสลับค่า สามารถนำมาประยุกต์ใช้ในการเปลี่ยนรหัสข้อมูลระหว่าง BCD และ ASCII ได้



คำถามท้ายบท

1. ถ้าหากมีข้อมูลในรีจิสเตอร์เป็นดังนี้ $A = F0$, $B = 56$ และ $R1 = 90$ จงหาผลลัพธ์จากการกระทำคำสั่งต่อไปนี้

- | | |
|-----------------|-----------------|
| ก. ANL A, #45H | ข. ORL A,B |
| ค. XRL A, #76H | ง. ANL A,B |
| จ. XRL A,R1 | ฉ. ORL A,R1 |
| ช. ANL A, #0FFH | ซ. ORL A, #99H |
| ณ. XRL A, #0EEH | จ. XRL A, #0AAH |

2. จงหาค่าในรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

- | | |
|-----------------|----------------|
| ก. MOV A, #65H | ข. MOV A, #70H |
| ANL A, #76H | ORL A, #6BH |
| ค. MOV A, #95H | ง. MOV A, #5DH |
| XRL A, #0AAH | MOV R3, #78H |
| จ. MOV A, #005H | ฉ. MOV A, #6AH |
| MOV R6, #12H | MOV R4, #6EH |
| ORL A, R6 | XRL A, R0 |

3. จริงหรือไม่ คำสั่ง CJNE จะทดสอบค่าในแอมคิวมเลเตอร์เท่านั้น

4. จงหาค่าบางบิตแฟลก CY หลังจากทำคำสั่ง CJNE ดังต่อไปนี้

- | | |
|---------------------|--------------------|
| ก. MOV A, #25H | ข. MOV A, #0FFH |
| CJNE A, #44H,OVER | CJNE A, #6FH,NEXT |
| ค. MOV A, #34 | ง. MOV R1, #0 |
| CJNE A, #34,NEXT | CJNE R1, #0,NEXT |
| จ. MOV R5, #54H | ฉ. MOV R3, #0AAH |
| CJNE R5, #0FFH,NEXT | ANL R3, #55H |
| | CJNE R3, #00H,NEXT |

5. จงหาค่าบางรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

- | | |
|----------------|----------------|
| ก. MOV A, #56H | ข. MOV A, #39H |
| SWAP A | CLR C |
| RR A | RL A |
| RR A | RL A |



ค. CLR	C	ง. SETB	C
MOV	A, #4DH	MOV	A, #7AH
SWAP	A		
RRC	A	SWAP	A
RRC	A	RLC	A
RRC	A	RLC	A

6. จงเขียนโปรแกรมหาจำนวนบิตที่เป็นศูนย์ในข้อมูล 8 บิต
7. ถ้าหากมีข้อมูล packed BCD อยู่ใน ROM ตำแหน่ง 300H จะเป็นโปรแกรมอ่านค่าออกมา และเปลี่ยนเป็นรหัส ASCII เก็บใน RAM ตั้งแต่ตำแหน่ง 40H เป็นต้นไป ถ้าหากข้อมูลใน ROM เป็นดังนี้


```

      ORG    300H
MYDATA:    DB    76H, 87H, 98H, 43H
      
```
8. จงเขียนโปรแกรมเปลี่ยนรหัส ASCII ให้เป็นข้อมูล packed BCD ถ้าหากข้อมูลรหัส ASCII อยู่ใน ROM ตั้งแต่ตำแหน่ง 300H และให้นำค่าที่เปลี่ยนได้ไปเก็บใน RAM ตั้งแต่ตำแหน่ง 60H


```

      ORG    300H
MYDATA:    DB    "87675649"
      
```
9. จงเขียนโปรแกรมรับข้อมูล 8 บิต ทางพอร์ต P1 และเปลี่ยนเป็นรหัส ASCII เก็บใน RAM ตำแหน่ง 40H, 41H และ 92H และถ้าหากค่าใน P1 เป็น 1000 1101 ค่าใน RAM จะมีค่าเป็นเท่าใด
10. จงแสดงการสร้างคำสั่ง SWAP จากคำสั่งต่อไปนี้
 - . หมุนไปทางซ้าย
 - . หมุนไปทางขวา

บทที่

7

คำสั่งกระทำต่อบิต และการโปรแกรม

ไมโครคอนโทรลเลอร์ MCS-51สามารถดำเนินการครั้งละบิตได้ เช่น เซตบิต เคลียร์บิต ย้ายบิตข้อมูล และกลับค่าบิตข้อมูลโดยการดำเนินการดังกล่าวสามารถกระทำกับหน่วยความจำพอร์ต หรือรีจิสเตอร์ก็ได้ ในบทนี้จะกล่าวถึงตัวอย่างการเขียนโปรแกรมและการประยุกต์การกระทำระดับบิต



7.1 การโปรแกรมแบบหนึ่งบิต

เมื่อไมโครคอนโทรลเลอร์ต้องการโอนย้ายข้อมูลระหว่างตัวมันกับหน่วยความจำ การทำงานประเภทนี้ไมโครคอนโทรลเลอร์ MCS-51 สามารถทำงานได้เป็นอย่างดีโดยจะโอนย้ายข้อมูลครั้งละเวิร์ด (word) หรือ ไบต์ (byte) แต่ในงานบางประเภทข้อมูลที่ได้รับเข้ามาหรือส่งออกไปยังพอร์ตจะไม่เต็มไบต์ อาจเป็นเพียงหนึ่งบิตหรือสองบิต เช่นการเขียนโปรแกรมไปปิดหรือเปิดอุปกรณ์ภายนอก หรือ การเขียนโปรแกรมไปแก้ไขข้อมูลบางบิตในหน่วยความจำ คำสั่งในการทำงานระดับบิตอยู่หลายคำสั่งดังต่อไปนี้

คำสั่งแบบหนึ่งบิต

คำสั่งที่ใช้ดำเนินการแบบหนึ่งบิตแสดงได้ดังตารางที่ 7.1 สำหรับสัญลักษณ์ที่เขียนว่า bit หมายถึง ตำแหน่งหน่วยความจำที่สามารถเข้าถึงข้อมูลระดับบิตได้



ตารางที่ 7.1 คำสั่งระดับบิต

คำสั่ง	การทำงาน
SETB bit	เซตบิต (bit = 1)
CLR bit	เคลียร์บิต (bit = 0)
CPL bit	กลับค่าบิต (bit = Not bit)
JB bit,target	กระโดดไปยัง target ถ้า bit = 1 (jump if bit)
JNB bit,target	กระโดดไปยัง target ถ้า bit = 0 (jump if no bit)
JBC bit,target	กระโดดไปยัง target ถ้า bit = 1 และเคลียร์บิต

พอร์ต I/O และตำแหน่งข้อมูลบิต

ไมโครคอนโทรลเลอร์ MCS-51 มีพอร์ตอินพุตเอาต์พุตแบบ 8 บิต จำนวน 4 พอร์ต คือ P0, P1, P2 และ P3 โดยพอร์ตทั้ง 4 พอร์ต เราสามารถเข้าถึงข้อมูลทีละบิตได้ ตัวอย่างเช่นถ้าหากต้องการเซตบิตของพอร์ตอาจเขียนรหัสคำสั่งได้ในรูปแบบดังนี้ “SETB X.Y” โดยที่ X เป็นพอร์ตหมายเลข 0, 1, 2 หรือ 3 ส่วน Y จะเป็นตำแหน่งบิตซึ่งมีค่าเท่ากับ 0 ถึง 7 ตัวอย่างเช่น ถ้าหากต้องการเซตบิตที่ 5 ของพอร์ต P1 จะเขียนคำสั่งได้ดังนี้ “SETB P1.5”

ตัวอย่างที่ 7.1 จงเขียนโปรแกรมให้ทำงานดังนี้

1. ร่างคำสั่งรูปสี่เหลี่ยมที่มีค่า duty cycle เท่ากับ 50% ออกมาทางบิต 0 ของพอร์ต 1
2. ร่างคำสั่งรูปสี่เหลี่ยมที่มีค่า duty cycle เท่ากับ 66% ออกมาทางบิต 3 ของพอร์ต 2

วิธีทำ 1. ค่า duty cycle เท่ากับ 50% หมายความว่าระยะเวลาที่เป็นลอจิก “1” และ “0” จะเท่ากัน ดังนั้นเราจะเขียนโปรแกรมโดยกลับค่า (toggle) ทางบิต P1.0 ได้ ดังนี้

```

HERE:   SETB    P1.0      ; เซตบิต 0 ของพอร์ต 1
        LCALL   DELAY     ; เรียกโปรแกรมน้อยหน่วยเวลา
        CLR     P1.0      ; เคลียร์พอร์ต P1.0 = 0
        LCALL   DELAY
        SJMP    HERE      ; ทำซ้ำ

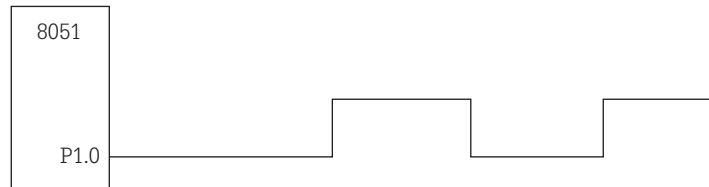
```

หรือเขียนโปรแกรมอีกแบบหนึ่งได้ดังนี้

```

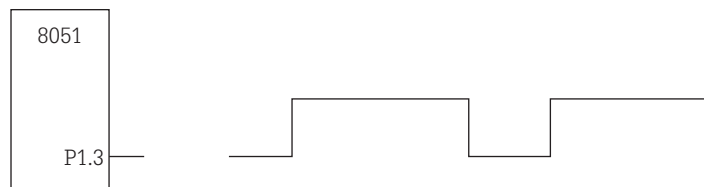
HERE:   CPL     P1.0      ; กลับค่าบิต 0 ของพอร์ต 1
        LCALL   DELAY     ; เรียกโปรแกรมน้อยหน่วยเวลา
        SJMP    HEEE      ; ทำซ้ำ

```

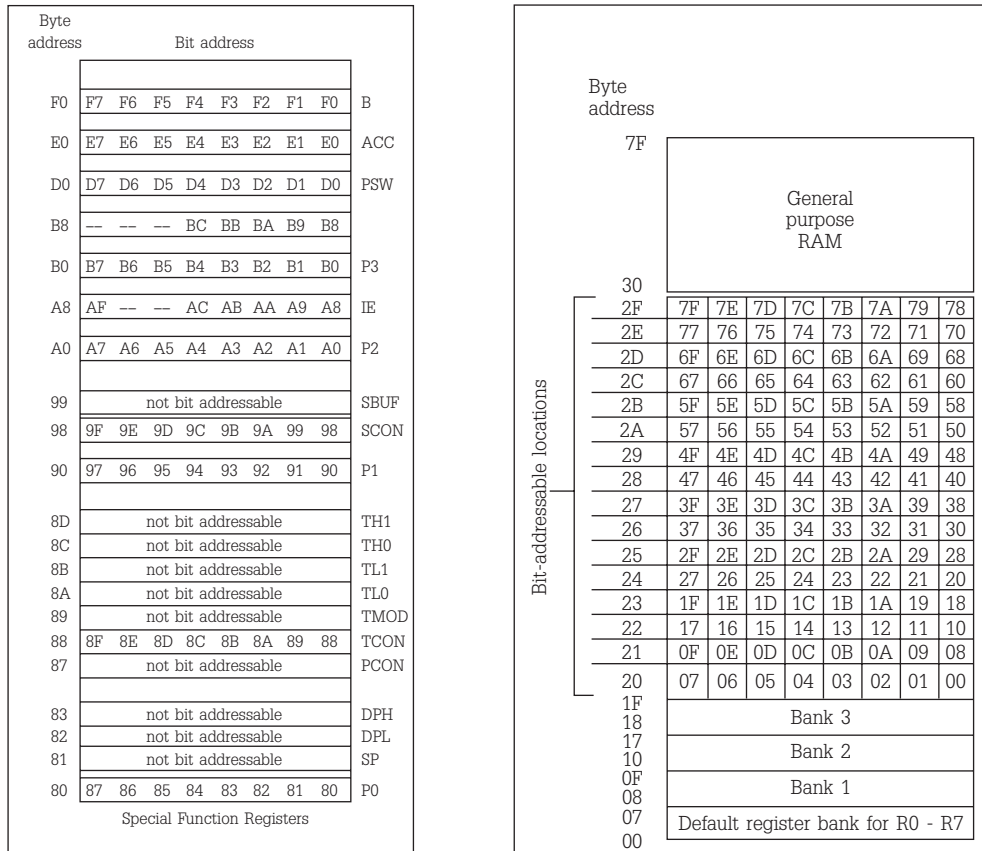


2. ค่า duty cycle 66% ทำได้โดยให้เป็นลอจิก “1” นานกว่าลอจิก “0” เป็นเวลาเท่าตัวซึ่งเขียนโปรแกรมได้ดังนี้

```
BACK:  SETB      P1.3          ; เซตพอร์ต 1 บิต 3
        LCALL    DELAY        ; เรียกโปรแกรมย่อยหน่วยเวลา
        LCALL    DELAY        ; เรียกโปรแกรมย่อยหน่วยเวลาอีกครั้ง
        CLR      P1.3         ; เคลียร์พอร์ต 1 บิต 3
        LCALL    DELAY        ; เรียกโปรแกรมย่อยหน่วยเวลา
        SJMP     BACK         ; ทำซ้ำ
```



แม้ว่าการเซตบิตจะเขียนคำสั่งเป็น “SETB P1.0” แต่เมื่อโปรแกรมทำการแปลเป็นรหัสภาษาเครื่องจะมีค่าเท่ากับคำสั่ง “SETB 90 H “ เนื่องจาก P1.0 มีค่าตรงกับ RAM ในตำแหน่ง 90 H จากรูปที่ 7.1 จะแสดงค่าแอดเดรสของหน่วยความจำภายในที่เป็นส่วนของรีจิสเตอร์ฟังก์ชันพิเศษ (SFR) โดยจะแสดงเป็นตำแหน่งแบบไบต์ และตำแหน่งแบบบิตของรีจิสเตอร์ต่าง ๆ ที่สามารถอ้างแอดเดรสระดับบิตได้ซึ่งจะเห็นว่า P0 แต่ละบิตจะเริ่มที่ตำแหน่งบิต 80H ถึง 87H ส่วน P1 จะเริ่มที่ตำแหน่งบิต 90H ถึง 97H



รูปที่ 7.1 แสดงแอดเดรสของหน่วยความจำแรม SFR ของ MCS-51

ตัวอย่างที่ 7.2 จากการเขียนคำสั่งต่อไปนี้จึงตอบว่ามีผลต่อบิตใดในรีจิสเตอร์ตัวใดโดยดูรูปที่ 7.1

ประกอบ

- | | | |
|--------------|-------------|--------------|
| ก. SETB 86H | ข. CLR 87H | ค. SETB 92H |
| ง. SETB 0A7H | จ. CLR 0F2H | ฉ. SETB 0E7H |

วิธีทำ

- | | |
|--------------|---------------------------------------|
| ก. SETB 86H | จะเป็นการเซตบิต P0.6 |
| ข. CLR 87H | จะเป็นการเคลียร์บิต P0.7 |
| ค. SETB 92H | จะเป็นการเซตบิต P1.2 |
| ง. SETB 0A7H | จะเป็นการเซตบิต P2.7 |
| จ. CLR 0F2H | จะเป็นการเคลียร์บิต D2 ในรีจิสเตอร์ B |
| ฉ. SETB 0E7H | จะเป็นการเซตบิตที่ 7 ในรีจิสเตอร์ A |



การทดสอบบิต

ถ้าหากมีข้อมูลหนึ่งบิตเราสามารถทดสอบข้อมูลในบิตนั้นได้ โดยการใช้คำสั่ง JNB (jump if no bit) และ JB (jump if bit = 1) ทั้งสองคำสั่งนี้จะดูค่าบิตที่กำหนดและจะกระโดดไปยังแอดเดรสที่กำหนดในคำสั่ง ขึ้นกับว่าข้อมูลในบิตนั้นเป็น “0” หรือ “1” คำสั่งนี้สามารถใช้ทดสอบค่าบิตจากพอร์ต P1, P2 และ P3 ได้

ตัวอย่างที่ 7.3 ถ้าหากมีข้อมูลเข้ามาทางบิต P2.3 จงเขียนโปรแกรมตรวจสอบค่าทางบิตนี้ถ้าเป็น “1” ให้ส่งสัญญาณที่มีการเปลี่ยนแปลงจาก “1” เป็น “0” ออกมาทางพอร์ต P1.5 เพื่อเปิด buzzer ให้ส่งเสียงออกมา

วิธีทำ สมมุติว่า บิต P1.5 ต่อกับ buzzer สำหรับส่งเสียง

```
HERE:    JNB  P2.3, HERE      ; monitoring จนค่าเป็น nigh
          SETB P1.5           ; เซตบิต P1.5 = 1
          CLR  P1.5           ; ทำให้เปลี่ยนจากลอจิก “1” ไปเป็นลอจิก “0”
```

ตัวอย่างที่ 7.4 จงเขียนโปรแกรมทดสอบว่าค่าข้อมูลในรีจิสเตอร์ A เป็นเลขคู่หรือไม่ ถ้าเป็นให้นำค่านั้นมาหารด้วยสอง ถ้าไม่เป็นให้ทำเป็นเลขคู่และนำค่ามาหารด้วยสอง

วิธีทำ การทดสอบว่าเป็นเลขคู่หรือไม่ทำได้โดยการดูว่าบิตต่ำสุด ถ้าเป็น 0 หมายความว่าเลขคู่ ถ้าเป็น 1 หมายความว่าเลขคี่ ซึ่งสามารถเขียนโปรแกรมได้ดังนี้

```
MOV  B, #2                ; B = 2
      JNB  AC.0, YES       ; ถ้าบิต D0 ใน A เป็น 0 จะเป็นเลขคู่ให้กระโดดไป
                           YES
      INC  A                ; ทำให้เป็นเลขคู่โดยการบวกด้วย 1
YES:  DIV  AB               ; A/B
```

ตัวอย่างที่ 7.5 จงเขียนโปรแกรมทดสอบบิต 0 และบิต 5 ของรีจิสเตอร์ B ว่าเป็น “1” หรือไม่ ถ้ายังไม่เป็นให้ทำเป็นลอจิก “1” และเก็บค่าลงรีจิสเตอร์ R0

วิธีทำ

```
JNB  0FH, NEXT_1          ; กระโดดถ้า B.0 เป็น “0”
SETB 0FH                  ; เซตบิต B.0 ให้เป็น “1”
NEXT_1: JNB  0FH, NEXT_2    ; กระโดดถ้า B.5 เป็น “0”
SETB 0FH                  ; เซตบิต B.5 ให้เป็น “1”
NEXT_2: MOV  R0, B          ; นำค่าในรีจิสเตอร์ B เก็บใน R0
```



ตัวอย่างที่ 7.6 จงเขียนโปรแกรมเก็บค่ารีจิสเตอร์แอกคิวมูเลเตอร์ลงในรีจิสเตอร์ R7 แบนก์ที่ 2

วิธีทำ

CLR PSW.3

SETB PSW.4

MOV R7,A

ในแต่ละบิตของรีจิสเตอร์ PSW (Program Status Word) สามารถอ้างแอดเดรสในระดับบิตได้เช่นกัน ถ้าหาก MCS-51 ถูกรีเซตจะทำให้บิตสำหรับเลือกแบงก์รีจิสเตอร์ RS1 และ RS0 มีค่าเป็น “0” ทั้งสองบิต ซึ่งจะทำให้ MCS-51 ติดต่อกับรีจิสเตอร์ R0 ถึง R7 ในแบงก์ที่ 0 สำหรับรีจิสเตอร์ PSW และการเลือกแบงก์ของรีจิสเตอร์สามารถแสดงได้ดังรูปที่ 7.2

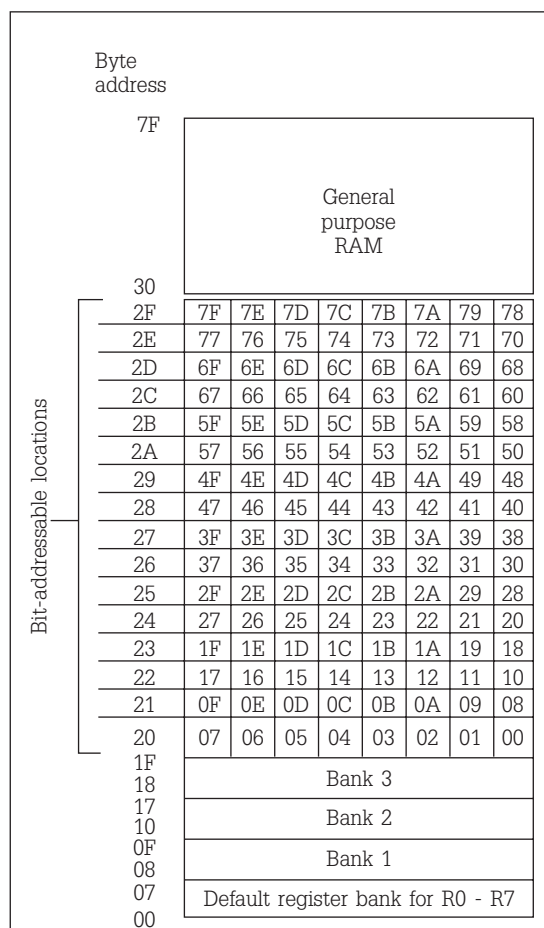
CY	AC	–	RS1	RS0	OV	–	P
----	----	---	-----	-----	----	---	---

RS1	RS0	Register Bank	Address
0	0	0	00H - 07H
0	1	1	08H - 0FH
1	0	2	10H - 17H
1	1	3	18H - 1FH

รูปที่ 7.2 บิตของรีจิสเตอร์ PSW

Bit-addressable RAM

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 ตัวที่เป็นพื้นฐานสำหรับศึกษากันทั่วไปคือเบอร์ 8031 หรือ 8051 ที่มีหน่วยความจำ RAM ภายใน 128 ไบটนั้นจะมีอยู่ 16 ไบต์ที่สามารถเข้าถึงข้อมูลในระดับบิตได้ (bit-addressable) โดย 16 ไบต์ดังกล่าวอยู่ที่ตำแหน่งแอดเดรสไบต์ 20H ถึง 2FH โดยจะอ้างตำแหน่งแบบบิตได้ $16 \times 8 = 128$ บิต ซึ่งจะเริ่มต้นที่แอดเดรสบิต 00H ถึง 7FH ดังรูปที่ 7.3 สำหรับบิต 0 ถึงบิต 7 จะเป็นการอ้างตำแหน่งบิตของไบต์แรกของหน่วยความจำส่วนนี้ (ไบต์แอดเดรส 20H) สำหรับบิต 8 ถึง 0FH จะเป็นตำแหน่งของไบต์ถัดไป จากรูปที่ 7.1 และรูปที่ 7.3 จะเห็นว่าตำแหน่งบิต 00-7F จะอยู่ใน RAM ไบต์ 20 - 2FH สำหรับตำแหน่งบิต 80 - 7FH จะอยู่ในรีจิสเตอร์ฟังก์ชันพิเศษ



รูปที่ 7.3 หน่วยความจำภายใน 128 ไบต์

ตัวอย่างที่ 7.7 จากชุดคำสั่งต่อไปนี้ จงแสดงว่าคำสั่งมีผลต่อบิตใด ในหน่วยความจำตำแหน่งใด

- ก. SETB 42H ; เซตบิตที่ 42H
- ข. CLR 67H ; เคลียร์บิตที่ 67H
- ค. CLR 0FH ; เคลียร์บิตที่ 0FH
- ง. SETB 28H ; เซตบิตที่ 28H
- จ. CLR 12 ; เคลียร์บิตที่ 12 (เลขฐานสิบ)
- ฉ. SETB 05 ; เซตบิตที่ 5

วิธีทำ

- ก. บิต D2 ของหน่วยความจำ RAM ตำแหน่ง 28H
- ข. บิต D7 ของหน่วยความจำ RAM ตำแหน่ง 2CH
- ค. บิต D7 ของหน่วยความจำ RAM ตำแหน่ง 21H



- ง. บิต D0 ของหน่วยความจำ RAM ตำแหน่ง 25H
- จ. บิต D4 ของหน่วยความจำ RAM ตำแหน่ง 21H
- ฉ. บิต D5 ของหน่วยความจำ RAM ตำแหน่ง 20H



คำถาม ?

1. จริงหรือไม่พอร์ตภายในทุกพอร์ตของ MCS-51สามารถอ้างตำแหน่งระดับบิตได้
2. จริงหรือไม่รีจิสเตอร์ทุกตัวของ MCS-51สามารถอ้างตำแหน่งระดับบิตได้
3. จริงหรือไม่หน่วยความจำ RAM ภายในทุกตำแหน่งของ MCS-51 สามารถอ้างตำแหน่งระดับบิตได้
4. รีจิสเตอร์ต่อไปนี้ ตัวใดเข้าถึงข้อมูลระดับบิตได้ และมีตำแหน่งบิตในช่วงแอดเดรสใด
 - ก. A ข. B ค. R4 ง. PSW จ. R7
5. หน่วยความจำ RAM 128 ไบต์ของ MCS-51 มีหน่วยความจำที่สามารถอ้างตำแหน่งระดับบิตได้กี่ไบต์
6. จะเขียนโปรแกรมตรวจสอบบิต D0 ของรีจิสเตอร์ R3 ได้อย่างไรว่าเป็นลอจิก “0” หรือ “1”
7. จงบอกเอาต์พุตจากการทำคำสั่งต่อไปนี้
 - ก. SETB 20 ข. CLR 32 ค. SETB 12H ง. SETB 95H จ. SETB 0E6H
8. บิตแอดเดรสในช่วงตำแหน่ง 00-7FH มีกี่ตำแหน่งอยู่ในหน่วยความจำช่วงใด และ บิตแอดเดรสในช่วงตำแหน่ง 80- F7H อยู่ในช่วงหน่วยความจำใด
9. จริงหรือไม่ พอร์ต P0, P1, P2 และ P3 อยู่ในช่วงรีจิสเตอร์ฟังก์ชันพิเศษ
10. จริงหรือไม่ รีจิสเตอร์ TCON สามารถอ้างแอดเดรสระดับบิตได้



7.2 การกระทำกับบิตตัวทวด

การเขียนโปรแกรมดำเนินการทางคณิตศาสตร์และลอจิกกับบิตนั้นจะมีบิตพิเศษบิตหนึ่งที่คำสั่งมากมาย มีผลกระทำต่อบิตนี้ และคำสั่งหลายคำสั่งจะดำเนินการทดสอบบิตนี้ โดยอาจเรียกบิตนี้ว่าเป็นบิตแอกคิวมูเลเตอร์ได้ บิตนี้ก็คือบิตแฟลกตัวทวด (carry flag : CY)

ในตารางที่ 7.2 จะแสดงคำสั่งต่าง ๆ ที่กระทำกับบิตแฟลกตัวทวด โดยมีทั้งคำสั่งย้ายข้อมูลและคำสั่งการกระทำทางลอจิก



ตารางที่ 7.2

คำสั่ง	การทำงาน
SETB C	ให้บิตแฟลกตัวทดเป็น 1
CLR C	เคลียร์บิตแฟลกตัวทด
CPL C	กลับค่าบิตแฟลกตัวทด
MOV b,C	ย้ายค่าจากบิตแฟลกตัวทดให้ กับตำแหน่งบิต
MOV C,b	ย้ายค่าจากตำแหน่งบิตให้กับแฟลกตัวทด
JNC target	กระโดดไปยังแอดเดรส target ถ้าแฟลกตัวทดเป็น 0
JC target	กระโดดไปยังแอดเดรส target ถ้าแฟลกตัวทดเป็น 1
ANL C,bit	AND ค่าบิตและเก็บไว้ในบิตตัวทด
ANL C,bit	กลับค่าบิตและนำมา AND กันและเก็บในบิตตัวทด
ORL C,bit	OR ค่าบิตและเก็บในบิตตัวทด
ORL C,bit	กลับค่าบิตแล้วนำมา OR และเก็บในบิตตัวทด

ตัวอย่างที่ 7.8 จงเขียนโปรแกรมอ่านค่าจากบิต P1.2 และ P1.3 จากนั้นนำไปเก็บใน RAM ตำแหน่งบิตที่ 6 และ 7

วิธีทำ

```

MOV  C,P1.2           ; อ่านค่าจาก P1.2 เก็บในบิตตัวทด
MOV  06,C             ; อ่านค่าจากบิตตัวทดเก็บในตำแหน่ง 6
MOV  C,P1.3           ; อ่านค่าจาก P1.3 เก็บในบิตตัวทด
MOV  07,C             ; อ่านค่าจากบิตตัวทดเก็บในตำแหน่ง 7

```

ตัวอย่างที่ 7.9 ระบบฝากข้อความทางโทรศัพท์ระบบหนึ่งใช้ MCS-51 เป็นตัวควบคุม โดยถ้าข้อมูลในตำแหน่งบิตที่ 12H มีค่าเป็น “1” แสดงว่ามีข้อความฝากไว้และให้แสดงคำว่า “New Messages” ทางจอ LCD แต่ถ้าในตำแหน่งนั้นเป็น “0” ให้แสดงคำว่า “No New Messegas” จงเขียนโปรแกรมให้ทำงานดังกล่าว

วิธีทำ

```

MOV  C,12H           ; อ่านค่าจากบิตตำแหน่ง 12H มาเก็บในแฟลก
JNC  NO              ; ตรวจสอบว่าเป็น “1” หรือไม่
MOV  DPTR, #400H     ; ถ้าเป็น “1” ให้โหลดค่าแอดเดรส Messages
LCALL DISPLAY        ; แสดงผลทาง LCD
SJMP EXIT            ; จบโปรแกรม
NO:  MOV  DPTR, #420H ; โหลดค่าแอดเดรส NO Messages
LCALL DISPLAY        ; แสดงผลทาง LCD

```



EXIT: ; จบโปรแกรม

ORG 400H

YES-MG: DB "New Messages"

ORG 420H

NO-MG: DB "No New Messages"

ตัวอย่างที่ 7.10 ถ้าหากบิต P2.2 ถูกใช้ในการควบคุมการเปิดไฟนอกอาคาร และบิต P2.5 ถูกใช้
ควบคุมการเปิดไฟภายในอาคาร จงเขียนโปรแกรมเปิดไฟนอกอาคาร และปิดไฟในอาคาร

วิธีทำ

```

SETB    C                ; CY = 1
ORL     C,P2.2           ; CY = P2.2 OR กับบิต CY
MOV     P2.2,C           ; เปิดไฟ ถ้าเดิมยังไม่ได้เปิด
CLR     C                ; CY = 0
ANL     C,P2.5           ; CY = P2.5 AND กับบิต กับบิต CY
MOV     P2.5,C           ; ปิดไฟถ้าเดิมยังไม่ได้ปิด

```



คำถาม ?

1. จงหาค่าบิตแฟล็ก CY ถ้ามีการทำคำสั่งต่อไปนี้

```

CLR     A
ADD     A, #OFFH
JNC     OVER
CPL     C

```

OVER:

2. จงหาค่าบิตแฟล็ก CY ถ้ามีการทำคำสั่งต่อไปนี้

```

CLR     C
JNC     OVER
SETB    C

```

OVER:



3. จงหาค่าบิตแฟลก CY ถ้ามีการทำคำสั่งต่อไปนี้

CLR C

JC OVER

CPL C

OVER:

4. จงเขียนคำสั่งอ่านค่าจากบิต P2.7 และเก็บใน RAM ตำแหน่งที่ 31

5. จงเขียนคำสั่งนำค่าใน RAM ตำแหน่ง 09 ส่งออกไปยัง P1.4

8

บทที่

การโปรแกรม ตัวนับตัวจับเวลา

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 เบอร์ที่นิยมใช้กันทั่วไปจะมีตัวนับตัวจับเวลาอยู่สองตัว โดยสามารถเขียนโปรแกรมให้มันนับสัญญาณนาฬิกาภายในหรือใช้เป็นตัวจับเวลาได้ ถ้าหากเขียนให้มันนับสัญญาณจากภายนอกจะเป็นการใช้งานเป็นตัวนับ ในหัวข้อที่ 8.1 จะกล่าวถึงการใช้งานเป็นตัวจับเวลา ส่วนหัวข้อที่ 8.2 จะกล่าวถึงการใช้งานเป็นตัวนับ



8.1 การโปรแกรมเป็นตัวจับเวลา

MCS-51 มีรีจิสเตอร์ตัวจับเวลาอยู่สองตัวเรียกชื่อว่า Timer 0 และ Timer 1 ทั้งสองตัวจะเป็นรีจิสเตอร์แบบ 16 บิต และแต่ละตัวสามารถแยกออกเป็นไบต์ต่ำและไบต์สูง เราสามารถเขียนโปรแกรมให้ทำงานเป็นตัวจับเวลาได้ โดยรีจิสเตอร์ที่เกี่ยวข้องได้แก่

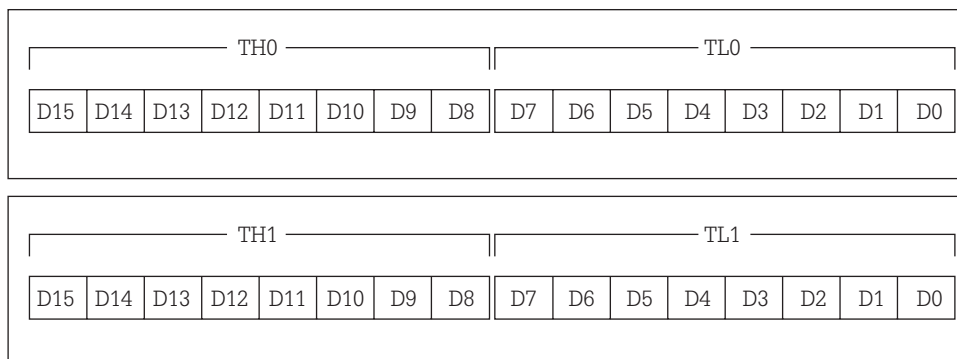
รีจิสเตอร์ Timer 0

รีจิสเตอร์ตัวนี้จะมีขนาด 16 บิต เกิดจากรีจิสเตอร์สองตัวประกอบกันคือรีจิสเตอร์ไบต์ต่ำชื่อว่า TL0 (Timer 0 low byte) และรีจิสเตอร์ไบต์สูงชื่อว่า TH0 (Timer 0 high byte) การใช้งานรีจิสเตอร์ตัวนี้จะคล้ายกับรีจิสเตอร์อื่น ๆ ทั่วไป เช่น เขียนคำสั่งว่า “MOV TL0, #4FH” จะเป็นการกำหนดค่า 4FH ใส่ในรีจิสเตอร์ไทมเมอร์ 0 ไบต์ต่ำ หรือถ้าเขียนคำสั่งเป็น MOV R5,TH0 จะเป็นการนำค่าในรีจิสเตอร์ไทมเมอร์ 0 ไบต์สูงมาใส่ในรีจิสเตอร์ R5



รีจิสเตอร์ Timer 1

เป็นรีจิสเตอร์ขนาด 16 บิต คล้ายกับรีจิสเตอร์ไทเมอร์ 0 โดยประกอบจากรีจิสเตอร์ 8 บิต สองตัว คือไบนารีต่ำ TL1 และ ไบนารีสูง TH1



รูปที่ 8.1 รีจิสเตอร์ Timer 0 และ Timer 1

รีจิสเตอร์ TMOD

การใช้งานไทเมอร์ 0 และไทเมอร์ 1 เราสามารถกำหนดโหมดการทำงานได้สี่รูปแบบ โดยการกำหนดค่าลงในรีจิสเตอร์ควบคุมที่ชื่อว่า TMOD (Timer mode) ซึ่งเป็นรีจิสเตอร์ขนาด 8 บิตโดย 4 บิตล่างใช้กำหนดการทำงานของรีจิสเตอร์ไทเมอร์ 0 และ 4 บิตบนใช้กำหนดการทำงานของรีจิสเตอร์ 1

(MSB)				(LSB)			
GATE	C/T	M1	M0	GATE	C/T	M1	M0
TIME 1				TIME 0			

GATE เป็นบิตเซตวงจรรให้ทำงาน

C/T ใช้เลือกเป็นตัวนับหรือเป็นตัวจับเวลา

M1 โหมดบิต 1

M0 โหมดบิต 0

M1	M0	Mode	การทำงาน
0	0	0	เป็น Timer แบบ 13 บิต โดยใช้ TH × 8 บิต และ TL × 5 บิต
0	1	1	เป็น Timer แบบ 16 บิต
1	0	2	เป็น Timer แบบ 16 บิต auto reload
1	1	3	

รูปที่ 8.2 รีจิสเตอร์ TMOD

**บิต M1, M0**

สองบิตนี้จะใช้เลือกโหมดการทำงานดังแสดงในรูปที่ 8.2 โดยโหมด 0 จะเป็นการทำงานแบบ 13 บิต โหมด 1 จะทำงานแบบ 16 บิต โหมด 2 ทำงานแบบ 8 บิตที่สามารถโหลดค่ากลับได้เองส่วนโหมด 3 จะใช้งานเป็นไทเมอร์ 8 บิต สองตัว และ 16 บิต หนึ่งตัว

C/T (clock / timer)

ในรีจิสเตอร์ TMOD จะมีบิตที่ใช้เลือกจะให้ไทเมอร์ทำงานเป็นตัวนับหรือตัวจับเวลาบิตนี้คือบิต C/T ถ้าหากมีค่าลอจิกเป็น “0” หมายความว่าใช้งานเป็นตัวจับเวลาโดยจะขึ้นกับค่าความถี่ของสัญญาณนาฬิกาของระบบ ถ้าเป็น “1” หมายความว่าใช้งานเป็นตัวนับสัญญาณจากภายนอก

ตัวอย่างที่ 8.1 จงบอกโหมดการทำงานของไทเมอร์ถ้าเขียนคำสั่งต่อไปนี้

ก. MOV TMOD,#01H ข. MOV TMOD,#20H ค. MOV TMOD,#12H

วิธีทำ จากเลขฐานสิบหกให้ตีความหมายของแต่ละบิตใน TMOD ออกมา โดยดูรูปที่ 8.2 ประกอบ

ก. TMOD = 00000001 ไทเมอร์ 0 ทำงานโหมด 1

ข. TMOD = 00100000 ไทเมอร์ 1 ทำงานโหมด 2

ค. TMOD = 00010010 ไทเมอร์ 0 ทำงานโหมด 2 และไทเมอร์ 1 ทำงานโหมด 1

สัญญาณนาฬิกาของไทเมอร์

ตามที่ทราบมาแล้วว่าตัวไทเมอร์นี้จะทำการนับสัญญาณนาฬิกาที่ถูกส่งเข้ามา ถ้า C/T = 0 จะนับสัญญาณนาฬิกาในระบบที่ได้จากค่าความถี่ของ crystal ที่ต่อให้กับ MCS-51 ทารด้วย 12 ถ้าหาก MCS-51 ทำงานด้วยความถี่สัญญาณนาฬิกา 12 MHz และให้ไทเมอร์ทำงานเป็นตัวจับเวลาจะทำให้สัญญาณที่เข้าสู่ไทเมอร์มีความถี่เท่ากับ 1 MHz โดยทั่วไปแล้วระบบที่ใช้ MCS-51 ในปัจจุบันจะใช้ XTAL ที่มีความถี่ตั้งแต่ 10 MHz ถึง 40 MHz แต่บางระบบจะต้องใช้ความถี่ 11.0592 MHz เพื่อให้สามารถสื่อสารข้อมูลแบบอนุกรมกับอุปกรณ์อื่น ๆ ได้อย่างถูกต้อง ซึ่งจะได้ศึกษารายละเอียดในบทต่อไป

ตัวอย่างที่ 8.2 จงหาค่าความถี่ของสัญญาณนาฬิกาที่เข้าสู่ไทเมอร์ ถ้าหาก MCS-51 ใช้ XTAL ที่มีความถี่ดังนี้

ก. 12 MHz ข. 16 MHz ค. 11.0592 MHz

วิธีทำ

ก. $1/12 \times 12 \text{ MHz} = 1 \text{ MHz}$ และ $T = 1/1 \text{ MHz} = 1 \mu\text{s}$

ข. $1/12 \times 16 \text{ MHz} = 1.333 \text{ MHz}$ และ $T = 1/1.333 \text{ MHz} = 0.75 \mu\text{s}$

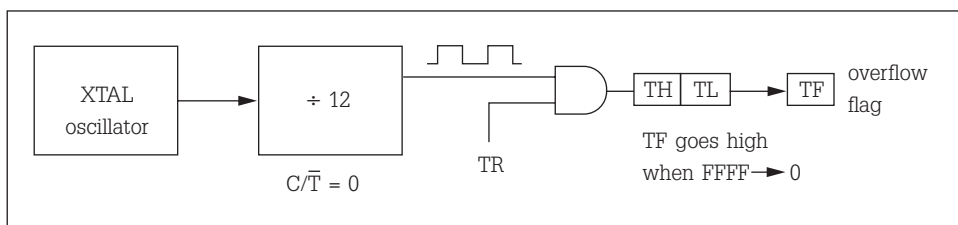
ค. $1/12 \times 11.2592 \text{ MHz} = 921.6 \text{ MHz}$ และ $T = 1/921.6 \text{ MHz} = 1.085 \mu\text{s}$



การโปรแกรมในโหมด 1

ลักษณะการทำงานของไทเมอร์โหมด 1 จะเป็นดังนี้

1. ทำงานเป็นรีจิสเตอร์แบบ 16 บิต ซึ่งจะเก็บค่าได้ตั้งแต่ 0000 ถึง FFFFH โดยจะกำหนดค่าได้โดยโปรแกรมข้อมูลลงไปในรีจิสเตอร์ TH และ TL
2. หลังจากโหลดข้อมูลขนาด 16 บิตลงไปแล้ว ตัวไทเมอร์จะเริ่มทำงานเมื่อได้รับคำสั่ง SETB TR0 สำหรับไทเมอร์ 0 และ SETB TR1 สำหรับไทเมอร์ 1
3. เมื่อให้ไทเมอร์เริ่มทำงานแล้ว ในตัวไทเมอร์จะเกิดการนับขึ้นตั้งแต่ค่าเริ่มต้นที่ป้อนเข้าไปจนถึงค่า FFFFH และเมื่อมีการเปลี่ยนจาก FFFFH เป็น 0000H จะมีการเซตบิต TF (timer flag) ให้เป็นลอจิก “1” ดังนั้นการใช้งานเราจะต้องเฝ้ามองว่าบิต TF นี้เป็นลอจิก “1” หรือยัง ถ้าเป็นแสดงว่าได้นับมาถึงค่าที่กำหนดแล้ว จากนั้นจะต้องหยุดไทเมอร์โดยการใช้คำสั่ง CLR TR0 หรือ CLR TR1 สำหรับไทเมอร์ 0 และไทเมอร์ 1 ตามลำดับ และจะต้องเคลียร์บิตแฟลก TF ด้วย เนื่องจากบิตนี้จะเซตโดยฮาร์ดแวร์เมื่อไทเมอร์มีการเปลี่ยนจาก FFFFH เป็น 0000H แต่ต้องเคลียร์โดยซอฟต์แวร์



4. หลังจากไทเมอร์ทำงานมาตามกำหนดแล้วจะต้องโหลดค่าให้กับรีจิสเตอร์ TH และ TL อีกครั้งเพื่อใช้งานครั้งต่อไป

ขั้นตอนการโปรแกรมในโหมด 1

ถ้าหากต้องการเขียนโปรแกรมจับเวลาโดยใช้ไทเมอร์โหมด 1 จะต้องเขียนโปรแกรมตามลำดับดังนี้

1. โหลดค่าลงรีจิสเตอร์ TMOD เพื่อกำหนดโหมดการทำงานของไทเมอร์ตัวที่ต้องการใช้งาน
2. โหลดค่าลงรีจิสเตอร์ TL และ TH เพื่อกำหนดค่าเริ่มต้นของการนับ
3. ให้ไทเมอร์เริ่มทำงาน
4. ตรวจสอบแฟลกไทเมอร์ (TF) ว่าเป็น “1” หรือยัง ถ้ายังให้วนลูปตรวจสอบจนกว่าจะเป็น “1”
5. หยุดไทเมอร์
6. เคลียร์บิตแฟลก TF เพื่อใช้งานครั้งต่อไป
7. กลับไปยังขั้นตอนที่ 2 เพื่อโหลดค่าสำหรับการทำงานครั้งต่อไป



จากขั้นตอนที่กล่าวมาสามารถเขียนชุดคำสั่งได้ดังตัวอย่างที่ 8.3 สำหรับการคำนวณค่าเริ่มต้นให้กับ ไทเมอร์เพื่อใช้ในการจับเวลาจะขึ้นอยู่กับความถี่ของ XTAL ด้วยโดยจะศึกษาได้ดังตัวอย่างที่ 8.4 ซึ่งจะสร้าง คลื่นสี่เหลี่ยม (square wave) ออกมาทางขา P1.5 และตัวอย่างที่ 8.5 จะเป็นการคำนวณค่าความถี่ของ สัญญาณที่ได้จากโปรแกรมในตัวอย่างที่ 8.4

ตัวอย่างที่ 8.3 ต่อไปจะแสดงการวิเคราะห์โปรแกรมการสร้างคำสั่งรูปสี่เหลี่ยมที่มีค่า duty cycle เท่ากับ 50 % (เวลาเป็น “1” เท่ากับเวลาเป็น “0”) โดยการใช้ไทเมอร์ 0 เป็นตัวจับเวลา

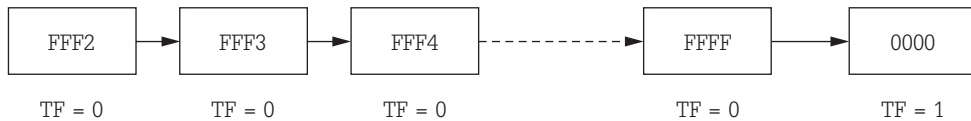
	MOV	TMOD, #01	; ไทเมอร์ 0 โหมด 1 (16 บิต)
HERE:	MOV	TL0, #0F2H	; ไบต์ต่ำ TL0 = F2H
	MOV	TH0, #0FFH	; ไบต์สูง TH0 = FFH
	CPL	P1.5	; toggle P1.5
	ACALL	DELAY	
	SJMP	HERE	; โหลดค่า TL และ TH อีกครั้ง
DELAY:	SETB	TR0	; ให้ไทเมอร์เริ่มทำงาน
AGAJN:	JNB	TF0, AGAJN	; เฝ้ามองบิตแฟลกจนเป็น “1”
	CLR	TR0	; หยุดไทเมอร์
	CLR	TF0	; เคลียร์บิตแฟลกให้เป็น “0”
	RET		

จากโปรแกรมวิเคราะห์การทำงานได้ดังนี้

1. โหลดข้อมูลลง TMOD เพื่อเลือกโหมดการทำงาน
2. โหลดค่า FFF2H ลงใน TH0-TL0
3. ทำให้บิต P1.5 toggle จาก “1” เป็น “0” จาก “0” เป็น “1”
4. เรียกโปรแกรมหน่วยเวลา DELAY เพื่อให้เป็น “0” หรือ “1” ชั่วขณะ
5. ในโปรแกรมหน่วยเวลา DELAY จะใช้ timer 0 โดยเริ่มจับเวลาด้วยคำสั่ง SETB TR0
6. ไทเมอร์ 0 เริ่มนับ 1 ซึ่งมีสัญญาณเข้ามาจาก FFF3, FFF4, FFF5, FFF6, FFF7, FFF8, ไปจน ถึง FFFFH เมื่อมีสัญญาณเข้ามาอีกลูกจะกลับเป็น 0 และบิตแฟลกไทเมอร์จะเซต (TF0=1) โดยใช้คำสั่ง JNB คอยดูบิตแฟลก
7. หยุดไทเมอร์ 0 ด้วยคำสั่ง CLR TR0 จะจบโปรแกรมย่อยหน่วยเวลา



การเปลี่ยนแปลงของรีจิสเตอร์ไทมเมอร์และบิตแฟลก TF เป็นดังรูปต่อไปนี้



ตัวอย่างที่ 8.4 จากตัวอย่างที่ 8.3 จงคำนวณหาเวลาหน่วยในโปรแกรมย่อย DELAY ถ้าหาก XTAL มีความถี่ 11.0592 MHz

วิธีทำ สัญญาณที่จะเข้าสู่ตัวไทมเมอร์จะเท่ากับ $1/12$ ของความถี่ XTAL ดังนั้นสัญญาณที่เข้าสู่ไทมเมอร์จะมีความถี่เท่ากับ $11.0592 \text{ MHz}/12 = 921.6 \text{ KHz}$ และคาบเวลาจะเท่ากับ $T = 1/921.6 \text{ KHz} = 1.085 \text{ us}$ ดังนั้นถ้าหาค่าเวลาที่ใช้ในการนับจะต้องนำค่าจำนวนสัญญาณที่ทำให้ไทมเมอร์เกิดโอเวอร์โฟลว์มาคูณกับค่าเวลา 1.085 us นี้

จำนวนของการนับที่ทำให้เกิดการโอเวอร์โฟลว์จะเท่ากับ $FFFFH - FFF2H = 0DH$ หรือ 13 D และรวมกับสัญญาณที่ทำให้ค่าในไทมเมอร์ FFFF กลับเป็น 0 อีกหนึ่งลูก รวมเป็น 14 ลูก ดังนั้นเวลาหน่วยจะเท่ากับ $14 \times 1.085 \text{ us} = 15.19 \text{ us}$ สำหรับสัญญาณพัลส์ครึ่งลูก ดังนั้นคาบเวลาจะเท่ากับ $T = 2 \times 15.19 \text{ us} = 30.38 \text{ us}$

ตัวอย่างที่ 8.5 จากโปรแกรมในตัวอย่างที่ผ่านมาจงหาความถี่ของคลื่นทางบิต P1.5

วิธีทำ จะต้องรวมเวลาทั้งหมดของการทำโปรแกรมและรวมเวลาหน่วยในโปรแกรมย่อย DELAY ด้วย โดยรวมไซเคิลของคำสั่งทั้งหมด โดยดูจากตารางเวลาของคำสั่งจะได้เป็น

		ไซเคิล	
HERE:	MOV TLO, #0F2H	2	
	MOV TH0, #0FFH	2	
	CPL P1.5	1	
	ACALL DELAY	2	
	SJMP HERE	2	
DELAY:	SETB TRO	1	
AGAIN:	JNB TF0, AGAIN	14	; มาจากตัวอย่างที่ 8.4
	CLR TRO	1	
	CLR TF0	1	
	RET	1	
	รวม	27	

ดังนั้นคาบเวลาจะได้เป็น $T = 2 \times 27 \times 1.085 \text{ us} = 58.59 \text{ us}$ และ $F = 17067.75 \text{ Hz}$



ตัวอย่างที่ 8.6 จากชุดคำสั่งต่อไปนี้จงหาเวลาหน่วงจากไทมเมอร์ 0 โดยไม่รวมเวลาที่เสียไปในโปรแกรม

```

CLR    P2.3                ; เคลียร์ P2.3
MOV    TMOD,#01            ; ใช้ไทมเมอร์ 0 โหมด 1
HERE:  MOV    TL0,#3EH      ; TL0 = 3EH โปรแกรมค่าไบต์ต่ำ
        MOV    TH0,#0B8H    ; TH0 = B8H ใส่ค่าไบต์สูง
        SETB   P2.3         ; ให้บิต P2.3 เป็น "1"
        STEB   TR0          ; ให้ไทมเมอร์ 0 เริ่มทำงาน
AGAIN:  JNB    TF0,AGAIN    ; ตรวจสอบบิตแฟลก
        CLR    TR0          ; หยุดไทมเมอร์ 0
        CLR    TF0          ; เคลียร์บิตแฟลก
        CLR    P2.3         ; ให้บิต P2.3 เป็น "0"

```

วิธีทำ หาจำนวนสัญญาณของการนับ

$(FFFF - B83E + 1) = 47C2H = 18370$ จะเป็นเวลาเท่ากับ $18370 \times 1.085 \text{ us} = 19.93145 \text{ ms}$

ตัวอย่างที่ 8.7 จากตัวอย่างที่ 8.6 จงแก้ไขค่าในรีจิสเตอร์ TH และ TL ให้หน่วงเวลาได้มากที่สุด

วิธีทำ จะต้องให้รีจิสเตอร์ TH และ TL มีค่าเป็น 0 เพื่อให้การนับเริ่มจาก 0000 ไปจนถึง FFFFH จะได้โปรแกรมหาดังนี้

```

CLR    P2.3                ; เคลียร์บิต P2.3
MOV    TMOD,#01            ; Timer 0 โหมด 1 (16-bit mode)
HERE:  MOV    TL0,#0        ; ใส่ค่าไบต์ต่ำ
        MOV    TH0,#0       ; ใส่ค่าไบต์สูง
        SETB   P2.3         ; เซตให้บิต P2.3 เป็น "1"
        SETB   TR0          ; ให้ไทมเมอร์ 0 เริ่มทำงาน
AGAIN:  JNB    TF0,AGAIN    ; ฝ้ามองบิตแฟลก 0
        CLR    TR0          ; หยุดไทมเมอร์ 0
        CLR    TF0          ; เคลียร์บิตแฟลกไทมเมอร์ 0
        CLR    P2.3

```

การนับตั้งแต่ 0000 ไปจนถึง FFFF และให้เกิดการทดที่แฟลก TF จะใช้สัญญาณทั้งหมด 65536 ลูก ดังนั้นเวลาที่เสียไปจะเท่ากับ $(65536 - 0) \times 1.085 \text{ us} = 71.1065 \text{ us}$



จากตัวอย่างที่ 8.6 และ 8.7 ที่ผ่านมามาดูเห็นว่าโหลดค่าเข้ารีจิสเตอร์ TH และ TL เพียงครั้งเดียวถ้า MCS-51 ทำงานตามนี้จะให้พัลส์ออกมาเพียงลูกเดียว ในตัวอย่างที่ 8.8 จะให้โหลดค่าเข้ารีจิสเตอร์ไทเมอร์อีกครั้ง และให้โปรแกรมทำซ้ำก็จะได้พัลส์ต่อเนื่องออกมา

ตัวอย่างที่ 8.8 โปรแกรมต่อไปนี้จะให้คลื่นรูปสี่เหลี่ยมออกมาทางพิน P1.5 โดยใช้ไทเมอร์ 1 เป็นตัวหน่วงเวลา จงหาค่าความถี่ของคลื่นถ้า XTAL ที่ใช้มีความถี่ 11.0592 MHz โดยไม่ต้องคำนวณเวลาที่เสียไปในคำสั่งอื่น ๆ ในรูป

```

MOV  TMOD, #10H          ; ไทเมอร์ 1 โหมด 1
AGAIN: MOV  TL1, #34H      ; TL1 = 34H ไบต์ต่ำ
      MOV  TH1, #76H      ; TH1 = 76H ไบต์สูง
                        ; ไทเมอร์ = 7634H
      SETB TR1            ; เริ่มทำงานไทเมอร์ 1
BACK:  JNB  TF1, BACK      ; รอจนตัวจับเวลาเกิดโอเวอร์โฟลว์
      CLR  TR1            ; หยุดไทเมอร์ 1
      CPL  P1.5           ; กลับค่าลอจิกพิน P1.5
      CLR  TF1            ; เคลียร์แฟล็กไทเมอร์
      SJMP AGAIN          ; โหลดค่าและเริ่มทำงานใหม่

```

วิธีทำ จากโปรแกรมจะทำงานในโหมด 1 และให้ทำซ้ำโหลดค่าเข้าไทเมอร์ใหม่ด้วยคำสั่ง SJMP จำนวนสัญญาณนาฬิกาที่ทำให้แฟล็กไทเมอร์เซตจะเท่ากับ $FFFFH - 7634H = 89CBH + 1 = 89CCH$ หรือ สัญญาณนาฬิกาจำนวน 35276 ลูก ซึ่งเป็นเวลาเท่ากับ $35276 \times 1.085 \text{ us} = 38.274 \text{ us}$ ทำให้ได้ความถี่ 26.127 Mz ออกมา

การคำนวณค่าสำหรับโหลดเข้าไทเมอร์

การที่จะให้ไทเมอร์ทำการหน่วงเวลาหรือจับเวลาตามที่เรากำหนดการนั้น จะต้องโหลดค่าให้กับรีจิสเตอร์ TH และ TL ก่อน โดยค่าในรีจิสเตอร์จะขึ้นกับความถี่สัญญาณนาฬิกาที่ใช้ในระบบด้วย ถ้าหาก 8051 ทำงานที่ความถี่ 11.0592 MHz การคำนวณค่าในรีจิสเตอร์สามารถทำได้ดังนี้

1. หาค่าเวลาที่ต้องการจับเวลาด้วยค่า 1.085 us
2. คำนวณค่า 65536 - N โดย N เป็นค่าที่ได้จากขั้นตอนที่ 1
3. แปลงค่าที่ได้จากขั้นตอนที่สองเป็นเลขฐานสิบหกในรูปของ yyxx ซึ่งเป็นค่าที่จะโหลดให้กับรีจิสเตอร์ไทเมอร์
4. กำหนดค่า TL = xx และ TH = yy



ตัวอย่างที่ 8.9 ถ้าหาก 8051 ทำงานกับ XTAL ที่มีความถี่ 11.0592 MHz อยากทราบว่าต้องโหลดค่าใดให้กับไทมเมอร์และจะต้องโปรแกรมอย่างไร ถ้าต้องการสร้างพัลส์ขนาด 5 ms ออกมาทางขา P2.3

วิธีทำ จาก XTAL ที่กำหนดจะทำให้ตัวไทมเมอร์เกิดการนับเมื่อเวลาผ่านไป 1.085 us ดังนั้นถ้าต้องการสร้าง pulse ขนาด 5 ms จะต้องใช้สัญญาณนาฬิกาจำนวน $5 \text{ ms} / 1.085 \text{ us} = 4608$ ลูก ค่าที่ใส่ให้รีจิสเตอร์ ไทมเมอร์จะเท่ากับ $65536 - 4608 = 60928 = \text{EE00H}$ โดยให้ TH เก็บ EE และ TL เก็บ 00 และเขียนโปรแกรมได้ดังนี้

```

CLR    P2.3                ; เคลียร์ P2.3
MOV    TMOD, #01           ; ไทมเมอร์ 0 โหมด 1
MERE:  MOV    TL0, #0       ; โหลดค่า TL0 = 0
        MOV    TH0, #0EEH   ; โหลดสูง TH0 = EE
        SETB   P2.3         ; ให้ P2.3 เป็นลอจิก "1"
        SETB   TR0          ; ให้ไทมเมอร์ 0 เริ่มทำงาน
AGAIN:  JNB    TF0, AGAIN   ; ตรวจสอบบิตแฟล็กไทมเมอร์ 0
        CLR    TR0          ; หยุดไทมเมอร์
        CLR    TF0          ; เคลียร์บิตแฟล็กไทมเมอร์ 0

```

ตัวอย่างที่ 8.10 ถ้าความถี่ XTAL เท่ากับ 11.0592 MHz จงเขียนโปรแกรมสร้างคลื่นสี่เหลี่ยมความถี่ 2 kHz ออกมาทางขา P1.5

วิธีทำ ตัวอย่างนี้จะคล้ายกับตัวอย่างที่ผ่านมา แต่จะทำให้บิต P1.5 เป็นค่าลอจิกกลับไปกลับมาโดยมีขั้นตอนดังนี้

1. คำนวณเวลาของคลื่น $T = 1 / f = 1 / 2 \text{ kHz} = 500 \text{ us}$
2. ให้เป็นลอจิก "High" และ "Low" เป็นเวลา 250 us
3. $250 \text{ us} / 1.085 \text{ us} = 230$ และ $65536 - 230 = 65306$ ซึ่งเท่ากับ FF1AH
4. ใส่ค่า TL = 1A และ TH = FF และเขียนโปรแกรมได้ดังนี้

```

MOV    TMOD, #10H          ; ไทมเมอร์ 1 โหมด 1
AGAIN:  MOV    TL1, #1AH    ; TL1 = 1A โหลดต่ำ
        MOV    TH1, #0FFH   ; TH1 = FF โหลดสูง
        SETB   TR1          ; ไทมเมอร์ 1 เริ่มทำงาน
BACK:   JNB    TF1, BACK    ; การตรวจสอบบิตแฟล็กไทมเมอร์
        CLR    TR1          ; หยุดไทมเมอร์
        CPL    P1.5         ; กลับค่าบิต P1.5
        CLR    TF1          ; เคลียร์บิตแฟล็ก

```



SJMP AGAIN

; โหลดค่าไทเมอร์ และทำซ้ำ

ตัวอย่างที่ 8.11 ถ้าหาก XTAL มีความถี่ 11.0592 MHz จงเขียนโปรแกรมสร้างคลื่นรูปสี่เหลี่ยม ความถี่ 50 Hz ออกมาทางขา P2.3

วิธีทำ สามารถทำได้ตามขั้นตอนดังนี้

1. คาบเวลาของคลื่นสี่เหลี่ยม $T = 1 / 50 \text{ Hz} = 20 \text{ ms}$
2. จะต้องเป็นลอจิก “High” และ “Low” เป็นเวลา 10 ms
3. $10 \text{ ms} / 1.085 \text{ us} = 9216$ และ $65536 - 9216 = 56320$ หรือ DC00H และโหลดเข้ารีจิสเตอร์ไทเมอร์

4. TL = 00H และ TH = DCH

	MOV TMOD, #10H	; ไทเมอร์ 1 โหมด 1
AGAIN:	MOV TL1, #00	; โหลดไบต์ต่ำ TL1 = 00
	MOV TH1, #0DCH	; โหลดไบต์สูง TH1 = DC
	SETB TR1	; ไทเมอร์ 1 เริ่มทำงาน
BACK:	JNB TF1, BACK	; ตรวจสอบบิตแฟลกไทเมอร์
	CLR TR1	; หยุดไทเมอร์
	CPL P2.3	; กลับค่าลอจิกบิต P2.3
	CLR TF1	; เคลียร์บิตแฟลก
	SJMP AGAIN	; โหลดค่าเข้าสู่ไทเมอร์และทำซ้ำ

ในการใช้ไทเมอร์เป็นตัวจับเวลาจะพบว่าไม่สามารถใช้ไทเมอร์นับสัญญาณนาฬิกามากกว่า 65536 ลูกได้ โดยการจับเวลานานที่สุดทำได้โดยโหลดค่า 0000 เข้ารีจิสเตอร์ไทเมอร์ แต่ถ้าหากต้องการจับเวลาที่นานขึ้นเราสามารถทำได้สองวิธีคือ เปลี่ยนความถี่สัญญาณนาฬิกาให้ต่ำลง อีกวิธีหนึ่งจะใช้ไทเมอร์แบบ 16 บิต และให้ทำลูปซ้ำหลาย ๆ ครั้ง ดังตัวอย่างโปรแกรมที่ 8.12

ตัวอย่างที่ 8.12 จากโปรแกรมต่อไปนี้จะคำนวณหาค่าเวลาหน่วย โดยไม่ต้องคำนวณเวลาย่อย ๆ ที่เสียไปในคำสั่งลูป

	MOV TMOD, #10 H	; ไทเมอร์ 1 โหมด 1
	MOV R3, #200	; ลูปของการจับเวลา
AGAIN:	MOV TL1, #08	; โหลดไบต์ต่ำ TL1 = 08
	MOV TH1, #01	; โหลดไบต์สูง TH1 = 01
	SETB TR1	; เริ่มต้นไทเมอร์ 1



BACK:	JNB	TF1, BACK	; ตรวจสอบบิต TF
	CLR	TR1	; หยดไทเมอร์ 1
	CLR	TF1	; เคลียร์บิตแฟล็กไทเมอร์ 1
	DJNZ	R3, AGAIN	; ทำซ้ำจน R3 เป็นศูนย์

วิธีทำ จากโปรแกรม จะพบว่าค่าในไทเมอร์เป็น 0108 H หรือ 264 จำนวนสัญญาณของไทเมอร์ จะเท่ากับ $65536 - 264 = 65272$ ซึ่งจะเสียเวลาไป $65272 \times 1.085 \text{ us}$ หรือ 70.820 ms และโปรแกรมทำซ้ำ 200 ครั้ง ดังนั้นจะเสียเวลาไป $200 \times 70.820 \text{ ms}$ เท่ากับ 14.164024 วินาที

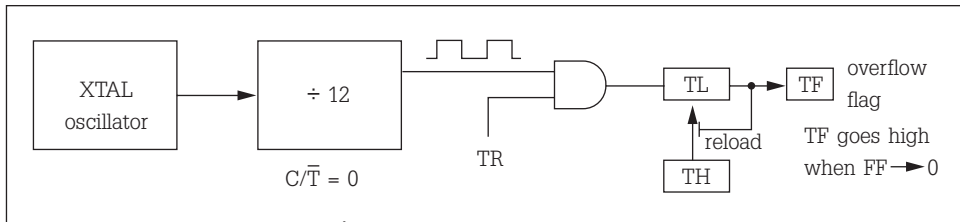
การโปรแกรมในโหมด 0

การทำงานของโหมด 0 จะคล้ายกับโหมด 1 แต่จะนับตัวนับตัวจับเวลาแบบ 13 บิต ทำให้ค่าในตัวนับ (TH - TL) อยู่ระหว่าง 0000 ถึง 1FFF และการนับจะทำให้เกิดการโอเวอร์โฟลว์โดยทดทางแฟล็ก TF เมื่อมีการเปลี่ยนจาก 1FFF เป็น 0000

การโปรแกรมในโหมด 2

ตัวนับตัวจับเวลาในโหมด 2 เป็นโหมดที่สามารถโหลดค่าการนับเข้าไปในรีจิสเตอร์ได้ (auto-reload) โดยจะมีการดำเนินการดังนี้

1. เป็นไทเมอร์แบบ 8 บิต โดยการนับจะอยู่ในช่วง 00 ถึง FFH โดยโหลดค่าการนับเข้าในรีจิสเตอร์ TH
2. หลังจากโหลดค่า 8 บิต เข้าในรีจิสเตอร์ TH แล้ว ตัว MCS-51 จะก๊อปปี้ข้อมูลเข้าสู่รีจิสเตอร์ TL และจะเริ่มนับเมื่อมีการทำคำสั่ง "SETB TR0" สำหรับไทเมอร์ 0 และ "SETB TR1" สำหรับไทเมอร์ 1 โดยมีการทำงานคล้ายกับโหมด 1
3. หลังจากไทเมอร์เริ่มทำงานจะมีการนับค่าในรีจิสเตอร์ TL ขึ้นไปจนถึง FFH เมื่อมีการเปลี่ยน FFH ไปเป็น 00H จะมีการเซตบิตแฟล็ก TF ให้เป็น "1"
4. เมื่อ TL มีค่าเปลี่ยนจาก FFH เป็น 00H และค่าแฟล็ก TF เป็น "1" รีจิสเตอร์ TL จะโหลดค่าจากรีจิสเตอร์ TH ลงไปใหม่เพื่อให้สามารถใช้ไทเมอร์อีกครั้งได้ แต่ถ้าเป็นโหมด 1 จะต้องเขียนคำสั่งโหลดข้อมูลเข้าไทเมอร์อีกครั้งถ้าจะใช้งานไทเมอร์ต่อไป



รูปที่ 8.3 การทำงานของไทมเมอร์ในโหมด 2

ขั้นตอนการโปรแกรมในโหมด 2

การใช้งานไทมเมอร์โหมด 2 มีขั้นตอนการเขียนคำสั่งดังนี้

1. โหลดค่ารูปแบบการทำงานเข้าสู่รีจิสเตอร์ TMOD โดยโปรแกรมให้เป็นโหมด 2
2. โหลดค่าเริ่มต้นของการนับเข้าสู่รีจิสเตอร์ TH
3. ให้ไทมเมอร์เริ่มทำงาน
4. ตรวจสอบบิตแฟล็ก (TF) ด้วยคำสั่ง “JNB” TFX,target” และให้ออกนอกกลุ่มเมื่อบิตแฟล็ก TF เป็นลอจิก “1”
5. เคลียร์บิตแฟล็ก TF
6. กลับเข้าสู่ขั้นตอนที่ 4

ตัวอย่างที่ 8.13 จากตัวอย่างโปรแกรมถ้าหาก MCS-51 ใช้ XTAL ที่มีความถี่ 11.0592 MHz จงหาค่า

ก. ความถี่ของคลื่นรูปสี่เหลี่ยมที่ออกมาทางขา P1.0

ข. คำนวณหาค่าความถี่ต่ำสุดที่สามารถทำได้โดยการเปลี่ยนค่าใน TH

MOV TMOD, #20H	; ไทมเมอร์ 1 โหมด 2 8-bit auto-reload
MOV TH1, #5	; TH1 = 5
SETB TR1	; เริ่มต้นให้ไทมเมอร์ 1 ทำงาน
BACK: JNB TF1, BACK	; ตรวจสอบบิตแฟล็ก TF
CPL P1.0	; กลับค่าบิต P1.0
CLR TF1	; เคลียร์บิตแฟล็ก TF
SJMP BACK	; ทำงานโหมด 2 ในแบบ auto-reload

วิธีทำ ก. จะเห็นว่าโปรแกรมการทำงานซ้ำ ๆ ด้วยคำสั่ง SJMP และเนื่องจากไทมเมอร์ทำงานในโหมด 2 ตัวมันจะโหลดค่าจากรีจิสเตอร์ TH ให้กับรีจิสเตอร์ TL เอง ส่วนเวลาที่เสียไปในไทมเมอร์จะเท่ากับ $(250 - 05) \times 1.085 \text{ us} = 251 \times 1.085 \text{ us} = 272.33 \text{ us}$ จากนั้นโปรแกรมจะกลับค่าบิต P1.0 ทำให้



สัญญาณคลื่นสี่เหลี่ยมที่ออกมาจะมีค่า duty cycle เท่ากับ 50% คาบเวลาที่ได้จะเท่ากับ $T = 2 \times 272.33 \text{ us} = 544.67$ และความถี่เท่ากับ 1.83597 kHz

ข. ความถี่ต่ำสุดจะเกิดจากการโหลดค่า 00 ให้กับรีจิสเตอร์ TH ดังนั้นคาบเวลาจะเท่ากับ $T = 2 \times 256 \times 1.085 \text{ us} = 555.52 \text{ us}$ และความถี่เท่ากับ 1.8 kHz

ตัวอย่างที่ 8.14 จงหาค่าความถี่ของคลื่นสี่เหลี่ยมที่เกิดจากขา P1.0 ถ้าทำโปรแกรมหาดังนี้

```

MOV    TMOD, #2H           ; ไทเมอร์ 0 ทำงานโหมด 2
MOV    TH0, #0              ; TH0 = 0
AGAIN: MOV    R5, #250       ; จำนวนการทำลูป
        ACALL DELAY
        CPL    P1.0
        SJMP   AGAIN

DELAY:  SETB    TR0           ; ให้ไทเมอร์ 0 เริ่มทำงาน
BACK:   JNB     TF0, BACK     ; ตรวจสอบบิตแฟลก
        CLR     TR0           ; หยุดไทเมอร์ 0
        CLR     TF0           ; เคลียร์บิตแฟลก
        DJNZ   R5, DELAY
        RET

```

ดังนั้นคาบเวลาสามารถหาได้จาก $T = 2 (250 \times 256 \times 1.085 \text{ us}) = 138.88 \text{ us}$ และความถี่ $f = 72$ Hz

ตัวอย่างที่ 8.15 ถ้าหากโปรแกรมให้ไทเมอร์ทำงานในโหมด 2 ด้วยค่าที่โหลดให้รีจิสเตอร์ TH ดังต่อไปนี้ จงหาค่าเลขฐานสิบหกที่อยู่ในรีจิสเตอร์ TH

ก. MOV TH1, # -200	ข. MOV TH0, # -60	ค. MOV TH1, # -3
ง. MOV TH1, # -12	จ. MOV TH0, # -48	

วิธีทำ จะได้ดังต่อไปนี้

เลขฐานสิบ	เลข 2's คอมพลีเมนต์ใน TH
-200	38H
-60	C4H
-3	FDH



-12 F4H
-48 D0H

ในการเขียนโปรแกรมภาษาแอสเซมบลีนั้นสามารถใส่เลขลบที่เป็นเลขฐานสิบโดยให้โปรแกรมแอสเซมเบเลอร์คำนวณเป็นเลขฐานสิบหกออกมาได้ ตัวอย่างเช่น “MOV TH1, # - 100” ตัวแอสเซมเบเลอร์จะคำนวณค่า - 100 เป็น 9C และให้ TH1 เท่ากับ 9C

ตัวอย่างที่ 8.16 จงหาค่าความถี่จากโปรแกรมต่อไปนี้พร้อมทั้งคำนวณหาค่า duty cycle

```

MOV    TMOD, #2H          ; ไทเมอร์ 0 โหมด 2
MOV    TH0, # -150         ; TH0 = 6AH
AGAIN: SETB  P1.3           ; P1.3 = 1
        ACALL DELAY
        ACALL DELEY
        CLR   P1.3          ; P1.3 = 0
        ACALL DELAY
        SJMP  AGAIN
DELAY:  SETB  TR0           ; ให้ไทเมอร์ 0 เริ่มทำงาน
BACK:   JNB   TF0, BACK     ; ตรวจสอบบิตแฟลก
        CLR   TR0          ; หยุดไทเมอร์ 0
        CLR   TF0          ; เคลียร์บิตแฟลก
        RET

```

วิธีทำ จากโปรแกรมจะเป็นการโปรแกรมให้ TH0 เกิดการโอเวอร์โฟลว์เมื่อมีสัญญาณนาฬิกาเข้าไป 150 ลูก ดังนั้นโปรแกรมย่อย DELAY จะหน่วงเวลาไปเท่ากับ $150 \times 1.085 \text{ us} = 162 \text{ us}$ และให้ P1.3 เป็นลอจิก “1” นานกว่าลอจิก “0” หนึ่งเท่า เนื่องจากหลังการเซตค่าบิต P1.3 จะเรียกโปรแกรมย่อย DELAY สองครั้ง ดังนั้นค่า duty cycle จะเท่ากับ 66% และเวลาที่เป็นลอจิก “1” รวมกับเวลาที่เป็นลอจิก “0” จะเท่ากับ $325.5 \text{ us} + 162.25 \text{ us} = 488.25 \text{ us}$ และความถี่เท่ากับ 2.048 kHz

**คำถาม ?**

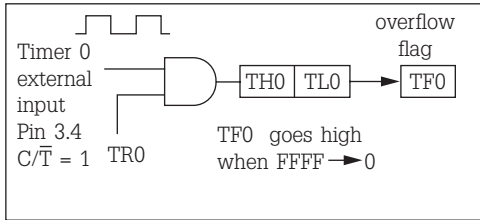
1. ไทเมอร์ของ 8051 ใช้ทำอะไร และมีอยู่ที่ตัว
2. รีจิสเตอร์ TMOD มีชื่อเต็มว่าอะไร มีขนาดกี่บิต
3. รีจิสเตอร์ TMOD สามารถอ้างแอดเดรสระดับบิตได้หรือไม่
4. คำสั่ง MOV TMOD, #20H จะเป็นการทำงานลักษณะใด
5. ถ้าไทเมอร์ทำงานในโหมด 1 จะมีการนับจากช่วงใดถึงช่วงใด
6. ถ้าไทเมอร์ทำงานในโหมด 2 จะมีการนับจากช่วงใดถึงช่วงใด
7. ถ้าเขียนคำสั่งเป็น MOV TH1, # - 200 ค่าในรีจิสเตอร์ TH จะเป็นเท่าใด
8. ถ้าต้องการช่วงเวลา 2 us และให้ไทเมอร์ทำงานในโหมด 1 จะต้องโหลดค่าอะไรในรีจิสเตอร์ TH และ TL ถ้า XTAL มีความถี่ 11.0592 MHz
9. ถ้าต้องการช่วงเวลา 100 us จะต้องโหลดค่าอะไรให้กับรีจิสเตอร์ TH ถ้าให้ไทเมอร์ทำงานในโหมด 2 ถ้าหาก XTAL มีความถี่ 11.0592 MHz

**8.2 การโปรแกรมเป็นตัวนับ**

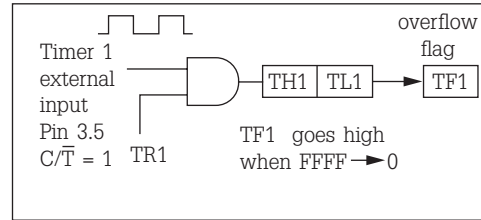
จากที่ผ่านมาเราได้ใช้ไทเมอร์คาน์เตอร์ของ 8051 เป็นตัวจับเวลามาแล้วถ้าหากใช้เป็นตัวนับหรือคาน์เตอร์ก็สามารถทำได้เช่นกัน โดยโปรแกรมให้นับสัญญาณพัลส์จากภายนอกตัว 8051 เมื่อมีสัญญาณพัลส์เข้ามาจะเกิดการนับขึ้นในรีจิสเตอร์ TH และ TL เช่นเดียวกับการนับเมื่อใช้เป็นตัวจับเวลา

บิต C / T ในรีจิสเตอร์ TMOD

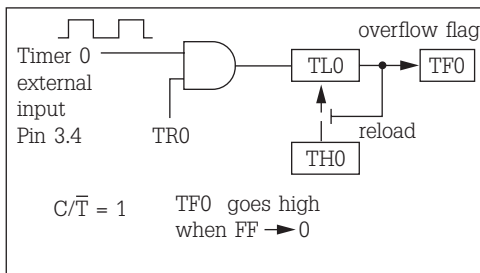
ในรีจิสเตอร์ควบคุม TMOD นั้นถ้าหากบิต C/T ถูกโปรแกรมให้เท่ากับ 0 จะเป็นการนับสัญญาณจาก crystal ถ้าเป็น 1 จะเป็นการนับสัญญาณกระตุ้นจากภายนอกซึ่งสามารถใช้เป็นตัวนับได้ ถ้าหากสัญญาณเข้ามาทางขา T0 (timer 0 input) หรือขา P3.4 จะเป็นการนับของตัวไทเมอร์ 0 ถ้าหากสัญญาณเข้ามาทางขา T1 (timer 1 input) หรือขา P3.5 จะเป็นการนับของตัวไทเมอร์ 1 และการเริ่มนับจะควบคุมที่บิต TR



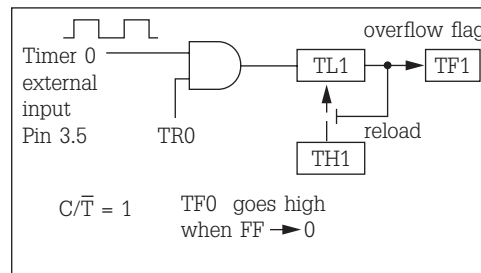
(ก) Timer 0 w/External Input (Mode 1)



(ข) Timer 1 with External Input (Mode 1)



(ค) Timer 0 with External Input (Mode 2)



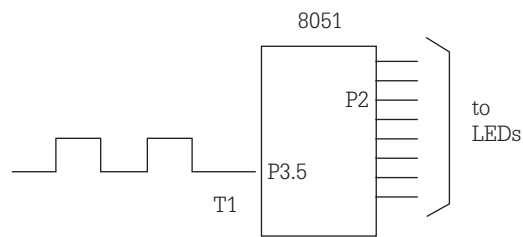
(จ) Timer 1 with External Input (Mode 2)

รูปที่ 8.4 การใช้ Timer เป็นตัวนับ

ตัวอย่างที่ 8.17 ถ้าหากมีสัญญาณเข้ามาทางขา T1 จงเขียนโปรแกรมให้เคาน์เตอร์ 1 ทำงานในโหมด 2 นับสัญญาณเข้ามา และแสดงผลค่าของ TL1 บนพอร์ต P2

วิธีทำ

MOV TMOD, #01100000B	; เคาน์เตอร์ 1 โหมด 2 C/T = 1
MOV TH1, #0	; นับพัลส์จากภายนอก
SETB P3.5	; โปรแกรมให้ T1 เป็นอินพุต
AGAIN: SETB TR1	; เริ่มนับ
BACK: MOV A, TL1	; อ่านค่าตัวนับ TL1
MOV P2, A	; แสดงผลทางพอร์ต P2
JNB TF1, BACK	; ตรวจสอบบิต และกระโดดถ้า TF = 0
CLR TR1	; หยุดนับ
CLR TF1	; เคลียร์บิตแฟล็ก
SJMP AGAIN	; นับอีกครั้ง



จากโปรแกรมจะเห็นว่ามีการใช้คำสั่ง SETB P3.5 เพราะว่าการจ่ายไฟครั้งแรกพอร์ตของ 8051 จะเป็นพอร์ตของ 8051 จะเป็นพอร์ตเอาต์พุต การใช้งาน P3.5 ให้เป็นพอร์ตอินพุตเพื่อรับสัญญาณจากภายนอกจะต้องส่งลอจิก “1” ออกไปก่อน

จากตัวอย่างที่ 8.17 จะเป็นการใช้ไทมเมอร์ 1 ทำงานเป็นตัวนับ เมื่อสัญญาณเข้ามาทางขา P3.5 ค่าที่ได้ในรีจิสเตอร์ TL1 จะเป็นจำนวนพัลส์จากภายนอกที่เข้ามาซึ่งเราสามารถนำค่านี้ไปแสดงผลทางอุปกรณ์แสดงผลได้ ในตัวอย่างที่ 8.18 จะเปลี่ยนข้อมูลในรีจิสเตอร์ TL ให้เป็นรหัส ACSII และแสดงผลทาง LCD

ตัวอย่างที่ 8.18 ถ้าหากมีความถี่ 1 Hz เข้ามาทางอินพุตขา P3.4 จงใช้ตัวนับจาก timer 0 ทำการนับและแสดงผลทาง LCD โดยกำหนดค่าเริ่มต้นให้ TH0 เป็น - 60

วิธีทำ การแสดงผลการนับใน TL บน LCD นั้นทำได้โดยเปลี่ยนข้อมูล 8 บิต ให้เป็นรหัส ASCII เสียก่อน แล้วส่งให้โปรแกรมแสดงผลทาง LCD

```

ACALL LCD_SET_UP          ; กำหนดค่าเริ่มต้นให้ LCD
MOV    TMOD, #00000110B   ; เคา์เตอร์ 0 โหมด 2 , C/T = 1
MOV    TH0, # - 60        ; ให้นับพัลส์ 60 ลูก
SETB   P3.4              ; ให้ T0 เป็นอินพุต
AGAIN: SETB   TR0          ; ให้เคา์เตอร์เริ่มทำงาน
BACK:  MOV    A, TL0       ; อ่านค่าจาก TL0 ให้กับ A
ACALL  CONV              ; เปลี่ยนเป็นรหัส BCD เก็บใน R2, R3, R4
ACALL  DISPLAY           ; แสดงผลทาง LCD
JNB    TF0, BACK         ; ทำซ้ำถ้า TF0 = 0
CLR    TR0              ; ให้ TF0 = 0
CLR    TF0
SJMP   AGAIN            ; ทำซ้ำ
; แปลงข้อมูล 8 บิต เป็นรหัส ACSII
; คินค่ารหัส ASCII ใน R4, R3, R2 (R2 เป็นหลักต่ำ )

```



```
CONV:  MOV    B, #10                ; ทหารด้วย 10
        DIV    AB
        MOV    R2,B                ; เก็บค่าหลักต่ำ
        MOV    B, #10                ; ทหารด้วย 10
        DIV    AB
        ORL    A, #30H              ; ทำเป็นรหัส ASCII
        MOV    R4,A                ; เก็บหลักสูงสุด
        MOV    A,B
        ORL    A, #30H              ; ทำรหัส ASCII หลักที่สอง
        MOV    R3,A                ; เก็บค่า
        MOV    A,R2
        ORL    A, #30H              ; ทำรหัส ASCII หลักต่ำสุด
        MOV    R2,A                ; เก็บค่ารหัส ASCII
        RET
```

จากตัวอย่างที่ผ่านมาเราสามารถนำตัวนับมาประยุกต์ใช้งานเป็นนาฬิกาได้ ถ้าป้อนความถี่ 1 Hz เข้าไปจะทำให้เกิดการนับทุก ๆ วินาที เราสามารถเขียนโปรแกรมให้แสดงผลเวลาเป็นชั่วโมง นาที และวินาที ได้



คำถามท้ายบท

1. ใน 8051 มีไทเมอร์กี่ตัว
2. ไทเมอร์ใน 8051 เป็นแบบกี่บิต
3. รีจิสเตอร์ TMOD เป็นรีจิสเตอร์แบบกี่บิต
4. รีจิสเตอร์ TMOD ใช้ทำอะไร
5. รีจิสเตอร์ TMOD สามารถเข้าถึงข้อมูลระดับบิตได้หรือไม่
6. จงหาค่าความถี่และคาบเวลาในการใช้งานเป็นไทเมอร์ เมื่อใช้ XTAL ที่มีความถี่ดังนี้
 - ก. 11.0592 MHz ข. 20 MHz
 - ค. 29 MHz ง. 30 MHz
7. การใช้งานไทเมอร์โหมดใดจับเวลาได้นานที่สุด
8. ถ้าหาก XTAL มีความถี่ 11.0592 MHz บิต TF0 จะเกิดการเซตเมื่อใดถ้าทำโปรแกรมต่อไปนี้

```
MOV    TMOD, #01
MOV    TL0, #12H
MOV    TH0, #1CH
SETB   TR0
```
9. ถ้าหาก XTAL มีความถี่ 16 MHz บิต TF0 จะเซตเมื่อใด ถ้าโปรแกรมทำคำสั่งต่อไปนี้

```
MOV    TMOD, #01
MOV    TL0, #12H
MOV    TH0, #1CH
SETB   TR0
```
10. ถ้าหาก XTAL มีความถี่ 11.0592 MHz บิต TF0 จะเซตเมื่อใด ถ้าโปรแกรมทำคำสั่งต่อไปนี้

```
MOV    TMOD, #01
MOV    TL0, #10H
MOV    TH0, #0F2H
SETB   TR0
```
11. ถ้าหาก XTAL มีความถี่ 11.0592 MHz จงหาค่า TH1 และ TL1 ถ้าต้องการใช้งานไทเมอร์ 1 โหมด 1 จับเวลา 2 ms

บทที่

๑

การสื่อสารข้อมูล แบบอนุกรม

ระบบคอมพิวเตอร์สามารถส่งข้อมูลได้สองรูปแบบคือแบบขนาน (parallel) ซึ่งจะส่งข้อมูลทุกบิตออกไปพร้อมกัน และแบบอนุกรม (serial) ซึ่งจะส่งข้อมูลออกไปครั้งละบิต การส่งข้อมูลแบบขนานนี้จะต้องใช้สายในการส่งข้อมูลจำนวนมากซึ่งไม่เหมาะกับการส่งระยะไกล โดยมากแล้วจะใช้ในการส่งระยะใกล้ เช่น ระหว่างไมโครโพรเซสเซอร์กับฮาร์ดดิสก์ เครื่องพิมพ์ เป็นต้น ส่วนการส่งข้อมูลระยะไกล ๆ ควรใช้การส่งข้อมูลแบบอนุกรมโดยจะส่งข้อมูลออกไปครั้งละบิต การส่งข้อมูลแบบอนุกรมนั้นจะส่งได้ช้ากว่าการส่งแบบขนานแต่ค่าใช้จ่ายจะต่ำกว่า

ในไมโครโพรเซสเซอร์ 8051 จะมีพอร์ตอนุกรมอยู่ภายในซึ่งจะรับส่งข้อมูลได้สองทิศทางในพอร์ตเดียวกัน สำหรับในบทนี้จะกล่าวถึงการสื่อสารข้อมูลของ 8051 โดยหัวข้อ 9.1 จะกล่าวถึงการสื่อสารทั่วไป ในหัวข้อ 9.2 จะกล่าวถึงการสื่อสารแบบ RS-232 โดยจะนำไอซี MAX232 มาร่วมใช้งานด้วย

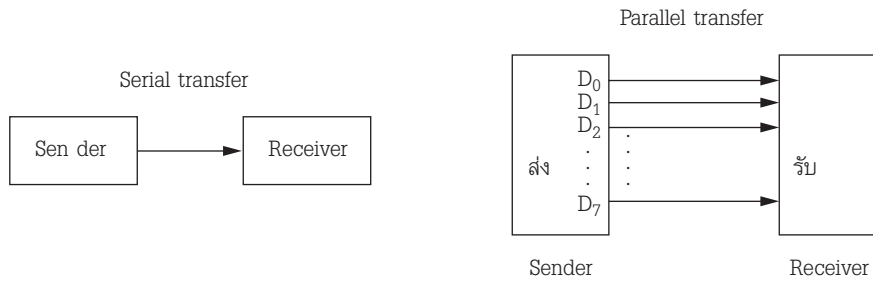


9.1 พื้นฐานการสื่อสารอนุกรม

เมื่อไมโครโพรเซสเซอร์ต้องการติดต่อกับอุปกรณ์ภายนอก ตัวมันจะส่งข้อมูลออกมาเป็นไบต์ หรือ 8 บิต แต่เนื่องจากไมโครโพรเซสเซอร์มีบัสข้อมูลขนาด 8 บิต การโอนถ่ายข้อมูลต่าง ๆ จะทำแบบขนาน ถ้าต้องการส่งข้อมูลออกไปแบบอนุกรมจะต้องเปลี่ยนข้อมูลแบบขนานนี้ให้เป็นข้อมูลอนุกรมเสียก่อนแล้วจึงส่งออกไป ส่วนการรับข้อมูลแบบอนุกรมนี้อาจรับข้อมูลเข้ามาครั้งละบิตและเปลี่ยนข้อมูลให้เป็นข้อมูลแบบขนาน



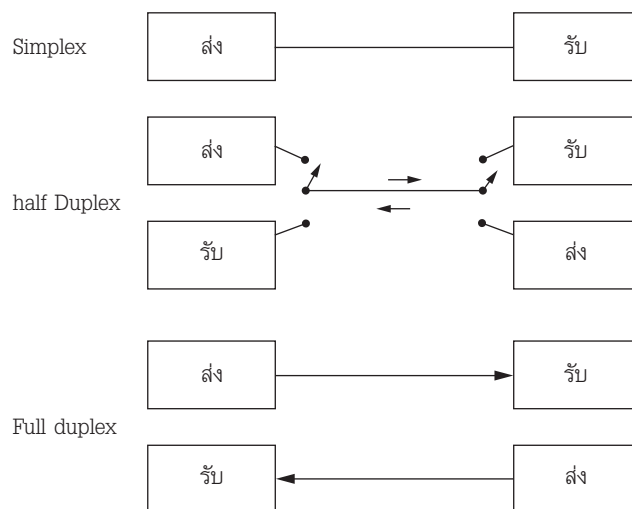
เพื่อส่งให้ไมโครโปรเซสเซอร์ประมวลผลต่อไป ในระบบคอมพิวเตอร์ตัวที่เปลี่ยนข้อมูลอนุกรมเป็นขนาน และเปลี่ยนข้อมูลขนานเป็นอนุกรมจะใช้อุปกรณ์ที่มีชื่อว่า UART (Universal asynchronous receiver - transmitter)



รูปที่ 9.1 ลักษณะการรับส่งข้อมูลแบบอนุกรมและแบบขนาน

รูปแบบการสื่อสารข้อมูล

การสื่อสารข้อมูลระหว่างตัวรับและตัวส่งนั้นมีหลายวิธี ถ้าหากตัวส่งทำหน้าที่ส่งอย่างเดียวและตัวรับทำหน้าที่รับอย่างเดียวจะเรียกว่าการสื่อสารแบบซิมเพล็กซ์ (simplex) เช่น การส่งข้อมูลระหว่างคอมพิวเตอร์กับเครื่องพิมพ์ แต่ถ้าหากตัวรับและตัวส่งสามารถรับและส่งข้อมูลได้แต่ทำในเวลาต่างกันเรียกว่าการสื่อสารแบบ ฮาล์ฟดูเพล็กซ์ (half duplex) เช่นการสื่อสารแบบเครื่องอินเตอร์คอมที่มีสายส่งสัญญาณได้ทางเดียว แต่ถ้าหากตัวรับและตัวส่งสามารถรับส่งข้อมูลได้สองทิศทางในเวลาเดียวกันจะเรียกว่าเป็นการสื่อสารแบบฟูลดูเพล็กซ์ (full duplex) การสื่อสารแต่ละแบบแสดงได้ดังรูปที่ 9.2



รูปที่ 9.2 การรับส่งข้อมูลแบบ Simplex, Half duplex และ Full duplex



สำหรับวิธีในการส่งข้อมูลแบบอนุกรมมีสองวิธีคือ การส่งแบบเข้าจังหวะเวลา (synchronous) และการส่งแบบไม่เข้าจังหวะเวลา (asynchronous) โดยการส่งแบบเข้าจังหวะเวลาจะต้องมีการส่งสัญญาณนาฬิกาพร้อมไปด้วยเพื่อควบคุมการรับส่งข้อมูล แต่การรับส่งแบบไม่เข้าจังหวะเวลาจะไม่ต้องมี แต่จะใช้เวลากำหนดอัตราเร็วในการรับส่งข้อมูลให้มีค่าเท่ากันที่เรียกว่า อัตราบอด หรือบอดเรต (baud rate) ซึ่งมีหน่วยเป็นบิตต่อวินาที (bit per second : bps)

การส่งข้อมูลแบบซิงโครนัส

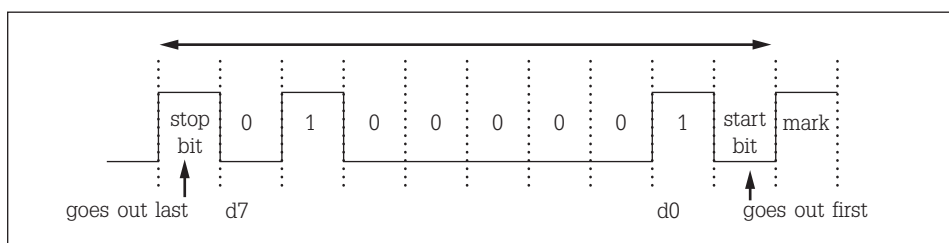
เมื่ออุปกรณ์สองตัวจะสื่อสารข้อมูลซึ่งกันและกันจะต้องกำหนดรูปแบบกฎเกณฑ์การสื่อสารซึ่งกันและกันเพื่อให้เข้าใจกันได้ รูปแบบการรับส่งข้อมูลนี้เรียกว่าโปรโตคอล (protocol) ซึ่งจะเป็นตัวบอกลักษณะของข้อมูล การเริ่มข้อมูล การจบข้อมูล เป็นต้น

การรับส่งข้อมูลแบบซิงโครนัสจะประกอบด้วย 4 ส่วนดังนี้

1. บิตเริ่มต้น (start bit)
2. ข้อมูลอนุกรม
3. บิตตรวจสอบความถูกต้อง
4. บิตสุดท้าย (stop bit)

บิตเริ่มต้นและบิตสุดท้าย

การส่งข้อมูลแบบอะซิงโครนัสนั้นจะส่งข้อมูลออกไปเป็นชุดเรียกว่า เฟรม (framing) ภายในเฟรมจะประกอบด้วยข้อมูลหรือรหัส ASCII ก็ได้ ในแต่ละเฟรมจะเริ่มต้นด้วยบิตเริ่มต้น (start bit) ที่จะบอกว่าสิ่งที่ตามมาคือข้อมูล และจะจบข้อมูลด้วยบิตสุดท้าย (stop bit) ที่จะบอกว่าข้อมูลในเฟรมนั้น ๆ สิ้นสุดลงแล้ว ในการส่งข้อมูลแบบนี้ถ้าหากยังไม่มีข้อมูลระดับลอจิกที่สายส่งจะเป็นลอจิก “1” เรียกว่าสภาวะรอ (waiting stage) ถ้าหากมีข้อมูลส่ง สัญญาณจะเป็นลอจิก “0” ในช่วงเวลาหนึ่ง บิตข้อมูลบิตนี้เรียกว่าบิตเริ่มต้น จากนั้นจะตามด้วยข้อมูล ถ้าหากจะส่งรหัส ASCII ของตัว “A” จะเป็นดังรูปที่ 9.3 โดยส่งค่าข้อมูล 8 บิต (01000001) ออกไปจากนั้นจะจบด้วยบิตปิดท้ายหรือบิตหยุดที่มีภาวะเป็นลอจิก “1” โดยบิตหยุดนี้อาจเป็นลอจิก “1” ในช่วงเวลา 1 บิต, 1.5 บิต หรือ 2 บิตก็ได้ขึ้นกับการกำหนดลักษณะการส่งข้อมูล



รูปที่ 9.3 เฟรมรหัส ASCII ของตัว “A”



สำหรับข้อมูลที่รับส่งกันนั้นอาจมีขนาด 7 บิต หรือ 8 บิตก็ได้ ขึ้นอยู่กับการออกแบบระบบ และข้อมูลสามารถทำการตรวจสอบความถูกต้องได้ระดับหนึ่งเรียกว่า การตรวจสอบบิตพาริตี (parity bit) ซึ่งจะเป็นบิตข้อมูลที่จะเพิ่มเข้าไปก่อนบิตปิดท้าย โดยการตรวจสอบบิตพาริตีนี้สามารถกำหนดเป็นพาริตีคี่ (odd) หรือ พาริตีคู่ (even) ก็ได้ การกำหนดบิตพาริตีคี่หมายความว่าบิตข้อมูลทั้ง 7 หรือ 8 บิต ถ้าหากรวมกับบิตพาริตีแล้วจะต้องเป็นลอจิก “1” คี่บิต ถ้าหากเป็นพาริตีคู่จะต้องรวมแล้วได้ลอจิก “1” คู่บิต ตัวอย่างเช่น ถ้าหากส่งรหัส ASCII ของตัว “A” ออกไป ซึ่งข้อมูลจะเป็น 0100 0001 ถ้ากำหนดเป็นพาริตีคู่ บิตพาริตีต้องมีลอจิกเป็น “0” แต่ถ้ากำหนดเป็นพาริตีคี่บิตพาริตีต้องเป็นลอจิก “1” การสร้างบิตพาริตีนี้จะถูกสร้างจากภาคส่งของ UART ซึ่งเราสามารถโปรแกรมได้ว่าการส่งนั้นจะเป็นแบบพาริตีคี่หรือคู่ หรือไม่มีพาริตี

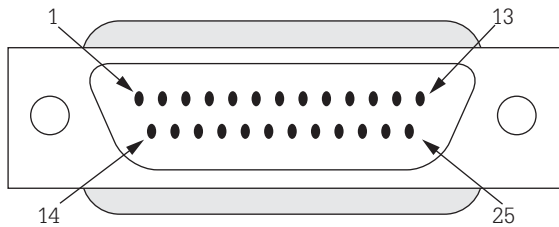
อัตราการรับส่งข้อมูล

ความเร็วในการรับส่งข้อมูลแบบอนุกรมจะบอกเป็นจำนวนบิตต่อวินาที (bits per second : bps) ที่เรียกว่าบอดเรต (baud rates) พอร์ตอนุกรมของ MCS-51 สามารถติดต่อกับเครื่องคอมพิวเตอร์ส่วนบุคคลได้ แต่ต้องกำหนดอัตราเร็วให้เท่ากัน ในการสื่อสารทางพอร์ตอนุกรม RS-232 ของคอมพิวเตอร์ PC ได้กำหนดอัตราเร็วไว้หลายค่าตั้งแต่ 100 ถึง 9600 bps สำหรับคอมพิวเตอร์ที่ใช้ไมโครโปรเซสเซอร์เพนเทียมสามารถส่งข้อมูลได้ความเร็วสูงถึง 56 กิโลบิตต่อวินาที

มาตรฐาน RS 232

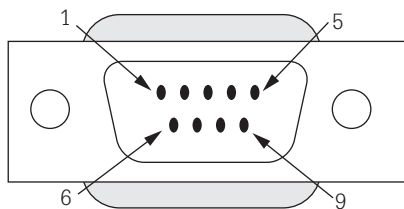
การสื่อสารแบบอนุกรมกับคอมพิวเตอร์ PC มักจะใช้รูปแบบมาตรฐาน RS-232 ซึ่งกำหนดโดย Electronics Industries Association หรือ EIA ในปี 1960 และได้มีการแก้ไขมาตรฐานในปี 1963, 1965 และ 1969 เรียกว่า RS-232A, RS-232B และ RS-232C ตามลำดับ ระดับแรงดันของลอจิกที่ใช้ในการสื่อสาร RS-232 นั้นลอจิก “1” จะแทนด้วยแรงดัน -3 ถึง -25 โวลต์ ส่วนลอจิก “0” จะแทนด้วยแรงดัน +3 ถึง +25 โวลต์ แรงดันในช่วง +3 จะไม่ถูกกำหนดให้ใช้งานซึ่งจะเห็นว่าแรงดันดังกล่าวไม่สามารถใช้กับไมโครคอนโทรลเลอร์ตระกูล MCS-51 ได้ โดยทั่วไปแล้วถ้าหากต้องการให้ไมโครคอนโทรลเลอร์ติดต่อกับคอมพิวเตอร์ PC ตามมาตรฐาน RS-232 จะต้องออกแบบวงจรอิเล็กทรอนิกส์เพิ่มเติม แต่ในปัจจุบันจะใช้ไอซี MAX232 ทำหน้าที่เปลี่ยนระดับแรงดันทางลอจิกให้อยู่ในมาตรฐาน RS-232 แต่ถ้าหากจะให้ไมโครคอนโทรลเลอร์สองตัวสื่อสารกันจะไม่ใช่มาตรฐานนี้ก็ได้

ในคอมพิวเตอร์ PC จะมีขั้วต่อ RS-232 หรือที่เรียกว่า คอนเนกเตอร์ (connector) อยู่สองแบบ คือ ขั้วต่อแบบ DB-25 และขั้วต่อแบบ DP-9 ขั้วต่อแบบ DB-25 และขาต่าง ๆ แสดงได้ดังรูปที่ 9.4 ส่วนขั้วต่อแบบ DB-9 และขาต่าง ๆ แสดงได้ดังรูปที่ 9.5



Pin	Description
1	Protective ground
2	Transmitted data (TxD)
3	Received data (R x D)
4	Request to send (RTS)
5	Clear to send (CTS)
6	Data set ready (DSR)
7	Signal ground (GND)
8	Data carrier detect (DCD)
9/10	Reserve for data testing
11	Unassigned
12	Secondary data carrier detect
13	Secondary clear to send
14	Secondary transmitted data
15	Transmit signal element timing
16	Secondary received data
17	Receive signal element timing
18	Unassigned
19	Secondary request to send
20	Data terminal ready ($\overline{\text{DTR}}$)
21	Signal quality detector
22	Ring indicator
23	Data signal rate select
24	Transmit signal element timing
25	Unassigned

รูปที่ 9.4 ขั้วต่อแบบ DB-25 และหน้าที่ของขาต่าง ๆ



Pin	Description
1	Data carrier detect ($\overline{\text{DCD}}$)
2	Received data (RxD)
3	Transmitted data (TxD)
4	Data terminal ready (DTR)
5	Signal ground (GND)
6	Data set ready ($\overline{\text{DSR}}$)
7	Request to send (RTS)
8	Clear to send (CTS)
9	Ring indicator (RI)

รูปที่ 9.5 ขั้วต่อแบบ DB-9 และหน้าที่ของขาต่าง ๆ

ในระบบ RS-232 มีขาต่าง ๆ ที่สำคัญดังนี้

1. DTR (data terminal ready) เป็นสายสำหรับการทำแฮนด์เช็กจาก DTE ไปยัง DCE
2. DSR (data set ready) เป็นสายสำหรับการทำแฮนด์เช็กจาก DCE ไปยัง DTE
3. RTS (request to send) เมื่ออุปกรณ์ DTE ต้องการส่งข้อมูลมันจะส่งสัญญาณที่ขานี้เป็นลอจิก

“Low” ออกไป



4. CTS (clear to send) เป็นสายสัญญาณสำหรับการทำแฮนด์เช็กจาก DCE ไปยัง DTE
5. DCD (carrier detect, หรือ data carrier detect)
6. RI (ring indicator) เป็นสายที่ใช้โดยโมเด็มเพื่อบอกว่าตัวมันเองได้รับสัญญาณเรียกเข้ามา

ในคอมพิวเตอร์ส่วนบุคคล IBM ที่ใช้ไมโครโพรเซสเซอร์ตระกูล X86 จะมีพอร์ตสื่อสารอนุกรมอยู่สองพอร์ตเรียกว่าพอร์ต COM1 และ COM2 โดยใช้คอนเนกเตอร์แบบ DB-25 และ DB-9 เราสามารถนำพอร์ตทั้งสองนี้มาเชื่อมต่อกับพอร์ตอนุกรมของ 8051 ได้ ดังตัวอย่างที่จะพบในโปรแกรมต่อไป



คำถาม ?

1. จงเปรียบเทียบความเร็วและราคาของการสื่อสารแบบอนุกรมและแบบขนาน
2. จริงหรือไม่ที่การส่งข้อมูลระหว่างคอมพิวเตอร์และเครื่องพิมพ์เป็นแบบ duplex
3. จริงหรือไม่การส่งข้อมูลแบบ full duplex จะใช้สายสองชุดคือ ชุดส่งและชุดรับ
4. การใช้บิตเริ่มต้นและบิตหยุดเป็นวิธีการส่งข้อมูลแบบซิงโครนัส หรือแบบอะซิงโครนัส
5. ถ้าหากจะส่งรหัส ASCII ของตัว “E” (01000101) แบบไม่มีพาริตีบิต และมีหนึ่งบิตหยุด จงแสดงลำดับการส่งข้อมูลแบบอนุกรมแต่ละบิต
6. ถ้าหากส่งข้อมูลแบบอนุกรมด้วยความเร็ว 9600 bps จงคำนวณเวลาการส่งข้อมูลจำนวน 10,000 ตัวอักษร
7. สัญญาณลอจิกระดับ TTL สอดคล้องกับสัญญาณ RS-232 หรือไม่
8. ในระบบ RS-232 จะใช้แรงดันไฟฟ้าเท่าใดแทนลอจิก “0”
9. จริงหรือไม่ใน 8051 มีชิป UART อยู่ภายใน
10. ไมโครคอมพิวเตอร์ที่ใช้ไมโครโพรเซสเซอร์ตระกูล X86 มีพอร์ตอนุกรมกี่พอร์ต



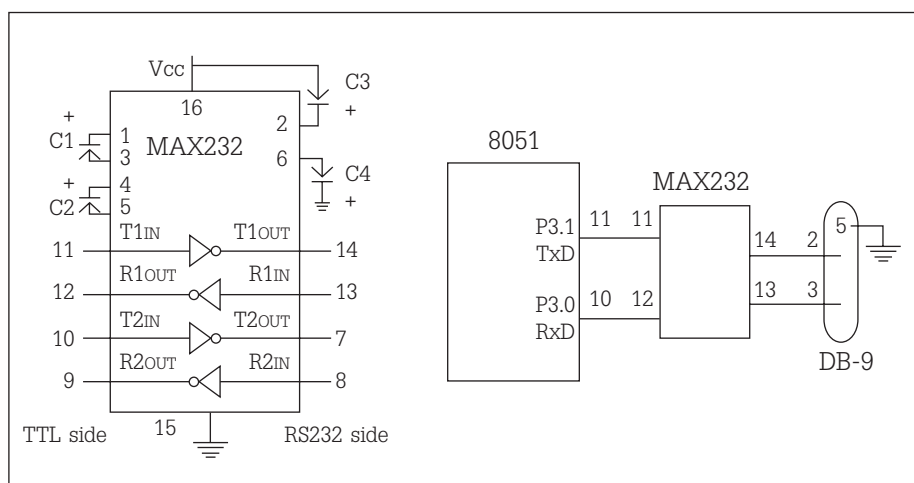
9.2 การต่อ MCS-51 กับ RS-232

ในหัวข้อนี้จะกล่าวถึงการต่อพอร์ตอนุกรมของ MCS-51 กับมาตรฐาน RS232 แต่เนื่องจากระดับสัญญาณของการสื่อสาร RS-232 ไม่เป็นไปตามมาตรฐานแรงดัน TTL เราสามารถนำชิป MAX232 มาช่วยปรับระดับแรงดันให้กับ 8051 ได้



ขา RxD และ TxD ของ 8051

พอร์ตที่ใช้ในการรับส่งข้อมูลแบบอนุกรมของ 8051 จะรับส่งออกมาทางขา RxD และ TxD ซึ่งอยู่ในพอร์ต 3 (P3.0 และ P3.1) โดย TxD จะเป็นขาที่ 9 และ RxD จะเป็นขาที่ 10 การรับส่งข้อมูลกับมาตรฐาน RS-232 เราจะต้องติดต่อผ่านขาทั้งสองนี้โดยสามารถนำไอซี MAX232 มาเชื่อมต่อได้ ดังวงจรในรูปที่ 9.6 ไอซีตัวนี้จะใช้ไฟเลี้ยง +5V เท่ากับไฟเลี้ยงของไมโครคอนโทรลเลอร์แต่สามารถยกแรงดันตั้งแต่ -25 V ถึง +25 V ได้โดยไม่ต้องใช้แหล่งจ่ายไฟแบบคู่ (dual power supplies)



รูปที่ 9.6 โครงสร้างภายใน MAX232 และการเชื่อมต่อกับ MCS-51



9.3 การโปรแกรมการสื่อสารแบบอนุกรมของ 8051

ในหัวข้อนี้จะกล่าวถึงรีจิสเตอร์ที่ใช้สื่อสารข้อมูลแบบอนุกรมของ 8051 และการเขียนโปรแกรมพื้นฐานในการรับส่งข้อมูล ถ้าหากคอมพิวเตอร์ IBM PC ต้องการติดต่อกับระบบ 8051 จะต้องสื่อสารแบบอนุกรมผ่านทาง COM พอร์ต โดยความเร็วที่ใช้ในการสื่อสารแสดงได้ดังตารางที่ 9.1 โดยเราสามารถเขียนโปรแกรมเลือกอัตราเร็วได้ หรือเซตในโปรแกรม WINDOWS ตั้งแต่รุ่น 3.1 ขึ้นไป



ตารางที่ 9.1 บอດเรตของคอมพิวเตอร์ PC

110
150
300
600
1200
2400
4800
9600
19200

บอດเรตของ 8051

ใน 8051 สามารถรับส่งข้อมูลได้ด้วยบอດเรตหลายค่าโดยเราสามารถเลือกได้โดยโปรแกรมเข้าไปทาง ไทเมอร์ 1 และความเร็วที่ได้จะสัมพันธ์กับความถี่ของ XTAL ที่ต่อกับระบบด้วย

จากบทที่ผ่านมาเราทราบว่าแมชชีนไซเคิลของ 8051 จะเท่ากับความถี่ของ XTAL ที่ต่ออยู่กับระบบหารด้วย 12 ดังนั้นถ้า XTAL มีความถี่ 11.0592 MHz ความถี่ของแมชชีนไซเคิลจะเท่ากับ 921.0 kHz ตัว UART ที่เป็นอุปกรณ์สื่อสารแบบอนุกรมที่อยู่ภายใน 8051 จะนำความถี่ของแมชชีนไซเคิลนี้มาหารด้วยค่า 32 หรือมากกว่านี้ขึ้นกับค่าที่โปรแกรมให้กับไทเมอร์ 1 ดังนั้นค่าอัตราการรับส่งข้อมูลหรือบอດเรตสูงสุดของ 8051 จะเท่ากับ 921.6 kHz หารด้วย 32 เท่ากับ 28,800 Hz ถ้าหากเราต้องการค่าอัตราบอດเรตค่าอื่น ๆ ทำได้โดยการโปรแกรมเข้าไปในไทเมอร์ 1 ตัวอย่างเช่น ถ้าให้ไทเมอร์ 1 ทำงานในโหมด 2 (auto - reload) อัตราบอດเรตและค่าในรีจิสเตอร์จะเป็นไปตามตารางที่ 9.2

ตารางที่ 9.2 ค่าของรีจิสเตอร์ TH1 ที่ใช้กำหนดบอດเรต

บอດเรต	TH1(Decimal)	TH1(Mex)
9600	-3	FD
4800	-6	FA
2400	-12	F4
1200	-24	E8

รีจิสเตอร์ SBUF

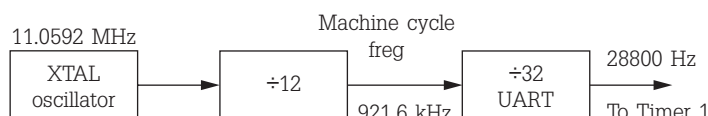
รีจิสเตอร์ตัวนี้เป็นรีจิสเตอร์บัฟเฟอร์พอร์ตอนุกรม (Serial data buffer register) ในการรับส่งข้อมูลจะต้องกระทำผ่านรีจิสเตอร์ตัวนี้ การส่งข้อมูลออกทางขา TxD จะต้องเขียนข้อมูลลงในรีจิสเตอร์ SBUF จากนั้น UART จะทำหน้าที่ส่งข้อมูลออกไปที่ละบิตส่วนการรับข้อมูลที่เข้ามาทางขา RxD ข้อมูลจะถูกเก็บอยู่ในรีจิสเตอร์ SBUF นี้ และจะต้องอ่านข้อมูลจากรีจิสเตอร์ SBUF ออกมาใช้งานดังตัวอย่างคำสั่งต่อไปนี้



```

MOV  SBUF, #D           ; สำหรับ ASCII ของตัว D ไปให้ SBUF
MOV  SBUF, A             ; ส่งค่าในรีจิสเตอร์ A ให้กับ SBUF
MOV  A, SBUF             ; อ่านค่าจากรีจิสเตอร์ SBUF ให้รีจิสเตอร์ A

```



รีจิสเตอร์ควบคุมพอร์ตอนุกรม

รีจิสเตอร์ตัวนี้ชื่อว่า Serial Control Register หรือ SCON เป็นรีจิสเตอร์แบบ 8 บิต ที่เข้าถึงข้อมูลระดับบิตได้ ใช้ในการโปรแกรมค่าบิตเริ่มต้น บิตสุดท้าย เลือกโหมดของการสื่อสาร เป็นต้น ชื่อบิตแต่ละบิต และ ดังรูปที่ 10.9 สำหรับความหมายแต่ละบิตเป็นดังนี้

SM0	SM1	SM2	REN	TB8	RB8	TI	RI
-----	-----	-----	-----	-----	-----	----	----

รูปที่ 9.7

SM0 - SM1: เป็นบิตที่ใช้เลือกโหมดการสื่อสารซึ่งมีอยู่ด้วยกัน 4 โหมด ถ้ามีค่าเป็น 00 จะเป็น การสื่อสารในโหมด 0 ซึ่งจะทำงานลักษณะของซีเรียลรีจิสเตอร์ ถ้าเป็น 01 จะเป็นโหมด 1 ทำงานรับส่งข้อมูลแบบ 8 บิต ที่สามารถเลือกอัตราบิตได้ ถ้าเป็น 10 จะทำงานโหมด 2 รับส่งข้อมูลแบบ 9 บิต มีอัตราบิตคงที่ และถ้าเป็น 11 จะทำงานโหมด 3 รับส่งข้อมูลแบบ 9 บิต ที่เลือกอัตราบิตได้

SM2 : เป็นบิตที่ใช้อำนวยการสื่อสารแบบมัลติโปรเซสเซอร์

REN : หรือ receive enable เป็นบิตที่ใช้อำนวยการรับข้อมูลของพอร์ตอนุกรม ถ้าบิตนี้ถูกเซตเป็นลอจิก “1” หมายความว่า จะรับข้อมูลเข้ามาได้

TB8 : หรือ transfer bit 8 เป็นบิตที่ใช้เก็บข้อมูลที่ 9 ที่จะส่งออกไปทางพอร์ตอนุกรมเมื่อทำงานใน โหมด 2 และโหมด 3

RB8 : หรือ receive bit 8 เป็นบิตที่ใช้เก็บข้อมูลที่ 9 เมื่อรับข้อมูลเข้ามาทางพอร์ตอนุกรมเมื่อ ทำงานในโหมด 2 และโหมด 3

TI : หรือ transmit interrupt เมื่อ 8051 ส่งข้อมูลหนึ่งไบต์ออกไปทางพอร์ตอนุกรมเรียบร้อยแล้ว บิตนี้จะมีลอจิกเป็น “1” ซึ่งสามารถนำไปอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ได้โดยบิตนี้จะเซตเมื่อมีการส่งบิต หยุดของข้อมูลออกไปและต้องเคลียร์ด้วยซอฟต์แวร์

RI : หรือ receive interrupt เมื่อ 8051 รับข้อมูลเข้ามาทางพอร์ตอนุกรมเรียบร้อยแล้วบิตนี้จะถูก เซตเป็นลอจิก “1” ซึ่งสามารถนำไปอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ได้ โดยบิตนี้จะถูกเซตเมื่อบิตหยุดของ ข้อมูลได้ส่งเข้ามาแล้ว และต้องเคลียร์ด้วยซอฟต์แวร์



การโปรแกรมให้ 8051 ส่งข้อมูล

การเขียนโปรแกรมให้ไมโครคอนโทรลเลอร์ส่งไบต์ข้อมูลออกไปสามารถทำได้ดังนี้

1. โหลดข้อมูลเข้ารีจิสเตอร์ TMOD ด้วยค่า 20H เพื่อให้ทำงานในโหมด 2 (8-bit auto-reload) และกำหนดด้วยบอดเรต
2. โหลดค่า TH1 ด้วยค่าใดค่าหนึ่งตามค่าบอดเรตที่กำหนดในตาราง 10.4 (ถ้าหาก XTAL มีความถี่ 11.0592 MHz)
3. โหลดข้อมูลเข้ารีจิสเตอร์ SCON ด้วยค่า 50H เพื่อให้ทำงานในโหมด 1 และส่งข้อมูลแบบ 8 บิต มีบิตเริ่มต้นและบิตหยุด
4. ให้ไทเมอร์ 1 ทำงานด้วยการเซตบิต TR1
5. เคลียร์บิต TI ด้วยคำสั่ง CLR TI
6. นำข้อมูลที่จะส่งเขียนลงรีจิสเตอร์ SBUF
7. วนลูปทดสอบแฟลก TI ด้วยคำสั่ง JNB TI, XX ถ้าแฟลกนี้เป็น “1” หมายความว่าส่งข้อมูลออกไปแล้ว
8. 'ส่งข้อมูลไบต์ต่อไปโดยทำซ้ำในขั้นตอนที่ 5

ตัวอย่างที่ 9.1 จงเขียนโปรแกรมให้ 8051 ส่งตัวอักขระตัว “A” ออกไปด้วยความเร็ว 4800 บิตต่อวินาที

วิธีทำ	MOV	TMOD, #20H	; ไทเมอร์ 1 โหมด 2 (auto-reload)
	MOV	TH1, #-6	; บอดเรต 4800
	MOV	SCON, #50H	; ส่ง 8 บิต, stop, REN อินาเปิล
	SETB	TR1	; ให้ไทเมอร์ 1 เริ่มทำงาน
AGAIN:	MOV	SBUF, #“A”	; ส่งตัว A ออกไป
HERE:	JNB	TI, HERE	; รอจนส่งออกไปทุกบิต
	CLR	TI	; เคลียร์บิต TI เพื่อส่งตัวต่อไป
	SJMP	AGAIN	; วนลูป

ตัวอย่างที่ 9.2 จงเขียนโปรแกรมให้ส่งคำว่า YES ออกไปทางพอร์ตอนุกรมแบบ 8 บิต บิตหยุด 1 บิต ด้วยความเร็ว 9600 บิตต่อวินาที

วิธีทำ	MOV	TMOD, #20H	; ไทเมอร์ 1 โหมด 2
	MOV	TH1, #-3	; บอดเรต 9600
	MOV	SCON, #50H	; ส่ง 8 บิต หนึ่งบิตหยุด อินาเปิล REN
	SETB	TR1	; ให้ไทเมอร์ 1 เริ่มทำงาน



```

AGAIN:  MOV    A, # "Y"                ; ส่งตัว Y
        ACALL  TRANS
        MOV    A, # "E"                ; ส่งตัว E
        ACALL  TRANS
        MOV    A, # "S"                ; ส่งตัว S
        ACALL  TRANS
        SJMP   AGAIN                  ; ทำซ้ำ
; โปรแกรมย่อยส่งข้อมูลพอร์ตอนุกรม
TRANS:  MOV    SBUF, A                  ; โหลดข้อมูลลงบัฟเฟอร์
HERE:   JNB     TI, HEREE               ; รอจนส่งออกไปทุกบิต
        CLR     TI                      ; เคลียร์บิตแฟลกเพื่อส่งไบต์ต่อไป
        RET

```

ความสำคัญของบิตแฟลก TI

การส่งข้อมูลแบบอนุกรมนี้เราจะพบว่าบิตแฟลก TI จะเป็นตัวบอกว่าข้อมูลได้ส่งไปเรียบร้อยแล้วหรือยัง ซึ่งขั้นตอนการส่งข้อมูลจะเป็นดังนี้

1. เขียนไบต์ข้อมูลที่จะส่งในรีจิสเตอร์ SBUF
2. 'งบิตเริ่มต้นออกไป
3. ข้อมูล 8 บิต ถูกส่งออกไปครั้งละบิต
4. 'งบิตหยุดตามออกไป เมื่อส่งบิตหยุดไปแล้วแฟลก TI จะเป็นลอจิก "1"
5. การส่งข้อมูลจะต้องเฝ้ามองบิตแฟลก TI นี้ ถ้าบิตนี้เป็นลอจิก "1" แสดงว่าส่งข้อมูลไปแล้ว
6. หลังจากส่งไบต์ต่อไปจะต้องเคลียร์บิตแฟลก TI ด้วยคำสั่ง "CLR TI" ให้เป็นลอจิก "0" ก่อนที่จะเขียนข้อมูลใหม่ลงในรีจิสเตอร์ SBUF

การเขียนโปรแกรมรับข้อมูล

การโปรแกรมให้ 8051 รับไบต์ข้อมูลสามารถทำได้ดังนี้

1. กำหนดให้รีจิสเตอร์ TMOD มีค่าเป็น 20H เพื่อใช้ไทเมอร์ 1 โหมด 2 (8-bit auto-reload) กำหนดค่าบอดเรต
2. โหลดรีจิสเตอร์ TH1 ด้วยค่าในตารางที่ 9.2 เพื่อกำหนดค่าบอดเรต
3. โหลดรีจิสเตอร์ SCON ด้วยค่า 50H เพื่อใช้พอร์ตอนุกรมโหมด 1
4. เซตบิต TR1 เพื่อให้ไทเมอร์ 1 เริ่มทำงาน
5. เคลียร์บิต RI ด้วยคำสั่ง CLR RI
6. เฝ้ามองบิตแฟลก RI ด้วยคำสั่ง JNB RI,XX ถ้าค่าเป็นลอจิก "1" แสดงว่ามีข้อมูลเข้ามาแล้ว



7. เมื่อค่า R1 เป็นลอจิก “1” ให้โหลดข้อมูลจากรีจิสเตอร์ SBUF มาใช้งาน
8. กลับไปขั้นตอนที่ 5 เพื่อรับค่าไบต์ต่อไป

ตัวอย่างที่ 9.3 จงเขียนโปรแกรมให้ 8051 รับข้อมูลที่เข้ามาทางพอร์ตอนุกรมและส่งออกไปทางพอร์ต P1 โดยเซตบอดเรตเท่ากับ 4800 รับข้อมูลแบบ 8 บิต และมีบิตหยุดหนึ่งบิต

วิธีทำ	MOV	TMOD, #20H	; ไทเมอร์ 1 โหมด 2
	MOV	TH1, # -6	; บอดเรต 4800
	MOV	SCON, #50H	; รับแบบ 8 บิต
	SETB	TR1	; ให้ไทเมอร์ 1 เริ่มทำงาน
HERE:	JNB	RI, HERE	; เฝ้ามองบิต RI
	MOV	A, SBUF	; เก็บข้อมูลที่รับเข้ามาใน A
	MOV	P1, A	; ส่งข้อมูลออกพอร์ต P1
	CLR	RI	; เคลียร์บิตแฟล็กเพื่อรับไบต์ต่อไป
	SJMP	HERE	

ตัวอย่างที่ 9.4 ถ้าหากพอร์ตอนุกรมของ 8051 ต่ออยู่กับ COM พอร์ตของ IBM PC และบนเครื่อง IBM ได้รับโปรแกรม terminal.exe สำหรับรับส่งข้อมูล สำหรับตัว 8051 นั้นพอร์ต P1 ต่ออยู่กับหลอด LED พอร์ต P2 ต่ออยู่กับสวิตช์ จงเขียนโปรแกรมให้ 8051 ทำงานดังนี้

- ก. ส่งข้อมูลคำว่า “We Are Ready” ให้กับคอมพิวเตอร์ PC
 - ข. รับข้อมูลจาก PC และแสดงผลทาง LED ที่ต่ออยู่ทางพอร์ต P1
 - ค. อ่านข้อมูลจากสวิตช์ที่ต่ออยู่ทางพอร์ต P2 และส่งให้คอมพิวเตอร์ PC
- โดยการสื่อสารทั้งหมดทำด้วยความเร็ว 4800 บิตต่อวินาที

	ORG	0000	
	MOV	P2, #0FFH	; โปรแกรมให้ P2 เป็นอินพุต
	MOV	TMOD, #20H	; ไทเมอร์ 1 ทำงานโหมด 2
	MOV	TH1, #0FAH	; บอดเรต 4800
	MOV	SCON, #50H	; 8 บิต, stop, อินาเบิล REN
	SETB	TR1	; ให้ไทเมอร์ 1 เริ่มทำงาน
	MOV	DPTR, #MYDATA	; โหลดตำแหน่งข้อมูล
H_1:	CLR	A	
	MOV	A, @A + DPTR	; อ่านตัวอักขระออกมา
	JZ	B_1	; ถ้าเป็นตัวสุดท้ายให้กระโดด

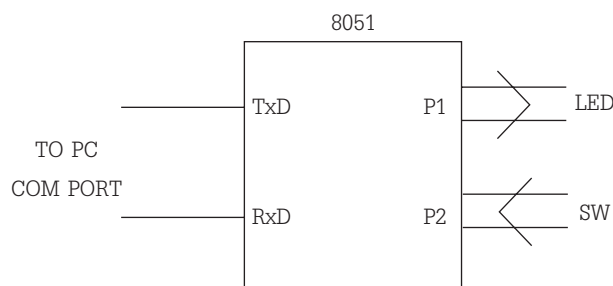


```

                ACALL SENP                ; เรียกโปรแกรมย่อยส่งข้อมูล
                JNC  DPTR                ; ชั่วตัวต่อไป
                SJMP H_1                ; ทำซ้ำ
B_1:            MOV  A, P2                ; อ่านค่าจากพอร์ต P2
                ACALL SEND                ; ส่งข้อมูลออกไป
                ACALL RECV                ; เรียกโปรแกรมย่อยรับข้อมูล
                MOV  P1, A                ; แสดงผลทาง LED
                SJMP B_1                ; ทำซ้ำ
; โปรแกรมย่อยส่งข้อมูล
SEND:           MOV  SBUF, A              ; เขียนข้อมูลบัฟเฟอร์
H_2:            JNB  TI, H_2              ; ฝ้ามองบิต TI
                CLR  TI                  ; เคลียร์บิตเพื่อส่งไบต์ต่อไป
                RET                       ; จบโปรแกรมย่อย
; โปรแกรมย่อยรับส่งข้อมูล
RECV:           JNB  RI, RECV              ; รอรับข้อมูล
                MOV  A, SBUF              ; อ่านข้อมูลที่รับเข้ามา
                CLR  RI                  ; เคลียร์แฟลกเพื่อรับข้อมูลต่อไป
                RET                       ; จบโปรแกรมย่อย

MYDATA:         DB  "We Are Ready" , 0
                END

```



ความสำคัญของบิตแฟลก RI

เมื่อ 8051 รับข้อมูลเข้ามาทางขา RxD การทำงานต่าง ๆ จะเป็นดังนี้

1. รับบิตเริ่มต้นเข้ามาเป็นบิตแรก
2. รับข้อมูลอีก 8 บิต เข้ามาครั้งละบิต เมื่อข้อมูลเข้ามาหมดแล้วจะเก็บอยู่ในรีจิสเตอร์ SBUF
3. เมื่อรับบิตหยุดเข้ามา บิตแฟลก RI จะถูกเซตเป็นลอจิก "1"



4. ตรวจสอบบิตแฟลก RI ถ้าบิตนี้เป็นลอจิก “1” หมายความว่าข้อมูลรับเข้ามาหมดแล้ว
5. หลังจากอ่านข้อมูลจากรีจิสเตอร์ SBUF ไปแล้วให้เคลียร์บิตแฟลก RI ด้วยคำสั่ง CLR RI ให้เป็นลอจิก “0” เพื่อรับข้อมูลไปต่อไป

การเปลี่ยนค่าบอดเรตของ 8051

การเปลี่ยนอัตราเร็วการรับส่งข้อมูลของ 8051 ทำได้สองวิธีคือ

1. เปลี่ยนความถี่ของ crystal
2. เปลี่ยนค่าบิตในรีจิสเตอร์ PCON โดยค่าในรีจิสเตอร์เป็นดังรูป

ถ้าหากปรับความเร็วโดยการเปลี่ยนค่าความถี่ของ crystal นั้นบางครั้งความถี่ที่ได้จะทำให้ความเร็วในการสื่อสารไม่สอดคล้องกับความเร็วของคอมพิวเตอร์ IBM PC โดยมากแล้วมักจะเปลี่ยนค่าความเร็วโดยเปลี่ยนค่าในรีจิสเตอร์ PCON (power control) ซึ่งเป็นรีจิสเตอร์แบบ 8 บิต ที่ใช้ควบคุมพลังงานของ 8051 โดยปกติแล้วบิต D7 (SMOD) จะเป็นลอจิก “0” ถ้าหากโปรแกรมให้บิต SMOD นี้เป็นลอจิก “1” อัตราบอดที่ได้จะมีความเร็วเพิ่มขึ้นหนึ่งเท่าตัวเนื่องจากรีจิสเตอร์ PCON เป็นรีจิสเตอร์ที่ไม่สามารถเข้าถึงข้อมูลระดับบิตได้แต่การเซตบิต SMOD ในรีจิสเตอร์ PCON ทำได้ดังนี้

MOV A, PCON	; อ่านค่าใน PCON มาเก็บใน A
SETB ACC.7	; ทำให้บิต D7 เป็น “1”
MOV PCON, A	; นำค่าใน A ใส่ใน PCON

อัตราเร็วเมื่อบิต SMOD = 0

เมื่อบิต SMOD มีค่าเป็น 0 ตัว 8051 จะนำความถี่ของ XTAL ที่ต่ออยู่กับระบบมาหารด้วย 12 ซึ่งจะได้ค่าความถี่ของแมกซิมัมไซเคิล จากนั้นนำมาหารด้วย 32 อีกครั้ง ค่าที่ได้จะถูกส่งให้ไทเมอร์ 1 คำนวณค่าอัตราบอด

ถ้า 8051 ทำงานกับ XTAL ที่มีความถี่ 11.0592 MHz จะได้

$$\begin{aligned}\text{ความถี่ของแมกซิมัมไซเคิล} &= 11.0592 \text{ MHz} / 12 \\ &= 921.6 \text{ kHz}\end{aligned}$$

และถ้า SMOD = 0 จะได้

$$921.6 \text{ kHz} / 32 = 28,800 \text{ Hz}$$

ค่าความถี่ 28,800 นี้จะถูกส่งให้ไทเมอร์ 1 ทำการเซตอัตราบอดโดยค่าที่ส่งให้รีจิสเตอร์ TH1 สำหรับอัตราบอดค่าต่าง ๆ จะเป็นดังตารางที่ 9.3



อัตราเร็วเมื่อบิต SMOD = 1

ถ้าหากระบบใช้ XTAL ที่มีความถี่ค่าเดิมและบิต SMOD ถูกเซตให้เป็นลอจิก “1” ความเร็วหรืออัตราบอดที่ได้จะเพิ่มขึ้นเท่าตัว เนื่องจากระบบจะนำค่าความถี่ของแมชชีนไซเคิลมาหารด้วย 16 และนำค่านี้นี้ให้ไทมเมอร์ 1 ทำการกำหนดค่าบอดเรต ตัวอย่างเช่น ถ้าระบบใช้ XTAL ที่มีความถี่ 11.0592 MHz จะได้

$$\text{ความถี่แมชชีนไซเคิล} = 11.0592 \text{ MHz} / 12$$

$$= 921.6 \text{ kHz}$$

$$\text{และถ้า SMOD} = 1 \text{ จะได้}$$

$$921.6 \text{ kHz} / 16 = 57,600 \text{ Hz}$$

ถ้าหากใช้ไทมเมอร์ 1 กำหนดค่าบอดเรตจะเป็นไปตามตารางที่ 10.5 ซึ่งจะแสดงค่าที่โหลดให้กับ TH1 สำหรับบอดเรตค่าต่าง ๆ และเปรียบเทียบระหว่างบิต SMOD เป็น “0” และ “1”

ตารางที่ 9.3 เปรียบเทียบบอดเรตเมื่อ SMOD = 0 และ SMOD = 1

TH1 (Dectmal)	Hex	SMOD = 0	SMOD = 1
-3	FD	9,600	19,200
-6	FA	4,800	9,600
-12	F4	2,400	4,800
-24	E8	1,200	2,400

ตัวอย่างที่ 9.5 ถ้า 8051 ต่อกับ XTAL ที่มีความถี่ 11.0592 MHz จากโปรแกรมตัวอย่างจงตอบคำถามต่อไปนี้

ก. โปรแกรมทำอะไร

ข. คำนวณค่าบอดเรต

MOV A,PCON	; A = PCON
SETB ACC.7	; ให้บิต D7 =1
MOV PCON, A	; ใส่ค่าลงใน PCON
MOV TMOD, #20H	; ให้ไทมเมอร์ 1 ทำงานโหมด 2
MOV TH1, - 3	; กำหนดค่าบอดเรตเป็น 19,200
	; (เกิดจาก 57,600 / 3 = 19,200)
MOV SCON, #50H	; ส่ง 8 บิต, บิตหยุด อินาเบิล R1
SETB TR1	; ให้ไทมเมอร์ 1 เริ่มทำงาน
MOV A, # 'B'	; ส่งตัว B ออกไป



```

A_1:    CLR    TI                ; เคลียร์บิต TI
        MOV    SBUF, A          ; ส่งข้อมูล
H_1:    JNB    TI,H_1
        SJMP   A_1

```

ก. โปรแกรมจะส่งรหัส ASCII ของตัว B (01000010) ออกไปอย่างต่อเนื่อง

ข. XTAL มีความถี่ 11.0592 MHz และ SMOD = 1 จะได้

$11.0592 / 12$	$= 921.6 \text{ kHz}$	เป็นความถี่ของแมชชีนไคลเคิล
$921.6 / 16$	$= 57,600 \text{ Hz}$	ความถี่ที่จะส่งให้ไทมเมอร์ 1 กำหนดบอดเรต
$57,600 / 3$	$= 19,200$	ค่าบอดเรตที่ได้

ตัวอย่างที่ 9.6 จงหาค่าในรีจิสเตอร์ TH1 ในรูปแบบของเลขฐานสิบหก ถ้ามีการกำหนดบอดเรต

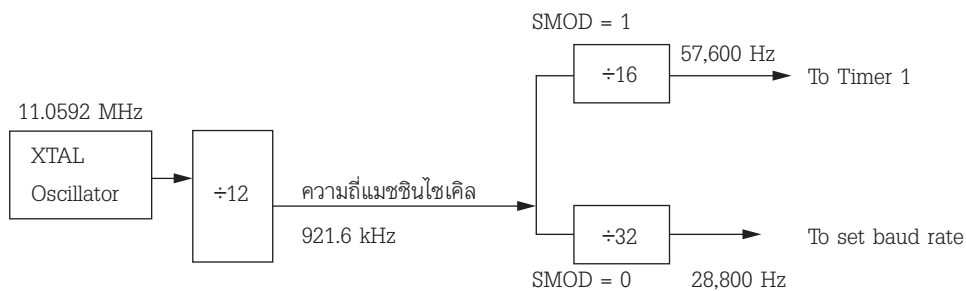
ดังนี้

ก. 9,600 ข. 4,800 ถ้าหาก SMOD = 1 และ XTAL มีความถี่ 11.0592 MHz

วิธีทำ ถ้า XTAL = 11.0592 และ SMOD = 1 จะได้ความถี่ของไทมเมอร์ 1 เท่ากับ 51,600 Hz

ก. $57,600 / 9,600 = 6$ ดังนั้น TH1 = -6 หรือ TH1 = FAH

ข. $57,600 / 4,800 = 12$ ดังนั้น TH1 = -12 หรือ TH1 = F4H



คำถาม ?

1. ไทมเมอร์ของ 8051 ใช้เซตบอดเรตได้อย่างไร
2. ถ้า XTAL มีความถี่ 11.0592 MHz ความถี่ของไทมเมอร์ที่ใช้กำหนดบอดเรตจะเป็นเท่าใด
3. เราใช้ไทมเมอร์ 1 ในโหมดใดสำหรับกำหนดค่าบอดเรต
4. บิต TI จะถูกเซตเมื่อใด
5. เมื่อจ่ายไฟให้ 8051 (Power Up) ค่าบิต SMOD จะเป็นเท่าใด



คำถามท้ายบท

1. การส่งข้อมูลแบบขนานและแบบอนุกรมแบบใดมีราคาแพงกว่ากัน
2. ถ้าหากต้องการส่งอักขระตัว “Z” ออกไปแบบอนุกรม ไม่มีพาริตีบิตและมี 1 บิตหยุดข้อมูลแต่ละบิตที่ออกไปจะเป็นอย่างไร
3. ไอซี MAX232 ใช้ทำอะไร เหตุใดต้องใช้
4. รีจิสเตอร์ SBUF คืออะไรมีกี่บิต
5. ถ้ากำหนดบอดเรตด้วยคำสั่ง MOV TH1, #-1 จะได้บอดเรตเท่าใด
6. บิต REN ในรีจิสเตอร์ SCON ใช้ทำอะไร
7. ถ้าหาก XTAL มีความถี่ 11.0592 MHz จงหาค่าใน TH1 ในรูปแบบเลขฐานสิบ และเลขฐานสิบหกในการกำหนดบอดเรตดังต่อไปนี้
ก. 9600 ข. 4800 ค. 1200 ง. 300 จ. 150
8. บิตแฟลก TI และ RI มีไว้ทำอะไร
9. จงเขียนโปรแกรมให้ 8051 ส่งตัวอักขระ “Z” ออกไปด้วยบอดเรต 1200 bps
10. จงเขียนโปรแกรมให้ 8051 ส่งคำว่า “My name of computer” ออกไปทางพอร์ตอนุกรมด้วยความเร็ว 57600 bps

บทที่ 10

การโปรแกรม อินเทอร์รัปต์

การที่อุปกรณ์ภายนอกที่ต่ออยู่ทางพอร์ตอินพุตเอาต์พุตจะติดต่อกับไมโครคอนโทรลเลอร์นั้นจะทำได้สองวิธีคือ การทำโพลลิง (polling) ซึ่งไมโครคอนโทรลเลอร์จะต้องทำการตรวจสอบเช็คพอร์ตต่าง ๆ อีกวิธีหนึ่งจะให้อุปกรณ์อินพุตเอาต์พุตบอกกับไมโครคอนโทรลเลอร์เองว่าต้องการจะติดต่อดังวิธีที่เรียกว่าการอินเทอร์รัปต์ (interrupt)

การทำงานแบบโพลลิงหรือการหยั่งสัญญาณนั้นไมโครโพรเซสเซอร์จะต้องมาคอยตรวจสอบว่าอุปกรณ์ภายนอกต้องการติดต่อกับไมโครโพรเซสเซอร์หรือไม่ การทำแบบนี้ไมโครโพรเซสเซอร์จะต้องเสียเวลามากมายตรวจสอบ แต่ถ้าเป็นการอินเทอร์รัปต์อุปกรณ์ภายนอกจะเป็นตัวบอกไมโครโพรเซสเซอร์เองว่าให้ไมโครโพรเซสเซอร์มาทำโปรแกรมสำหรับติดต่อกับตัวมัน การทำงานแบบนี้ไมโครโพรเซสเซอร์ไม่ต้องมาเสียเวลาตรวจสอบอุปกรณ์ภายนอก



10.1 การอินเทอร์รัปต์ของ MCS-51

การอินเทอร์รัปต์ (interrupt) อาจจะเรียกอีกอย่างหนึ่งว่าเป็นการขัดจังหวะการทำงานปกติของไมโครโพรเซสเซอร์ ถ้าหากไมโครโพรเซสเซอร์ทำงานปกติอยู่หรือทำงานโปรแกรมหลักอยู่ เมื่อถูกอินเทอร์รัปต์ไมโครโพรเซสเซอร์จะต้องมาทำโปรแกรมตอบสนองการอินเทอร์รัปต์ (interrupt service routine : ISR) เมื่อทำจบแล้วไมโครโพรเซสเซอร์ก็จะมาทำโปรแกรมหลักในคำสั่งต่อจากขณะที่ถูกอินเทอร์รัปต์ต่อไป สำหรับตำแหน่งที่เก็บโปรแกรมตอบสนองการอินเทอร์รัปต์นั้นเรียกว่า อินเทอร์รัปต์เวกเตอร์ (interrupt vector) ซึ่งเป็นตำแหน่งที่มีค่าแอดเดรสแน่นอนในหน่วยความจำโปรแกรม



ขั้นตอนการอินเทอร์รัปต์

เมื่อไมโครคอนโทรลเลอร์ถูกอินเทอร์รัปต์การทำงานต่าง ๆ จะเป็นไปตามขั้นตอนดังนี้

1. ทำคำสั่งที่กำลังทำอยู่ให้จบและเก็บค่าแอดเดรสของคำสั่งถัดไป (ค่าของ PC) ลงหน่วยความจำแตกเองโดยอัตโนมัติ
2. เก็บค่าสถานะต่าง ๆ ของไมโครคอนโทรลเลอร์
3. กระโดดไปยังตำแหน่งหน่วยความจำที่เป็นค่าอินเทอร์รัปต์เวกเตอร์
4. ทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่อยู่ในอินเทอร์รัปต์เวกเตอร์จนพบคำสั่ง RETI (return from interrupt)
5. เมื่อไมโครคอนโทรลเลอร์พบคำสั่ง RETI จะกลับมาทำโปรแกรมตำแหน่งต่อจากที่ถูกอินเทอร์รัปต์ โดยจะอ่านค่าจากหน่วยความจำสแต็กสองไบต์มาทำให้โปรแกรมเคาน์เตอร์ และทำแอดเดรสตำแหน่งนั้นเองโดยอัตโนมัติ

ในการเขียนโปรแกรมตอบสนองการอินเทอร์รัปต์นั้นควรเก็บค่าของรีจิสเตอร์และแฟล็กต่าง ๆ ที่ใช้งานในโปรแกรมการอินเทอร์รัปต์ของหน่วยความจำสแต็กด้วยคำสั่ง PUSH และเมื่อจบโปรแกรมตอบสนองการอินเทอร์รัปต์ให้อ่านค่าที่เก็บเอาไว้ด้วยคำสั่ง PCP ก่อนทำคำสั่ง RETI การทำแบบนี้เพื่อป้องกันความผิดพลาดของรีจิสเตอร์ต่าง ๆ ที่อาจเกิดขึ้นได้ขณะทำโปรแกรมตอบสนองการอินเทอร์รัปต์

แหล่งกำเนิดสัญญาณอินเทอร์รัปต์

สัญญาณอินเทอร์รัปต์ของ 8051 จะมาจากสองระบบใหญ่ ๆ คือ การอินเทอร์รัปต์ด้วยสัญญาณจากภายนอกซึ่งมีอยู่สามแหล่ง และการอินเทอร์รัปต์ด้วยสัญญาณจากภายในซึ่งมีอยู่สามแหล่ง โดยสัญญาณต่าง ๆ เป็นดังนี้

1. สัญญาณรีเซต (reset) เป็นสัญญาณจากภายนอกเมื่อ MCS-51 ถูกรีเซตจะกระโดดไปทำงานยังแอดเดรส 0000 เข้าสู่ขั้นตอนการทำ power-up
2. สัญญาณอินเทอร์รัปต์จากภายในซึ่งเกิดจากไทเมอร์สองแหล่งคือ ไทเมอร์ 0 และไทเมอร์ 1 โดยจะกระโดดไปทำงานโปรแกรมที่อยู่ในแอดเดรส 000BH และ 001BH ของไทเมอร์ 0 และไทเมอร์ 1 ตามลำดับ โดยสัญญาณที่อินเทอร์รัปต์คือบิตแฟล็กของไทเมอร์ (TF0 และ TF1)
3. สัญญาณอินเทอร์รัปต์จากภายนอกสองแหล่งที่เกิดจากสัญญาณที่เข้ามาทางขา 12 (P3.2) และสัญญาณที่เข้ามาทางขา 13 (P3.3) ของพอร์ต 3 สัญญาณนี้มีชื่อทางฮาร์ดแวร์ว่า INTO และ INT1 ตามลำดับ เมื่อ 8051 ถูกอินเทอร์รัปต์นี้จะกระโดดไปทำโปรแกรมที่อยู่ในแอดเดรส 0003H และ 0013H สำหรับ INTO และ INT1 ตามลำดับ
4. สัญญาณอินเทอร์รัปต์จากพอร์ตอนุกรม ถ้าหากมีการรับข้อมูลแบบอนุกรมเข้ามา (RI) หรือหลังจากที่จะส่งข้อมูลแบบอนุกรมออกไป (TI) จะมีสัญญาณมาอินเทอร์รัปต์ 8051 และจะกระโดดไปทำงานโปรแกรมที่อยู่ในตำแหน่ง 0023H



ในตารางที่ 10.1 จะแสดงตำแหน่งของอินเทอร์รับต์เวกเตอร์ของการอินเทอร์รับต์จากสัญญาณต่าง ๆ จะเห็นว่าแอดเดรสของอินเทอร์รับต์เวกเตอร์แต่ละตัวจะห่างกัน 8 ไบต์ ดังนั้นถ้าเขียนโปรแกรมตอบสนองการอินเทอร์รับต์ของสัญญาณบางสัญญาณอาจทำให้โปรแกรมไปทับตำแหน่งอินเทอร์รับต์เวกเตอร์ของอินเทอร์รับต์อื่น ๆ ได้ ตัวอย่างเช่นถ้าโปรแกรมตอบสนองการอินเทอร์รับต์ของไทเมอร์ 0 ไปวางไว้ที่แอดเดรส 000BH ถ้าหากโปรแกรมนี้นี้นยาวเกิน 8 ไบต์ก็จะไปทับตำแหน่งอินเทอร์รับต์เวกเตอร์ของ INT1 (ตำแหน่ง 0013)

อินเทอร์รับต์	ตำแหน่งโปรแกรมใน ROM	ขา
รีเซต	0000	9
อินเทอร์รับต์ภายนอก INTO	0003	P3.2 (12)
อินเทอร์รับต์ไทเมอร์ TF0	000B	
อินเทอร์รับต์ภายนอก INT1	0013	P3.3 (13)
อินเทอร์รับต์ไทเมอร์ TF1	001B	
อินเทอร์รับต์พอร์ตอนุกรม RI และ TI	0023	

เมื่อ MCS-51 เริ่มทำงานจะต้องทำงานโปรแกรมหลักที่อยู่ในแอดเดรส 0000H และจะเห็นว่าโปรแกรมหลักถ้ามีความยาวเกิน 3 ไบต์ก็จะทับตารางอินเทอร์รับต์เวกเตอร์แล้ว ดังนั้นในการเขียนโปรแกรมจะต้องใช้คำสั่ง LJMP ในตำแหน่ง 0000 เพื่อให้โปรแกรมหลักไปอยู่ในแอดเดรสต่อจากตารางอินเทอร์รับต์เวกเตอร์และถ้าโปรแกรมตอบสนองการอินเทอร์รับต์มีความยาวเกิน 8 ไบต์ก็จะใช้คำสั่งกระโดดไปวางไว้ที่ตำแหน่งอินเทอร์รับต์เวกเตอร์และให้ไปทำโปรแกรมตอบสนองการอินเทอร์รับต์ที่อยู่ตำแหน่งอื่น ๆ แทน

```

ORG 0000                ; ตำแหน่งเริ่มต้นของโปรแกรมใน ROM
LJMP MAIN                ; กระโดดข้ามอินเทอร์รับต์เวกเตอร์ไปโปรแกรมหลัก
.....
; พื้นที่เขียนโปรแกรมหลัก
ORG 0030H

MAIN: .....
.....
END

```

รูปที่ 10.1 ลักษณะการเขียนโปรแกรมให้กระโดดข้ามอินเทอร์รับต์เวกเตอร์



การอีนาเบิลอินเทอร์รัปต์

เมื่อ MCS-51 ถูกรีเซตการอินเทอร์รัปต์ต่าง ๆ จะอยู่ในภาวะดิสเอเบิล (disables) คือไม่สามารถถูกอินเทอร์รัปต์ได้ เราสามารถอีนาเบิลการอินเทอร์รัปต์ได้ด้วยการโปรแกรมไปนรีจิสเตอร์ IE (interrupt enable) ซึ่งเป็นรีจิสเตอร์ที่สามารถเข้าถึงระดับบิตได้โดยขั้นตอนเป็นดังนี้

1. ให้บิต D7 เป็น “1” หมายความว่าอินเทอร์รัปต์ได้
2. เลือกบิตของการอินเทอร์รัปต์แต่ละประเภท ถ้าต้องการอินเทอร์รัปต์ประเภทใดให้บิตนั้นเป็น 1

ความหมายของบิตต่าง ๆ ในรีจิสเตอร์ IE แสดงได้ดังรูปที่ 10.2

D7				D0			
EA		ET2	ES	ET1	EX1	ET0	EX0

EA	IE.7	ถ้า EA เป็น “0” หมายความว่าไม่สามารถอินเทอร์รัปต์ได้ ถ้าเป็น “1” หมายความว่ายอมให้อินเทอร์รัปต์จากแหล่งต่าง ๆ ได้ โดยกำหนดแหล่งกำเนิดอินเทอร์รัปต์ในบิตที่ตามมา
....	IE.6	ไม่ใช้งาน
ET2	IE.5	อีนาเบิลหรือดิสเอเบิลของอินเทอร์รัปต์ไทเมอร์ 2 (เฉพาะเบอร์ที่มี เช่น 8952)
ES	IE.4	อีนาเบิลหรือดิสเอเบิลของอินเทอร์รัปต์พอร์ตอนุกรม
ET1	IE.3	อีนาเบิลหรือดิสเอเบิลของอินเทอร์รัปต์ไทเมอร์ 1
EX1	IE.2	อีนาเบิลหรือดิสเอเบิลของอินเทอร์รัปต์ภายนอกหมายเลข 1
ET0	IE.1	อีนาเบิลหรือดิสเอเบิลของอินเทอร์รัปต์ไทเมอร์ 0
EX0	IE.0	อีนาเบิลหรือดิสเอเบิลของอินเทอร์รัปต์ภายนอกหมายเลข 0

รูปที่ 10.2 ความหมายของแต่ละบิตในรีจิสเตอร์ IE

ตัวอย่างที่ 10.1 จงแสดงการเขียนคำสั่งต่อไปนี้

- ก. อีนาเบิลอินเทอร์รัปต์พอร์ตอนุกรมไทเมอร์ 0 และอินเทอร์รัปต์จากภายนอก
- ข. ดิสเอเบิลอินเทอร์รัปต์ไทเมอร์ 0
- ค. ดิสเอเบิลอินเทอร์รัปต์ทุกชนิด

วิธีทำ ก. MOV IE, # 10010110B

ข. CLR IE.1 ; ดิสเอเบิลอินเทอร์รัปต์ไทเมอร์ 0 เท่านั้น

ค. CLR IE.7 ; ดิสเอเบิลอินเทอร์รัปต์ทุกชนิด



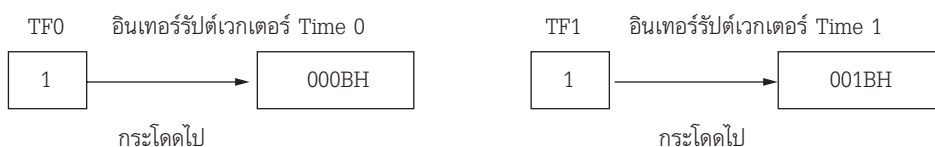
การเขียนคำสั่งในข้อ ก. นั้นอาจใช้คำสั่งกระทำที่ละบิตก็ได้ดังนี้

SETB IE.7	; อินาเบิลอินเทอร์รัปต์
SETB IE.4	; อินาเบิลอินเทอร์รัปต์ฟอร์ตอณูกรม
SETB IE.1	; อินาเบิลอินเทอร์รัปต์ไทมเมอร์ 0
SETB IE.2	; อินาเบิลอินเทอร์รัปต์ภายนอก



10.2 การโปรแกรมไทมเมอร์อินเทอร์รัปต์

ในบทที่ 8 ได้ศึกษาการใช้งานไทมเมอร์มาแล้วซึ่งการตรวจสอบว่าไทมเมอร์ได้นับมาถึงเวลาที่กำหนดแล้วจะทำได้โดยการตรวจสอบบิตแฟลก TF ด้วยคำสั่ง JNB TF, TERQUT โดยการทำแบบนี้ไมโคร-คอนโทรลเลอร์จะไม่สามารถไปทำงานอื่นได้สะดวกนักเราสามารถตรวจสอบการนับของไทมเมอร์ว่านับมาถึงค่าที่กำหนดหรือยังโดยใช้การอินเทอร์รัปต์ได้ โดยการอินเทอร์รัปต์เวกเตอร์ของไทมเมอร์ 0 และไทมเมอร์ 1 แสดงได้ดังรูปที่ 10.3



รูปที่ 10.3 อินเทอร์รัปต์ TF

การใช้อินเทอร์รัปต์ไทมเมอร์ของไมโครคอนโทรลเลอร์ทำได้โดยเซตริจิสเตอร์ IE ให้สามารถอินเทอร์รัปต์ไทมเมอร์ได้เมื่อตัวไทมเมอร์เกิดการโอเวอร์โฟลว์ค่าในบิต TF จะเป็น “1” และจะอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ทำให้ไมโครคอนโทรลเลอร์กระโดดไปทำงานยังตำแหน่งอินเทอร์รัปต์เวกเตอร์ของไทมเมอร์ได้ ดังตัวอย่างโปรแกรมที่ 10.2 ซึ่งมีขั้นตอนการทำงานดังนี้

1. นำโปรแกรมหลักไปเก็บยังแอดเดรสเริ่มต้นที่อยู่หลังตารางอินเทอร์รัปต์เวกเตอร์ จากโปรแกรมจะให้กระโดดไปทำโปรแกรมหลักที่อยู่ในแอดเดรส 30H เมื่อระบบเริ่มทำงานจะทำคำสั่ง LJMP ก่อนแล้วกระโดดไปยังหน่วยความจำที่อยู่หลังตารางอินเทอร์รัปต์เวกเตอร์
2. โปรแกรมตอบสนองการอินเทอร์รัปต์ของไทมเมอร์ 0 ให้เริ่มต้นที่แอดเดรส 000BH ซึ่งเป็นอินเทอร์รัปต์เวกเตอร์ของไทมเมอร์ 0 และให้จบโปรแกรมด้วยคำสั่ง RETI
3. อินาเบิลอินเทอร์รัปต์ไทมเมอร์ 0 ด้วยคำสั่ง MOV IE, #10000010B หรือ MOV IE, #82H ในโปรแกรมหลัก (MAIN)



4. ในโปรแกรมหลักเขียนคำสั่งอ่านข้อมูลจากพอร์ต P0 และส่งไปยังพอร์ต P1 และให้ทำซ้ำถ้าหากบิต TFO ถูกเซตเมื่อใดจะเป็นการอินเทอร์รัปต์ไมโครคอนโทรลเลอร์และกระโดดไปทำคำสั่งที่ตำแหน่ง 000BH

5. ในโปรแกรมตอบสนองการอินเทอร์รัปต์ของไทมเมอร์ 0 ไม่ต้องใช้คำสั่ง CLR TFO ก่อนคำสั่ง RETI

ตัวอย่างที่ 10.2 จงเขียนโปรแกรมให้ 8051 อ่านข้อมูลจากพอร์ต 0 และส่งข้อมูลที่อ่านได้ออกทางพอร์ต 1 รวมทั้งสร้างสัญญาณคลีนส์เฮลล์ยิมที่มีคาบเวลา 200 us ออกทางขา P2.1 โดยใช้ไทมเมอร์ 0 ถ้า XTAL ที่ใช้มีความถี่ 11.0592 MHz

วิธีทำ ให้ไทมเมอร์ 0 ทำงานในโหมด 2 (auto-reload) โดย

$$TH0 = 100/1.085 \text{ us} = 92$$

```

ORG 0000H
LJMP MAIN          ; By-Pass interrupt vector
ORG 000BH          ; อินเทอร์รัปต์ไทมเมอร์ 0
CPL P2.1          ; toggle P2.1
RETI               ; กลับจากอินเทอร์รัปต์
ORG 0030H          ; โปรแกรมหลักต่อจากตาราง
                  ; อินเทอร์รัปต์เวกเตอร์

MAIN  MOV TMOD, #02H ; ไทมเมอร์ 0 โหมด 2
      MOV P0, #0FFH  ; ให้ P0 เป็นพอร์ตอินพุต
      MOV TH0, # - 92 ; TH0 = A4H
      MOV IE, #82H   ; อนุญาตไทมเมอร์ 0
      SETB TR0       ; ให้ไทมเมอร์ 0 เริ่มทำงาน

BACK:  MOV A, P0      ; อ่านข้อมูลจาก P0
      MOV P1,A        ; ส่งข้อมูลออกทาง Py
      SJMP BACK      ; ทำซ้ำ

END.
```



ตัวอย่างที่ 10.3 จงนำโปรแกรมในตัวอย่าง 10.2 มาเขียนใหม่ ให้สร้างคลื่นสี่เหลี่ยมที่เป็น High

1085 us และเป็น Low 15 us

วิธีทำ เวลา 1085 us เกิดจาก 1000×1.085 us โดยใช้ไทเมอร์ 1 โหมด 1

```

ORG 0000H
LJMP MAIN

;ISR ของไทเมอร์ 1
ORG 001BH ; อินเทอร์รัปต์เวกเตอร์ไทเมอร์ 1
LJMP ISR_T1 ; กระโดดไป ISR
ORG 0030H ; กระโดดไปหลังอินเทอร์รัปต์เวกเตอร์

MAIN: MOV TMOD, #10H ; ไทเมอร์ 1 โหมด 1
      MOV P0, #0FFH ; ให้ P0 เป็นอินพุต
      MOV TL1, #018H ; TL1 = 18H
                        ; ไบต์ต่ำของ - 1000
      MOV TH1, #0FCH ; TH1 = 0FCH ไบต์สูงของ - 1000
      MOV IE, # 88 H ; IE = 10001000 อินาเบิลไทเมอร์ 1
      SETB TR1 ; เริ่มไทเมอร์ 1

BACK: MOV A,P0 ; อ่านข้อมูลจาก P0
      MOV P1,A ; ส่งข้อมูลออกไป P1
      SJMP BACK

ISR_T1: CLR TR1 ; หยุดไทเมอร์ 1
        CLR P2.1 ; P2.1 = 0 ให้เป็น Low
        MOV R2, # 4 ; 2 MC
HERE:  DJNZ R2, HERE ; 4 x 2 แมกซีนไซเคิล 8 MC
        MOV TL1, #18H ; โหลดไบต์ต่ำ 2 MC
        MOV TH1, #0FCH ; โหลดไบต์สูง 2 MC
        SETB TR1 ; เริ่มไทเมอร์ 1 1 MC
        SETB P2.1 ; P2.1 กลับเป็น high 1 MC
        RETI ; กลับโปรแกรมหลัก
END.
```

เวลาในช่วงที่เป็น Low จะเท่ากับ 14 แมกซีนไซเคิล โดย $MC = 1.085$ us และ 14×1.085 us =

15.19 us



ตัวอย่างที่ 10.4 จงเขียนโปรแกรมสร้างคลื่นรูปสี่เหลี่ยมความถี่ 50 Hz ออกทางขา P1.2 โดยใช้อินเทอร์รัปต์ของไทมเมอร์ 0 ถ้า XTAL มีความถี่ 11.0592 MHz

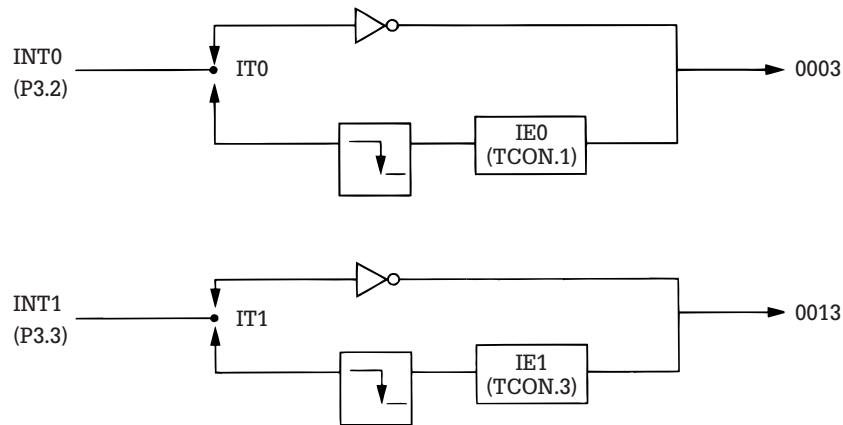
```
วิธีทำ      ORG 0000
            LJMP MAIN
            ORG 000BH                ; ISR ของไทมเมอร์ 0
            CPL P1.2
            MOV TL0, #00
            MOV TH0, #0DCH
            RETI
            ORG 0030H                ; เริ่มต้นโปรแกรมหลัก
MAIN:      MOV TMOD, #00000001B      ; ไทมเมอร์ 0 โหมด 1
            MOV TL0, #00
            MOV TH0, #0DCH
            MOV IE, #82 H            ; อินาเบิลอินเทอร์รัปต์ไทมเมอร์ 0
            SETB TR0
HERE:      SJMP HERE
            END
```



10.3 การโปรแกรมอินเทอร์รัปต์ภายนอก

ไมโครคอนโทรลเลอร์ MCS-51 สามารถรับอินเทอร์รัปต์จากภายนอกได้สองทางคือ INTO และ INT1 ทางขา P3.2 และขา P3.3 ตามลำดับ เมื่อตัวมันถูกอินเทอร์รัปต์ก็จะกระโดดไปทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่ถูกเก็บอยู่ในตำแหน่งอินเทอร์รัปต์เวกเตอร์ที่กำหนด การรับสัญญาณอินเทอร์รัปต์จากภายนอกแสดงได้ดังรูปที่ 10.4 ซึ่งจะเห็นว่า MCS-51 สามารถรับสัญญาณอินเทอร์รัปต์จากภายนอกได้ 2 รูปแบบคือ

1. รับด้วยระดับแรงดัน (level triggered)
2. รับด้วยขอบการเปลี่ยนแปลงของสัญญาณ (edge triggered)



รูปที่ 10.4 การอินเทอร์รัปต์ทาง INT0 และ INT1

การอินเทอร์รัปต์ด้วยระดับแรงดันไฟฟ้า

ในโหมดการอินเทอร์รัปต์ด้วยระดับแรงดันไฟฟ้านั้น ตัว MCS-51 ในภาวะปกติขา INT0 และ INT1 จะเป็นลอจิก High ถ้าหากขาใดขาหนึ่งเปลี่ยนแปลงเป็นระดับแรงดัน Low จะมีการอินเทอร์รัปต์เกิดขึ้น และไมโครคอนโทรลเลอร์จะกระโดดไปทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่อยู่ในตำแหน่งอินเทอร์รัปต์เวกเตอร์ที่กำหนด การทำงานลักษณะนี้เรียกว่า level-triggered หรือ level-activated หลังจาก MCS-51 ถูกอินเทอร์รัปต์สัญญาณทางขา INT จะต้องกลับเป็นลอจิก high ก่อนที่โปรแกรมตอบสนองการอินเทอร์รัปต์จะทำคำสั่ง RETI เพื่อไม่ให้เกิดการอินเทอร์รัปต์ซ้ำอีก

ตัวอย่างที่ 10.5 ถ้าหากขา INT1 ต่ออยู่กับสวิตช์ที่อยู่ในภาวะปกติเป็นลอจิก “high” จงเขียนโปรแกรม ถ้าหากกดสวิตช์นี้จะทำให้หลอด LED ที่ต่ออยู่ทาง P1.3 สว่าง

```
วิธีทำ      ORG 0000H
            LJMP MAIN                ; กระโดดไปโปรแกรมหลัก
;โปรแกรม   ISR ของ INT1
            ORG 0013H
            SETB P1.3                ; เปิดหลอด LED
            MOV R3, #255
BACK:      DJNZ R3, BACK              ; หน่วงเวลา
            CLR P1.3                 ; ปิดหลอด LED
            RETI                     ; กลับสู่โปรแกรมหลัก
            ORG 0030H
MAIN:      MOV IE, #10000100B        ; อนุญาต INT1
```



HERE: SJMP HERE

END

จากตัวอย่างโปรแกรมจะวนลูป HERE ถ้าหากมีการอินเทอร์รัปต์เข้ามาทาง INT1 (ขา P3.3) ไมโครคอนโทรลเลอร์จะกระโดดออกจากลูปไปยังอินเทอร์รัปต์เวกเตอร์ที่ตำแหน่ง 0013H ซึ่งจะเขียนโปรแกรมเปิดหลอดไฟเองได้

การอินเทอร์รัปต์ด้วยการเปลี่ยนแปลงของสัญญาณ

จากที่ผ่านมาเราได้ทดลองให้ 8051 รับสัญญาณอินเทอร์รัปต์ด้วยระดับแรงดันจากภายนอกมาแล้วเราสามารถให้ 8051 รับอินเทอร์รัปต์จากภายนอกทาง INT0 และ INT1 เมื่อมีการเปลี่ยนแปลงจากลอจิก High ไปลอจิก Low ได้อีกด้วย การอินเทอร์รัปต์ด้วยสัญญาณแบบนี้เรียกว่าการอินเทอร์รัปต์ด้วยขอบของสัญญาณ (Edge-triggered interrupt) การที่จะให้ 8051 ถูกอินเทอร์รัปต์ด้วยสัญญาณแบบนี้ เราจะต้องเซตบิต IT0 และ IT1 ในรีจิสเตอร์ TCON เสียก่อน โดย IT0 เป็นบิต D0 ในรีจิสเตอร์ TCON ส่วน IT1 เป็นบิต D2 ในรีจิสเตอร์ TCON ดังนั้นถ้าต้องการให้ 8051 รับอินเทอร์รัปต์ภายนอกแบบการอินเทอร์รัปต์ด้วยขอบจะต้องใช้คำสั่ง SETB TCON.0 สำหรับ INT0 และ SETB TCON.2 สำหรับ INT1

ตัวอย่างที่ 10.6 ถ้าหากขา INT1 ของ 8051 ต่ออยู่กับตัวกำเนิดสัญญาณพลัส จงเขียนโปรแกรมให้ 8051 ตรวจสอบว่าถ้ามีพัลส์เข้ามาให้บิต P1.3 เป็นลอจิก “high” ซึ่งมี LED ต่ออยู่ก็ให้ LED หลอดนั้นสว่าง

วิธีทำ	ORG 0000H	
	LJMP MAIN	; กระโดดไปโปรแกรมหลัก
	ORG 0013H	; ISR ของ INT1
	SETB P1.3	; ให้ไฟติด
	MOV R3, #255	
BACK:	DJNZ R3, BACK	; ทำซ้ำจน R3 เป็นศูนย์
	CLR P1.3	; ให้ไฟดับ
	RETI	; กลับสู่โปรแกรมหลัก
	ORG 0030H	; ตำแหน่งเริ่มต้นโปรแกรมหลัก
MAIN:	SETB TCON.2	; ให้อินเทอร์รัปต์ด้วยขอบสัญญาณ
	MOV IE, #10000100B	; อินาเบิล INT1
HERE:	SJMP HERE	; ทำซ้ำ
	END	



จากโปรแกรมที่ 10.6 จะแตกต่างจากโปรแกรมที่ 10.5 ตรงบรรทัดแรกของโปรแกรมหลัก MAIN ซึ่งจะใช้คำสั่ง SETB TCON.2 เพื่อให้ INT1 เป็นการอินเทอร์รัปต์ด้วยขอบของสัญญาณ

ถ้าหากให้ 8051 ถูกอินเทอร์รัปต์ด้วยระดับแรงดันนั้นถ้าหากออกแบบสัญญาณอินเทอร์รัปต์ไม่เหมาะสมกับโปรแกรมจะทำให้ระบบทำงานผิดพลาดได้ จากที่ทราบมาแล้วว่าการอินเทอร์รัปต์ด้วยระดับแรงดันระบบจะถูกอินเทอร์รัปต์ด้วยลอจิก “Low” ถ้าหากสัญญาณที่เข้ามาทางขา INT เป็น Low นานจนกระทั่งโปรแกรมตอบสนองการอินเทอร์รัปต์ทำคำสั่ง RETI แล้ว สัญญาณทาง INT ยังไม่กลับเป็น “High” ก็จะทำให้ระบบถูกอินเทอร์รัปต์อีกครั้งได้ แต่ถ้าหากเป็นการอินเทอร์รัปต์ด้วยขอบของสัญญาณเมื่อค่าสัญญาณทางขา INT จะเป็นลอจิก “Low” นานกว่าที่โปรแกรมตอบสนองการอินเทอร์รัปต์ทำคำสั่ง RETI ระบบก็จะถือว่าถูกอินเทอร์รัปต์เพียงครั้งเดียว



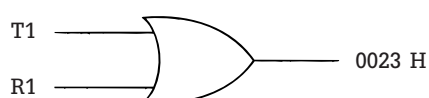
10.4 การโปรแกรมการอินเทอร์รัปต์พอร์ตอนุกรม

ในบทที่ 9 ได้กล่าวถึงการรับส่งข้อมูลแบบอนุกรมของ MCS-51 มาแล้ว โดยทุกตัวอย่างในบทที่ 9 จะใช้วิธีพอลลิง (polling method) ในการติดต่อกับพอร์ตอนุกรมในบทนี้จะศึกษาเกี่ยวกับการอินเทอร์รัปต์ของพอร์ตอนุกรม โดยถ้ามีข้อมูลส่งเข้ามาจะมีสัญญาณไปอินเทอร์รัปต์ MCS-51 เพื่อให้ MCS-51 ทำการอ่านข้อมูลที่ถูกส่งเข้าไป

บิตแฟลก RI และ TI

เมื่อพอร์ตอนุกรมของ MCS-51 ได้รับไบต์ข้อมูลเข้ามาจะใช้บิตแฟลก RI (Received Interrupt) เป็นบิตอินเทอร์รัปต์ ถ้าหากพอร์ตอนุกรมทำงานในโหมด 0 บิตนี้จะเซตเมื่อรับข้อมูลบิตที่ 7 หรือบิตที่ 8 เข้ามาเสร็จสมบูรณ์ แต่ถ้าพอร์ตอนุกรมทำงานในโหมดอื่น บิตนี้จะเซตเมื่อรับข้อมูลบิตสุดท้ายเข้ามา สำหรับการเคลียร์บิตนี้จะต้องทำโดยการเขียนคำสั่ง สำหรับการรับข้อมูลเข้ามาทางพอร์ตอนุกรมนั้นจะใช้บิตแฟลก TI (Transmit Interrupt) เป็นบิตอินเทอร์รัปต์ เมื่อพอร์ตอนุกรมทำงานในโหมด 0 บิตนี้จะเซตเมื่อมีการส่งข้อมูลบิตที่ 7 หรือบิตที่ 8 ออกไป แต่ถ้าหากทำงานในโหมดอื่น บิตนี้จะเซตเมื่อมีการส่งบิตหยุดหรือบิตสุดท้ายออกไป สำหรับการเคลียร์บิตนี้จะต้องทำโดยการเขียนคำสั่ง

ดังนั้นถ้าหากจะต้องใช้การอินเทอร์รัปต์พอร์ตอนุกรมจะต้องนำโปรแกรมตอบสนองการอินเทอร์รัปต์ไปเก็บยังตำแหน่งอินเทอร์รัปต์เวกเตอร์ของอินเทอร์รัปต์พอร์ตอนุกรมคือตำแหน่ง 0023H โดยลักษณะการทำงานของบิตอินเทอร์รัปต์พอร์ตอนุกรมจะเป็นดังรูปที่ 10.5



รูปที่ 10.5 สัญญาณอินเทอร์รัปต์จาก TI และ RI



ตัวอย่างที่ 10.7 จงเขียนโปรแกรมให้ 8051 อ่านข้อมูลจากพอร์ต P1 และส่งออกไปทางพอร์ตอนุกรม และพอร์ต P2 ถ้าหาก XTAL มีความถี่ 11.0592 และมีค่า baud rate เท่ากับ 9600

```

วิธีทำ      ORG 0000H
               LJMP MAIN           ; กระโดดไปโปรแกรมหลัก
               ORG 0023H
               LJMP SERIAL         ; กระโดดไปโปรแกรมตอบสนองการอินเทอร์รัปต์
               ORG 0030H
MAIN:          MOV P1, #0FFH       ; ให้พอร์ต P1 เป็นพอร์ตอินพุต
               MOV TMOD, #20H      ; ใช้งานไทเมอร์ 1 โหมด 2
               MOV TH1, #0FDH      ; baud rate 9600
               MOV SCON, #50H      ; ส่งข้อมูล 8 บิต 1 บิตหยุด
               MOV IE, #10010000B  ; อินาเบิลอินเทอร์รัปต์พอร์ตอนุกรม
               SETB TR1             ; ให้ไทเมอร์ 1 เริ่มทำงาน
BACK:          MOV A, P1           ; อ่านค่าจาก P1 มาเก็บใน A
               MOV SBUF, A         ; นำค่า A ส่งทางพอร์ตอนุกรม
               MOV P2, A           ; นำค่า A ส่งออกทางพอร์ต P2
               SJMP BACK           ; ทำลูปซ้ำ
                                   ; โปรแกรม ISR พอร์ตอนุกรม

               ORG 0100H
SERIAL:        JB TI, TRANS        ; กระโดดถ้าบิต TI เป็น high
               MOV A, SBUF         ; อ่านข้อมูลที่รับเข้ามา
               CLR RI              ; เคลียร์บิต RI
               RETI               ; จบโปรแกรม ISR
TRANS:         CLR TI             ; เคลียร์บิต TI
               RETI               ; จบโปรแกรม ISR
               END
  
```

ตัวอย่างที่ 10.8 จงเขียนโปรแกรมให้ 8051 อ่านข้อมูลจากพอร์ต P1 และส่งไปยังพอร์ต P2 ถ้าหากมีข้อมูลส่งเข้ามาทางพอร์ตอนุกรมให้นำข้อมูลนั้นส่งไปทางพอร์ต P0 ถ้าหาก XTAL มีความถี่ 11.0592 MHz และ baud rate เท่ากับ 9600

```

วิธีทำ      ORG 0000
               LJMP MAIN
  
```



```

ORG 23H
LJMP SERIAL          ; กระโดดไป ISR พอร์ตอนุกรม
ORO 0030H

MAIN: MOV P1, #0FFH   ; ให้พอร์ต P1 เป็นพอร์ตอินพุต
      MOV TMOD, #20H  ; ให้ไทเมอร์ 1 ทำงานโหมด 2
      MOV TH1, #0FDH  ; baud rate 9600
      MOV SCON, #50H  ; รับ 8 บิต 1 stop , REN อีนาเบิล
      MOV IE, #10010000B ; อีนาเบิลอินเทอร์รับต์พอร์ตอนุกรม
      SETB TR1        ; ให้ไทเมอร์ 1 เริ่มทำงาน

BACK: MOV A, P1       ; อ่านค่าจากพอร์ต P1
      MOV P2, A       ; ส่งข้อมูลออกไปพอร์ต P2
      SJMP BACK       ; ทำซ้ำ
                        ; โปรแกรม ISR พอร์ตอนุกรม

ORG
100H

SERIAL: JB TI,TRANS   ; กระโดดถ้าแฟลก TI เป็น high
        MOV A, SBUF   ; อ่านค่าที่รับเข้ามา
        MOV P0, A     ; ส่งค่าออกไปทางพอร์ต P0
        CLR RI        ; เคลียร์บิต RI
        RETI          ; กลับโปรแกรมหลัก

TRANS: CLR TI         ; เคลียร์บิต TI
        RETI          ; กลับโปรแกรมหลัก

END

```

ตัวอย่างที่ 10.9 จงเขียนโปรแกรมโดยใช้อินเทอร์รับต์ให้ทำงานต่อไปนี้

- ก. รับข้อมูลจากพอร์ตอนุกรม และส่งออกไปทางพอร์ต P0
- ข. อ่านข้อมูลจากพอร์ต P1 และส่งออกไปทาง P2 และพอร์ตอนุกรม
- ค. ให้ไทเมอร์ 0 สร้างความถี่ 5 kHz ทาง P0.1

ถ้าหาก XTAL มีความถี่ 11.0592 MHz และการสื่อสารอนุกรมมี baud rate 4800



```
วิธีทำ      ORG 0000
            LJMP MAIN                ; กระโดดไปโปรแกรมหลัก
            ORG 000BH                ; ISR ของไทมเมอร์ 0
            CPL P0.1                 ; กลับค่า P0.1
            RETI                     ; กลับโปรแกรมหลัก
            ORG 0023 H
            LJMP SERIAL              ; กระโดดไปโปรแกรม JSR พอร์ตอนุกรม
            ORG 0030H
MAIN:        MOV P1, #0FFH           ; ให้พอร์ต P1 เป็นอินพุต
            MOV TMOD, #22H           ; ไทมเมอร์ 0 ทำงานโหมด 2
            MOV TH1, #0F6H           ; baud rate 4800
            MOV SCON, #50H           ; 8 บิต 1 stop
            MOV TH0, # - 92           ; ความถี่ 5 KHz
            MOV IE, #10010010B       ; อินทิเอนเตอร์รัปต์
            SETB TR1                  ; เริ่มไทมเมอร์ 1
            SETB TR0                  ; เริ่มไทมเมอร์ 0
BACK:        MOV A, P1                ; อ่านค่าจากพอร์ต P1
            MOV SBUF, A               ; ส่งค่าออกพอร์ตอนุกรม 2
            MOV P2,A                 ; ส่งค่าออกพอร์ต P2
            SJMP BACK                ; ทำซ้ำ
            ORG 100H
SERIAL:      JB TI, TRANS
            MOV A,SBUF
            MOV P0, A
            CLR RI
            RETI
TRANS:       CLR TI
            ERT1
            END
```

ก่อนที่จะจบหัวข้อนี้จะขอสรุปปิดแฟลกของการอินเทอร์รัปต์จากแหล่งต่าง ๆ ซึ่งจะทำหน้าที่อินเทอร์รัปต์ไมโครคอนโทรลเลอร์โดยบิตต่าง ๆ จะอยู่ในรีจิสเตอร์ TCON ดังตารางที่ 10.2



ตารางที่ 10.2 บิตแฟลกอินเทอร์รับต์

อินเทอร์รับต์	แฟลก	บิตในรีจิสเตอร์
External o	IE 0	TCON 1
External 1	IE 1	TCON 3
Timer 0	TF0	TCON 5
Timer 1	TF1	TCON 7
Serial port	T1	SCON 1
Timer 2	TF 2	T2CON 7 (AT89C52)



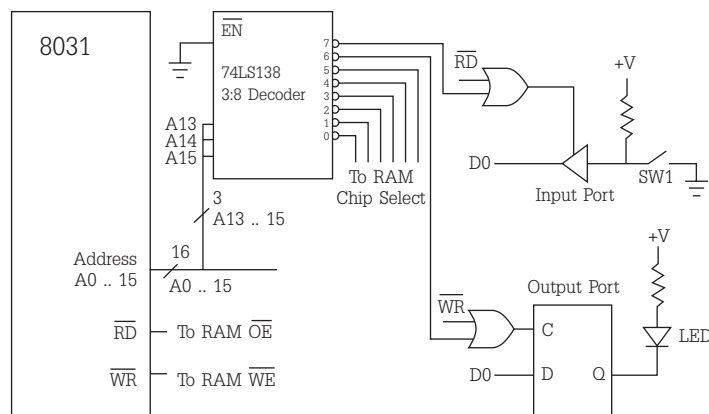
ไมโครคอนโทรลเลอร์มักจะถูกใช้งานในการควบคุมอุปกรณ์ภายนอก โดยอุปกรณ์แต่ละชิ้นจะต่ออยู่กับพอร์ตของไมโครคอนโทรลเลอร์ สำหรับ MCS-51 จะมีพอร์ตขนานอยู่ 4 พอร์ต แต่จะพบว่าบางพอร์ตจะต้องใช้งานในหน้าที่อื่นด้วย ถ้าหากต้องการขยายพอร์ตเพิ่มก็สามารถทำได้ โดย MCS-51 จะใช้แอดเดรสของพอร์ตเป็นแอดเดรสเดียวกับหน่วยความจำ RAM โดยชุดคำสั่งและสัญญาณที่ใช้กับพอร์ตจะเป็นเหมือนกับการติดต่อกับหน่วยความจำ RAM

จากโครงสร้างของ MCS-51 จะพบว่าถ้าต้องการขยายหน่วยความจำเพิ่มเติมภายนอก จะต้องนำพอร์ต 0 และพอร์ต 2 มาใช้เป็นบัสแอดเดรสและบัสข้อมูลด้วย สำหรับพอร์ต 0 นั้นจะทำงานสองหน้าที่คือเป็นแอดเดรสไบต์ต่ำ (A0 - A7) และเป็นบัสข้อมูลและจะต้องใช้อุปกรณ์ภายนอกมาช่วยในการแลตช์ส่วนที่เป็นแอดเดรสให้ค้างเอาไว้ โดยมีสัญญาณ ALE เป็นสัญญาณควบคุม ดังนั้นถ้าหากต้องการขยายพอร์ตเพิ่มเติมจะต้องนำสัญญาณดังกล่าวมาถอดรหัสเป็นตำแหน่งของพอร์ตด้วย จากนั้นจึงต่ออุปกรณ์เพิ่มเติมสำหรับพอร์ตลงไป เช่น การใช้งานเป็นพอร์ตอินพุตจะต้องใช้ไอซีบัฟเฟอร์สามสถานะ (tri-state buffers) มาต่อเป็นพอร์ต ถ้าใช้งานเป็นพอร์ตเอาต์พุตจะต้องนำเอาฟลิปฟล็อปที่สามารถแลตช์ (latches) ข้อมูลได้มาต่อเป็นอุปกรณ์เอาต์พุต



11.1 การถอดรหัสตำแหน่งพอร์ต

สำหรับการถอดรหัสตำแหน่งพอร์ตนั้นจะต้องไม่ให้มีตำแหน่งตรงกับตำแหน่งของหน่วยความจำ RAM ส่วนวงจรถอดรหัสสามารถทำได้หลายลักษณะ แต่โดยทั่วไปแล้วมักจะใช้ไอซีถอดรหัสมาเป็นตัวถอดรหัสตำแหน่งพอร์ต ดังตัวอย่างในรูปที่ 11.1



รูปที่ 11.1 การต่อ 8031 กับพอร์ตภายนอกเพิ่มเติม

เนื่องจาก 8031 สามารถต่อหน่วยความจำ RAM ภายนอกได้ 64k ตำแหน่ง คือตั้งแต่แอดเดรส 0000H ไปจนถึง FFFFH จากวงจรในรูปที่ 11.1 จะนำไอซีถอดรหัสเบอร์ 74LS138 มาต่อ โดยจะรับสัญญาณอินพุต เข้าไป 3 ค่า และให้เอาต์พุตออกมา 8 ค่า โดยค่าทางเอาต์พุตจะเป็นลอจิก “0” ที่ละบิต ขึ้นกับสัญญาณที่ เข้าไปทางอินพุต จากวงจรดังรูปที่ 11.1 ถ้าหากให้ขาเอาต์พุต 0 ถึงเอาต์พุต 5 ของไอซี 74LS138 ต่ออยู่กับ สัญญาณเลือก RAM ซึ่งต่ออยู่กับ RAM จำนวน 6 ตัว และให้ขาเอาต์พุต 6 เป็นตัวถอดรหัสเอาต์พุต และ ให้เอาต์พุต 7 เป็นตัวถอดรหัสอินพุต โดยแอดเดรสของตำแหน่งต่าง ๆ รวมทั้งสัญญาณอินพุตและเอาต์พุต ของตัวถอดรหัสแสดงได้ดังตารางที่ 11.1

ตารางที่ 11.1 แอดเดรสของหน่วยความจำและพอร์ตของวงจรในรูปที่ 11.1

ช่วงแอดเดรส (HEX)	บิตแอดเดรส A15 A14 A13	เอาต์พุตตัวถอดรหัส 7 6 5 4 3 2 1 0	อุปกรณ์ที่ถูกเลือก
0000 - 1FFF	0	0 0 1 1 1 1 1 1 0	RAM 0
2000 - 3FFF	0	0 1 1 1 1 1 1 1 0 1	RAM 1
4000 - 5FFF	0	1 0 1 1 1 1 1 0 1 1	RAM 2
6000 - 7FFF	0	1 1 1 1 1 1 0 1 1 1	RAM 3
8000 - 9FFF	1	0 0 1 1 1 0 1 1 1 1	RAM 4
A000 - BFFF	1	0 1 1 1 0 1 1 1 1 1	RAM 5
C000 - DFFF	1	1 0 1 0 1 1 1 1 1 1	Output Port
E000 - FFFF	1	1 1 0 1 1 1 1 1 1 1	Input Port

ถ้าหาก MCS-51 ทำคำสั่งที่ใช้ส่งข้อมูลไปยังแอดเดรสภายนอกจะทำให้สัญญาณ WR เป็นลอจิก “0” ดังนั้นจะต้องใช้สัญญาณนี้ในการถอดรหัสของพอร์ตเอาต์พุตด้วย และถ้าหาก MCS-51 ทำคำสั่งอ่านข้อมูล จากหน่วยความจำภายนอกจะทำให้สัญญาณ RD เป็นลอจิก “0” ดังนั้นจะต้องใช้สัญญาณนี้ในการถอดรหัส

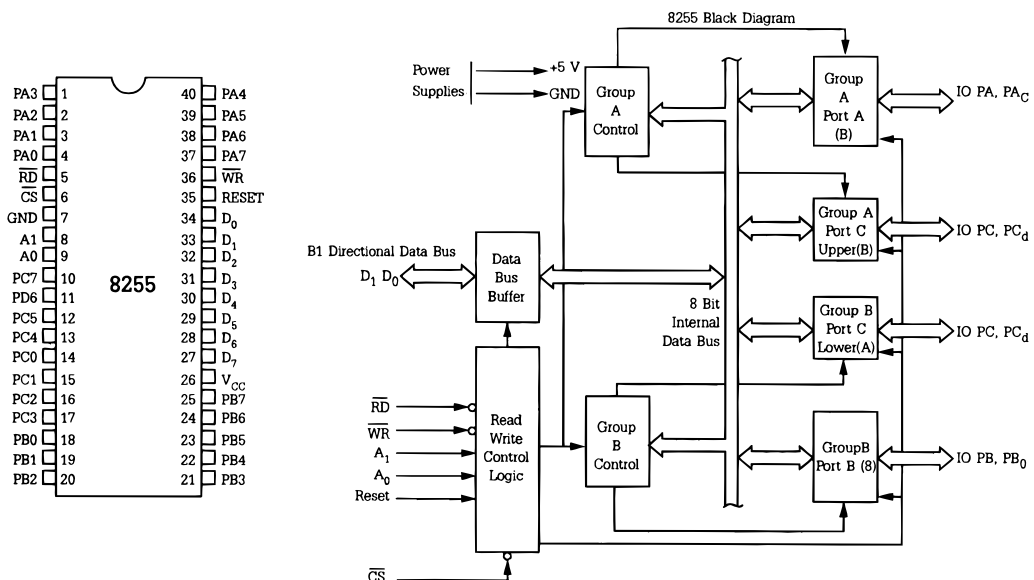


พอร์ตอินพุตด้วย (สำหรับไทม์อะแอมเวลการทำการคำสั่งศึกษาได้จากคู่มือของไมโครคอนโทรลเลอร์ MCS-51) การถอดรหัสแอดเดรสของพอร์ตประเภทนี้เรียกว่า memory mapped I/O จากวงจรจะเห็นว่าถ้าส่งข้อมูลไปยังแอดเดรสในช่วง C000H ถึง DFFFH จะเป็นการติดต่อไปยังพอร์ตเอาต์พุตซึ่งมีฟลิปฟล็อปต่ออยู่สำหรับแลตช์ข้อมูลทางเอาต์พุต แต่ถ้าอ่านข้อมูลจากแอดเดรสในช่วง E000H ถึง FFFFH จะเป็นการอ่านข้อมูลเข้ามาทางอินพุต



11.2 ไอซี 8255 และการโปรแกรม

จากที่กล่าวมาแล้วว่าอุปกรณ์ที่ใช้เป็นพอร์ตอินพุตจะต้องใช้บัฟเฟอร์แบบ 3 สถานะ และอุปกรณ์ที่ใช้เป็นพอร์ตเอาต์พุตจะต้องใช้ฟลิปฟล็อปที่สามารถแลตช์ข้อมูลได้ นอกจากนี้งานบางประเภทจะต้องมีการใช้พอร์ตขนานหลายพอร์ต ในหัวข้อนี้จะกล่าวถึงไอซีพอร์ตขนานที่ภายในตัวมันมีพอร์ตที่ใช้งานได้ถึง 3 พอร์ต โดยแต่ละพอร์ตสามารถโปรแกรมให้เป็นพอร์ตอินพุตหรือพอร์ตเอาต์พุตได้ตามต้องการ ความสามารถนี้เรียกว่า Programmable ลักษณะตัวถังและโครงสร้างภายในของไอซี 8255 แสดงได้ดังรูปที่ 11.2



รูปที่ 11.2 ไอซี 8255

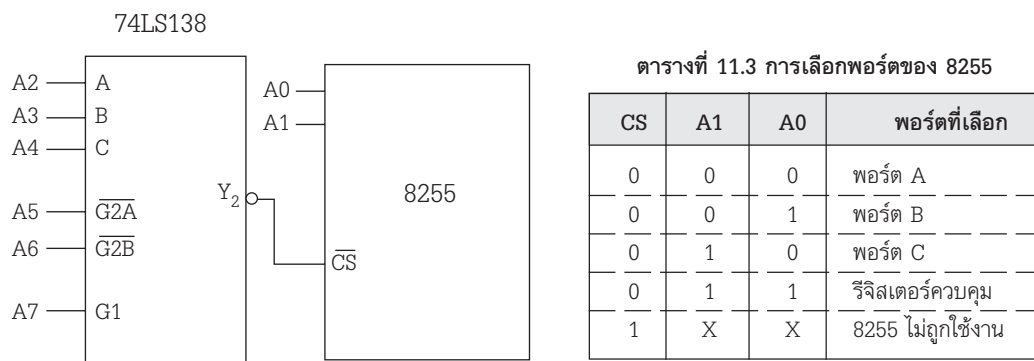
ไอซี 8255 จะมีพอร์ขนานอยู่ 3 พอร์ต คือพอร์ต A พอร์ต B และพอร์ต C โดยแต่ละพอร์ตจะเป็นแบบ 8 บิต พอร์ตเหล่านี้สามารถแบ่งออกได้สองกลุ่มคือ กลุ่ม A และกลุ่ม B โดยกลุ่ม A ประกอบด้วยพอร์ต A 8 บิต และพอร์ต C 4 บิต (4 บิตบน) กลุ่ม B ประกอบด้วยพอร์ต B 8 บิต และพอร์ต C 4 บิต (4 บิตล่าง) สำหรับขาต่าง ๆ แสดงได้ดังตารางที่ 11.2



ตารางที่ 11.2 หน้าที่การทำงานของขาไอซี 8255

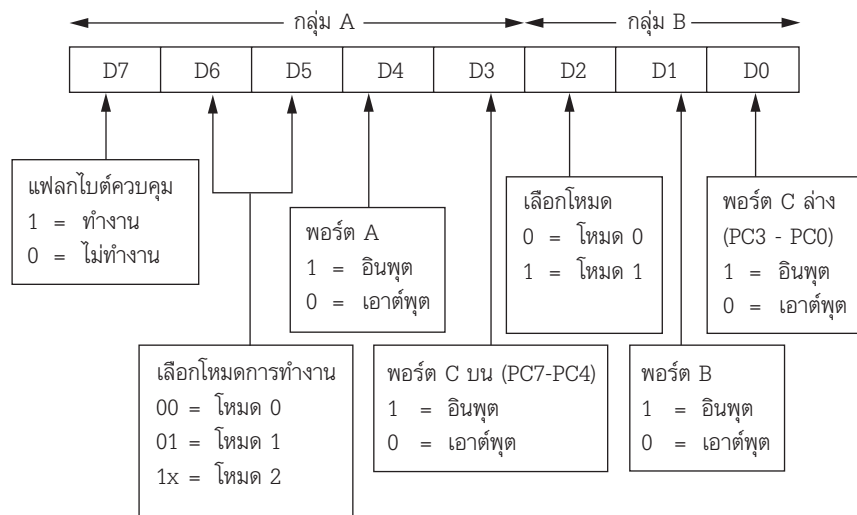
ขาสัญญาณ	การทำงาน
D0 - D7	กลุ่มสัญญาณที่ใช้อ่านเขียนข้อมูลให้กับ 8255
PA0 - PA7	กลุ่มสัญญาณพอร์ต A ขนาด 8 บิต
PB0 - PB7	กลุ่มสัญญาณพอร์ต B ขนาด 8 บิต
PC0 - PC7	กลุ่มสัญญาณพอร์ต C ขนาด 8 บิต
CS\	สัญญาณเลือกชิป 8255 ให้สามารถอ่านเขียนข้อมูลได้ จะแอกทีฟลอจิกต่ำ
RD\	สัญญาณการอ่านข้อมูลออกจาก 8255
WR\	สัญญาณการเขียนข้อมูลให้ 8255
A0 - A1	สัญญาณเลือกพอร์ตของ 8255
RESET	สัญญาณรีเซ็ตการทำงานของ 8255

การเชื่อมต่อ 8255 กับ MCS-51 นั้นจะใช้สัญญาณดังนี้ D0-D7, RD, WR, CS, A0 และ A1 ดังรูปที่ 11.3 เมื่อขา CS เป็นลอจิก “0” จะสามารถใช้งาน 8255 ได้ โดยใช้สัญญาณ A1 กับ A0 เป็นตัวเลือกว่าจะติดต่อกับพอร์ตใด ดังตารางที่ 11.3



รูปที่ 11.3 โค้ดแอมของ 8255

การใช้งาน 8255 เป็นพอร์ตขนานได้นั้นจะต้องโปรแกรมการทำงานให้กับรีจิสเตอร์ควบคุมเสียก่อน ว่าต้องการใช้พอร์ตใดเป็นอินพุตพอร์ตใดเป็นเอาต์พุต สำหรับรีจิสเตอร์ควบคุมแสดงได้ดังรูปที่ 11.4



รูปที่ 11.4 ความหมายแต่ละบิตของรีจิสเตอร์ควบคุม

การศึกษการใช้งาน 8255 ในขณะนี้จะกล่าวถึงการทำงานในโหมดอินพุตเอาต์พุตอย่างง่ายหรือโหมด 0 เท่านั้น การโปรแกรมการทำงานให้กับ 8255 จะต้องส่งไบต์รีจิสเตอร์ควบคุมเข้าไปตามรูปที่ 11.4 โดยส่งไปยังแอดเดรสที่ขา A0 และ A1 เป็นลอจิก “1” ทั้งคู่

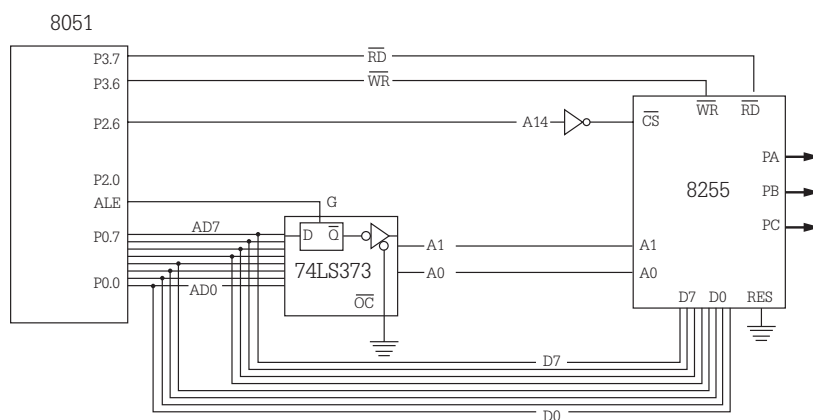
ตัวอย่างที่ 11.1 จงหาค่าของไบต์ควบคุมของ 8255 ถ้าหากต้องการให้ทำงานในลักษณะต่อไปนี้

- พอร์ต A , B และ C เป็นพอร์ตเอาต์พุตทั้งหมด
- PA เป็นอินพุต , PB เป็นเอาต์พุต , PCL เป็นเอาต์พุต และ PCH เป็นอินพุต

วิธีทำ

จากรูปที่ 11.4 จะได้ดังนี้

- $1000\ 0000 = 80H$
- $1001\ 1000 = 98H$



รูปที่ 11.5 การต่อ 8255 เข้ากับ 8051



ตัวอย่างที่ 11.2 จากวงจรการต่อ 8255 ดังรูปที่ 11.5 จงตอบคำถามต่อไปนี้

- ก. จงหาแอดเดรสของพอร์ต A, B, C และรีจิสเตอร์ควบคุม (control register)
- ข. จงหาไบต์ควบคุมที่จะโปรแกรมให้ทุกพอร์ตเป็นพอร์ตเอาต์พุต
- ค. จงเขียนโปรแกรมส่งค่า 55H สลับกับค่า 0AAH ออกไปยังพอร์ตทุกพอร์ต

วิธีทำ ก. แอดเดรสของพอร์ตสามารถหาได้ดังนี้

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
X	1	X	X	X	X	X	X	X	X	x	X	x	X	0	0	PA = 4000H
X	1	X	X	X	X	X	X	X	X	X	X	X	X	0	1	PB = 4001H
X	1	X	X	X	X	X	X	X	X	X	X	X	X	1	0	PC = 4002H
X	1	X	X	X	X	x	X	X	X	X	x	X	x	1	1	CR = 4003H

- ข. ไบต์ควบคุมที่จะให้ทุกพอร์ตเป็นเอาต์พุตคือ 80H ตามตัวอย่างที่ 11.1

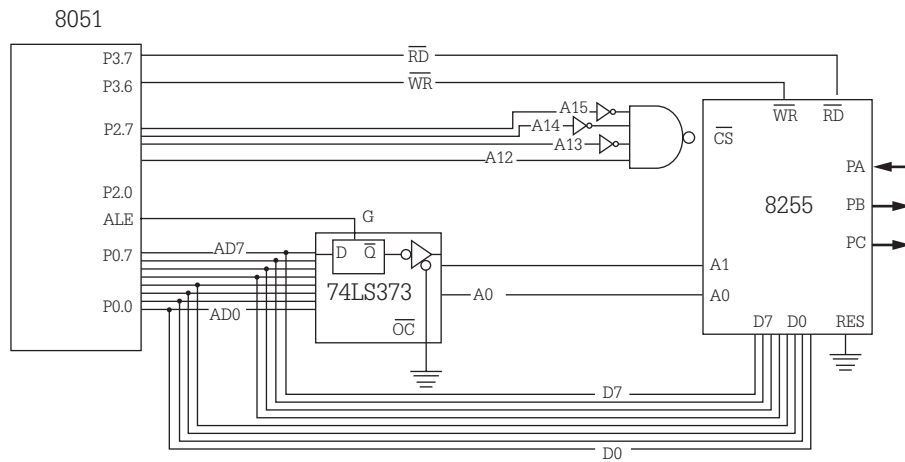
- ค. สามารถเขียนโปรแกรมได้ดังนี้

```

MOV    A , #80H           ; กำหนดไบต์ควบคุมให้ทุกพอร์ตเป็นพอร์ตเอาต์พุต
MOV    DPTR , #4003H      ; โหลดตำแหน่งพอร์ตของไบต์ควบคุม
MOVX   @DPTR , A          ; ส่งไบต์ควบคุมออกไป
MOV    A , #55H           ; ให้ A = 55H
AGAIN: MOV    DPTR , #4000H ; โหลดแอดเดรสของพอร์ต A
MOVX   @DPTR , A          ; ส่งข้อมูลออกไปยังพอร์ต A
INC    DPTR               ; กำหนดแอดเดรสของพอร์ต B
MOVX   @DPTR , A          ; ส่งข้อมูลออกไปยังพอร์ต B
INC    DPTR               ; กำหนดแอดเดรสของพอร์ต C
MOVX   @DPTR , A          ; ส่งข้อมูลออกไปยังพอร์ต C
CPL    A                  ; กลับค่าบิตในรีจิสเตอร์ A จะได้ A = 0AAH
ACALL  DELAY              ; เรียกโปรแกรมย่อยหน่วงเวลา
SJMP   AGAIN              ; ทำซ้ำ
  
```

ตัวอย่างที่ 11.3 จากวงจรรูปที่ 11.6 จงตอบคำถามต่อไปนี้

- ก. หาค่าตำแหน่งของพอร์ต A, B, C และพอร์ตรีจิสเตอร์ควบคุม
- ข. หาไบต์ควบคุมถ้าต้องการให้ PA = อินพุต, PB = เอาต์พุต และ PC = เอาต์พุต
- ค. จงเขียนโปรแกรมอ่านข้อมูลจากพอร์ต A และส่งข้อมูลออกไปยังพอร์ต B และพอร์ต c



รูปที่ 11.6 ตัวอย่างการต่อ 8255 กับ 8051

วิธีทำ ก. สมมุติว่าบิตแอดเดรสที่ไม่ได้ใช้งานเป็น “0” ทุกบิต จะได้ว่าแอดเดรสเริ่มต้นของ 8255 คือ 1000H

ดังนั้น	PA	เป็นแอดเดรส	1000H
	PB	เป็นแอดเดรส	1001H
	PC	เป็นแอดเดรส	1002H
	พอร์ตควบคุม	เป็นแอดเดรส	1003H

ข. ไบต์ควบคุมคือ 10010000 หรือ 90H

ค. MOV A , #90H	; ให้ PA = IN, PB = OUT, PC = OUT
MOV DPTR , #1003H	; โหลดแอดเดรสของพอร์ตควบคุม
MOVX @DPTR , A	; ส่งไบต์ควบคุมออกไป
MOV DPTR , #1000H	; โหลดแอดเดรสของพอร์ต PA
MOVX A , @DPTR	; อ่านข้อมูลจากพอร์ต PA เข้ามา
INC DPTR	; กำหนดแอดเดรสของพอร์ต PB
MOVX @DPTR , A	; ส่งข้อมูลออกไปยังพอร์ต PB
INC DPTR	; กำหนดแอดเดรสของพอร์ต PC
MOVX @DPTR , A	; ส่งข้อมูลออกไปยังพอร์ต PC

จากตัวอย่างที่ 11.3 สามารถนำคำสั่งเทียม EQU มาใช้แทนชื่อพอร์ตได้ ดังการเขียนโปรแกรมต่อไปนี้

```

PORT_A EQU 1000H
PORT_B EQU 1001H
PORT_C EQU 1002H

```



```

CON_PORT EQU 1003H

MOV A, #90H ; ให้ PA = IN, PB = OUT, PC = OUT
MOV DPTR, #CON_PORT ; โหลดแอดเดรสของพอร์ตควบคุม
MOVBX @DPTR, A ; ส่งไบต์ควบคุมออกไป
MOV DPTR, #PORT_A ; โหลดแอดเดรสของพอร์ต PA
MOVBX A, @DPTR ; อ่านข้อมูลจากพอร์ต PA เข้ามา
INC DPTR ; กำหนดแอดเดรสของพอร์ต PB
MOVBX @DPTR, A ; ส่งข้อมูลออกไปยังพอร์ต PB
INC DPTR ; กำหนดแอดเดรสของพอร์ต PC
MOVBX @DPTR, A ; ส่งข้อมูลออกไปยังพอร์ต PC

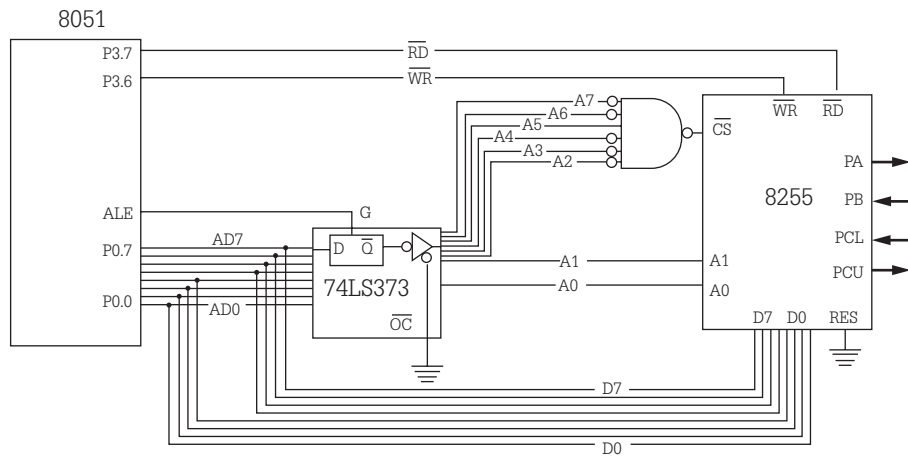
```

จากตัวอย่างโปรแกรมจะพบว่าการอ้างตำแหน่งของพอร์ตที่ 8255 ต่ออยู่ จะต้องใช้รีจิสเตอร์ DPTR เป็นตัวชี้ เนื่องจากแอดเดรสที่ถอดรหัสเอาไว้เป็นแอดเดรสแบบ 16 บิต แต่ถ้าหากต้องการอ้างตำแหน่งพอร์ตในช่วงแอดเดรส 8 บิต สามารถใช้รีจิสเตอร์ R0 หรือ R1 เป็นตัวชี้ได้ เช่นการเขียนคำสั่ง `MOVBX A, @R0` หรือ `MOVBX @R0, A`

จากตัวอย่างที่ 11.2 และ 11.3 จะเห็นว่าแอดเดรสของพอร์ตที่ถอดรหัสออกมาจะอยู่ในช่วงที่กว้าง ถ้าพิจารณาตัวอย่างที่ 11.2 ในตารางตัว x จะเป็นตำแหน่งที่ไม่ได้เอามาถอดรหัส ซึ่งบิตนี้จะมีค่าลอจิกเป็นค่าใดก็ได้ ถ้า x เป็นลอจิก “0” ทุกบิต พอร์ต PA จะมีแอดเดรสเป็น 4000H แต่ถ้าอ้างตำแหน่งพอร์ตเป็นตำแหน่ง 4004H ก็ยังเป็นพอร์ต PA เช่นกัน เนื่องจากเราไม่สนใจค่า x สำหรับตัวอย่างที่ 11.4 และ 11.5 จะเป็นการนำ A0 - A7 มาถอดรหัสด้วย ซึ่งจะทำให้แอดเดรสอยู่ในช่วงที่แคบลงมา

ตัวอย่างที่ 11.4 จากวงจรในรูปที่ 11.7 จงตอบคำถามต่อไปนี้

- ก. จงหาแอดเดรสของพอร์ต A, B, C และพอร์ตควบคุม
- ข. จงหาไบต์ควบคุมที่ทำให้ PA = OUT, PB = IN, PC0 - PC3 = IN และ PC4 - PC7 = OUT
- ค. จงเขียนโปรแกรมอ่านข้อมูลจากพอร์ต PB และส่งข้อมูลให้พอร์ต PA รวมทั้งอ่านข้อมูลจากพอร์ต PC ล้างและส่งข้อมูลออกทางพอร์ต PC บน



รูปที่ 11.7 ตัวอย่างการต่อ 8255 กับ 8051

วิธีทำ ก. แอดเดรสของพอร์ตจะเป็นดังนี้

CS	A1	A0	Address	Port
0010 00	0	0	20H	พอร์ต A
0010 00	0	1	21H	พอร์ต B
0010 00	1	0	22H	พอร์ต C
0010 00	1	1	23H	พอร์ตควบคุม

ข. ไบต์ควบคุมคือ 1000 0011 หรือ 83H

ค.

CONTRBYT EQU 83H ; กำหนดให้ A = OUT, PB = IN, PCL = IN, PCU = OUT

A_PORT EQU 20H ; พอร์ต PA

B_PORT EQU 21H ; พอร์ต PB

C_PORT EQU 22H ; พอร์ต PC

CNT_PORT EQU 23H ; พอร์ตควบคุม

.....

MOV A, #CONTRBYT ; PA = OUT, PB = IN, PCL = IN, PCU = OUT

MOV R0, #CNTPORT ; โหลดแอดเดรสของพอร์ตควบคุมให้รีจิสเตอร์ R0

MOVX @R0, A ; ส่งไบต์ควบคุมไปยังตำแหน่งที่ R0 ชี้อยู่

MOV R0, #B_PORT ; ให้ R0 เก็บตำแหน่งของพอร์ต B

MOVX A, @R0 ; อ่านข้อมูลจากพอร์ต B เข้ามาในรีจิสเตอร์ A

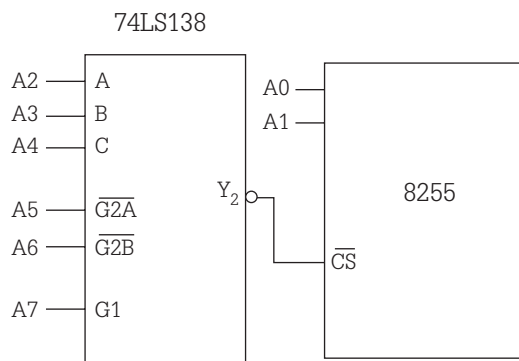


```

DEC    R0                ; ลดค่า R0 ให้ชี้ที่พอร์ต A
MOVX   @R0 , A           ; ส่งข้อมูลออกไปยังพอร์ต A
MOV    R0 , #C_PORT      ; ให้ R0 ชี้ที่แอดเดรสของพอร์ต C
MOVX   A , @R0           ; อ่านข้อมูลจากพอร์ต C ล่างเข้ามา
ANL    A , #0FH          ; เลือกเฉพาะพอร์ต C ล่าง
SWAP   A                 ; สลับข้อมูลระหว่าง 4 บิตบน และ 4 บิตล่าง
MOVX   @R0 , A           ; ส่งข้อมูลออกไปยังพอร์ต C บน

```

ตัวอย่างที่ 11.5 จากวงจรในรูปที่ 11.8 จงหาแอดเดรสแรกของ 8255



รูปที่ 11.8 การถอดรหัสแอดเดรส 8255 โดยใช้ 74LS138

วิธีทำ

G1	G2B	G2A	C	B	A			แอดเดรส
A7	A6	A5	A4	A3	A2	A1	A0	
1	0	0	0	1	0	0	0	88H

ดังนั้นแอดเดรสของพอร์ต A ของ 8255 คือตำแหน่ง 88H

ในระบบไมโครคอนโทรลเลอร์ถ้าหากต้องการใช้พอร์ตเป็นจำนวนมาก สามารถนำ 8255 หลาย ๆ ตัวมาต่อกันได้ โดยออกแบบวงจรถอดรหัสพอร์ตให้กับ 8255 แต่ละตัว จากนั้นพอร์ตของ 8255 จะถูกนำไปใช้ในการควบคุมอุปกรณ์อื่น ๆ ต่อไป ดังตัวอย่างในรูปที่ 11.9 จะเป็นการนำ 8255 ไปควบคุม stepper motor และสามารถเขียนชุดคำสั่งควบคุมได้ดังต่อไปนี้

```

MOV    A , #80H          ; กำหนดไบต์ควบคุมให้ PA เป็นเอาต์พุต
MOV    R1 , CRPORT       ; ให้ R1 ชี้ไปที่แอดเดรสพอร์ตควบคุม

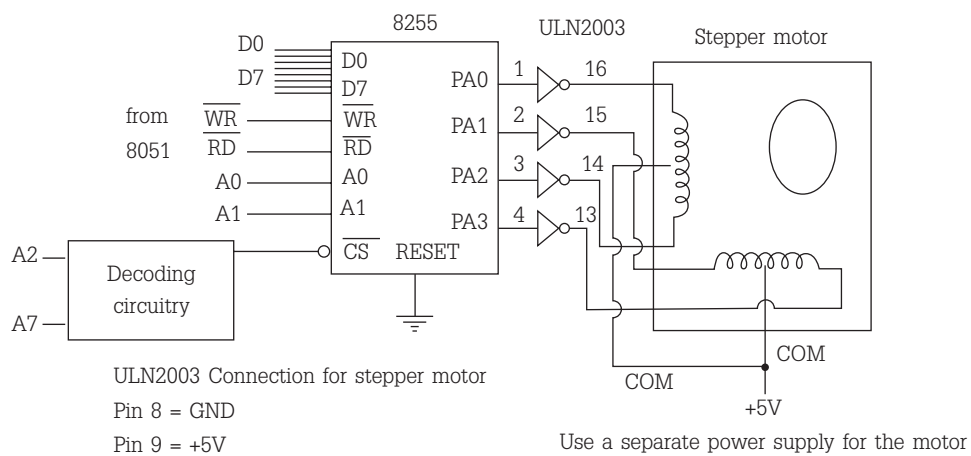
```



```

MOVX  @R1 , A           ; ส่งไบต์ควบคุมให้ PA เป็นเอาต์พุต
MOV   R1 , #A_PORT      ; โหลดค่าแอดเดรสของพอร์ต PA
MOV   A , #66H           ; ให้ A = 66H เพื่อเป็นตัวควบคุมสแต็ปมอเตอร์
AGAIN: MOVX  @R1 , A      ; ส่งข้อมูลขับเคลื่อนมอเตอร์ทางพอร์ต PA
RR    A                  ; หมุนลอจิกที่ควบคุมมอเตอร์แต่ละเฟส
ACALL DELAY              ; หน่วงเวลา
SJMP  AGAIN              ; ทำซ้ำ

```



รูปที่ 11.9 การต่อ 8255 เข้ากับ stepper motor

ไอซีที่ใช้เปลี่ยนสัญญาณแอนะล็อกเป็นดิจิตอล (ADC) ตัวหนึ่งที่ใช้งานง่ายและมีราคาถูกคือเบอร์ ADC804 เมื่อวงจรภายในตัวไอซีสามารถแปลงสัญญาณแอนะล็อกเป็นดิจิตอลได้สำเร็จมันจะส่งสัญญาณ INTR เป็นลอจิก “0” ออกมา สำหรับระบบที่ต้องการแปลงสัญญาณแอนะล็อกเป็นดิจิตอลนั้นสามารถนำไอซีเบอร์นี้ มาต่อกับไอซีพอร์ต 8255 ได้เช่นกัน ดังตัวอย่างวงจรในรูปที่ 11.10 โดยที่ ADC804 จะรับสัญญาณแอนะล็อกเข้ามาแปลงเป็นสัญญาณดิจิตอลขนาด 8 บิต และส่งให้ทางพอร์ต PA ของ 8255 โดยใช้บิตต่ำสุดของพอร์ต PC เป็นตัวตรวจสอบสัญญาณ INTR เมื่อระบบไมโครคอนโทรลเลอร์ต้องการอ่านข้อมูลนี้จะโปรแกรมให้ 8255 ทำงานในโหมด 0 และอ่านข้อมูลเข้ามาในระบบตามต้องการ โดยเขียนคำสั่งได้ดังนี้

```

MOV   A , #80H           ; กำหนดไบต์ควบคุมให้ PA เป็นเอาต์พุต PC เป็นอินพุต
MOV   R1 , #CRPORT       ; กำหนดแอดเดรสของพอร์ตควบคุม
MOVX  @R1 , A             ; โปรแกรมให้ PA เป็นเอาต์พุต PC เป็นอินพุต
BACK: MOV   R1 , #C_PORT  ; โหลดค่าแอดเดรสของพอร์ต PC
MOVX  A , @R1             ; อ่านข้อมูลจากพอร์ต PC

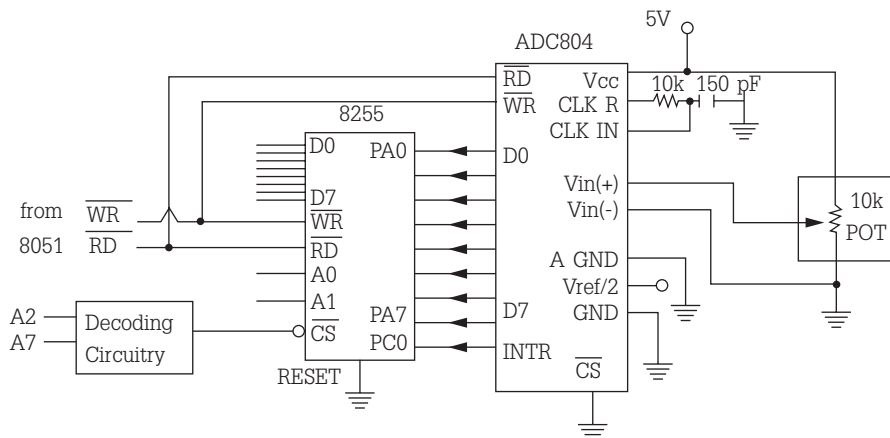
```



```

ANL    A , #00000001B    ; ตรวจสอบบิตต่ำสุดที่ต่ออยู่กับ INTR
JNZ    BACK              ; วนลูปจนกว่าสัญญาณจาก INTR เป็นลอจิก "0"
; รอจนกว่าวงจรแปลงสัญญาณ ADC แปลงสัญญาณได้สำเร็จ
MOV    R1 , #A_PORT      ; กำหนดค่าแอดเดรสของพอร์ต PA
MOVX   A , @R1            ; อ่านข้อมูลแอนะล็อกที่แปลงได้เข้ามาสู่ MCS-51

```

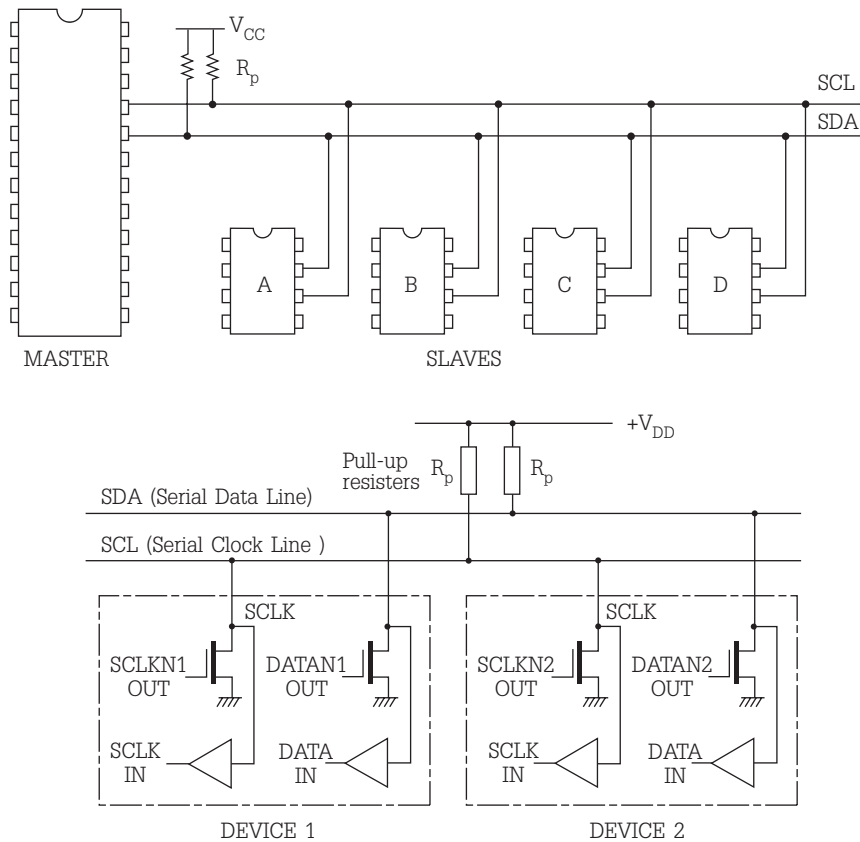


รูปที่ 11.10 การต่อ ADC804 เข้ากับ 8255



11.3 ระบบบัสแบบ I²C และการเชื่อมต่อ

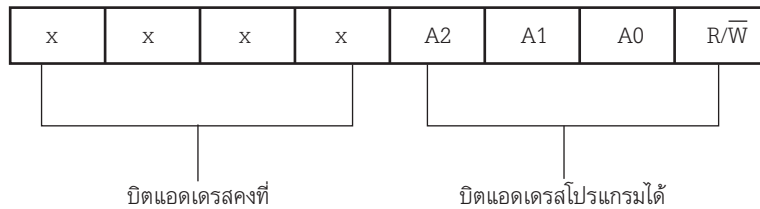
จากการต่อวงจรขยายพอร์ตที่ผ่านมาจะเห็นว่าการต่อบัสข้อมูลของ MCS-51 กับชิปที่ใช้เป็นพอร์ตจะต้องใช้บัสข้อมูลแบบขนาน ซึ่งจะทำให้ต้องใช้สัญญาณในการเชื่อมต่อเป็นจำนวนมาก ปัจจุบันได้มีการพัฒนาวิธีการเชื่อมต่อชิปไอซีต่าง ๆ เข้าด้วยกันโดยใช้การสื่อสารแบบพอร์ตอนุกรม ซึ่งจะทำให้ช่วยลดสายสัญญาณที่ใช้ในการสื่อสารข้อมูลลงได้มาก ระบบบัสแบบหนึ่งที่นิยมใช้กันคือระบบบัสแบบ I²C ซึ่งย่อมาจากคำว่า Inter-IC Communication ที่พัฒนาโดยบริษัท Philips สำหรับใช้ในการเชื่อมต่อชิปต่าง ๆ กับระบบไมโครคอนโทรลเลอร์โดยใช้สัญญาณแบบอนุกรมเพียงสองเส้น โดยเส้นหนึ่งเป็นสายข้อมูลมีชื่อว่า Serial Data Line (SDA) อีกเส้นหนึ่งเป็นสายสัญญาณนาฬิกาสำหรับกำหนดจังหวะการสื่อสารข้อมูลมีชื่อว่า Serial Clock Line (SCL) ตัวอย่างการเชื่อมต่อแสดงได้ดังรูปที่ 11.11

รูปที่ 11.11 การเชื่อมต่อด้วยระบบบัส I²C และวงจรทางเอาต์พุต

สายสัญญาณสองเส้นที่เชื่อมต่อกันสามารถต่ออุปกรณ์ได้หลายตัว โดยแต่ละตัวจะมีแอดเดรสของมันเอง ระบบบัสแบบนี้สามารถรับส่งข้อมูลได้สองทิศทางด้วยความเร็วสูงถึง 100 กิโลบิตต่อวินาที และสามารถใช้กับไอซีที่ใช้แรงดันไฟฟ้าต่างกันได้ การติดต่อกับอุปกรณ์แต่ละตัวจะมีรูปแบบการรับส่งข้อมูลหรือโพรโทคอล (protocol) ที่อุปกรณ์ทุกตัวรู้จัก ถ้าหากต้องการให้อุปกรณ์ตัวใดรับข้อมูล ตัวส่งจะส่งแอดเดรสของอุปกรณ์ตัวนั้นไปก่อน ถ้าหากอุปกรณ์ตัวใดมีแอดเดรสตรงกันก็จะรับข้อมูลนั้นไป สำหรับตัวอุปกรณ์ที่ต้องการส่งข้อมูลจะเรียกว่ามาสเตอร์ (master) ซึ่งจะเป็นตัวที่สร้างจังหวะสัญญาณต่าง ๆ บนระบบบัส ส่วนอุปกรณ์ที่ถูกควบคุมหรือเป็นตัวรับข้อมูลจะเรียกว่าสเลฟ (slave)

ถ้าหากต้องการให้ไมโครคอนโทรลเลอร์เป็นตัวส่งข้อมูล ตัวมันจะทำหน้าที่เป็นมาสเตอร์ และจะส่งสัญญาณไปบนบัส โดยจะส่งค่าแอดเดรสออกไปก่อน ถ้าแอดเดรสนี้ตรงกับแอดเดรสของชิปตัวใด ชิปตัวนั้นจะส่งสัญญาณตอบรับ (ACK) ออกมา และชิปตัวนั้นก็จะถูกไมโครคอนโทรลเลอร์ควบคุม แต่ถ้าค่าแอดเดรสไม่ตรงกับชิปตัวใดก็จะมีสิ่งใดเกิดขึ้น

การส่งข้อมูลต่าง ๆ จะต้องเกิดขึ้นเมื่อระบบบัสว่างเท่านั้น โดยทั่วไปแล้วสถานะที่มีบนระบบบัสจะมีอยู่ 5 สถานะ ดังรูปที่ 11.12



รูปที่ 11.13 ไบต์แอดเดรส

จากรูปจะเห็นว่าบิตต่ำสุดจะเป็นตัวระบุว่าการดำเนินการที่ต้องการดำเนินการนั้นเราจะใช้อ่านหรือเขียนข้อมูลลงไป ถ้าบิตนี้เป็นลอจิก “0” หมายความว่าเราจะใช้เขียนข้อมูลลงไป ถ้าเป็นลอจิก “1” หมายความว่าเราจะใช้อ่านข้อมูล ส่วน 7 บิตบนจะเป็นบิตแอดเดรส ซึ่งแบ่งออกเป็นบิตแอดเดรสคงที่ (fixed address bit) จำนวน 4 บิต โดยจะถูกโปรแกรมมาจากโรงงานที่ผลิตชิปแต่ละประเภท ส่วนอีก 3 บิตต่อมาจะเป็นบิตแอดเดรสที่ผู้ใช้สามารถโปรแกรมได้ (programmable address bit) ในกรณีที่มีการต่อชิปเบอร์เดียวกันหลาย ๆ ตัวในระบบบัลแบบ I²C นี้ จะต้องโปรแกรมให้บิตแอดเดรสที่ผู้ใช้โปรแกรมได้มีค่าแอดเดรสต่างกัน

2. การเขียนไบต์ควบคุม (control byte) ข้อมูลไบต์นี้จะขึ้นอยู่กับอุปกรณ์แต่ละประเภท เพื่อกำหนดการทำงานต่าง ๆ ของตัวมัน อุปกรณ์บางประเภทอาจจะไม่ต้องการเขียนไบต์นี้ก็ได้

3. การเขียนไบต์ข้อมูล (data byte) เป็นข้อมูลที่จะโอนถ่ายในระบบ ระหว่างสภาวะเริ่มต้นและสภาวะหยุด จะมีการเขียนข้อมูลจำนวนกี่ไบต์ก็ได้

12

บทที่

การทดลองกับ หน่วยแสดงผล

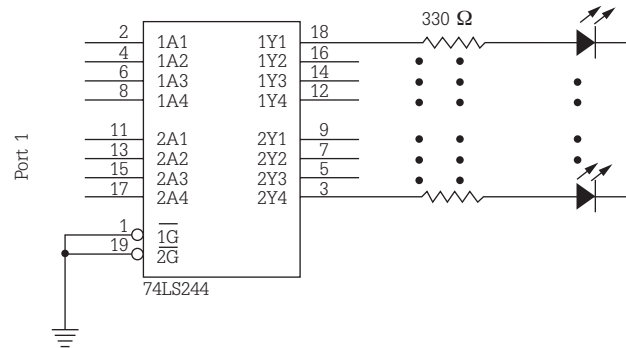
เมื่อต้องการให้ MCS-51 แสดงผลข้อมูลจะต้องนำหน่วยแสดงผลมาเชื่อมต่อกับพอร์ตของ MCS-51 การแสดงผลพัลส์ที่เป็นลอจิก “0” หรือลอจิก “1” สามารถทำได้โดยการนำหลอด LED มาต่อ หรือถ้าต้องการแสดงผลเป็นตัวเลขสามารถทำได้โดยนำ LED แบบ 7 ส่วนมาเชื่อมต่อ แต่ถ้าต้องการแสดงผลเป็นตัวอักขระหรือตัวเลขอาจทำได้โดยการนำอุปกรณ์แสดงผลแบบ LCD มาต่อ สำหรับในบทนี้จะแสดงการเชื่อมต่อกับอุปกรณ์แสดงผลอย่างง่าย พร้อมทั้งอธิบายหลักการทำงานของโปรแกรมแสดงผลข้อมูล



12.1 การเชื่อมต่อกับไดโอดเปล่งแสง

ไดโอดเปล่งแสงหรือ LED (Light Emitting Diode) เป็นอุปกรณ์ที่สามารถใช้แสดงผลลอจิก “1” หรือลอจิก “0” ได้ การทำให้หลอด LED สว่างจะต้องไบแอสมันด้วยความต่างศักย์ประมาณ 1.5 - 3 โวลต์ และให้กระแสไหลผ่านหลอดในช่วง 5 ถึง 25 มิลลิแอมป์

ถ้าหากต้องการนำหลอด LED มาต่อกับพอร์ตขนาน P1 สามารถทำได้ดังรูปที่ 12.1 ถ้าหากบิตใดเป็นลอจิก “1” จะทำให้มีแรงดันตกคร่อมที่ขานั้นเท่ากับ 5 โวลต์ และเนื่องจาก LED แต่ละหลอดจะต้องใช้กระแสไฟฟ้าในการทำงาน แต่โดยทั่วไปแล้วแต่ละบิตของ MCS-51 จะจ่ายกระแสได้น้อย ดังนั้นจึงมีการนำบัฟเฟอร์มาช่วยในการขับกระแสด้วย และจะมีตัวต้านทาน 330 โอห์มต่ออยู่เพื่อควบคุมกระแสให้ไหลผ่าน LED ประมาณ 20 มิลลิแอมป์



รูปที่ 12.1 การเชื่อมต่อหลอด LED เข้ากับพอร์ต P1

ถ้าหากต้องการให้หลอด LED ที่ต่ออยู่กับพอร์ต P1 สว่างทุกหลอดก็ทำได้โดยส่งค่า 0FFH ออกไปทางพอร์ต P1 แต่ถ้าส่งค่า 00H ออกไปที่พอร์ต P1 จะทำให้ LED ทุกดวงดับ ถ้าหากต้องการให้ LED สว่างในรูปแบบของไฟวิ่ง เราสามารถออกแบบรูปแบบการสว่างของ LED ได้ดังตารางที่ 12.1 และเขียนโปรแกรมส่งค่าข้อมูลออกไป

ตารางที่ 12.1 รูปแบบการแสดงผลของ LED และค่าข้อมูลที่ส่งออกไป

ลำดับ	รูปแบบ	ค่าข้อมูล	เลขฐานสิบหก
1	● ○ ○ ○ ○ ○ ○ ○ ●	1000 0001	81H
2	○ ● ○ ○ ○ ○ ○ ● ○	0100 0010	42H
3	○ ○ ● ○ ○ ● ○ ○	0010 0100	24H
4	○ ○ ○ ● ● ○ ○ ○	0001 1000	18H
5	○ ○ ● ○ ○ ● ○ ○	0010 0100	24H
6	○ ● ○ ○ ○ ○ ○ ● ○	0100 0010	42H
7	● ○ ○ ○ ○ ○ ○ ○ ●	1000 0001	81H
8	○ ○ ○ ○ ○ ○ ○ ●	0000 0001	01H
9	● ○ ○ ○ ○ ○ ○ ○	1000 0000	80H
10	○ ○ ○ ● ● ○ ○ ○	0001 1000	18H

ตัวอย่างที่ 12.1 การเขียนโปรแกรมให้ MCS-51 ทำงานเป็นไฟรั้งที่มีรูปแบบตั้งแต่ลำดับที่ 1 ไปถึงลำดับที่ 10 และวนลูปซ้ำตามตารางที่ 12.1

การเขียนโปรแกรมดังกล่าวสามารถทำได้โดยนำรูปแบบข้อมูลของไฟวิ่งไปเก็บไว้ในตาราง และเขียนโปรแกรมให้อ่านค่าในตารางออกมาและส่งออกไปยังพอร์ท P1 ครั้งละค่า หลังจากส่งข้อมูลออกไปแล้ว ให้นั่งเวลาสักครู่ก่อนที่จะส่งค่าใหม่ออกไป โดยสามารถเขียนโปรแกรมได้ดังนี้



```
;***** MAIN *****  
  
MAIN:  
  
SHOW:  MOV    R0,#10D          ; มีรูปแบบแสดงผล 10 รูปแบบ  
        MOV    R1,#00H        ; รูปแบบแรก  
LOOP:  MOV    DPTR,#DATA_SHOW ; ให้ DPTR ชี้ไปที่ตารางรูปแบบแสดงผล  
        MOV    A,R1           ; ให้ A เก็บลำดับของรูปแบบ  
        MOVC   A,@A+DPTR      ; อ่านค่าจากตารางมาใส่ใน A  
        MOV    P1,A           ; ส่งออกไปทางพอร์ต P1  
        INC    R1             ; กำหนดค่าลำดับถัดไป  
        LCALL  DELAY          ; เรียกโปรแกรมย่อยหน่วงเวลา  
        DJNZ   R0,LOOP        ; ทำซ้ำ 10 รูปแบบ  
        SJMP   SHOW          ; เริ่มต้นทำใหม่  
  
;*****  
DELAY:  MOV    R3,#10D          ; โปรแกรมย่อยหน่วงเวลาหนึ่งวินาที  
DE1:    MOV    R4,#100D        ;  
DE2:    LCALL  DESEG           ; เรียกโปรแกรมย่อยหน่วงเวลา 1 มิลลิวินาที  
        DJNZ   R4,DE2  
        DJNZ   R3,DE1  
        RET                  ; จบโปรแกรมย่อยหน่วงเวลา  
  
;*****  
DESEG:  MOV    TMOD,#01H       ; โปรแกรมย่อยหน่วงเวลา 1 มิลลิวินาที  
        MOV    TH0,#0FCH       ; โดยใช้ไทเมอร์ 0  
        MOV    TL0,#66H  
        SETB   TR0  
LOOP1:  JNB    TF0,LOOP1  
        CLR    TR0  
        CLR    TF0  
        RET  
  
DATA_SHOW: DB 81H,42H,24H,18H,24H ; ตารางเก็บรูปแบบการแสดงผลทาง LED  
            DB 42H,81H,01H,80H,18H  
            END
```

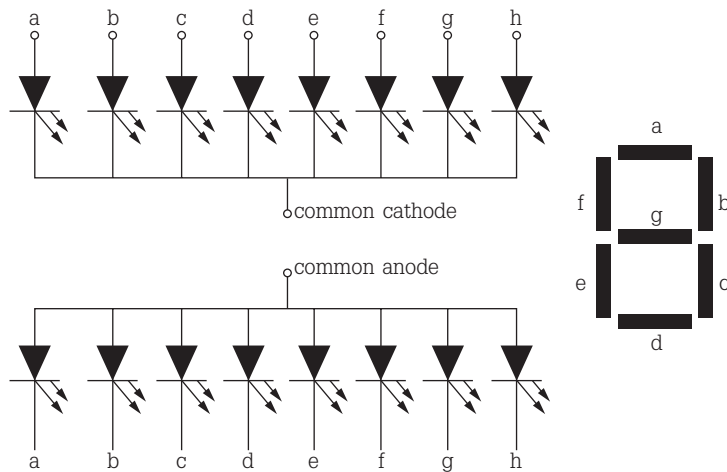


การอ่านค่าจากตารางและส่งออกมาทางพอร์ต P1 จะทำได้เร็วมาก ดังนั้นจะต้องหน่วงเวลาในการแสดงผลแต่ละครั้งเพื่อให้สามารถมองเห็น LED สว่างได้ จากโปรแกรมในตัวอย่างที่ 12.1 จะสร้างโปรแกรมน้อยหน่วงเวลาขนาด 1 วินาทีขึ้นมาโดยให้ชื่อว่า DESEG และให้โปรแกรมน้อยเรียกโปรแกรม DESEG ขึ้นมา 1000 ครั้ง ก็จะทำให้มีการหน่วงเวลาในการแสดงผลแต่ละครั้งเท่ากับ 1 วินาที



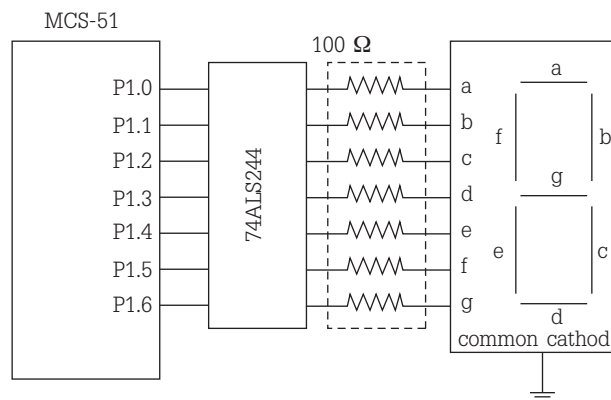
12.2 การแสดงผลทาง LED แบบ 7 ส่วน

หลอด LED 7 ส่วน หรือที่เรียกกันว่า LED 7 เซกเมนต์ (LED 7-segment) เกิดจากการนำหลอด LED จำนวน 7 หลอดมาต่อเรียงกันเพื่อให้สามารถแสดงผลเป็นตัวเลขได้ ตัว LED แต่ละหลอดหรือแต่ละเซกเมนต์จะมีชื่อประจำคือ a, b, c, d, e, f และ ถ้าหลอด LED ทั้ง 7 หลอดต่อกันโดยนำขาแอนโอดมาต่อร่วมกันจะเรียกว่าแบบแอนโอดร่วม (common anode) ถ้าเป็นแบบแคโทดต่อร่วมกันจะเรียกว่าแบบแคโทดร่วม (common cathode) ดังรูปที่ 12.2 ถ้าหากต้องการให้หลอด LED สว่างเป็นเลขใดก็ได้ทำได้โดยให้หลอดแต่ละหลอดสว่างประกอบกันเป็นเลขนั้น เช่นถ้าหาก LED ทุกหลอดสว่างพร้อมกันหมดจะแสดงเป็นเลข 8



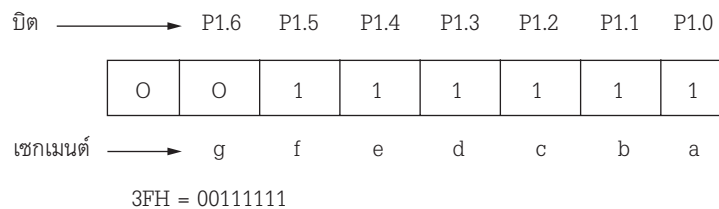
รูปที่ 12.2 หน่วยแสดงผลแบบ LED 7 เซกเมนต์

หลอด LED 7 เซกเมนต์นี้สามารถนำมาต่อกับพอร์ตขนานของระบบไมโครคอนโทรลเลอร์ได้ แต่เนื่องจากการสว่างของหลอด LED จะต้องใช้กระแสไฟฟ้าจำนวนหนึ่ง และถ้าให้ LED ทุกหลอดสว่างพร้อมกันหมดจะต้องใช้กระแสไฟฟ้าจำนวนมาก ดังนั้นถ้าหากต่อกับพอร์ต P1 ของ MCS-51 จะต้องมียุโรปเฟอร์ช่วยในการจ่ายกระแสได้ แต่ถ้าหากต่อกับพอร์ต 8255 ก็สามารถต่อได้ทันที สำหรับการต่อ LED 7 เซกเมนต์แบบแคโทดร่วมกับพอร์ต P1 สามารถทำได้ดังรูปที่ 12.3



รูปที่ 12.3 การต่อ LED 7 เซกเมนต์แบบแคโทดร่วมกับพอร์ต P1

จากรูปที่ 12.3 ถ้าหากส่งข้อมูลออกมาทางพอร์ต P1 เป็นค่า 3FH หรือ 00111111 จะทำให้ LED 7 เซกเมนต์สว่างเป็นเลขศูนย์



รูปที่ 12.4 ค่าแต่ละบิตของพอร์ต P1 และการแสดงผล

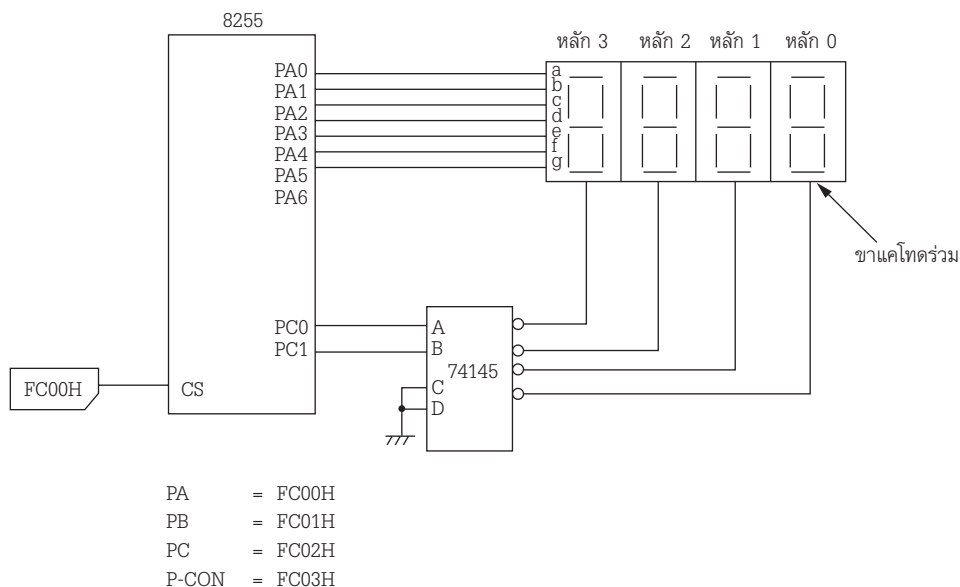
ถ้านำ LED 7 เซกเมนต์ไปต่อกับพอร์ตโดยให้บิตต่ำสุดต่อกับเซกเมนต์ a บิตต่อมาต่อกับเซกเมนต์ b ไปเรื่อย ๆ จนถึงเซกเมนต์ g ถ้าต้องการแสดงผลเป็นเลขต่าง ๆ จะต้องส่งข้อมูลออกไปยังพอร์ตดังตารางที่ 12.2

ตารางที่ 12.2 รูปแบบการแสดงผลและค่าของพอร์ต

ตัวเลข	g	f	e	d	c	b	a	ค่าพอร์ต
0	0	1	1	1	1	1	1	3FH
1	0	0	0	0	1	1	0	06H
2	1	0	1	1	0	1	1	5BH
3	1	0	0	1	1	1	1	4FH
4	1	1	0	0	1	1	0	66H
5	1	1	0	1	1	0	1	6DH
6	1	1	1	1	1	0	1	7DH
7	0	0	0	0	1	1	1	07H
8	1	1	1	1	1	1	1	7FH
9	1	1	0	1	1	1	1	6FH



จากตัวอย่างการต่อ LED 7 เซกเมนต์ที่ผ่านมาจะพบว่า การต่อ LED 7 เซกเมนต์ให้แสดงตัวเลขหนึ่งหลัก จะต้องใช้พอร์ตจำนวนหนึ่งพอร์ต ถ้าหากมีหลายหลักจะต้องใช้พอร์ตหลายพอร์ตตามไปด้วย โดยทั่วไปแล้วถ้าหากต้องการออกแบบระบบไมโครคอนโทรลเลอร์ให้สามารถแสดงผลทาง LED 7 เซกเมนต์มากกว่าหนึ่งหลัก จะใช้วิธีการแสดงผลแบบมัลติเพล็กซ์ (multiplex) โดยจะให้ LED สว่างทีละหลักเรียงกันไปจนครบทุกหลัก ด้วยอัตราที่เร็วกว่าที่คนเราจะสังเกตเห็นได้ซึ่งจะทำให้มองเห็นว่า LED 7 เซกเมนต์นั้นสว่างพร้อมกันทุกหลัก การใช้วิธีการมัลติเพล็กซ์นี้จะช่วยลดจำนวนพอร์ตที่ใช้ต่อกับ LED 7 เซกเมนต์ได้ และยังช่วยลดพลังงานที่ต้องจ่ายให้กับระบบอีกด้วย แต่การต่อวงจรแบบนี้จะต้องใช้เทคนิคการเขียนโปรแกรมเข้ามาช่วย เพื่อให้สามารถแสดงผลตามที่ต้องการได้ ตัวอย่างในรูปที่ 12.5 จะเป็นตัวอย่างการต่อ LED 7 เซกเมนต์แบบแคโทดรวมจำนวน 4 หลัก เข้ากับพอร์ต 8255



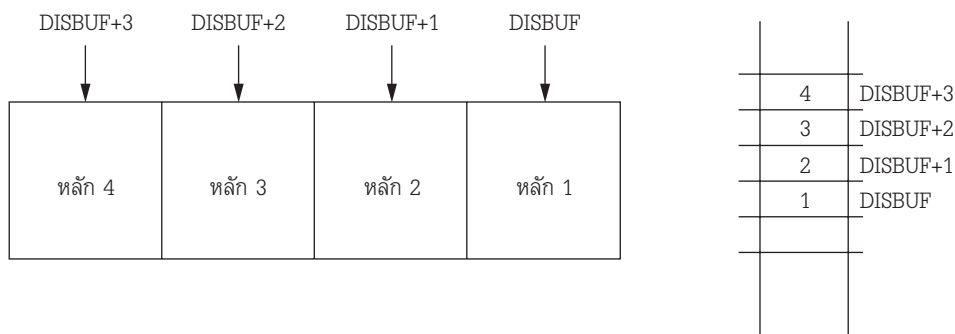
รูปที่ 12.5 ตัวอย่างการต่อ LED 7 เซกเมนต์ด้วยวิธีการมัลติเพล็กซ์

จากวงจรดังรูปที่ 12.5 จะพบว่าพอร์ต A จะต่อ LED แต่ละเซกเมนต์ และพอร์ต C จะถูกต่อกับวงจรถอดรหัสเพื่อทำการสแกน LED แต่ละหลัก วงจรถอดรหัสของการสแกนนี้จะต่อเข้ากับขาแคโทดรวมของ LED ดังนั้นจะเห็นว่าถ้าหากจะให้ LED ทั้งสี่หลักสว่างพร้อมกันจะทำได้ แต่จะใช้วิธีการสแกนให้สว่างทีละหลัก เริ่มจากหลักที่ 0 หลักที่ 1 หลักที่ 2 หลักที่ 3 จากนั้นวนกลับมายังหลักที่ 0 อีกครั้ง โดยการสแกนนี้จะทำด้วยความเร็วสูงทำให้ดูเหมือนว่า LED ทุกหลักสว่างพร้อมกัน เนื่องจากไมโครคอนโทรลเลอร์ทำงานด้วยความเร็วสูงมาก ดังนั้นการแสดงผลแต่ละหลักนั้นจะหน่วงเวลาไว้จำนวนหนึ่งเพื่อให้กระแสไฟฟ้าสามารถจ่ายเข้าไปใน LED และทำให้หลอดสว่างได้

สำหรับตัวอย่างที่ 12.2 จะเป็นตัวอย่างโปรแกรมการสแกนอย่างง่าย โดยจะมีการประกาศหน่วย



ความจำภายในชื่อ DISBUF เอาไว้สำหรับเก็บตัวเลขที่ต้องการแสดงผล โดยหน่วยความจำส่วนนี้จะใช้จำนวน 4 ไบต์ เพื่อแสดงผล 4 หลัก โปรแกรมจะอ่านค่าจากหน่วยความจำ DISBUF นี้และส่งไปยัง LED 7 เซกเมนต์ที่ละหลักถ้าหากต้องการแสดงผลเป็นเลขใดก็สามารถทำได้โดยนำตัวเลขไปใส่ในหน่วยความจำ DISBUF นี้ สำหรับตำแหน่งหน่วยความจำและตัวเลขแต่ละหลักแสดงได้ดังนี้



ตัวอย่างที่ 12.2 โปรแกรมต่อไปนี้จะแสดงผลตัวเลขแบบมัลติเพล็กซ์ที่ใช้กับวงจรดังรูปที่ 12.5 โดยจะออกแบบหน่วยความจำแสดงผลชื่อ DISBUF เอาไว้ และสร้างโปรแกรมน้อย scan สำหรับแสดงผลเป็นตัวเลข โปรแกรมจะนำค่าจากหน่วยความจำนี้ไปเปิดตารางและแสดงออกทาง LED 7 เซกเมนต์ที่ละหลัก และวนลูปซ้ำ

```
PORT_A      EQU    0FC00H          ; แอดเดรสพอร์ตต่าง ๆ ของไอซี 8255
PORT_B      EQU    0FC01H
PORT_C      EQU    0FC02H
PORT_CON    EQU    0FC03H
```

```
DISBUF EQU    20H                  ; ให้หน่วยความจำ DISBUF เริ่มที่ตำแหน่ง 20H
```

```
;***** MAIN *****
```

```
MAIN:      MOV A,#80H              ; กำหนดไบต์ควบคุม
           MOVD PTR,#PORT_CON      ; กำหนดแอดเดรสของพอร์ตควบคุม
           MOVX @DPTR,A            ; ส่งไบต์ควบคุมให้ทุกพอร์ตเป็นเอาต์พุต
           MOV DISBUF+3,#8         ; ส่งเลข 8 ให้กับหน่วยความจำหลักที่ 4
           MOV DISBUF+2,#6         ; ส่งเลข 6 ให้กับหน่วยความจำหลักที่ 3
           MOV DISBUF+1,#4         ; ส่งเลข 4 ให้กับหน่วยความจำหลักที่ 2
           MOV DISBUF,#2           ; ส่งเลข 2 ให้กับหน่วยความจำหลักที่ 1
```



```

DIS:      ACALL    SCAN          ; เรียกโปรแกรมย่อยแสดงผล
          SJMPDIS              ; ทำซ้ำ

,*****
; SCAN LED 7                  ; โปรแกรมย่อย SCAN จะมีการใช้รีจิสเตอร์
                                DPTR
; DPTR,R0,R1,R2              ; R0 R1 และ R2
,*****

SCAN:     MOV      R1,#4          ; R1 เก็บจำนวนหลักของการสแกน
          MOV      R2,#3          ; R2 เก็บค่าสำหรับส่งออกไปทางหลัก
          MOV      R0,#DISBUF+3   ; R0 เก็บตำแหน่งของหน่วยความจำแสดงผล
LOOP:     MOV      DPTR,#SEG_CODE ; DPTR ชี้ไปที่ตารางเก็บรูปแบบการแสดงผล
          MOV      A,@R0          ; อ่านค่าตัวเลขจากหน่วยความจำแสดงผล
          MOVC     A,@A+DPTR       ; เปิดค่าจากตารางว่าตัวเลขนั้นมีรูปแบบอย่างไร
          MOV      DPTR,#PORT_A    ; กำหนดแอดเดรสของพอร์ต A
          MOVX     @DPTR,A         ; ส่งรูปแบบการแสดงผลออกไปทางพอร์ต A
          MOV      A,R2           ; ให้ A เก็บหลักที่ต้องการแสดงผล
          MOV      DPTR,#PORT_C    ; กำหนดแอดเดรสของพอร์ต C
          MOVX     @DPTR,A         ; ส่งค่าหลักที่ต้องการแสดงผลออกไป
          LCALL    DELAY           ; เรียกโปรแกรมย่อยหน่วงเวลา
          DEC      R0              ; ลดค่า R0 เพื่อชี้หน่วยความจำหลักถัดมา
          DEC      R2              ; ลดค่า R2 เพื่อแสดงผลหลักถัดมา
          DJNZ     R1,LOOP         ; ทำซ้ำจำนวน 4 ครั้ง
          RET                    ; จบโปรแกรมย่อย

,*****

DELAY:     MOV      R3,#0FFH       ; หน่วงเวลาสำหรับการแสดงผลแต่ละหลัก
DE1:       DJNZ     R3,DE1
          RET

,*****

SEG_CODE:  DB       3FH,06H,5BH,4FH,66H ; ตารางเก็บรูปแบบการแสดงผลตัวเลข
          DB       6DH,7DH,07H,7FH,6FH
          END

```



จากโปรแกรมในตัวอย่างที่ 12.2 จะเห็นว่าโปรแกรมน้อย SCAN ที่สร้างขึ้นมานั้น สามารถนำไปประยุกต์ใช้ในงานต่าง ๆ ได้ เพียงแต่ใส่ตัวเลขแต่ละหลักเข้าไปในหน่วยความจำ DISBUF และเรียกโปรแกรมน้อย SCAN ก็สามารถแสดงผลเป็นตัวเลขออกมาได้ ถ้าค่าใน DISBUF เปลี่ยนไปก็จะทำให้การแสดงผลเปลี่ยนไปด้วย

ในงานทางคอมพิวเตอร์อิเล็กทรอนิกส์นั้นการแสดงผลข้อมูลที่เป็นตัวเลขมักจะทำให้ข้อมูลออกมาในรูปแบบของเลข BCD เนื่องจากเลขฐานสองที่อยู่ในรูปแบบเลข BCD สามารถนำมาแสดงได้สะดวก อย่างเช่นถ้ามีเลขฐานสองที่อยู่ในรูปแบบ BCD และมีค่า 12H จะเป็นการแทนเลขฐานสิบค่าสิบสอง

ข้อมูลในหน่วยความจำขนาดหนึ่งไบต์นั้นสามารถแทนเลข BCD ได้สองหลัก ถ้าหากมีข้อมูลเลข BCD อยู่เราสามารถนำมาถอดออกเป็นเลขฐานสิบแต่ละหลักได้ ตัวอย่างเช่นถ้ามีข้อมูลเป็น 0011 0111 หรือ 37H ซึ่งเป็นเลข BCD ที่แทนเลขฐานสิบที่มีค่าเท่ากับ 37 สามารถถอดตัวเลขออกมาทีละหลักได้ดังนี้

- นำข้อมูลที่ได้อมา AND กับค่า 0FH จะได้ค่า 7 ออกมา

$$\begin{array}{rcl} 37 & = & 0011 \ 0111 \\ 0F & = & 0000 \ 1111 \\ \text{ผลการ AND} & & 0000 \ 0111 = 7 \end{array}$$

- สับข้อมูลระหว่างสปีตล่างกับสปีตบน และนำมา AND กับค่า 0FH จะได้เลข 3 ออกมา

$$\begin{array}{rcl} & 0011 \ 0111 & 0111 \ 0011 \\ \text{สลับ} \longrightarrow & 0111 \ 0011 & \text{AND กับ } 0FH \\ & & \text{ผลลัพธ์} \end{array} \quad \begin{array}{rcl} & & 0000 \ 1111 \\ & & \underline{0000 \ 0011} = 3 \end{array}$$

สำหรับตัวอย่างที่ 12.3 จะเป็นการแสดงผลทาง LED 7 เซกเมนต์ดังวงจรที่เคยศึกษามา โดยจะสร้างหน่วยความจำ DISBUF สำหรับเก็บเลขฐานสิบแต่ละหลักที่ต้องการแสดงผล และสร้างหน่วยความจำชื่อ HEXBUF สำหรับเก็บเลข BCD ในโปรแกรมจะสร้างโปรแกรมน้อยชื่อ BCD2DEC ซึ่งจะทำหน้าที่อ่านข้อมูลจากหน่วยความจำ HEXBUF ออกมา และถอดรหัสเป็นเลขฐานสิบแต่ละหลักจากนั้นนำค่าที่ได้ใส่ในหน่วยความจำ DISBUF เพื่อแสดงผลต่อไป

ตัวอย่างที่ 12.3 โปรแกรมนี้จะนำข้อมูลที่เป็นเลข BCD ใส่ในหน่วยความจำ HEXBUF และจะเรียกโปรแกรมน้อย BCD2DEC และ SCAN เพื่อแสดงผลทาง LED 7 เซกเมนต์

```
PORT_A    EQU    0FC00H
PORT_B    EQU    0FC01H
PORT_C    EQU    0FC02H
PORT_CON  EQU    0FC03H
```



```
DISBUF    EQU    20H                ; หน่วยความจำ DISBUF สำหรับเก็บข้อมูลแสดงผล
HEXBUF    EQU    30H                ; หน่วยความจำ HEXBUF สำหรับเก็บเลข BCD
,***** MAIN *****
MAIN:     MOV     A,#80H
          MOV     DPTR,#PORT_CON
          MOVX    @DPTR,A
          MOV     HEXBUF,#15H        ; ใส่เลข BCD ค่า 15H ลงในหน่วยความจำหลักแรก
          MOV     HEXBUF+1,#16H      ; ใส่เลข BCD ค่า 16H ลงในหน่วยความจำหลักที่สอง
          ACALL   BCD2DEC            ; เรียกโปรแกรมย่อย BCD2DEC เพื่อปรับค่าตัวเลข
DIS:      ACALL   SCAN              ; เรียกโปรแกรมย่อย SCAN เพื่อแสดงผล
          SJMP    DIS              ; ทำซ้ำ
,*****
; SCAN LED 7
; DPTR,R0,R1,R2
,*****
SCAN:     MOV     R1,#4              ; หน่วยความจำแสดงผล
          MOV     R2,#3
          MOV     R0,#DISBUF+3
LOOP:     MOV     DPTR,#SEG_CODE
          MOV     A,@R0
          MOVC    A,@A+DPTR
          MOV     DPTR,#PORT_A
          MOVX    @DPTR,A
          MOV     A,R2
          MOV     DPTR,#PORT_C
          MOVX    @DPTR,A
          LCALL   DELAY
          DEC     R0
          DEC     R2
          DJNZ    R1,LOOP
          RET
```



```
,*****
```

```
DELAY:  MOV    R3,#0FFH
```

```
DE1:    DJNZ   R3,DE1
```

```
RET
```

```
,*****
```

```
BCD2DEC:
```

```

        PUSH   ACC                ; เก็บค่ารีจิสเตอร์ A ลงหน่วยความจำสแตก
        PUSH   B                  ; เก็บค่ารีจิสเตอร์ B ลงหน่วยความจำสแตก
        MOV    A,HEXBUF           ; อ่านค่าเลข BCD หลักแรกออกมา (ค่า 15H)
        MOV    B,A               ; สำเนาเก็บไว้ในรีจิสเตอร์ B
        SWAP   A                 ; สลับค่าสี่บิตบนกับสี่บิตล่าง
        ANL    A,#00001111B       ; เอาเฉพาะค่า 4 บิตล่าง จะได้ค่า 1 ออกมา
        MOV    DISBUF,A          ; นำเลข 1 ไปเก็บในหน่วยความจำแสดงผล
        MOV    A,B               ; อ่านเลข BCD ค่า 15H เข้ามาใหม่
        ANL    A,#00001111B       ; เอาเฉพาะค่า 4 บิตล่าง จะได้ค่า 5 ออกมา
        MOV    DISBUF+1,A        ; นำไปเก็บในหน่วยความจำแสดงผลหลักที่สอง
        MOV    A,HEXBUF+1        ; อ่านค่าเลข BCD หลักที่สอง
        MOV    B,A               ; สำเนาเก็บไว้ใน B
        SWAP   A                 ; สลับระหว่าง 4 บิตบน และ 4 บิตล่าง
        ANL    A,#00001111B       ; จะได้ค่า 0000 0001 หรือ 1 ออกมา
        MOV    DISBUF+2,A        ; นำไปเก็บในหน่วยความจำแสดงผลหลักที่สาม
        MOV    A,B               ;
        ANL    A,#00001111B       ; จะได้ค่า 0000 0110 หรือค่า 6 ออกมา
        MOV    DISBUF+3,A        ; นำไปเก็บในหลักที่สี่
        POP    B                 ; คืนค่าจากสแตกให้ B
        POP    ACC               ; คืนค่าจากสแตกให้ A
        RET                      ; จบโปรแกรมย่อย

```

```
SEG_CODE:  DB    3FH,06H,5BH,4FH,66H
```

```
           DB    6DH,7DH,07H,7FH,6FH
```

```
           END
```

เมื่อโปรแกรมในตัวอย่างที่ 12.3 ทำงาน ระบบจะแสดงค่าตัวเลข 1516 ออกมาทาง LED 7

เชกเมนต์

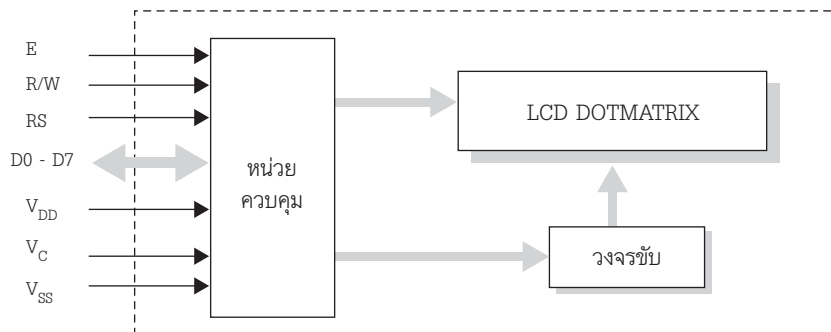


12.3 การแสดงผลทาง LCD

อุปกรณ์ที่ใช้เป็นหน่วยแสดงผลของระบบไมโครคอนโทรลเลอร์มีอยู่หลายชนิด หน่วยแสดงผลแบบ LCD (Liquid Crystal Display) ที่มีโครงสร้างเป็นแบบดอตเมทริกซ์ เป็นอุปกรณ์อีกชนิดหนึ่งที่นิยมนำมาใช้ เนื่องจากเป็นหน่วยแสดงผลที่กินพลังงานต่ำ เหมาะกับระบบที่ต้องพลังใช้งานจากแบตเตอรี่ นอกจากนี้ยังเป็นอุปกรณ์ที่มีขนาดเล็ก มีความละเอียดสูง สามารถแสดงตัวอักษรและรูปภาพได้หลายแบบ การที่จะทำให้ LCD แต่ละดอตสว่างเป็นรูปร่างหรือตัวอักษรต่าง ๆ นั้นจะต้องใช้วงจรขับที่มีความซับซ้อนมาก ในปัจจุบันจึงมีการสร้างเป็นบอร์ด LCD สำเร็จรูป (LCD Module) เพื่อให้สะดวกต่อการใช้งาน โดยโครงสร้างภายในประกอบด้วย จอภาพแบบ LCD วงจรขับสัญญาณ และตัวควบคุม LCD การทำให้บอร์ด LCD แสดงผลลักษณะต่าง ๆ จะทำได้โดยการเขียนคำสั่งควบคุม และเขียนข้อมูลให้กับ LCD โมดูลเท่านั้น

โครงสร้างทั่วไปของ LCD โมดูล

หน่วยแสดงผลแบบ LCD โมดูลที่พบทั่วไปในท้องตลาดจะมีโครงสร้างที่สะดวกต่อการใช้งาน โดยส่วนประกอบทั่วไปเป็นดังรูปที่ 12.6



รูปที่ 12.6 โครงสร้างทั่วไปของ LCD โมดูล

ส่วนที่เป็น LCD DOT MATRIX จะประกอบด้วยจุดขนาดเล็กจำนวนมาก โดยแต่ละจุดจะถูกบังคับให้ติดสว่างเป็นรูปร่างต่าง ๆ ด้วยวงจรขับ ส่วนหน่วยควบคุมจะประกอบด้วยหน่วยความจำและรีจิสเตอร์ต่าง ๆ ทำหน้าที่กำหนดลักษณะการทำงานของ LCD รูปภาพของการแสดงผล รวมทั้งควบคุมส่วนต่าง ๆ ภายในโมดูล

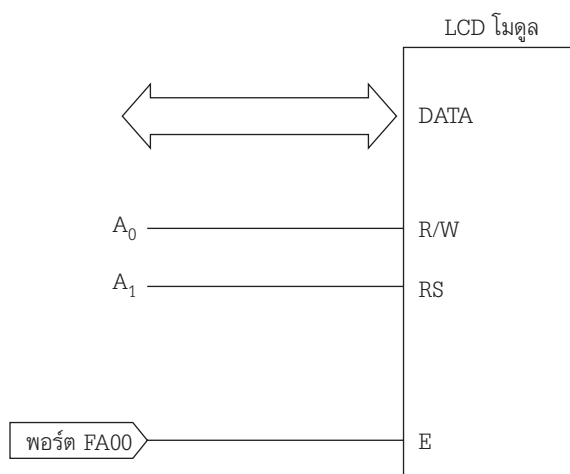
โดยทั่วไปแล้วขาต่าง ๆ ของ LCD โมดูลประกอบด้วย

1. ขา V_{DD} ขานี้จะต่อกับแหล่งจ่ายไฟ +5 โวลต์
2. ขา V_{SS} ขานี้จะต่อกับกราวด์ของแหล่งจ่ายไฟ



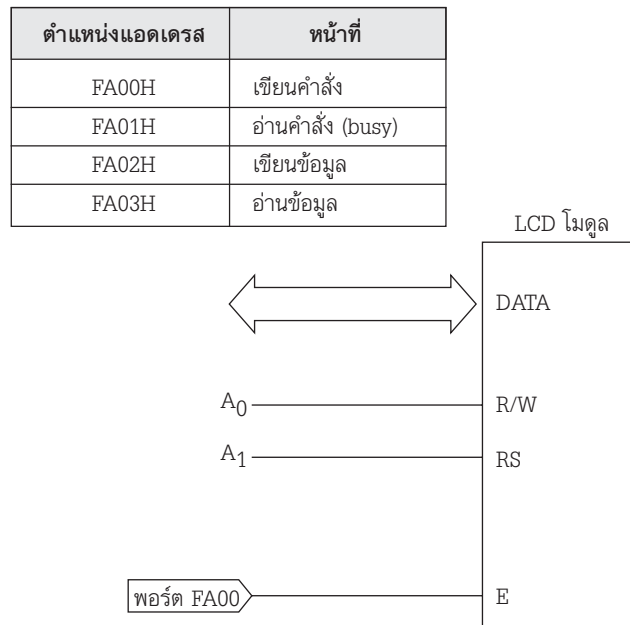
3. ขา V_c ขานี้เป็นขาที่ควบคุมความสว่างของ LCD
4. ขา RS เป็นขาที่ใช้ควบคุมรีจิสเตอร์ภายในหน่วยควบคุม เนื่องจากมีรีจิสเตอร์อยู่สองประเภท คือรีจิสเตอร์คำสั่ง (command register) และรีจิสเตอร์ข้อมูล (data register) ถ้าหากขานี้เป็นลอจิก “0” หมายความว่าเลือกรีจิสเตอร์ข้อมูล ถ้าเป็นลอจิก “1” หมายความว่าเลือกรีจิสเตอร์คำสั่ง
5. ขา R/W เป็นขาสัญญาณที่ใช้เลือกว่าจะอ่านหรือเขียนข้อมูลให้กับ LCD ถ้าเป็นลอจิก “0” หมายความว่าอ่านข้อมูล แต่ถ้าเป็นลอจิก “1” หมายความว่าเขียนข้อมูล
6. ขา E (enable) เป็นขากำหนดลักษณะการอ่านเขียนข้อมูล

สำหรับการเชื่อมต่อ LCD โมดูลเข้ากับระบบไมโครคอนโทรลเลอร์นั้นทำได้หลายวิธี โดยอาจต่อ LCD โมดูลเข้ากับไมโครคอนโทรลเลอร์โดยตรง โดยให้ไมโครคอนโทรลเลอร์มองว่าเป็นชิปตัวหนึ่งคล้ายกับหน่วยความจำ การต่อแบบนี้เรียกว่าการต่อแบบ memory map แต่จะทำให้เสียเนื้อที่หน่วยความจำบางส่วนไป การติดต่อระหว่างไมโครคอนโทรลเลอร์กับ LCD โมดูลจะใช้คำสั่งเหมือนกับการติดต่อกับหน่วยความจำ อีกวิธีหนึ่งเราอาจต่อ LCD โมดูลเข้ากับพอร์ตอินพุตเอาต์พุตของระบบที่มีอยู่แล้ว จากนั้นใช้การเขียนโปรแกรมสร้างสัญญาณขึ้นมาเพื่อติดต่อกับ LCD โมดูล นอกจากนี้สัญญาณข้อมูลที่จะส่งให้กับ LCD โมดูลสามารถเลือกใช้แบบ 8 บิตหรือ 7 บิตได้อีกด้วย ตัวอย่างการออกแบบวงจรการเชื่อมต่อ LCD โมดูลดังรูปที่ 12.7



รูปที่ 12.7 วงจรอย่างง่ายสำหรับการเชื่อมต่อ LCD โมดูลกับระบบ JAZZ-31

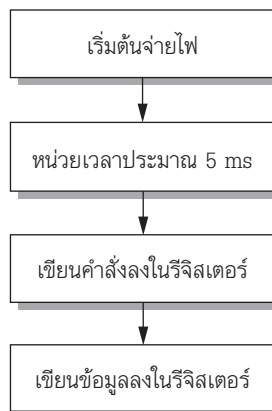
ถ้าหากมีการถอดรหัสพอร์ตสำหรับต่อกับ LCD โมดูลเป็นแบบ memory map ดังรูปที่ 12.8 จะพบว่าแอดเดรสต่าง ๆ สำหรับติดต่อกับ LCD โมดูลจะเป็น FA00 - FB00H แอดเดรสต่าง ๆ จะใช้ติดต่อกับ LCD โมดูลจะเป็นดังตารางต่อไปนี้



รูปที่ 12.8 ตัวอย่างการออกแบบพอร์ตเชื่อมต่อกับ LCD โมดูล

การใช้งาน LCD โมดูลเบื้องต้น

ในการใช้งาน LCD โมดูลเราจะต้องทำความเข้าใจคำสั่งต่าง ๆ ของมันด้วย โดยการเริ่มต้นใช้งานจะต้องส่งรหัสควบคุมให้กับรีจิสเตอร์คำสั่ง (instruction register) และส่งข้อมูลให้กับรีจิสเตอร์ข้อมูล (data register) โดยมีลำดับการทำงานดังรูปที่ 12.9



รูปที่ 12.9 ขั้นตอนการควบคุม LCD โมดูล



ในรูปที่ 12.9 จะพบว่าหลังจากจ่ายไฟให้กับ LCD โมดูลจะต้องหน่วยเวลาประมาณ 15 ms เนื่องจากเมื่อเริ่มจ่ายไฟจะต้องรอให้แรงดันไฟเปลี่ยนจาก 0 โวลต์ไปจนถึง 4.5 โวลต์เสียก่อนจึงทำงานได้ แต่โดยทั่วไปแล้วขั้นตอนนี้อาจไม่จำเป็นเสมอไป เพราะเมื่อนำ LCD ไปต่อกับระบบไมโครคอนโทรลเลอร์ เมื่อระบบได้รับพลังงานไฟฟ้า ไมโครคอนโทรลเลอร์จะเสียเวลาการทำงานต่าง ๆ ไปก่อน ซึ่งเป็นช่วงเวลาที่นานพอที่ LCD โมดูลจะรับคำสั่งต่าง ๆ ได้ ขั้นตอนต่อไปจะต้องเขียนคำสั่งควบคุมต่าง ๆ ให้กับรีจิสเตอร์คำสั่งของ LCD โมดูล โดยมีคำสั่งหลายคำสั่งสำหรับกำหนดการทำงานของ LCD โมดูล เช่นต้องการให้รับข้อมูลแบบ 4 บิตหรือ 8 บิต ต้องการให้มี cursor หรือไม่ เมื่อแสดงข้อมูลแล้วจะให้ cursor เลื่อนไปทางใด โดยคำสั่งต่าง ๆ แ่ ดังได้ตั้งตารางที่ 12.3 และการทำงานแต่ละคำสั่งจะต้องใช้เวลาในการทำงานด้วย ถ้าหาก LCD โมดูลทำคำสั่งยังไม่เรียบร้อยจะไม่สามารถเขียนคำสั่งใหม่เข้าไปได้ ต่อจากนั้นเมื่อต้องการแสดงข้อมูลหรือตัวอักษรในตำแหน่งต่าง ๆ ของจอภาพ จะต้องเขียนข้อมูลนั้นให้กับรีจิสเตอร์ข้อมูล (data register) เพื่อให้ส่งข้อมูลไปยังหน่วยความจำสำหรับแสดงผลของ LCD โมดูล

ตารางที่ 12.3 รูปแบบของคำสั่งควบคุมและการเขียนข้อมูล รวมทั้งสถานะของขาสัญญาณต่าง ๆ ที่จำเป็น

คำสั่ง	RS	R/W	บิตข้อมูล								เวลาทำงาน
			7	6	5	4	3	2	1	0	
CLEAR DISPLAY	0	0	0	0	0	0	0	0	0	1	1640 us
CURSOR AT HOME	0	0	0	0	0	0	0	0	1	*	1640 us
ENTRY MODE SET	0	0	0	0	0	0	0	1	I/D	S	40 us
DISPLAY ON/OFF	0	0	0	0	0	0	1	D	C	B	40 us
DISPLAY SHIFT	0	0	0	0	0	1	S/C	R/L	*	*	40 us
FUNCTION SET	0	0	0	0	1	DL	N	F	*	*	40 us
SET CGRAM ADD	0	0	0	1	CGRAM ADDRESS						40 us
SET DDRAM ADD	0	0	1	DDRAM ADDRESS							0 us
BUSY, ADD. READ	0	1	BF	ADDRESS							0 us
WR CGRAM,DDRAM	1	0	WRITE DATA								40 us
RD CGRAM,DDRAM	1	1	READ DATA								40 us

เราจะสรุปได้ว่าการเขียนข้อมูลให้กับ LCD โมดูล จะมีสองลักษณะคือเป็นคำสั่ง และเป็นข้อมูล ซึ่งจะถูกกำหนดด้วยขา RS ถ้าขา RS = 0 หมายความว่า เป็นคำสั่งควบคุม ถ้า RS = 1 หมายความว่า เป็นข้อมูล และในการเขียนข้อมูลตัว LCD โมดูลจะต้องใช้เวลาในการทำงานดังที่แสดงในตารางที่ 12.3 โดยระบบไมโครคอนโทรลเลอร์สามารถตรวจสอบเวลาการทำงานของ LCD โมดูลได้ทางแฟลก BF (Busy Flag) ถ้าแฟลก BF เป็นลอจิก “0” หมายความว่า LCD โมดูลสามารถรับข้อมูลต่อไปได้ ดังนั้นการเขียนข้อมูลทุกครั้งควรตรวจสอบแฟลกนี้ด้วย

รหัสควบคุมในตารางที่ 12.3 สามารถเขียนอีกแบบหนึ่งได้ดังตารางที่ 12.4



ตารางที่ 12.4 รหัสคำสั่งและการทำงานของ LCD โมดูล

คำสั่ง (HEX)	การทำงาน
01	เคลียร์หน่วยแสดงผล
02	ให้เคอร์เซอร์กลับสู่ตำแหน่งซ้ายสุด (HOME)
04	แ ดงผลโดยเลื่อนเคอร์เซอร์ไปทางซ้าย
05	เลื่อนไปทางขวา
06	แ ดงผลโดยเลื่อนเคอร์เซอร์ไปทางขวา
07	เลื่อนไปทางซ้าย
08	ปิดการแสดงผล ไม่แสดงเคอร์เซอร์
0A	ปิดการแสดงผล แต่แสดงเคอร์เซอร์
0C	แ ดงผลแต่ไม่แสดงเคอร์เซอร์
0E	แ ดงผลและแสดงเคอร์เซอร์
0F	แ ดงผล และแสดงเคอร์เซอร์กระพริบ
10	เลื่อนเคอร์เซอร์ไปทางซ้าย
14	เลื่อนเคอร์เซอร์ไปทางขวา
18	เลื่อนตัวอักษรตัวใหม่ไปทางซ้าย
1C	เลื่อนตัวอักษรตัวใหม่ไปทางขวา
80	ตำแหน่งเริ่มต้นบรรทัดแรก
C0	ตำแหน่งเริ่มต้นบรรทัดที่สอง
38	เป็นแบบ 2 บรรทัด ขนาดตัวอักษร 5 × 7 จุด

สำหรับหน่วยความจำภายใน LCD โมดูลมีอยู่สองชนิดคือ DDRAM (Display Data RAM) และ CGRAM (Character Generator RAM) โดย DDRAM เป็นหน่วยความจำภายในตัว LCD โมดูล ถ้าหากเขียนค่ารหัส ASCII ลงไปในหน่วยความจำจะทำให้ LCD โมดูลแสดงตัวอักษร ส่วน CGRAM เป็นหน่วยความจำที่ผู้ใช้สามารถสร้างภาพตัวอักษรเองได้

ดังนั้นถ้าหากเราต้องการให้ LCD โมดูลแสดงข้อความต่าง ๆ หลังจากเขียนคำสั่งควบคุมต่าง ๆ ลงไปแล้ว จะต้องเขียนข้อมูลรหัส ASCII ที่จะแสดงผลให้กับแอดเดรสของตำแหน่งต่าง ๆ บน LCD โมดูล สำหรับการกำหนดแอดเดรสของ DDRAM เมื่อทำการกำหนดไว้แล้วการอ่านเขียนข้อมูลต่อจากนี้ จะเป็นไปตามแอดเดรสที่กำหนดทันที โดยแอดเดรสของ LCD โมดูลแต่ละรุ่นจะแตกต่างกันบ้าง แอดเดรสของ LCD โมดูลบางรุ่นเป็นดังนี้

รุ่น DMC 161 จะแสดงผลแบบบรรทัดเดียว 16 ตัวอักษร

80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

รุ่น DMC 162 จะแสดงผลแบบสองบรรทัด บรรทัดละ 16 ตัวอักษร

80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF



จากตารางคำสั่งจะเห็นว่าการเขียนข้อมูลแอดเดรสให้ DDRAM จะกำหนดให้บิต 7 เป็นลอจิก “1” เสมอ เพื่อให้สะดวกต่อการเรียกใช้และเป็นแอดเดรสที่สอดคล้องกับ LCD โมดูลแต่ละรุ่น สำหรับการเขียนข้อมูลลงหน่วยความจำ DDRAM เมื่อเขียนข้อมูลลงไปแล้วค่าแอดเดรสจะเพิ่มขึ้นหรือลดลงโดยอัตโนมัติ โดยถูกกำหนดจากค่า I/D ในคำสั่ง Entry mode set

ถ้าหากระบบ MCS- 51 มี LED โมดูลแบบสองบรรทัดต่อกับพอร์ตดังรูปที่ 12.8 การเขียนโปรแกรมแสดงผลทาง LCD โมดูลจะทำได้โดยเขียนคำสั่งควบคุมต่าง ๆ ลงไปก่อน จากนั้นเขียนรหัส ASCII ของตัวอักษรที่ต้องการแสดงผลออกไปยัง DDRAM ของ LCD โดยที่ตัวอักษรตัวแรกของบรรทัดแรกจะเริ่มที่แอดเดรส 80H และตัวอักษรตัวแรกของบรรทัดที่สองจะเริ่มที่แอดเดรส C0H ดังตัวอย่างที่ 12.4

ตัวอย่างที่ 12.4 โปรแกรมในตัวอย่างนี้จะแสดงผลทาง LCD โมดูลแบบสองบรรทัดที่ต่อกับวงจรดังรูปที่ 12.8 ถ้าหากต้องการใช้กับ LCD ที่ต่อกับพอร์ตหมายเลขอื่น ๆ ก็สามารถทำได้โดยการเปลี่ยนค่าแอดเดรสของพอร์ต จากโปรแกรมเมื่อระบบทำงานจะแสดงคำว่า TEERAWAT KMITL.. ออกทางบรรทัดแรกของ LCD โดยในโปรแกรมจะเก็บตัวอักษรที่ต้องการแสดงผลในแอดเดรสที่มีลาเบลชื่อ SHOW_DATA และสร้างโปรแกรมน้อยชื่อ COMMAND สำหรับเขียนคำสั่งลงบน LCD โมดูล โปรแกรมย่อยชื่อ DATA_DISPLAY สำหรับเขียนข้อมูลที่ต้องการแสดงผล

```

LCDCOM EQU 0FA00H ; แอดเดรสของพอร์ตเขียนคำสั่ง
LCDBUS EQU 0FA01H ; แอดเดรสของพอร์ตอ่านคำสั่ง
LCDWR EQU 0FA02H ; แอดเดรสของพอร์ตเขียนข้อมูล
LCDRD EQU 0FA03H ; แอดเดรสของพอร์ตอ่านข้อมูล
ORG 0000H

MAIN: CALL LCD_INT ; เรียกโปรแกรมน้อยเขียนคำสั่งควบคุม
      MOV A,#80H ; กำหนดค่าแอดเดรสของบรรทัดแรก
      CALL COMMAND ; เรียกโปรแกรมน้อยเขียนคำสั่ง
      MOV DPTR,#SHOW_DATA ; แอดเดรสของข้อมูลที่ต้องการแสดงผล
      CALL SHOW_LINE ; เรียกโปรแกรมน้อยแสดงผลหนึ่งบรรทัด
      SJMP $ ; หยุดโปรแกรม

;*****
SHOW_LINE: ; โปรแกรมย่อยแสดงผลหนึ่งบรรทัด
      MOV R3,#16D ; หนึ่งบรรทัดมี 16 ตัวอักษร
LOOP: CLR A
      MOVC A,@A+DPTR ; อ่านรหัส ASCII ตัวอักษรออกมาหนึ่งตัว

```



```
CALL DATA_DISPLAY      ; เรียกโปรแกรมย่อยแสดงผลหนึ่งตัวอักษร
INC DPTR                ; ให้ DPTR ชี้ที่ตัวอักษรถัดไป
DJNZ R3,LOOP            ; ทำซ้ำจนครบ 16 ตัวอักษร
RET                     ; จบโปรแกรมย่อย
```

; โปรแกรมย่อย LCD_INT สำหรับกำหนดค่าเริ่มต้นของ LCD โดยเป็นคำสั่งตามตารางที่ 12.4

```
LCD_INT: MOV A,#00111000B      ; ให้ LCD ทำงานแบบ 8 บิต ขนาด 5 × 7 จุด
CALL COMMAND            ; เขียนคำสั่งลงไป
MOV A,#00001110B        ; ให้แสดงผลและแสดงเคอร์เซอร์
CALL COMMAND            ; เขียนคำสั่งลงไป
MOV A,#01H              ; ให้เคลียร์หน่วยความจำแสดงผล
CALL COMMAND            ;
MOV A,#06H              ; ให้แสดงผลโดยเลื่อนเคอร์เซอร์ไปทางขวา
CALL COMMAND            ;
CALL DELAY              ; เรียกโปรแกรมย่อยหน่วงเวลา
RET                     ; จบโปรแกรมย่อย
```

; โปรแกรมย่อยสำหรับเขียนคำสั่ง โดยรับคำสั่งเข้ามาทางรีจิสเตอร์ A และใช้ DPTR ชี้ตำแหน่ง

```
COMMAND: PUSH DPH
        PUSH DPL
        PUSH ACC          ; เก็บค่ารีจิสเตอร์ A ลงสแตก
        MOV DPTR,#LCDBUS  ; ชี้ไปที่แอดเดรสอ่านคำสั่ง
READY1: MOVX A,@DPTR       ; อ่านความพร้อมของ LCD โดยดูว่าบิต D7
        JB ACC.7,READY1   ; เป็นลอจิก “1” หรือไม่ ถ้าเป็นให้ทำซ้ำ
        POP ACC           ; อ่านค่ารีจิสเตอร์ A กลับมา
        MOV DPTR,#LCDCOM  ; ชี้ไปที่แอดเดรสพอร์ตคำสั่ง
        MOVX @DPTR,A      ; เขียนคำสั่งลงไป
        POP DPL
        POP DPH
        RET
```

; โปรแกรมย่อยแสดงผล โดยจะเขียนรหัส ASCII ที่รับเข้ามาทางรีจิสเตอร์ A ลง LCD โมดูล

```
DATA_DISPLAY: PUSH DPH
              PUSH DPL
```



```

                                PUSH    ACC                ; เก็บค่ารีจิสเตอร์ A ลงสแตก
                                MOV     DPTR,#LCDBUS
READY2:  MOVX    A,@DPTR
                                JB      ACC.7,READY2       ; ตรวจสอบความพร้อมของ LCD
                                POP     ACC
                                MOV     DPTR,#LCDWR
                                MOVX    @DPTR,A           ; เขียนข้อมูลรหัส ASCII ลงไป
                                POP     DPL
                                POP     DPH
                                RET
                                RET

DELAY:   MOV     R1,#00
DE1:     MOV     R2,#00
                                DJNZ    R2,$
                                DJNZ    R1,DE1
                                RET

SHOW_DATA: DB    'TEERAWAT KMITL..' ; เก็บรหัส ASCII ที่ต้องการแสดงผล
                                END

```

จากตัวอย่างที่ผ่านมาจะเห็นว่า การแสดงผลทาง LCD โมดูลนั้นจะต้องเขียนคำสั่งต่าง ๆ หลายขั้นตอน และการแสดงผลทาง LCD โมดูลนี้สามารถนำไปประยุกต์กับงานต่าง ๆ ได้จำนวนมาก ในตัวอย่างที่ 12.5 จะเป็นการสร้างไฟล์โปรแกรมรวม หรือ INCLUDE ไฟล์ เก็บโปรแกรมย่อยต่าง ๆ ในการติดต่อกับ LCD โมดูล ซึ่งสามารถนำไปเรียกใช้ในโปรแกรมอื่น ๆ ได้

ตัวอย่างที่ 12.5 ไฟล์ต่อไปนี้จะ เป็นโปรแกรมรวม หรือ INCLUDE FILE โดยจะให้ชื่อว่า LCDIOH.ASM ซึ่งสามารถนำไปเรียกใช้ด้วยคำสั่ง “\$INCLUDE LCDIOH.ASM” ในโปรแกรมต่าง ๆ ที่ต้องการติดต่อกับ LCD โมดูลได้ โดยจะสร้างโปรแกรมย่อยต่าง ๆ ดังนี้

LCD_INT	สำหรับกำหนดค่าเริ่มต้นการทำงานของ LCD โมดูล
COMMAND	สำหรับเขียนคำสั่ง โดยจะรับรหัสคำสั่งเข้ามาทางรีจิสเตอร์ A
DATA_DISPLAY	ใช้ในการเขียนไบนารีรหัส ASCII ลงไปบน LCD โมดูล
SHOW_LINE	ใช้สำหรับการแสดงผลหนึ่งบรรทัด หรือ 16 ตัวอักษรบน LCD โมดูล



; โปรแกรม LCDIO.ASM

```
LCD_INT:      MOV    A,#00111000B
               CALL   COMMAND
               MOV    A,#00001110B
               CALL   COMMAND
               MOV    A,#01H
               CALL   COMMAND
               MOV    A,#06H
               CALL   COMMAND
               CALL   DELAY
               RET

COMMAND:       PUSH   DPH
               PUSH   DPL
               PUSH   ACC
               MOV    DPTR,#LCDBUS

READY1:        MOVX   A,@DPTR
               JB     ACC.7,READY1
               POP    ACC
               MOV    DPTR,#LCDCOM
               MOVX   @DPTR,A
               POP    DPL
               POP    DPH
               RET

DATA_DISPLAY:  PUSH   DPH
               PUSH   DPL
               PUSH   ACC
               MOV    DPTR,#LCDBUS

READY2:        MOVX   A,@DPTR
               JB     ACC.7,READY2
               POP    ACC
               MOV    DPTR,#LCDWR
               MOVX   @DPTR,A
```



```

                                POP    DPL
                                POP    DPH
                                RET

DELAY:MOV    R1,#00
DE1:        MOV    R2,#00
            DJNZ    R2,$
            DJNZ    R1,DE1
            RET

SHOW_LINE:
            MOV    R3,#16D           ; แสดงผล 16 ตัวอักษร
LOOP:       CLR    A
            MOV    A,@R0
            CALL   DATA_DISPLAY
            INC    R0
            DJNZ    R3,LOOP
            RET
```



บทที่ 13

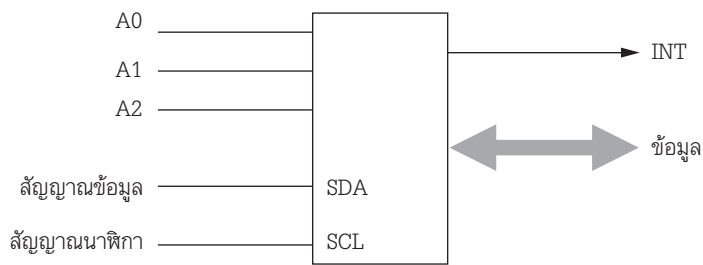
การทดลองกับ พอร์แบบ I²C

ตามที่ทราบมาแล้วว่าไมโครคอนโทรลเลอร์จะติดต่อกับอุปกรณ์ภายนอกโดยใช้พอร์ตอินพุตเอาต์พุต โดยพอร์ตอินพุตจะต้องเลือกอุปกรณ์ที่ทำหน้าที่เป็นบัฟเฟอร์ ส่วนเอาต์พุตจะต้องเลือกอุปกรณ์ที่สามารถ แลตซ์ข้อมูลได้ ซึ่งโดยทั่วไปแล้วจะเลือกใช้ฟลิปฟล็อป นอกจากนี้ยังมีไอซีสำเร็จรูปที่นิยมนำมาใช้เป็นพอร์ต โดยสามารถโปรแกรมให้เป็นอินพุตเอาต์พุตได้เช่นเบอร์ 8255 แต่การเชื่อมต่อไอซีที่ทำหน้าที่เป็นพอร์ต หรือ ไอซี 8255 ส่วนใหญ่แล้วจะต้องใช้สายสัญญาณบัสข้อมูลที่มีจำนวนเท่ากับจำนวนบิตของพอร์ต ในปัจจุบัน ได้มีการพัฒนาไอซีที่ทำหน้าที่เป็นอินพุตเอาต์พุตได้โดยเชื่อมต่อกับไมโครคอนโทรลเลอร์แบบอนุกรมได้ โดยระบบบัสนิยมนิยมนำมาใช้จะเป็นระบบบัสแบบ I²C ในที่นี้จะแนะนำไอซีเบอร์ PCF8574



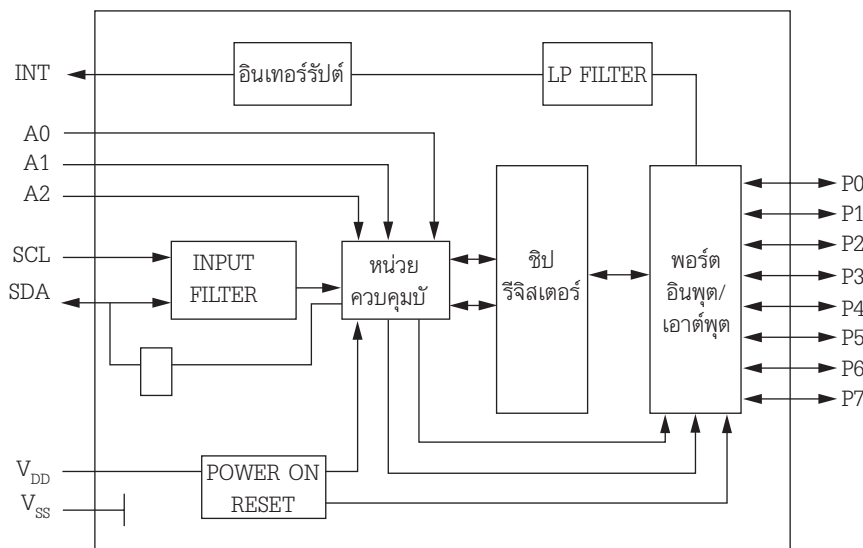
13.1 การใช้งานไอซี PCF8574

ไอซี PCF8574 เป็นไอซีที่ใช้เป็นพอร์ตอินพุตเอาต์พุตขนาด 8 บิต ที่กินพลังงานต่ำมาก โดยจะเชื่อมต่อกับไมโครคอนโทรลเลอร์แบบอนุกรมโดยใช้สัญญาณสองเส้น เส้นหนึ่งเป็นสัญญาณข้อมูลแบบสองทิศทาง อีกเส้นหนึ่งเป็นสัญญาณนาฬิกา เอาต์พุตสามารถแลตซ์ข้อมูลได้และสามารถขับกระแสสำหรับหลอด LED ได้ นอกจากนี้ยังมีสัญญาณอินเทอร์รัปต์ที่สามารถต่อกับไมโครคอนโทรลเลอร์ได้อีกด้วย ขาที่สำคัญของไอซีเบอร์นี้แสดงได้ดังรูปที่ 13.1 สำหรับรายละเอียดการใช้งานต่าง ๆ ให้ศึกษาจากคู่มือของไอซีโดยตรง



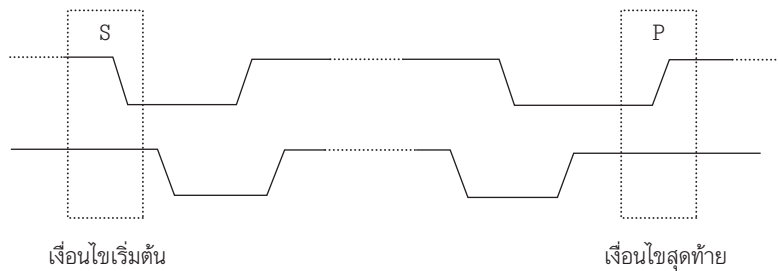
รูปที่ 13.1 ลักษณะขาของ PCF8574

สำหรับไดอะแกรมโครงสร้างภายในของ PCF8574 เป็นดังรูปที่ 13.2 โดยขา SCL จะเป็นตัวกำหนดจังหวะของข้อมูลที่เข้าหรือออกทางขา SDA ส่วนในการติดต่อกับอุปกรณ์อินพุตเอาต์พุตแบบขนานนั้นจะมีตัวชิปรีจิสเตอร์ทำหน้าที่เปลี่ยนข้อมูลระหว่างข้อมูลแบบขนานกับแบบอนุกรม สำหรับขาอินเทอร์รัปต์จะเป็นสัญญาณเอาต์พุตออกจากไอซีเพื่อต่อกับไมโครคอนโทรลเลอร์ แต่ในการใช้งานอย่างง่ายมักจะไม่ใช้ขา

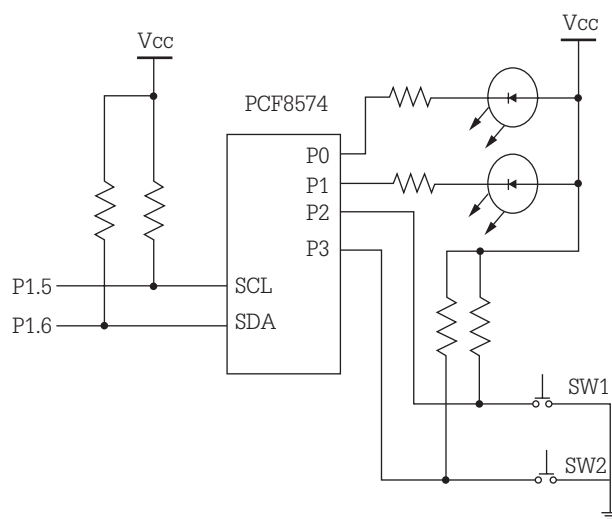


รูปที่ 13.2 ไดอะแกรมโครงสร้างภายในของ PCF8574

การเชื่อมต่อขาสัญญาณนาฬิกาและขาข้อมูลกับไมโครคอนโทรลเลอร์นั้นสามารถเชื่อมต่อผ่านทางพอร์ต P1 ได้ ดังตัวอย่างการเชื่อมต่อในรูปที่ 13.4 จะเชื่อมต่อทางขา P1.5 และ P1.6 โดยใช้ขา P1.6 เป็นขารับส่งข้อมูล และเขียนโปรแกรมสร้างสัญญาณนาฬิกาออกมาทางขา P1.5 โดยการติดต่อระหว่างไมโครคอนโทรลเลอร์กับ PCF8574 เงื่อนไขเริ่มต้นและเงื่อนไขสุดท้ายจะต้องเป็นไปตามไดอะแกรมรูปที่ 13.3 ดังนั้นการเขียนโปรแกรมติดต่อกับ PCF8574 จะต้องสร้างสัญญาณตามรูปนี้ออกมาซึ่งเป็นไปตามโปรโตคอลของระบบบัสแบบ I²C



รูปที่ 13.3 ไตอะแกรมเงื่อนไขการติดต่อกับ PCF8574



รูปที่ 13.4 การเชื่อมต่อพอร์ต PCF8574 กับพอร์ต P1 ของ MCS-51

การอ่านเขียนข้อมูลให้กับ PCF8574 เราต้องเขียนโปรแกรมติดต่อทางขา P1.5 และ P1.6 โดยการอ่านเขียนข้อมูล 1 ไบต์หรือ 8 บิตนั้นไมโครคอนโทรลเลอร์จะต้องส่งสัญญาณนาฬิกาออกไป 8 ลูก เมื่อรับหรือส่งข้อมูลครบแล้ว และ PCF8574 ทำงานถูกต้องมันจะส่งสัญญาณตอบรับออกมาเป็นลอจิก “0” โดยการอ่านสัญญาณตอบรับนี้จะต้องส่งสัญญาณนาฬิกาเข้าไปอีก 1 ลูก ไตอะแกรมเวลาการติดต่อกับ PCF8574 แด่ ดังได้ดังรูปที่ 13.5

เนื่องจาก PCF8574 สามารถเชื่อมต่อโดยใช้โปรโตคอลการส่งข้อมูลแบบสองทิศทางได้เป็นอย่างดี โดยอุปกรณ์ที่ทำหน้าที่ควบคุมการรับส่งข้อมูลจะเรียกว่า master ส่วนตัวที่ถูกควบคุมจะเรียกว่า slaves ในที่นี้ PCF8574 จะถูกควบคุมจึงทำหน้าที่เป็น slaves ในการเขียนข้อมูลแต่ละครั้งจะต้องใส่ค่าแอดเดรสของ PCF8574 ที่ต่ออยู่กับระบบเข้าไปก่อน จากนั้นจึงจะส่งหรือรับข้อมูลได้ จากคู่มือของ PCF8574 ค่าแอดเดรสของ slaves จะเป็นดังรูปข้างล่างนี้ โดยที่ A2, A1 และ A0 เป็นแอดเดรสที่สามารถโปรแกรมได้ ซึ่งจะเป็นค่าทางลอจิกที่ขาของ PCF8574 ต่ออยู่ในวงจร ถ้าหากระบบ MCS-51 ที่ออกแบบขึ้นมีไอซีที่เชื่อมต่อแบบ I²C เพียงตัวเดียวควรต่อให้ขาแอดเดรสที่โปรแกรมได้นี้เป็นลอจิก “0” ทั้งหมด



0	1	1	1	A2	A1	A0	0
---	---	---	---	----	----	----	---

สำหรับการทดลองในเรื่องนี้จะเขียนโปรแกรมย่อยสำหรับการอ่านเขียนข้อมูลขึ้นมาสำหรับติดต่อกับพอร์ต PCF8574 สามารถใช้โปรแกรมย่อยนี้ได้โดยตรง โดยจะสร้างเป็นโปรแกรมร่วมชื่อ IO8574H โดยในโปรแกรมร่วมนี้จะเขียนโปรแกรมย่อยสำหรับอ่านเขียนข้อมูลกับพอร์ต รวมทั้งการสร้างสัญญาณต่าง ๆ ในการติดต่อกับระบบบัสแบบ I²C

ตัวอย่างที่ 13.1 โปรแกรมร่วม IO8574H.ASM

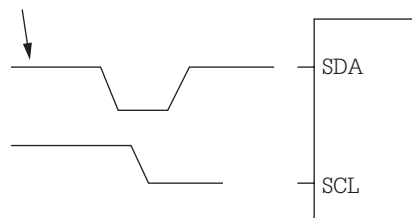
,***** IPW SUB *****

```
; PCF8574 WRITE PORT                ; โปรแกรมย่อยเขียนข้อมูลลงพอร์ต โดยให้
; IN  = R2 ADDRESS (00000XXXB)      ; R2 เป็นค่าแอดเดรส และให้
;          R3 DATA                  ; R3 เป็นค่าข้อมูล
; OUT = ERRFAG 0 = OK 1 = ERROR
; REG = A,R2
```

,*****

```
IPW:  SETB  RTCSDA
      SETB  RTCSCL
      CLR   RTCSDA
      CALL  CDEL
      CLR   RTCSCL
      SETB  RTCSDA
      MOV   A,R2
      RL    A
      ANL   A,#00001110B
      ORL   A,#01110000B
      CALL  IPWRB
      JB    ERRFAG,IPW9
      MOV   A,R3
      CALL  IPWRB
```

; สร้างสัญญาณลักษณะดังรูป



; กำหนดค่าแอดเดรสของ slave

; เรียกโปรแกรมย่อยเขียนไบต์แอดเดรสลงไป

; ถ้ามีการผิดพลาดให้กระโดดไปที่ IPW9

; นำค่าข้อมูลใส่ในรีจิสเตอร์ A

; เรียกโปรแกรมย่อยเขียนไบต์ข้อมูลลงไป



```

IPW9:    CLR    RTCSDA
          SETB   RTCSCL
          CALL   CDEL
          SETB   RTCSDA
          RET

```

ร่างเงื่อนไขสุดท้ายตามรูปที่ 13.3

```

;***** IPWRB SUB *****
;

```

```

; PCF8574 WRITE BYTE
; IN    =  A
; OUT   =  ERRFAG  0 = OK  1 = ERROR
; REG   =  A,R2

```

โปรแกรมย่อยเขียนข้อมูลลงไปที่ละบิต
โดยรับข้อมูลมาเก็บใน A

```

;*****
;

```

```

IPWRB:    CLR    ERRFAG
          MOV     R2,#8
IPWRB1:   RLC     A
          MOV     RTCSDA,C
          CALL    CHIGH
          CALL    CLOW
          DJNZ    R2,IPWRB1
          SETB    RTCSDA
          CALL    CHIGH
          JNB     RTCSDA,IPWRB2
          SETB    ERRFAG
IPWRB2:   CALL    CLOW
          RET

```

; ข้อมูลหนึ่งไบต์มี 8 บิต
; เลื่อนข้อมูลในรีจิสเตอร์ A เข้าไปในแฟลค
; ส่งข้อมูลจากแฟลคออกไป
; สัญญาณนาฬิกาหนึ่งลูก
; ทำจนครบ 8 บิต
; เซตให้เป็นลอจิก “1”
; ใส่สัญญาณนาฬิกาลอจิก “1”
; รอสัญญาณตอบรับจาก PCF8574

```

;***** IPR SUB *****
;

```

```

; PCF8574 READ PORT
; IN    =  R2  ADDRESS (00000XXB)
; OUT   =  R3  DATA : ERRFAG 0 = OK  1 = ERROR
; REG   =  A,R2,R3

```

โปรแกรมย่อยใช้อ่านข้อมูล
โดยส่งค่าแอดเดรสเข้าไป
และอ่านข้อมูลออกมาทาง R3

```

;*****
;

```



```

IPR:    SETB    RTCSDA
        SETB    RTCSCL
        CLR     RTCSDA
        CALL    CDEL
        CLR     RTCSCL
        MOV     R3,#0
        MOV     A,R2
        RL      A
        ANL     A,#00001110B
        ORL     A,#01110001B
        CALL    IPWRB                ; เขียนค่าแอดเดรส slave ลงไป
        JB      ERRFAG,IPR9
        CALL    IPRDB                ; เรียกโปรแกรมย่อยอ่านค่าข้อมูลจากตัวไอซี
        JB      ERRFAG,IPR9
        MOV     R3,A                ; นำข้อมูลที่อ่านได้มาเก็บใน R3
IPR9:    CLR     RTCSDA
        SETB    RTCSCL
        CALL    CDEL
        SETB    RTCSDA
        RET

```

```

,***** IPRDB SUB *****
,

```

```

; PCF8574 READ BYTE                ; โปรแกรมย่อยอ่านข้อมูลออกมา 1 ไบต์
; OUT = A : ERRFAG 0 = OK 1 = ERROR
; REG = A,R2

```

```

,*****
,

```

```

IPRDB:    CLR     ERRFAG
        MOV     R2,#8                ; ข้อมูล 1 ไบต์มี 8 บิต
IPRDB1:   CALL    CHIGH              ; ส่งสัญญาณนาฬิกาให้เป็น "1"
        MOV     C,RTCSDA            ; อ่านค่าข้อมูลเข้ามาในแฟลก
        RLC     A                    ; หมุนค่าในแฟลกเข้ามาในรีจิสเตอร์ A
        CALL    CLOW                ; ส่งสัญญาณนาฬิกาให้เป็น "0"

```



```
DJNZ    R2,IPRDB1           ; ทำจนครบ 8 บิต
SETB    RTCSDA
CALL     CHIGH
JB       RTCSDA,IPRDB2
SETB     ERRFAG
IPRDB2:  CALL    CLOW
RET

;***** โปรแกรมย่อยสร้างสัญญาณนาฬิกาและหน่วงเวลา *****
CHIGH:   SETB    RTCSCL
NOP
NOP
NOP
NOP
RET

CLOW:    CLR     RTCSCL
NOP
NOP
NOP
RET

CDEL:    NOP
NOP
NOP
RET
```

ในตัวอย่างที่ 13.1 จะเป็นโปรแกรมรวมที่สามารถนำไปใช้อ่านเขียนข้อมูลกับชิป PCF8574 ได้ โดยการเรียกใช้โปรแกรมย่อยสองโปรแกรมคือ IPW สำหรับเขียนข้อมูลลงไปในชิป โดยให้ R2 เป็นแอดเดรสของชิป และ R3 เป็นข้อมูลที่ต้องการเขียน หลังจากการเขียนแล้วให้ตรวจสอบค่าในบิตแฟลก ERRFAG ที่สร้างขึ้นมา ถ้าเป็นลอจิก “0” หมายความว่าเขียนข้อมูลลงไปได้ แต่ถ้าเป็นลอจิก “1” หมายความว่ามีความผิดพลาดจากการเขียนข้อมูล สำหรับการอ่านข้อมูลจากชิป PCF8574 จะใช้โปรแกรมย่อย IPR โดยค่า R2 จะเป็นแอดเดรสของชิปที่ต้องการอ่านข้อมูล และข้อมูลที่จะอ่านได้จะอยู่ในรีจิสเตอร์ R3 หลังการอ่านจะตรวจสอบค่าบิตแฟลก ERRFAG ถ้าค่าบิตนี้เป็น “0” หมายความว่าอ่านข้อมูลได้ ถ้าบิตนี้เป็น “1” หมายความว่ามีความผิดพลาดในการอ่านข้อมูล



ตัวอย่างที่ 13.2 เป็นตัวอย่างการเขียนโปรแกรมใช้งานพอร์ต PCF8574 เป็นพอร์ตเอาต์พุต จากวงจรในรูปที่ 13.4 จะเห็นว่ามี LED ต่ออยู่กับพอร์ตทางบิต P0 และ P1 ถ้าหากบิตนี้เป็นลอจิก “0” จะทำให้หลอด LED ที่ต่ออยู่สว่าง โปรแกรมในตัวอย่างนี้จะทำให้ LED ทั้งสองหลอดที่ต่ออยู่ติดและดับสลับกัน โดยจะใช้โปรแกรมรวม IO8574H.ASM ในการเขียนข้อมูลลงชิป

จากวงจรในรูปที่ 13.4 ถ้าหากส่งข้อมูล 1111 1110 หรือ FEH ออกไปจะทำให้ LED ที่ต่ออยู่กับ P0 สว่าง แต่ถ้าหากส่งข้อมูล 1111 1101 หรือ FDH ออกไปจะทำให้ LED ที่ต่ออยู่กับ P1 สว่าง ดังนั้นในการเขียนโปรแกรมจะต้องส่งค่าทั้งสองนี้สลับกันไปมา ดังโปรแกรมต่อไปนี้

```

RTCSCL EQU P1.5 ; กำหนดขาสัญญาณนาฬิกาสำหรับติดต่อกับชิป
RTCSDA EQU P1.6 ; กำหนดขาข้อมูลสำหรับรับส่งข้อมูลกับชิป
ERRFAG EQU 7AH ; กำหนดตำแหน่งหน่วยความจำแบบบิตที่ใช้เป็น แฟลก
ORG 0000H

MAIN:
MOV R2,#00H ; กำหนดค่าแอดเดรสของชิป
MOV R3,#0FEH ; กำหนดค่าข้อมูลที่ต้องการเขียน
CALL IPW ; เรียกโปรแกรมย่อยเขียนข้อมูลลงชิป
JB ERRFAG,$ ; ตรวจสอบความถูกต้องในการเขียนข้อมูล
CALL DELAY ; เรียกโปรแกรมย่อยหน่วงเวลา
MOV R2,#00H ; กำหนดแอดเดรสของชิป
MOV R3,#0FDH ; กำหนดค่าของข้อมูลที่ต้องการเขียน
CALL IPW ; เรียกโปรแกรมย่อยเขียนข้อมูลลงชิป
JB ERRFAG,$ ; ตรวจสอบความถูกต้องในการเขียนข้อมูล
CALL DELAY ; เรียกโปรแกรมย่อยหน่วงเวลา
SJMP MAIN ; ทำซ้ำ

DELAY: MOV R1,#0FH ; โปรแกรมย่อยหน่วงเวลา
DE1: MOV R2,#0FFH
DE2: MOV R3,#0FFH
DJNZ R3,$
DJNZ R2,DE2
DJNZ R1,DE1
RET

```



```
$INCLUDE "IO8574H.ASM" ; เรียกโปรแกรมร่วมในการติดต่อกับชิป
END
```

นอกจากการใช้ชิป PCF8574 เป็นเอาต์พุตแล้ว ยังสามารถใช้เป็นพอร์ตอินพุตได้อีกด้วย การใช้งานเป็นพอร์ตอินพุตสามารถทำได้โดยการอ่านข้อมูลเข้ามา แล้วตรวจสอบว่าข้อมูลที่อ่านเข้ามานั้นเป็นค่าใด ใน การอ่านข้อมูลสามารถใช้โปรแกรมย่อย IPR จากโปรแกรมร่วม IO8574H.ASM ได้ โดยให้ R2 เก็บค่า แอดเดรส และจะได้ข้อมูลที่อ่านได้ทางรีจิสเตอร์ R3

ตัวอย่างที่ 13.3 โปรแกรมในตัวอย่างนี้จะเป็นการอ่านค่าสวิตช์จากอินพุต และแสดงหมายเลข สวิตช์ที่กดทาง LED 7 เซกเมนต์ โดยใช้วงจรการต่อสวิตช์ในรูปที่ 13.4 และการต่อหน่วยแสดงผลดังวงจร ในรูปที่ 12.5

จากวงจรจะเห็นว่าสวิตช์ SW1 ต่ออยู่กับบิต P2 และสวิตช์ SW2 ต่ออยู่กับบิต P3 ถ้าหากสวิตช์ ไม่กดจะเป็นลอจิก “1” ถ้าหากกดสวิตช์จะเป็นลอจิก “0” ดังนั้นในการเขียนโปรแกรมจะต้องตรวจสอบค่าที่อ่าน ได้จาก PCD8574 ถ้าหากได้ค่าเป็น 1111 1011 ให้ส่งเลข 1 ไปยังหน่วยความจำแสดงผล แต่ถ้าหากได้ค่าเป็น 1111 0111 ให้ส่งเลข 2 ไปยังหน่วยความจำแสดงผล ดังตัวอย่างโปรแกรมต่อไปนี้

```

                                ORG    0000H
PORT_A      EQU    0FC00H      ; กำหนดค่าพอร์ตต่าง ๆ ของวงจรแสดงผล
PORT_B      EQU    0FC01H      ; ทาง LED 7 เซกเมนต์แบบมัลติเพล็กซ์
PORT_C      EQU    0FC02H
PORT_CON    EQU    0FC03H      ; พอร์ตควบคุมของ 8255
                                ; กำหนดค่าต่าง ๆ ของวงจรที่ต่อกับชิป PCF8574
RTCSCL      EQU    P1.5        ; ขาลัญญานนาฬิกาควบคุม
RTCSDA      EQU    P1.6        ; ขารับส่งข้อมูล
ERRFAG      EQU    7AH         ; กำหนดหน่วยความจำที่ใช้เป็นบิตแฟลก

DISBUF      EQU    20H         ; กำหนดหน่วยความจำภายในที่ใช้ในการแสดงผล
;***** MAIN *****
MAIN:        MOV    A,#80H
              MOV    DPTR,#PORT_CON
              MOVX   @DPTR,A    ; โปรแกรมการทำงานของพอร์ต 8255
DIS:         MOV    R2,#00H     ; กำหนดค่าแอดเดรสของชิป PCF8574

```



```

CALL    IPR                ; อ่านไบต์ข้อมูลเข้ามา
JB      ERRFAG,$           ; ตรวจสอบการอ่านว่าถูกต้องหรือไม่
MOV     A,R3               ; นำค่าข้อมูลที่อ่านได้เก็บในรีจิสเตอร์ A
SHOW1:  ORL     A,#1111011B ; นำมา OR กับค่า FBH ถ้า SW1 ไม่กดจะได้ FFH
        CJNE   A,#1111011B,SHOW2 ; เปรียบเทียบ ถ้า SW1 ไม่กดให้ไป SHOW2
        MOV    DISBUF+3,#1   ; ถ้า SW1 กดให้ใส่เลข 1 ลงในหน่วยความจำ
                                แ ดงผล
        JMP    GO            ; กระโดดไป GO
SHOW2:  MOV     A,R3         ; นำค่าข้อมูลที่อ่านได้มาเก็บใน A
        ORL     A,#1111011B ; ถ้า SW2 ไม่กดจะได้ค่า FFH
        CJNE   A,#1111011B,GO ; เปรียบเทียบ ถ้า SW2 ไม่กดให้ไป GO
        MOV    DISBUF+3,#2   ; ถ้า SW2 กดให้ใส่เลข 2 ลงในหน่วยความจำ
                                แ ดงผล
GO:      ACALL  SCAN         ; เรียกโปรแกรมย่อยแสดงผลทาง LED 7 เซกเมนต์
        SJMP   DIS          ; วนลูปทำซ้ำ
,*****
; SCAN LED 7                โปรแกรมย่อยแสดงผลทาง LED 7 เซกเมนต์
; DPTR,R0,R1,R2            โดยจะใช้รีจิสเตอร์ DPTR , R0 , R1 และ R2
,*****
SCAN:    MOV     R1,#4
        MOV     R2,#3
        MOV     R0,#DISBUF+3
LOOP:    MOV     DPTR,#SEG_CODE
        MOV     A,@R0
        MOVC    A,@A+DPTR
        MOV     DPTR,#PORT_A
        MOVX    @DPTR,A
        MOV     A,R2
        MOV     DPTR,#PORT_C
        MOVX    @DPTR,A
        LCALL   DELAY

```



```
        DEC    R0
        DEC    R2
        DJNZ   R1,LOOP
        RET

.*****
,
DELAY:    MOV    R3,#0FFH
DE1:      DJNZ   R3,DE1
        RET

.*****
,
SEG_CODE: DB     3FH,06H,5BH,4FH,66H
          DB     6DH,7DH,07H,7FH,6FH
          $INCLUDE "IO8574H.ASM"
          END
```

บทที่ 14

การทดลอง

สร้างฐานเวลา

การสร้างฐานเวลาให้กับระบบไมโครคอนโทรลเลอร์เราอาจทำได้ง่าย ๆ โดยใช้วงจรกำเนิดความถี่ป้อนให้กับตัวนับ/ตัวจับเวลาในไมโครคอนโทรลเลอร์ แต่การใช้งานแบบนี้ผู้ใช้จะต้องเขียนโปรแกรมควบคุมการทำงาน มีการคำนวณคาบเวลาให้ถูกต้อง และต้องตั้งค่าใหม่ทุกครั้งเมื่อมีการเริ่มทำงาน ระบบการสร้างฐานเวลาแบบนี้สามารถใช้งานง่าย ๆ ได้ แต่ไม่สามารถบอกเวลาจริงให้กับระบบได้

ระบบที่ใช้ไมโครคอนโทรลเลอร์ในการควบคุมบางระบบจะต้องมีเวลาเข้ามาเกี่ยวข้องกับ โดยจะมีการบอกเวลาเป็นชั่วโมง นาที และวินาที หรือมีการบอกวัน เดือน และปีให้กับระบบด้วย ดังนั้นการสร้างฐานเวลาจะต้องถูกต้องและแม่นยำ แม้ว่าระบบหยุดทำงานและเริ่มทำงานใหม่ค่าเวลาต่าง ๆ จะต้องถูกต้องด้วย ในปัจจุบันการทำงานประเภทนี้จะใช้ชิปไอซีที่ทำหน้าที่สร้างฐานเวลาจริงให้กับระบบ (RTC หรือ Real Time Clock) ไอซีประเภทนี้จะมีวงจรจัดการด้านเวลาจริงอยู่ภายใน การใช้งานเพียงต่อคริสตอลให้กับมัน การทำงานด้านเวลาจะเป็นไปอย่างอัตโนมัติ เสมือนกับว่าตัวมันเป็นนาฬิกาและปฏิทินให้กับระบบ ถ้าหากเมื่อใดต้องการทราบเวลาเราสามารถอ่านจากหน่วยความจำภายในของมันได้โดยตรง

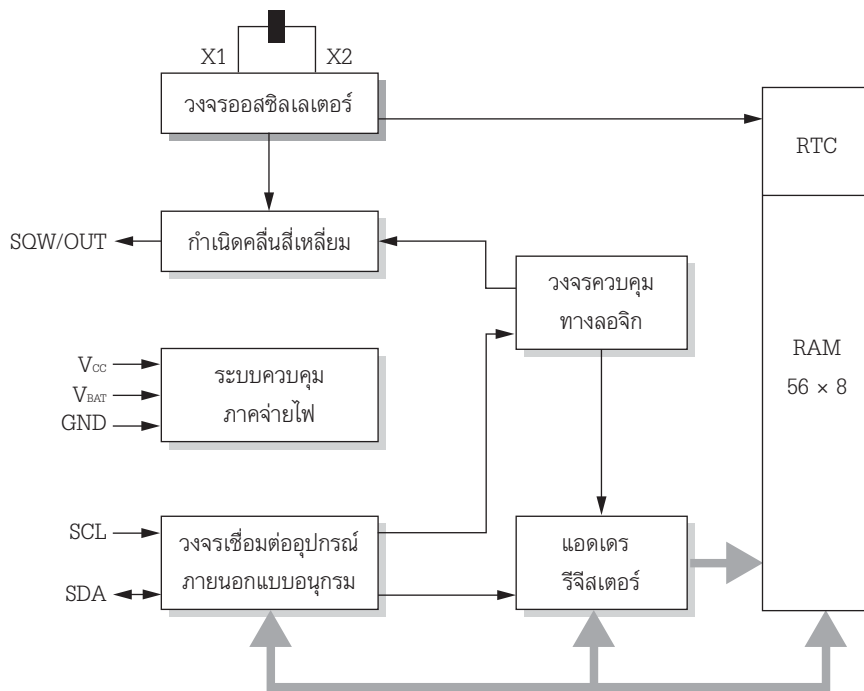


14.1 ไอซีฐานเวลาจริง DS1307

โดยทั่วไปแล้วชิปไอซีที่ทำหน้าที่เป็น RTC จะมีอยู่หลายเบอร์ บางเบอร์เมื่อเวลาเดินมาถึงค่าที่กำหนดก็สามารถอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ได้ ในที่นี้จะแนะนำชิปเบอร์ DS1307 ของบริษัท Dallas Semiconductor ซึ่งเป็นชิปที่มีความแม่นยำมาก โดยเป็นไอซีแบบ 8 ขา การเชื่อมต่อกับไมโครคอนโทรลเลอร์จะใช้การอินเทอร์เฟสแบบอนุกรม 2 สาย หรือแบบ I²C ไอซีเบอร์นี้กินพลังงานต่ำมาก พร้อมทั้งมีปฏิทินเวลา



แบบ BCD สามารถใช้ข้อมูลเกี่ยวกับเวลา เช่น วินาที นาที ชั่วโมง (ทั้งแบบ 24 ชั่วโมง / 12 ชั่วโมง พร้อมทั้งระบุค่า AM/PM) และบอกวัน เดือน ปี ได้ โดยจะมีการปรับวันที่โดยอัตโนมัติ ในแต่ละเดือนจะแสดงวันได้สูงสุดไม่เกิน 31 วัน และจะปรับวันต่าง ๆ อย่างถูกต้องเมื่อครบปี การใช้งานกับไมโครคอนโทรลเลอร์จะใช้การส่งข้อมูลและแอดเดรสของค่าต่าง ๆ แบบอนุกรม โดยมีขาหนึ่งเป็นขาสัญญาณ อีกขาหนึ่งเป็นตัวกำหนดสัญญาณนาฬิกา โครงสร้างภายในของไอซี DS1307 แสดงได้ดังรูปที่ 14.1



รูปที่ 14.1 โครงสร้างภายในของไอซี DS1307

ไอซี DS1307 เป็นไอซีแบบ 8 ขา ขาต่าง ๆ ของ DS 1307 เป็นดังต่อไปนี้

ขา SDA (Serial Data Input/Output) เป็นขารับส่งข้อมูลแบบอนุกรม ในการอินเตอร์เฟสจะต้องมีตัวต้านทานต่อพูล์อัพภายนอกด้วย

ขา SCL (Serial Clock Input) เป็นขาอินพุตสัญญาณนาฬิกาอนุกรม เพื่อให้เกิดการซิงโครไนซ์ในการรับส่งข้อมูลแบบอนุกรม

ขา SQU/OUT เป็นขาส่งสัญญาณคลื่นสี่เหลี่ยมออกมาทางเอาต์พุต เมื่อเริ่มทำงานบิตนี้จะถูกเซตเป็นลอจิก “1” สัญญาณเอาต์พุตที่ได้จะมีอยู่ 4 ค่า คือ 1 Hz, 4 kHz, 8 kHz และ 32 kHz โดยเราสามารถเลือกได้ในการใช้งานจะต้องมีตัวต้านทานต่อพูล์อัพภายนอกด้วย ในการใช้งานบางประเภทไม่จำเป็นต้องใช้ขานี้



ขา X1, X2 เป็นขาที่ใช้ต่อกับคริสตอลภายนอก โดยใช้ความถี่ 32.768 kHz วงจรสร้างสัญญาณนาฬิกาภายในออกแบบให้ทำงานร่วมกับคริสตอลที่มีตัวเก็บประจุ 12.5 พิโกฟารัดต่อรวมอยู่ด้วย

ขา V_{cc} และ GND เป็นขาที่ใช้ต่อกับไฟเลี้ยง โดยทั่วไปแล้วจะต่อกับแรงดันไฟ 5 โวลต์ ถ้าหากแรงดันไฟต่อดังกล่าว $1.25 \times V_{BAT}$ การรักษาเวลาสำรองภายในจะทำงานต่อ โดยจะรับพลังงานจากแบตเตอรี่สำรอง

ขา V_{BAT} เป็นขาต่อกับแรงดันไฟเลี้ยงสำรอง โดยทั่วไปแล้วจะใช้แบตเตอรี่แรงดัน 3 โวลต์

หน่วยความจำ RAM ภายใน และส่วนที่ใช้เป็น RTC มีการจัดแอดเดรสดังรูปที่ 14.2 โดยตำแหน่ง 00H ถึง 07H จะใช้เป็น RTC ส่วนตำแหน่ง 08H ถึง 3FH จะเป็นหน่วยความจำ RAM จากรูปจะเห็นว่าที่ตำแหน่ง 00H จะเก็บเวลาเป็นวินาที ตำแหน่ง 01 จะเก็บเวลาเป็นนาฬิกา โดยข้อมูลที่เก็บจะอยู่ในรูปแบบรหัส BCD

00H	วินาที
	นาฬิกา
	ชั่วโมง
	วัน
	วันที่
	เดือน
	ปี
07H	รหัสควบคุม
08H	RAM 58 × 8
0FH	

รูปที่ 14.2 แผนผังหน่วยความจำ

สำหรับการอ่านและเขียนค่าเวลากับไอซีตัวนี้ทำได้โดยการและเขียนข้อมูลในลักษณะตัวเลข BCD กับรีจิสเตอร์ที่เก็บค่าต่าง ๆ ตามที่กำหนดเอาไว้ รายละเอียดของแอดเดรสตำแหน่งต่าง ๆ แสดงในรูปที่ 14.3



	บิต 7					บิต 0			
00H	CH		วินาที หลักสิบ			วินาที			00-59
01H	X		นาทีกหลักสิบ			นาทีก			00-59
02H	X		12,24	A/P	10 HP	ชั่วโมง			01-12 , 00-23
03H	X		X	X	X	วัน			0-7
04H	X		X	วันที่ หลักสิบ		วันที่			01-31
05H	X		X	เดือน หลักสิบ		เดือน			01-12
06H	ปี หลักสิบ					ปี			00-99
07H	OUT	X	X	SQWE	X	X	RS1	RS2	

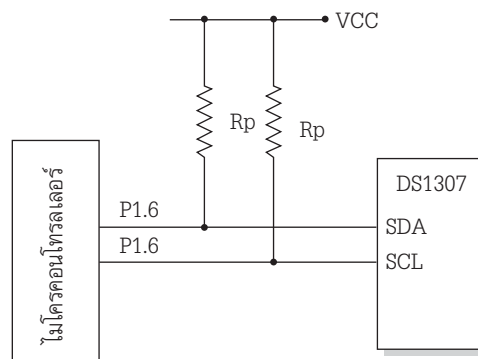
รูปที่ 14.3 แผนผังรีจิสเตอร์ตำแหน่งต่าง ๆ ในการจัดการด้านฐานข้อมูลเวลา

ในรีจิสเตอร์ตำแหน่ง 00H ซึ่งเป็นรีจิสเตอร์วินาที ถ้าบิตที่ 7 ถูกเซตให้เป็นลอจิก “1” วงจรออสซิลเลเตอร์จะหยุดทำงาน ถ้าเป็น “0” วงจรออสซิลเลเตอร์จะทำงานต่อไป สำหรับรีจิสเตอร์ตำแหน่ง 01H เป็นรีจิสเตอร์นาทีก ซึ่งเวลาในหลักสิบของวินาทีจะมีค่าตั้งแต่ 0 ถึง 5 ดังนั้นบิตที่ 7 จะไม่ใช่ สำหรับรีจิสเตอร์ตำแหน่ง 02H ซึ่งเป็นรีจิสเตอร์ชั่วโมง สามารถเซตได้ว่าจะให้เก็บข้อมูลแบบ 12 ชั่วโมงหรือ 24 ชั่วโมง และยังบอกค่า AM/PM ได้อีกด้วย ส่วนรีจิสเตอร์ตำแหน่ง 07H จะเป็นรีจิสเตอร์ควบคุมเพื่อกำหนดว่าจะให้มีเอาต์พุตออกมาทางขา SQW/OUT หรือไม่ และเอาต์พุตที่ออกมาจะมีความถี่เท่าใด รายละเอียดต่าง ๆ สามารถศึกษาได้จากคู่มือของไอซีได้โดยตรง ซึ่งจะไม่กล่าวถึงในที่นี้

การจัดหน่วยความจำภายในของชิปไอซีฐานเวลาในลักษณะนี้ทำให้ผู้ใช้สามารถเลือกอ่านหรือเขียนข้อมูลใด ๆ ในตำแหน่งที่กำหนดได้ เช่นถ้าหากอ่านค่าในตำแหน่ง 01H ออกมาได้เป็น 0011 0010 หรือ 32 ในระบบเลข BCD จะหมายความว่าเป็นเวลา 32 นาทีก ถ้าอ่านค่าในตำแหน่ง 02H ออกมาได้เป็น 0010 0010 หรือ 22 ในระบบเลข BCD หมายความว่าเป็นเวลา 22 ชั่วโมง เป็นต้น



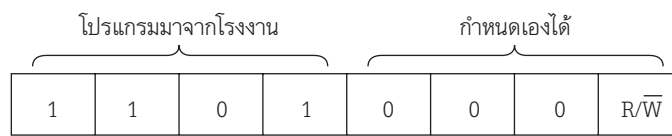
14.2 การเชื่อมต่อ DS1307 กับไมโครคอนโทรลเลอร์ MCS-51



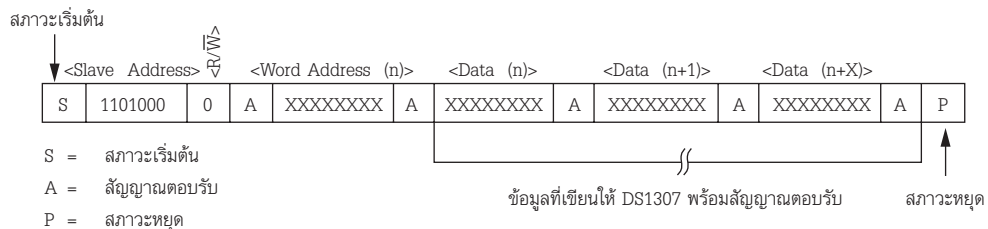
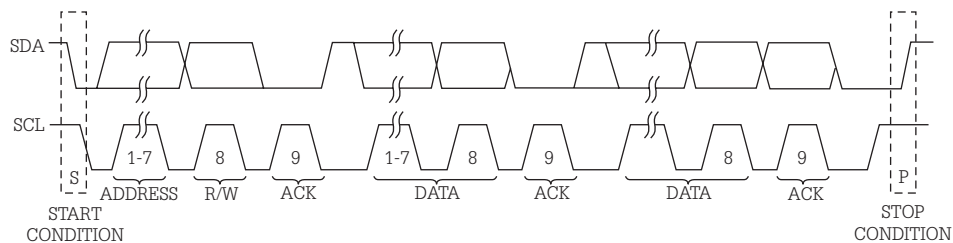
รูปที่ 14.4 วงจรการเชื่อมต่อกับ MCS-51 อย่างง่าย



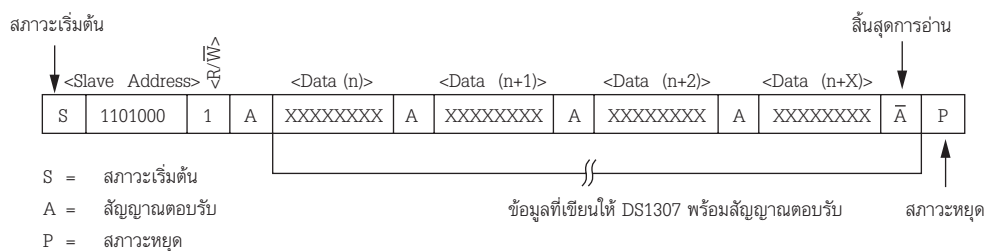
การเชื่อมต่ออย่างง่ายระหว่าง DS1307 กับไมโครคอนโทรลเลอร์แสดงได้ดังรูปที่ 14.4 โดยใช้ P1.6 เป็นขารับส่งข้อมูลแบบอนุกรม ส่วน P1.5 ใช้เป็นขาสัญญาณนาฬิกาควบคุม ตัว DS1307 สามารถรับส่งข้อมูลแบบสองทิศทางได้ และสามารถต่อกับอุปกรณ์อื่น ๆ หลายตัวได้โดยมีอุปกรณ์ที่ทำหน้าที่ควบคุมการสื่อสารข้อมูล เรียกว่า master สำหรับตัวที่ถูกควบคุมจะเรียกว่า slaves ในการทดลองนี้จะให้ไมโครคอนโทรลเลอร์เป็นตัวควบคุมการอ่านเขียนข้อมูล โดยบิต P1.5 จะเป็นตัวกำหนดสัญญาณนาฬิกาเพื่อควบคุมข้อมูลที่จะอ่านเขียนทาง P1.6 ที่เป็นขาข้อมูล สำหรับแอดเดรสของชิป DS1307 และไคอะแกรมการอ่านเขียนข้อมูลแสดงได้ดังรูปที่ 14.5



(ก) แอดเดรสของชิป DS1307



(ข) ไคอะแกรมเวลาการเขียนข้อมูลลงบนชิป DS1307



(ค) ไคอะแกรมเวลาการอ่านข้อมูลลงบนชิป DS1307

รูปที่ 14.5 แอดเดรสของชิปและไคอะแกรมเวลาในการติดต่อกับชิป



จากไดอะแกรมเวลาจะพบว่าการเขียนข้อมูลลงบน DS1307 จะต้องทำดังนี้

1. เขียนไบต์แอดเดรสของชิปลงไป
2. เขียนไบต์ระบุแอดเดรสภายในชิปที่ต้องการ
3. เขียนข้อมูลลงบนชิปที่กำหนด

สำหรับการอ่านไบต์ข้อมูลสามารถทำได้ดังขั้นตอนดังนี้

1. เขียนไบต์แอดเดรสของชิปลงไป
2. เขียนไบต์แอดเดรสภายในชิปที่ต้องการอ่าน
3. เขียนไบต์แอดเดรสระบุว่าการอ่านโดยให้บิตต่ำสุดเป็นลอจิก “1”
4. อ่านไบต์ข้อมูลออกมาเก็บในหน่วยความจำ

ในการเขียนข้อมูลเพื่อกำหนดวันเวลาต่าง ๆ ให้กับ DS1307 ทำได้โดยขั้นตอนแรกให้เขียนข้อมูล 1 ไบต์ บอกว่า DS1307 เป็น slaves โดยให้เขียนข้อมูลแอดเดรสของชิปลงไป โดยข้อมูล 7 บิตบนเป็น 1101000 ส่วนบิตที่ 0 เป็น “0” เพื่อบอกว่าเขียนข้อมูล จากนั้นเมื่อ DS1307 รับทราบแล้วจะส่งสัญญาณตอบรับออกมาทางขา SDA จากนั้นให้เขียนข้อมูลไบต์ต่อไปซึ่งเป็นค่าแอดเดรส และเขียนข้อมูลเวลาต่าง ๆ ลงไป โดยข้อมูลที่เขียนจะเรียงลำดับตั้งแต่รีจิสเตอร์แอดเดรสต่ำไปรีจิสเตอร์ที่มีแอดเดรสสูง และการเขียนข้อมูลทุกครั้งจะมีสัญญาณตอบรับออกมาทางขา SDA

สำหรับการอ่านข้อมูลวันเวลาต่าง ๆ จาก DS1307 ทำได้โดยขั้นตอนแรกให้เขียนข้อมูล 1 ไบต์ลงไปก่อนเพื่อบอกว่า DS1307 เป็น slaves โดยมีบิตข้อมูล 7 บิตบนเป็น 1101000 ส่วนบิตที่ 0 เป็น “1” เพื่อบอกว่าจะอ่านข้อมูล หลังจาก DS1307 รับทราบแล้วจะส่งสัญญาณตอบรับออกมาทางขา SDA จากนั้นจะเริ่มส่งข้อมูลวันเวลาต่าง ๆ ออกมา โดยข้อมูลจากรีจิสเตอร์แอดเดรสต่ำจะถูกส่งออกมาก่อน

แนวทางการเขียนโปรแกรมควบคุมการใช้งาน DS1307 อาจทำได้โดย

1. เขียนโปรแกรมสร้างลอจิก “0” และลอจิก “1” ส่งออกไปทาง P1.5 เพื่อเป็นสัญญาณนาฬิกาควบคุม
2. ینگหรือรับข้อมูลที่ละบิตทางขา P1.6 โดยใช้การเลื่อนข้อมูลไปทางซ้าย เพื่อติดต่อกับ SDA
3. ร้างหน่วยความจำบัพเฟอร์ขนาด 7 ไบต์เพื่อเป็นที่เก็บค่าเวลาในการติดต่อกับ DS1307 โดยในการตั้งเวลาและอ่านค่าเวลาโปรแกรมหลักจะติดต่อกับหน่วยความจำส่วนนี้ จากนั้นจะมีโปรแกรมน้อยรับส่งข้อมูลจากหน่วยความจำบัพเฟอร์นี้เพื่อสื่อสารทางอนุกรมกับ DS1307

ในตัวอย่างที่ 14.1 จะแสดงการสร้างโปรแกรมร่วมสำหรับอ่านเขียนข้อมูลกับชิป DS1307 โดยโปรแกรมร่วมนี้สามารถนำไปประยุกต์กับการเชื่อมต่อ DS1307 กับ MCS-51 ในการใช้งานลักษณะต่าง ๆ ได้



ตัวอย่างที่ 14.1 โปรแกรมรวมสำหรับการอ่านเขียนข้อมูลเวลากับชิป DS1307 โดยออกแบบหน่วยความจำสำหรับเก็บเวลาขนาด 8 ไบต์ชื่อ TIMEBUF ไว้ที่ตำแหน่ง 8500H ของหน่วยความจำภายนอก โดยแต่ละไบต์ของหน่วยความจำนี้จะสอดคล้องข้อมูลเวลาที่เก็บอยู่ภายในชิป

```

;***** RTWR SUB *****
; DS1307 RTC WRITE
; IN  = TIMBUF                      โปรแกรมย่อยนี้มีอินพุตเป็นหน่วยความจำบัฟเฟอร์เวลา
; OUT = RTCERR 0 = OK 1 = ERROR      บิตแฟล็ก RTCERR = "0" หมายถึงทำงาน
ถูกต้อง
; REG = A,R0,R2,R3,DPTR
;*****
RTWR:  CALL  RTSET                    ; เรียกโปรแกรมย่อยกำหนดค่าเริ่มต้น
      CLR   RTCSDA                    ; เคลียร์บิต SDA
      CALL  CDEL                      ; หน่วยเวลา
      CALL  CLOW                      ; ให้ SCL เป็น "0"
      MOV   A,#11010000B              ; กำหนดแอดเดรส Slave
      CALL  RTWRB                    ; เขียนลงในชิป DS1307
      JB    RTCERR,RTWR9              ; ตรวจสอบความถูกต้อง
      MOV   A,#0                      ; กำหนดแอดเดรส
      CALL  RTWRB                    ; เขียนลง DS1307
      JB    RTCERR,RTWR9              ;
      MOV   DPTR,#TIMEBUF             ; ชี้ที่หน่วยความจำบัฟเฟอร์เวลา
      MOV   R3,#8                     ;
RTWR2: MOVX  A,@DPTR
      CALL  RTWRB
      JB    RTCERR,RTWR9              ; เขียนลง DS1307 จำนวน 8 ไบต์
      INC   DPTR
      DJNZ  R3,RTWR2
RTWR9: CLR   RTCSDA
      CALL  CHIGH
      CALL  CDEL
      RET

```



```

;***** RTRD SUB *****
; DS1307 RTC READ โปรแกรมย่อยอ่านข้อมูลจาก DS1307 ส่งให้หน่วยความจำบัฟเฟอร์เวลา
; OUT = TIMEBUF RTCERR 0 = OK 1 = ERROR
; REG = A,R0,R2,R3,DPTR
;*****
RTRD:  CLR    RTCSDA
        CALL  CDEL
        CALL  CLOW
        MOV   A,#11010000B      ; กำหนดเป็น Slave
        CALL  RTWRB             ; เขียนลงใน DS1307
        JB    RTCERR,RTRD9
        MOV   A,#0
        CALL  RTWRB
        JB    RTCERR,RTRD9
        CLR   RTCSDA
        CALL  CHIGH
        CALL  CDEL
        SETB  RTCSDA
        CALL  CDEL
        CALL  CDEL
        CLR   RTCSDA
        CALL  CDEL
        CALL  CLOW
        MOV   A,#11010001B      ; กำหนดว่าต้องการอ่านข้อมูล
        CALL  RTWRB             ; เขียนลงบนชิป DS1307
        JB    RTCERR,RTRD9
        MOV   DPTR,#TIMEBUF     ; ชี้ตำแหน่งหน่วยความจำบัฟเฟอร์เวลา
        MOV   R3,#7
RTRD2:  CALL  RTRDB
        MOVB  @DPTR,A           ; อ่านข้อมูล 7 ไบต์ มาเก็บในหน่วยความจำ
        INC   DPTR              ; บัฟเฟอร์เวลา
        DJNZ  R3,RTRD2

```



```
CALL RTRDE ; อ่านข้อมูลไบต์สุดท้าย
MOVX @DPTR,A
RTRD9: CLR RTCSDA
CALL CHIGH
CALL CDEL
SETB RTCSDA
RET

;***** RTSET SUB *****
; SET RTC STATUS BIT โปรแกรมย่อยกำหนดค่าเริ่มต้นการทำงานให้กับDS1307 โดย
เขียนลงในหน่วยความจำบัฟเฟอร์เวลาจากนั้นจะถูกเขียนลงใน
DS1307

; IN = TIMEBUF
; OUT = TIMEBUF
; REG = A,DPTR

;*****
RTSET: MOV DPTR,#TIMEBUF ; ชี้ที่หน่วยความจำบัฟเฟอร์เวลา
MOVX A,@DPTR
CLR ACC.7 ; เคลียร์บิต 7 ให้ฮอสซิลเลเตอร์ทำงาน
MOVX @DPTR,A
MOV DPTR,#TIMEBUF+2 ; ชี้ที่หน่วยความจำบัฟเฟอร์ชั่วโมง
MOVX A,@DPTR
CLR ACC.6 ; เคลียร์บิต 6 ให้ทำงานแบบ 24 ชั่วโมง
MOVX @DPTR,A
MOV DPTR,#TIMEBUF+7
MOVX A,@DPTR
ANL A,#11101100B ; กำหนดรีจิสเตอร์ควบคุม
ORL A,#00000000B
MOVX @DPTR,A ; เขียนค่ารีจิสเตอร์ควบคุมลงบัฟเฟอร์
RET

;***** RTWRB SUB *****
; DS1307 RTC WRITE BYTE โปรแกรมย่อยเขียนข้อมูล 1 ไบต์ โดยนำค่าในรีจิสเตอร์ A
; IN = A เลื่อนออกไปทีละบิต
; OUT = RTCERR 0 = OK 1 = ERROR
```



```

; REG = A,R2
;*****
RTWRB:  CLR   RTCERR
        MOV   R2,#8           ; ทำซ้ำ 8 ครั้ง เพราะข้อมูล 1 ไบต์มี 8 บิต
RTWRB1:  RLC   A               ; หมุนบิตสูงเข้าแฟลกตัวทด
        MOV   RTCSDA,C        ; นำค่าแฟลกตัวทดส่งออกไปให้ SDA
        CALL  CHIGH            ; ให้ SCL เป็น "1"
        CALL  CLOW             ; ให้ SCL เป็น "0"
        DJNZ  R2,RTWRB1        ; ทำซ้ำจนครบ 8 บิต
        SETB  RTCSDA
        CALL  CHIGH
        JNB   RTCSDA,RTWRB2    ; ตรวจสอบสัญญาณตอบรับ
        SETB  RTCERR
RTWRB2:  CALL  CLOW
        RET
;***** RTRDB SUB *****
; DS1307 RTC READ BYTE
; OUT = A โปรแกรมย่อยอ่านข้อมูล 1 ไบต์ โดยรับทีละบิต มาเก็บในรีจิสเตอร์ A
; REG = A,R2
;*****
RTRDB:  MOV   R2,#8           ; กำหนดลูปการทำงาน 8 ครั้ง
RTRDB1:  CALL  CHIGH
        MOV   C,RTCSDA        ; อ่านข้อมูลจาก SDA เข้ามาในบิตแฟลก
        RLC   A
        CALL  CLOW
        DJNZ  R2,RTRDB1        ; ทำจนครบ 8 บิต
        CLR   RTCSDA
        CALL  CHIGH
        CALL  CLOW
        SETB  RTCSDA
        RET
RTRDE:  MOV   R2,#8           ; อ่านข้อมูลไบต์สุดท้าย

```



```
RTRDE1:  CALL    CHIGH
          MOV     C,RTCSDA
          RLC     A
          CALL    CLOW
          DJNZ    R2,RTRDE1
          SETB    RTCSDA
          CALL    CHIGH
          CALL    CLOW
          RET

CHIGH:    SETB    RTCSCL          ; โปรแกรมย่อยสร้างสัญญาณนาฬิกา SCl ลอจิก “1”
          NOP
          NOP
          NOP
          RET

CLOW:     CLR     RTCSCL          ; โปรแกรมย่อยสร้างสัญญาณนาฬิกาลอจิก “0”
          NOP
          NOP
          NOP
          RET

CDEL:     NOP                    ; โปรแกรมย่อยหน่วงเวลา
          NOP
          NOP
          RET

          ORG     8500H          ; กำหนดตำแหน่งของหน่วยความจำบัฟเฟอร์เวลา
TIMEBUF:  DS      8
```

ถ้าหากโปรแกรมในตัวอย่างที่ 14.2 ถูกตั้งชื่อไว้ว่า RTC_IOH.ASM ถ้าหากมีระบบ MCS-51 ที่มีชิป RTC DS1307 ต่ออยู่ จะสามารถเรียกโปรแกรมย่อยต่าง ๆ มาใช้ได้ โดยใช้คำสั่ง INCLUDE เรียกขึ้นมาก่อน โดยในโปรแกรมนี้อ้างอิงโปรแกรมย่อยที่ต้องเรียกใช้ดังนี้

```
RTWR      ใช้สำหรับนำข้อมูลจากหน่วยความจำบัฟเฟอร์เวลาเขียนลงบนชิป DS1307
RTRD      ใช้สำหรับอ่านข้อมูลจากชิป DS1307 มาเก็บในหน่วยความจำบัฟเฟอร์เวลา
```



จากที่ศึกษามาจะพบว่า การตั้งเวลาให้กับชิป DS1307 จะทำได้โดยเขียนค่าเวลาต่าง ๆ ลงในหน่วยความจำฟเฟอร์เวลา จากนั้นนำข้อมูลจากหน่วยความจำฟเฟอร์เวลาเขียนลงบนชิปด้วยโปรแกรมย่อย RTWR และถ้าหากต้องการอ่านค่าเวลา สามารถทำได้โดยอ่านข้อมูลจากชิป DS1307 ด้วยโปรแกรมย่อย RTRD จากนั้นมาตรวจสอบในหน่วยความจำฟเฟอร์เวลาว่าเวลานั้นเป็นเวลาใด ดังโปรแกรมในตัวอย่างที่ 14.2

สำหรับการอ่านข้อมูลออกจาก DS1307 นั้นข้อมูลที่ได้จะอยู่ในรูปรหัส ASCII ดังนั้นถ้าหากต้องการนำมาแสดงผล จะต้องทำการปรับค่าให้เหมาะสมกับอุปกรณ์แสดงผลในระบบนั้น ๆ ตัวอย่างเช่นวงจรแสดงผลทาง LCD 7 แบบมัลติเพล็กซ์ที่ออกแบบไว้ในบทที่ 12 จะแสดงผลได้โดยการนำตัวเลขไปใส่ในหน่วยความจำฟเฟอร์เวลา ดังนั้นถ้าหากอ่านข้อมูลเวลาออกมาจากชิป DS1307 ขนาดหนึ่งไบต์ จะได้ข้อมูลเป็นเลข BCD ซึ่งจะต้องแปลงเลข BCD นั้นเป็นเลขฐานสิบสองหลักเสียก่อน ดังในตัวอย่างที่ 14.2

ตัวอย่างที่ 14.2 โปรแกรมในตัวอย่างนี้จะแสดงเวลาเป็นนาฬิกา และวินาทีออกทาง LED 7 เซกเมนต์สี่หลัก ที่ต่อแบบมัลติเพล็กซ์กับ 8255 ทางพอร์ต 0FC00H ถึง 0FC03H โดยที่ไอซีฐานเวลา DS1307 ต่อกับ MCS-51 ทางขา P1.5 และ P1.6 เมื่อโปรแกรมนีทำงานเวลาจะเริ่มเดินตั้งแต่ 00 นาฬิกา 00 วินาที

```

PORT_A    EQU    0FC00H           ; กำหนดแอดเดรสพอร์ต A ของ 8255
PORT_B    EQU    0FC01H
PORT_C    EQU    0FC02H
PORT_CON  EQU    0FC03H

RTCSCL    EQU    P1.5             ; ขาสัญญาณนาฬิกาที่ต่อกับ DS1307
RTCSDA    EQU    P1.6             ; ขาข้อมูลที่ต่อกับ DS1307
DISBUF    EQU    20H              ; กำหนดหน่วยความจำฟเฟอร์แสดงผล
HEXBUF    EQU    30H              ; กำหนดหน่วยความจำเก็บเลข BCD
RTCERR    EQU    79H              ; กำหนดหน่วยความจำของบิตแฟล็ก

;***** MAIN *****
MAIN:     MOV     A,#80H           ; กำหนดไบต์ควบคุมให้เป็นเอาต์พุตทุกพอร์ต
          MOV     DPTR,#PORT_CON
          MOVX    @DPTR,A         ; เขียนไบต์ควบคุมลงไป
          MOV     HEXBUF,#00H
          MOV     HEXBUF+1,#00H   ; กำหนดเลข BCD จำนวน 4 หลักให้เป็น 0
          ACALL   BCD2DEC          ; แปลงเลข BCD เป็นเลขฐานสิบ
          MOV     DPTR,#TIMEBUF    ; ชี้ไปที่หน่วยความจำฟเฟอร์เวลา

```



```

MOV    A,#00H          ; กำหนดเวลาวินาทีเป็น 00
MOVX   @DPTR,A         ; เขียนค่าเวลาวินาทีลงไป
INC    DPTR            ; ซี่ที่หน่วยความจำของนาฬิกา
MOV    A,#00H          ; กำหนดให้นาทีเป็น 00
MOVX   @DPTR,A         ; เขียนเวลานาทีลงไป
INC    DPTR            ; ซี่ที่หน่วยความจำชั่วโมง
MOV    A,#00H          ; กำหนดให้ชั่วโมงเป็น 00
MOVX   @DPTR,A         ; เขียนเวลาชั่วโมงลงไป
CALL   RTWR            ; นำข้อมูลเวลาทั้งหมดเขียนลงบนชิป
DIS:   CALL   RTRD      ; อ่านค่าเวลาจากชิป
MOV    DPTR,#TIMEBUF   ; ซี่ที่หน่วยความจำเวลา
MOVX   A,@DPTR         ; อ่านข้อมูลวินาที
MOV    HEXBUF+1,A      ; นำข้อมูลวินาทีที่เป็นเลข BCD ใส่หน่วยความจำ
INC    DPTR            ; ซี่ที่หน่วยความจำนาฬิกา
MOVX   A,@DPTR         ; อ่านค่านาที
MOV    HEXBUF,A        ; นำข้อมูลนาฬิกาใส่หน่วยความจำ HEXBUF
ACALL  BCD2DEC         ; เปลี่ยนข้อมูลนาฬิกาที่เป็นเลข BCD ให้เป็นเลขฐานสิบ
ACALL  SCAN            ; เรียกโปรแกรมย่อยแสดงผล
SJMP   DIS             ; วนลูปอ่านค่าเวลาและแสดงผลใหม่

;*****
; SCAN LED 7          โปรแกรมย่อยแสดงผลทาง LED 7 ส่วนแบบมัลติเพล็กซ์ โดยอ่านค่า
; DPTR,R0,R1,R2      ตัวเลขจากหน่วยความจำ DISBUF ใช้รีจิสเตอร์ DPTR , R0 , R1 และ R2
;*****

SCAN:   MOV    R1,#4
        MOV    R2,#3
        MOV    R0,#DISBUF+3

LOOP:   MOV    DPTR,#SEG_CODE
        MOV    A,@R0
        MOVC   A,@A+DPTR
        MOV    DPTR,#PORT_A
        MOVX   @DPTR,A
        MOV    A,R2

```



```
MOV    DPTR,#PORT_C
MOVX   @DPTR,A
LCALL  DELAY
DEC    R0
DEC    R2
DJNZ   R1,LOOP
RET

,*****
DELAY:  MOV    R3,#0FFH
DE1:    DJNZ   R3,DE1
RET

,*****
BCD2DEC: ; โปรแกรมย่อยเปลี่ยนเลข BCD สองหลักหรือ 1 ไบต์ เป็นเลขฐานสิบสองหลัก
        PUSH   ACC                ; เก็บค่ารีจิสเตอร์ A และ B
        PUSH   B
        MOV    A,HEXBUF           ; อ่านค่าตัวเลข BCD ของนาที่ออกมา
        MOV    B,A                ; สำเนาเก็บในรีจิสเตอร์ B
        SWAP   A                  ; สลับบิตบนและบิตล่าง
        ANL    A,#00001111B       ; เลือกเฉพาะบิตล่าง
        MOV    DISBUF,A           ; นำตัวเลขหลักสิบที่ได้ใส่ใน DISBUF
        MOV    A,B                ; อ่านค่าตัวเลข BCD ค่าเดิม
        ANL    A,#00001111B       ; เอาเฉพาะหลักหน่วย
        MOV    DISBUF+1,A         ; นำหลักหน่วยใส่ในหน่วยความจำ
        MOV    A,HEXBUF+1         ; อ่านค่าเลข BCD ของวินาทีออกมา
        MOV    B,A
        SWAP   A
        ANL    A,#00001111B
        MOV    DISBUF+2,A         ; เก็บตัวเลขวินาทีหลักสิบ
        MOV    A,B
        ANL    A,#00001111B
        MOV    DISBUF+3,A         ; เก็บตัวเลขวินาทีหลักหน่วย
        POP    B
```



```

POP      ACC
RET
$INCLUDE "RTC_IOH.ASM" ; เรียกโปรแกรมการติดต่อกับ RTC
SEG_CODE: DB    3FH,06H,5BH,4FH,66H ; รูปแบบการแสดงผลทาง LED 7 เซกเมนต์
           DB    6DH,7DH,07H,7FH,6FH
           END

```

เนื่องจากวงจรการแสดงผลทาง LED 7 เซกเมนต์ที่ออกแบบไว้แสดงได้เพียงสี่หลัก โปรแกรมในตัวอย่างที่ 14.2 จึงเขียนให้แสดงผลแบบนาฬิกา และวินาที โดยตั้งเวลาให้เริ่มที่ 0 นาฬิกา 0 วินาที ถ้าหากต้องการตั้งเวลาเป็นเวลาใด ๆ ก็สามารถกำหนดในหน่วยความจำแฟลชได้

ตัวอย่างที่ 14.3 โปรแกรมต่อไปนี้จะแสดงเวลาเป็นชั่วโมง นาฬิกา และวินาทีทางหน่วยแสดงผลแบบ LCD โมดูล ที่ต่ออยู่ทางพอร์ต 0FA00H ถึง 0FA03H โดยจะแสดงผลทางบรรทัดที่สองของ LCD ในรูปของ 00:00:00

เนื่องจากการแสดงผลทาง LCD โมดูลจะต้องส่งข้อมูลที่เป็นรหัส ASCII เข้าไป ดังนั้นข้อมูลตัวเลข BCD ที่อ่านได้จากไอซีฐานเวลาจะต้องเปลี่ยนให้เป็นรหัส ASCII เสียก่อน โดยในโปรแกรมจะประกาศตัวแปรชื่อ DISBUF สำหรับหน่วยความจำแสดงผลของ LCD หน่วยความจำสำหรับเก็บชั่วโมง นาฬิกา และวินาที และเนื่องจากรหัส ASCII ของเลข 0 ถึง 9 จะมีค่าเป็น 30H ถึง 39H ดังนั้นการเปลี่ยนตัวเลขเป็นรหัส ASCII จะทำได้โดยนำค่า 30H ไปบวกกับค่าตัวเลขของแต่ละหลัก

LCDCOM	EQU	0FA00H	; ประกาศตำแหน่งพอร์ตต่าง ๆ ของ LCD
LCDBUS	EQU	0FA01H	
LCDWR	EQU	0FA02H	
LCDRD	EQU	0FA03H	
RTCSCL	EQU	P1.5	; ขาสัญญาณนาฬิกา
RTCSDA	EQU	P1.6	; ขารับส่งข้อมูลกับชิปฐานเวลา
DISBUF	EQU	32H	; ตำแหน่งหน่วยความจำแสดงผลของ LCD
RTCERR	EQU	79H	; ตำแหน่งของแฟลชตรวจสอบความผิดพลาด
HOURS	EQU	20H	; หน่วยความจำเก็บเลข BCD ของชั่วโมง
DOT1	EQU	21H	; หน่วยความจำเก็บสัญลักษณ์ :
MIN	EQU	22H	; หน่วยความจำเก็บเลข BCD ของนาฬิกา



DOT2	EQU	23H	; หน่วยความจำเก็บสัญลักษณ์ :
SEC	EQU	24H	; หน่วยความจำเก็บเลข BCD ของวินาที
,*****			
	ORG	0000H	
MAIN:	CALL	LCD_INT	; เรียกโปรแกรมย่อยกำหนดค่าเริ่มต้นของ LCD
	CALL	CLRSCR	; เคลียร์หน่วยความจำแสดงผล
	MOV	DOT1,#3AH	; ใส่รหัส ASCII ของ :
	MOV	DOT2,#3AH	
	MOV	DPTR,#TIMEBUF	; ชี้ที่ตำแหน่งหน่วยความจำเวลา
	MOV	A,#00H	; กำหนดเลข BCD ของวินาทีเป็น 00
	MOVBX	@DPTR,A	; เขียนลงหน่วยความจำเวลา
	INC	DPTR	; ชี้ที่หน่วยความจำนาที
	MOV	A,#00H	; กำหนดให้นาทีเป็น 00
	MOVBX	@DPTR,A	
	INC	DPTR	; ชี้ที่หน่วยความจำชั่วโมง
	MOV	A,#00H	; กำหนดให้ชั่วโมงเป็น 00
	MOVBX	@DPTR,A	
	CALL	RTWR	; เขียนข้อมูลเวลาลงไปบนชิป
AA:			
	CALL	RTRD	; อ่านข้อมูลเวลาออกมาจากชิป
	MOV	DPTR,#TIMEBUF	; ชี้ที่หน่วยความจำบัฟเฟอร์เวลา
	MOVBX	A,@DPTR	; อ่านเลข BCD ของวินาทีออกมา
	MOV	SEC,A	; เก็บเลข BCD ของวินาทีลงหน่วยความจำ SEC
	INC	DPTR	; ชี้ที่หน่วยความจำนาที
	MOVBX	A,@DPTR	
	MOV	MIN,A	; เก็บเลข BCD ของนาทึกลงหน่วยความจำ MIN
	INC	DPTR	; ชี้ที่หน่วยความจำชั่วโมง
	MOVBX	A,@DPTR	
	MOV	HOURS,A	; เก็บเลข BCD ของชั่วโมงลงหน่วยความจำ
		HOURL	
	CALL	SHOW_TIME	; เรียกโปรแกรมย่อยแสดงเวลา
	SJMP	AA	; วนลูปอ่านค่าเวลา



```

SHOW_TIME:                                ; โปรแกรมย่อยแสดงเวลาในรูปแบบของ
                                           00:00:00
MOV    A,#0C0H                            ; หน่วยความจำตำแหน่งแรกในบรรทัดที่สอง
                                           ของ LCD
CALL   COMMAND                            ; เขียนคำสั่งลงบน LCD
CALL   TIME_TO_DISBUF                      ; นำข้อมูลเวลาใส่ในหน่วยความจำแสดงผล
MOV    R0,#DISBUF                          ; ให้ R0 ซี่ที่หน่วยความจำแสดงผล
CALL   SHOW_LINE                           ; เรียกโปรแกรมย่อยแสดงผลทาง LCD หนึ่ง
                                           บรรทัด
RET                                         ; จบโปรแกรมย่อย

TIME_TO_DISBUF:                          ; โปรแกรมย่อยนำข้อมูลเวลาใส่ในหน่วยความจำแสดงผล
MOV    A,HOURS                             ; อ่านเลข BCD ของชั่วโมง
CALL   BCD_TO_ASCII                       ; เปลี่ยนเลข BCD ให้เป็นรหัส ASCII
MOV    DISBUF+4,R2                         ; ใส่หลักสิบของชั่วโมงในคอลัมน์ที่ 5 ของ LCD
MOV    DISBUF+5,R3                         ; ใส่หลักหน่วยของชั่วโมง
MOV    DISBUF+6,DOT1                       ; ใส่เครื่องหมาย :
MOV    A,MIN                               ; อ่านเลข BCD ของนาที
CALL   BCD_TO_ASCII                       ; เปลี่ยนให้เป็นรหัส ASCII
MOV    DISBUF+7,R2                         ; ใส่หลักสิบของนาที
MOV    DISBUF+8,R3                         ; ใส่หลักหน่วยของนาที
MOV    DISBUF+9,DOT2                       ; ใส่เครื่องหมาย :
MOV    A,SEC                               ; อ่านค่าเลข BCD ของวินาที
CALL   BCD_TO_ASCII                       ; เปลี่ยนเลข BCD เป็นรหัส ASCII
MOV    DISBUF+10,R2                        ; ใส่หลักสิบของวินาที
MOV    DISBUF+11,R3                       ; ใส่หลักหน่วยของวินาที
RET

; *****
;   BCD TO ASCII   โปรแกรมอ่านเลข BCD หนึ่งไบต์ และให้ข้อมูลเป็นรหัส ASCII สองหลัก
;   INPUT A : OUTPUT R2,R3 โดยอินพุตเป็นรีจิสเตอร์ A และให้เอาต์พุตออกมาทาง R2 และ R3
; *****
BCD_TO_ASCII:

```



```
PUSH    B
MOV     B,A                ; คัดลอกเลข BCD มาเก็บใน B
ANL     A,#11110000B      ; ทำเลข BCD หลักหน่วยให้เป็น 0
SWAP    A                  ; สลับระหว่างสปีตบนและสปีตล่าง
ADD     A,#30H             ; บวกด้วย 30H จะได้รหัส ASCII ของหลักสิบ
MOV     R2,A               ; ให้ R2 เก็บรหัส ASCII ของหลักสิบ
MOV     A,B                ; อ่านเลข BCD ค่าเดิมมาเก็บใน A อีกครั้ง
ANL     A,#00001111B      ; อ่านเลข BCD หลักหน่วย
ADD     A,#30H             ; บวกด้วย 30H จะได้รหัส ASCII ของหลักหน่วย
MOV     R3,A               ; ให้ R3 เก็บรหัส ASCII ของหลักหน่วย
POP     B
RET                        ; จบโปรแกรมย่อย

,*****
CLRSCR:  MOV     R0,#DISBUF ; โปรแกรมย่อยเคลียร์หน่วยความจำแสดงผล
          MOV     R1,#16D   ; หนึ่งบรรทัดมี 16 ตัว
L_CLR:   MOV     @R0,#20H   ; ใส่รหัส ASCII ของที่ว่าง (20H)
          INC     R0         ; ชี้หน่วยความจำถัดไป
          DJNZ    R1,L_CLR   ; ทำจนครบ 16 ตัว
          RET
$INCLUDE "LCDIOH.ASM" ; เรียกโปรแกรมย่อยติดต่อกับ LCD
$INCLUDE "RTC_IOH.ASM"; เรียกโปรแกรมย่อยติดต่อกับชิป RTC
END
```

เครื่องมือการทดลอง ไมโครคอนโทรลเลอร์

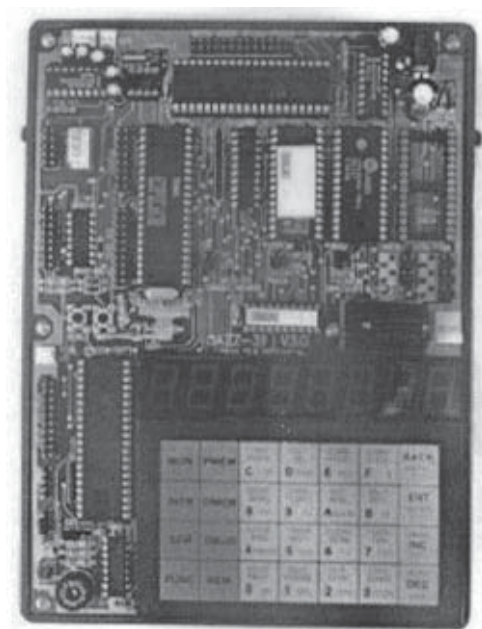
ไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีอยู่ด้วยกันหลายเบอร์ ทำให้มีเครื่องมือที่ใช้ช่วยในการพัฒนางานมากมาย ในบทนี้จะอธิบายถึงแนวทางและตัวอย่างการพัฒนางานทางด้านไมโครคอนโทรลเลอร์ รวมทั้งตัวอย่างเครื่องมือในการพัฒนาโปรแกรม สำหรับการทดลองต่าง ๆ ในบทนี้ผู้เขียนได้เลือกใช้ชุดทดลองด้านไมโครคอนโทรลเลอร์ของบริษัทซิลาร์เรซ (Sila Research) โดยรายละเอียดของเครื่องมือต่าง ๆ ผู้สนใจสามารถเข้าไปดูได้ที่ www.silaresearch.com ซึ่งในเว็บไซต์นี้ยังมีตัวอย่างโปรแกรมให้ดาวน์โหลดมาศึกษาอีกด้วย

สำหรับการทดลองโปรแกรมในบทนี้จะเป็นอย่างโปรแกรมที่ใช้กับชิป 89C51RD2 ของบริษัท Philips ซึ่งเป็นไมโครคอนโทรลเลอร์ที่มีความเร็วเป็นสองเท่าของ MCS-51 มาตรฐาน โดยที่หนึ่งเมกเฮิรตซ์-ไซเคิลจะใช้สัญญาณนาฬิกา 6 ลูก นอกจากนี้ชิปตัวนี้ยังมีหน่วยความจำแบบ FLASH MEMORY ขนาด 64 กิโลไบต์อยู่ภายในชิป ที่สามารถโปรแกรมและล้างได้มากถึง 10,000 ครั้ง และการโปรแกรมให้กับตัวชิปสามารถโปรแกรมโดยไม่จำเป็นต้องถอดไมโครคอนโทรลเลอร์ออกมาจากบอร์ดทดลอง ที่เรียกว่าการโปรแกรมในวงจร หรือการโปรแกรมในระบบ (ISP : In-system programming) โดยภายในตัวชิปเองจะมีโปรแกรมบูตรอม (Boot ROM) สำหรับติดต่อกับเครื่องคอมพิวเตอร์ PC เพื่อโปรแกรมให้กับตัวมัน



15.1 การพัฒนาด้วยซิงเกิลบอร์ด

สำหรับผู้เริ่มศึกษาทางด้านไมโครคอนโทรลเลอร์ การสร้างวงจรขึ้นมาทำการทดลองเองอาจทำให้เสียเวลามาก ปัจจุบันได้มีการพัฒนาอุปกรณ์ทดลองไมโครคอนโทรลเลอร์ในรูปของซิงเกิลบอร์ดซึ่งมีทั้งของต่างประเทศ และของที่ผลิตภายในประเทศ โดยอุปกรณ์ประเภทนี้จะมีคีย์บอร์ด และหน่วยแสดงผลติดมากับตัวเครื่อง ทำให้สามารถแก้ไขโปรแกรมและดูผลลัพธ์จากการรันโปรแกรมได้ง่าย นอกจากนี้ซิงเกิลบอร์ดยังมีพอร์ตสำหรับเชื่อมต่อกับอุปกรณ์ภายนอกได้อีกด้วย สำหรับการเขียนโปรแกรมจะเขียนด้วยภาษาแอสเซมบลีบนคอมพิวเตอร์ PC จากนั้นจะใช้โปรแกรมแอสเซมเบลอร์แปลให้เป็นรหัสในรูปแบบของ hexadecimal file (ในที่นี้ผู้เขียนใช้โปรแกรม SXA51 ในการแอสเซมเบลอร์) แล้วจึงโหลดรหัสที่ได้ลงบนหน่วยความจำของซิงเกิลบอร์ด แต่โปรแกรมที่เขียนขึ้นจะไม่สามารถเริ่มต้นที่แอดเดรสแรกได้ (แอดเดรส 0000H) เนื่องจากซิงเกิลบอร์ดจะต้องใช้หน่วยความจำส่วนหนึ่งเป็นหน่วยความจำของโปรแกรมมอนิเตอร์เพื่อควบคุมการทำงานของบอร์ด สำหรับบอร์ดที่ผู้เขียนได้ทดลองใช้คือบอร์ด JAZZ-31 ดังรูปที่ 15.1

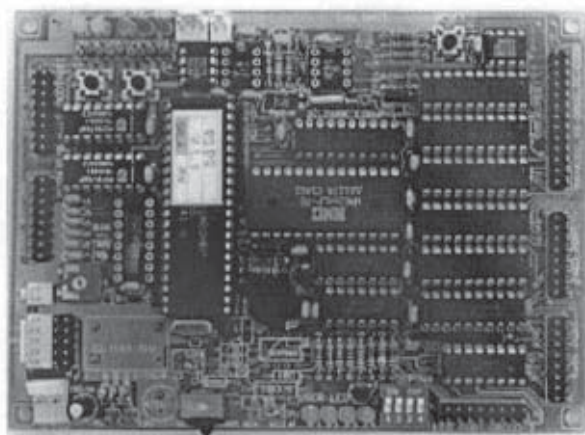


รูปที่ 15.1 บอร์ด JAZZ-31



15.2 การพัฒนาด้วย Embedded Board

ตัวซิงเกิลบอร์ดนั้นเป็นเครื่องมือที่เหมาะสมสำหรับการเรียนรู้ แต่ถ้าหากนำไปใช้งานจริง ๆ จะไม่คุ้มเนื่องจากซิงเกิลบอร์ดมีราคาแพงเพราะจะมีส่วนของโปรแกรมมोनิตอร์และอุปกรณ์ต่าง ๆ ติดไปด้วย สำหรับการนำระบบไมโครคอนโทรลเลอร์ไปใช้งานจริงมักจะใช้บอร์ดแบบ Embedded Board เนื่องจากมีราคาถูกเพราะว่าภายในบอร์ดจะประกอบด้วยชิปไมโครคอนโทรลเลอร์และอุปกรณ์ที่สำคัญ ๆ เท่านั้น ในการทดลองนี้ผู้เขียนได้ใช้บอร์ด ISP-1000 ซึ่งเป็นบอร์ดที่ใช้ชิปเบอร์ 89C51RD2 เป็นบอร์ดทดลอง โดยลักษณะของบอร์ดแสดงได้ดังรูปที่ 15.2 ซึ่งจะเป็นว่ามีคอนเนกเตอร์สำหรับต่อกับอุปกรณ์ภายนอกด้วย



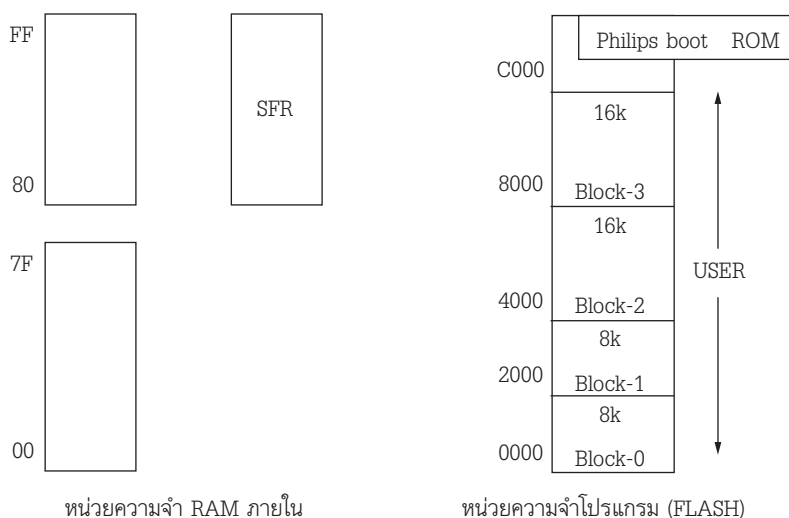
รูปที่ 15.2 วงจรบอร์ด ISP-1000

วงจรของบอร์ด ISP-1000 แสดงได้ดังรูปที่ 15.3 จากรูปจะเห็นว่าพอร์ตขนานสำหรับเป็นพอร์ตเอาต์พุตจะใช้ชิปเบอร์ 74HC374 ซึ่งภายในจะประกอบด้วยฟลิปฟล็อปจำนวน 8 ตัว สำหรับพอร์ตอินพุตแบบขนาน 8 บิตจะใช้ชิปเบอร์ 74HC245 ซึ่งภายในประกอบด้วยบัฟเฟอร์จำนวน 8 ตัว นอกจากนี้ในแผงวงจรยังมีการนำชิปที่ใช้ระบบบัสแบบ I²C มาใช้งานด้วย เช่นใช้ชิป PCF8574 จำนวนสองตัวมาสร้างเป็นพอร์ตขนานใช้ชิป PCF8591 สำหรับเป็นวงจรแปลงสัญญาณแอนะล็อกเป็นดิจิทัล ใช้ชิป DS1307 เป็นชิปบอกเวลาจริง สำหรับสัญญาณนาฬิกาที่ใช้บนบอร์ด ISP-1000 นี้จะสามารถเลือกได้ทั้งแบบ 11.0592 MHz และ 22.1184 MHz โดยเลือกจาก Jumper ที่ต่อเอาไว้



15.3 หน่วยความจำของ 89C51RD2 และการโปรแกรม

ไมโครคอนโทรลเลอร์ 89C51RD2 ของบริษัท Philips มีหน่วยความจำหลัก ๆ อยู่ 3 ส่วนคือ หน่วยความจำโปรแกรม (Program memory) ขนาด 64 กิโลไบต์ หน่วยความจำข้อมูล (data memory) ขนาด 1 กิโลไบต์ และหน่วยความจำบูตโรม (Boot ROM) ขนาด 1 กิโลไบต์ โดยโปรแกรมที่อยู่ในบูตโรมนี้อาจใช้ในการติดต่อแบบ ISP โดยมีแอดเดรสอยู่ที่ตำแหน่ง FC00H - FFFFH ทำให้สามารถโปรแกรมใช้งานได้จริง ๆ 63 กิโลไบต์ สำหรับหน่วยความจำโปรแกรมนั้นได้แบ่งออกเป็นบล็อก ๆ ดังรูปที่ 15.4 ซึ่งผู้ใช้สามารถเลือกโปรแกรมหรือลบข้อมูลในบล็อกใด ๆ ได้ สำหรับหน่วยความจำข้อมูลขนาด 1 กิโลไบต์นั้นจะมีการแบ่งออกเป็นหน่วยความจำสำหรับเก็บข้อมูลทั่วไปขนาด 128 ไบต์ตั้งแต่แอดเดรส 00H-7FH หน่วยความจำที่เป็นพื้นที่ของรีจิสเตอร์ฟังก์ชันพิเศษขนาด 128 ไบต์ตั้งแต่แอดเดรส 80H - FFH และหน่วยความจำแรมเพิ่มเติม (Expanded RAM) อีก 768 ไบต์



รูปที่ 15.4 การจัดหน่วยความจำของ 89C51RD2

สำหรับการโปรแกรมให้กับ 89C51RD2 นั้นสามารถกระทำผ่านทางโปรแกรม WinISP ของบริษัท Philips ที่ทำงานภายใต้ระบบปฏิบัติการวินโดวส์ได้ โปรแกรมจะติดต่อกับชิปผ่านทางพอร์ตอนุกรมของเครื่องคอมพิวเตอร์ PC และพอร์ตอนุกรมของชิป โดยโปรแกรมนี้อาจสามารถลบข้อมูลบนชิป และใช้ดาวน์โหลดไฟล์ .HEX ลงบนตัวชิปได้ เมื่อเรียกโปรแกรม WinISP ขึ้นมาจะเห็นหน้าต่างของโปรแกรมเป็นดังรูปที่ 15.5 เมื่อเรียกโปรแกรมขึ้นมาจะต้องตั้งค่าต่าง ๆ ดังนี้

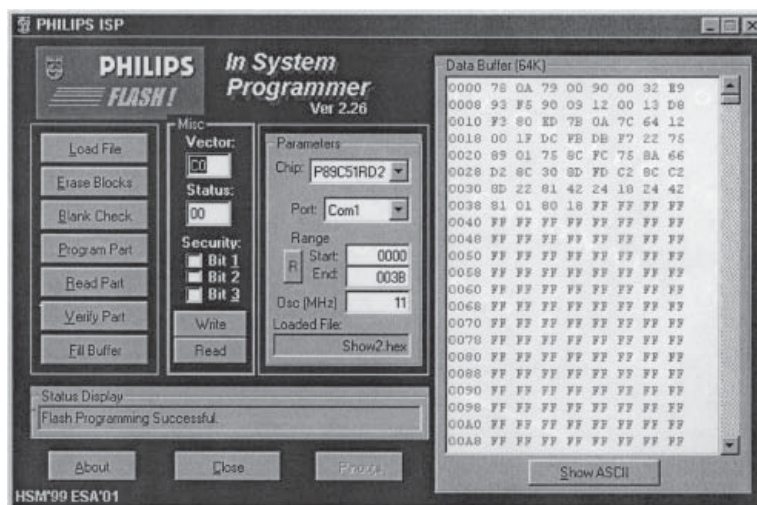
1. เลือกชิปเป็นเบอร์ 89C51RD2
2. เลือกพอร์ตสื่อสารอนุกรมให้ตรงกับพอร์ตที่ใช้งานจริง (พอร์ต COM1 หรือ COM2)



3. ใส่ค่าความถี่ของ OSC ที่ใช้ เช่นถ้าใช้สัญญาณนาฬิกา 11.0592 MHz ก็ให้ใส่ค่า 11 ลงไป
4. ตั้งค่า Vector เป็น C0 และค่า Status เป็น 00

สำหรับคำสั่งต่าง ๆ ของโปรแกรมเป็นดังนี้

Load File	ใช้สำหรับโหลดไฟล์ .HEX มาเก็บในหน่วยความจำ เพื่อเตรียมโปรแกรมลงบนชิป โดยเมื่อโหลดไฟล์เข้ามาแล้วค่าตำแหน่ง Start และ End จะเป็นไปตามข้อมูลของไฟล์นั้น ๆ
Erase Blocks	ใช้ลบข้อมูลที่อยู่ในชิป โดยสามารถใช้เมาส์เลือกบล็อกที่ต้องการลบได้
Blank Check	ใช้ตรวจสอบค่า Blank หรือค่า FFH ตามตำแหน่ง Start ถึงตำแหน่ง End
Program Part	ใช้โหลดไฟล์ .HEX ที่เก็บอยู่ในหน่วยความจำโปรแกรมลงบนตัวชิป ตามตำแหน่ง Start ถึง End
Read Part	ใช้อ่านข้อมูลจากชิปมาเก็บในหน่วยความจำบัฟเฟอร์
Verify Part	ใช้เปรียบเทียบข้อมูลที่อยู่ในชิปกับข้อมูลที่อยู่ในหน่วยความจำ
Fill Buffer	ใช้สำหรับใส่ค่าข้อมูลใด ๆ ลงใน Data Buffer ตามตำแหน่ง Start ถึง End



รูปที่ 15.5 โปรแกรม WinISP



15.4 การเขียนโปรแกรมส่งข้อมูลทางพอร์ตอนุกรม

ตัวอย่างต่อไปเป็นโปรแกรมส่งข้อมูลที่เป็นรหัส ASCII ออกจากตัว 89C51RD2 ทางพอร์ตอนุกรมให้กับคอมพิวเตอร์ PC โดยใช้โปรแกรม Hyper Terminal ของวินโดวส์เป็นตัวรับข้อมูลและแสดงผลทางหน้าจอคอมพิวเตอร์ โดยโปรแกรมจะกำหนดให้ MCS-51 รับส่งข้อมูลด้วยอัตรา 9600 bps โดยกำหนดให้ SCON รับส่งพอร์ตอนุกรมในโหมด 1 โดยใช้ไทมเมอร์ 1 โหมด 2 เป็นตัวกำหนดการรับส่งข้อมูล โดยข้อมูลที่จะส่งจะถูกอ่านออกมาจากตารางแบบ Look up table และนำข้อมูลแต่ละไบต์ที่อ่านออกมาได้ส่งให้กับรีจิสเตอร์ SBUF เพื่อส่งให้ไมโครคอมพิวเตอร์ที่ละไบต์ โดยโปรแกรมที่เขียนขึ้นเป็นดังนี้

```

CR      EQU    0DH      ; รหัสของ Carriage return
LF      EQU    0AH      ; รหัสของ Line feed
ORG     0000H          ; ให้โปรแกรมเริ่มต้นที่แอดเดรส 0000H
MOV     PCON,#00000000B ; กำหนดอัตราการรับส่งข้อมูล
MOV     SCON,#01000000B ; ใช้พอร์ตอนุกรมโหมด 1
MOV     TMOD,#21H      ; ไทมเมอร์ 1 แบบ 8 บิต Auto reload
MOV     TL1,#0FAH
MOV     TH1,#0FAH      ; กำหนดอัตราการรับส่งข้อมูล 9600 bps
SETB    TR1            ; ให้ไทมเมอร์ 1 เริ่มทำงาน

;*****
MOV     DPTR,#DATA      ; ชี้ไปที่ตารางของข้อมูลที่จะส่ง
MOV     R0,00H          ; ให้ R0 มีค่าแรกเป็น 00
SEND_D: MOV     A,R0      ; นำค่า R0 มาเก็บใน A
        MOVC    A,@A+DPTR ; อ่านข้อมูลรหัส ASCII จากตารางมาเก็บใน A
        MOV     SBUF,A    ; นำข้อมูลที่ได้ส่งให้รีจิสเตอร์ SBUF เพื่อส่งออกไป
        JNB     TI,$      ; รอจนส่งครบทุกบิต
        CLR     TI        ; เคลียร์บิต TI เพื่อส่งข้อมูลไบต์ต่อไป
        INC     R0        ; เพิ่มค่า R0 เพื่อชี้ตำแหน่งข้อมูลต่อไป
        JNZ     SEND_D    ; ทำซ้ำส่งข้อมูลไบต์ต่อไปจนกว่าข้อมูลเป็น 00H

;*****
DATA:   DB      CR,LF,"SHOW MCS-51 SEND DATA",CR
        DB      CR,LF,"By TEERAWAT PRAKOBPHON",CR,LF
        DB      00H
        END

```



หลังจากเขียนโปรแกรมด้วยภาษาแอสเซมบลีแล้วให้แปลงข้อมูลที่ได้อให้เป็นไฟล์ .HEX ซึ่งจะได้ข้อมูลดังต่อไปนี้

:10000000758700759840758921758BFA758DFAD2C0

:100010008E900022A800E893F5993099FDC29908C6

:1000200070F40D0A53484F57204D43532D3531205E

:1000300053454E4420444154410D0D0A4279205409

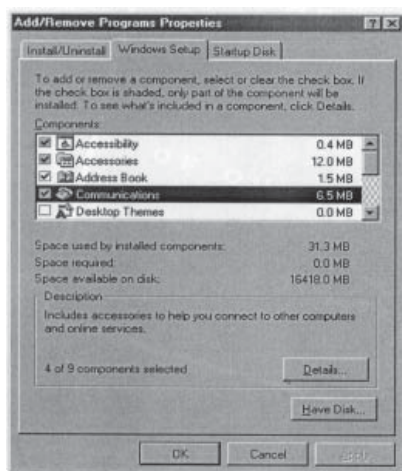
:1000400045455241574154205052414B4F42504830

:050050004F4E0D0A00F7

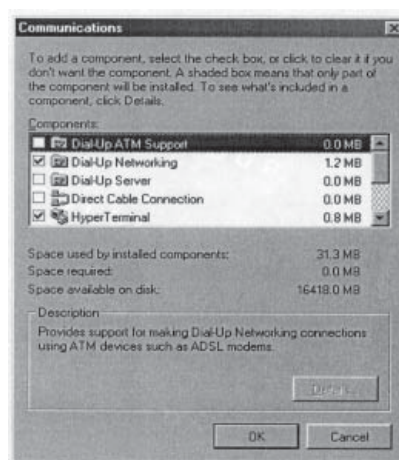
:00000001FF

ขั้นตอนต่อไปให้โปรแกรมไฟล์ .HEX ที่ได้ลงบนชิป 89C51RD2 โดยใช้โปรแกรม WinISP ก็จะทำให้ไมโครคอนโทรลเลอร์พร้อมที่จะส่งข้อมูลออกจากตัวมันให้กับเครื่องคอมพิวเตอร์ PC สำหรับบนเครื่องคอมพิวเตอร์ PC จะต้องติดตั้งโปรแกรมรับส่งข้อมูล ในที่นี่จะใช้โปรแกรม Hyper Terminal ถ้าหากยังไม่ได้ติดตั้งสามารถทำได้ตามขั้นตอนต่อไปนี้

1. เลือกไอคอน Add/Remove Program ใน Control Panel ของโปรแกรมวินโดวส์
2. เลือก Communications ดังรูปที่ 15.6
3. ติดตั้งโปรแกรม Hyper Terminal ดังรูปที่ 15.7



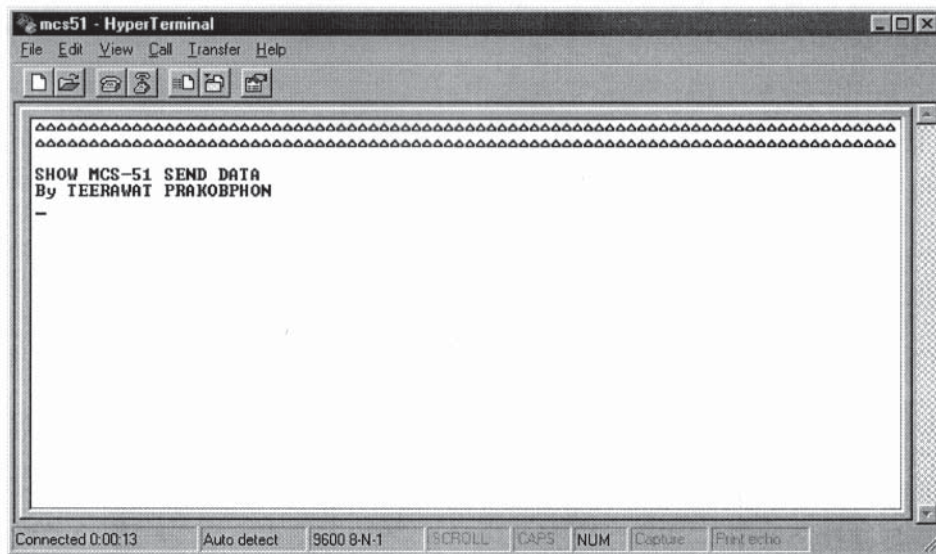
รูปที่ 15.6 เลือกไปที่ Communications



รูปที่ 15.7 คลิกเลือก HyperTerminal



หลังจากติดตั้งโปรแกรมแล้วสามารถรันโปรแกรม Hyper Terminal ได้โดยเลือกที่ Programs> Accessories> Communications> HyperTerminal และให้เลือกพอร์ตการสื่อสารเป็น Com1 หรือ Com2 ตามสายที่ต่อเอาไว้ และกำหนดอัตราการรับส่งข้อมูลเป็น 9600 เมื่อให้ไมโครคอนโทรลเลอร์ทำงานตามโปรแกรมเครื่องคอมพิวเตอร์ PC จะแสดงผลที่ได้ออกมาดังรูปที่ 15.8

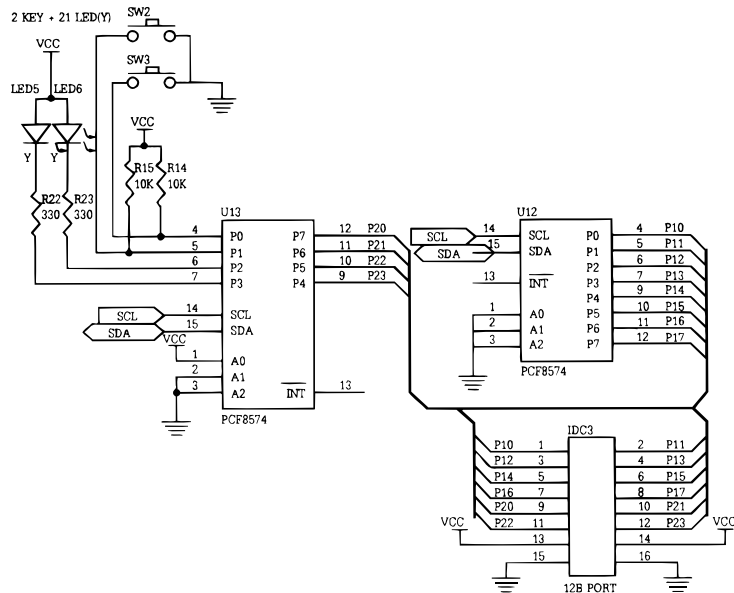


รูปที่ 15.8 ผลลัพธ์จากการรันโปรแกรม



15.5 การขยายพอร์ตและการเขียนโปรแกรม

จากตัวอย่างวงจรของบอร์ด ISP-1000 ในรูปที่ 15.3 จะพบว่ามีการใช้ไอซีฟลิปฟล็อปเบอร์ 74HC374 เป็นพอร์ตเอาต์พุตแบบขนาน และใช้ไอซีชิฟเฟอร์เบอร์ 74HC245 เป็นพอร์ตอินพุตแบบขนาน โดยสามารถเขียนโปรแกรมติดต่อได้ตามแอดเดรสที่กำหนดเอาไว้ นอกจากนี้ยังมีใช้พอร์ตแบบ I²C เบอร์ PCF8574 จำนวนสองตัวมาต่อเป็นพอร์ตขนานอีกด้วย โดยชิปทั้งสองถูกกำหนดแอดเดรสไว้ต่างกัน โดยวงจรสามารถเขียนใหม่ได้เป็นดังรูปที่ 15.9



รูปที่ 15.9 การใช้ไอซี PCF8574 เป็นพอร์ต

จากวงจรจะพบว่า PCF8574 ตัวแรกจะกำหนดแอดเดรสแบบโปรแกรมได้ไว้เป็น 000 โดยไอซีตัวนี้จะถูกใช้เป็นพอร์ตขนานขนาด 8 บิต ส่วน PCF8574 ตัวที่สองจะถูกกำหนดแอดเดรสแบบโปรแกรมได้ไว้เป็น 001 โดยที่สองบิตต่ำ (บิต P0 และ P1) จะถูกใช้งานเป็นพอร์ตอินพุต โดยต่อสวิตช์เอาไว้ สำหรับบิต P2 และ P3 จะต่อกับหลอด LED ถ้าสองบิตนี้เป็นลอจิก 0 จะทำให้หลอด LED สว่าง สำหรับสปีดบนจะถูกใช้งานเป็นพอร์ตอินพุตเอาต์พุตทั่วไป

การเขียนโปรแกรมติดต่อกับ PCF8574 นี้สามารถใช้โปรแกรมรวม IO8574H.ASM ตามที่ได้ศึกษามาในบทที่ 13 ได้ ถ้าหากต้องการใช้ PCF8574 เป็นพอร์ตเอาต์พุต สามารถทำได้โดยเรียกใช้โปรแกรมน้อย IPW และกำหนดแอดเดรสของชิปในรีจิสเตอร์ R2 สำหรับข้อมูลที่ต้องการเขียนลงชิปให้กำหนดในรีจิสเตอร์ R3

ถ้าหากมีการต่อหลอด LED เข้ากับพอร์ตของ PCF8574 และต้องการเขียนโปรแกรมควบคุมการติดดับของหลอด LED ครั้งละสี่หลอดสามารถเขียนโปรแกรมได้ดังต่อไปนี้

```

*****
;
;*  ISP-1000 v2.0      *
;*  test 12B port      *
;*  PCF8574A           *
*****
;

```

```

RTCSCL EQU P1.5

```

; ขาสัญญาณนาฬิกาที่ต่อกับ 89C51RD2



```

RTCSDA EQU P1.6 ; ขาสัญญาณข้อมูลที่ต่อกับ 89C51RD2
ERRFAG EQU 7AH
ORG 0000H ; กำหนดแอดเดรสเริ่มต้นเป็น 0000H

MAIN:
MOV R2,#00H ; กำหนดแอดเดรสของชิปตัวแรก
MOV R3,#0F0H ; กำหนดข้อมูลให้สปีดบนเป็น "1" สปีดล่างเป็น "0"
CALL IPW ; เขียนข้อมูลลงบนชิป
JB ERRFAG,$ ; ตรวจสอบความถูกต้องในการเขียนข้อมูล
CALL DELAY ; เรียกโปรแกรมย่อยหน่วงเวลา
MOV R2,#00H ; กำหนดแอดเดรสของชิปตัวแรก
MOV R3,#0FH ; ให้สปีดล่างเป็นลอจิก "1"
CALL IPW ; เขียนข้อมูลลงบนชิป
JB ERRFAG,$ ; ตรวจสอบความถูกต้องในการเขียนข้อมูล
CALL DELAY ; เรียกโปรแกรมย่อยหน่วงเวลา
MOV R2,#01H ; กำหนดแอดเดรสของชิปตัวที่สอง
MOV R3,#0F0H ; กำหนดข้อมูลให้สปีดบนเป็นลอจิก "1"
CALL IPW ; เขียนข้อมูลลงบนชิป
JB ERRFAG,$ ; ตรวจสอบความถูกต้องของการเขียนข้อมูล
CALL DELAY ; เรียกโปรแกรมย่อยหน่วงเวลา
MOV R2,#01H ; กำหนดแอดเดรสของชิปตัวที่สอง
MOV R3,#00H ; กำหนดข้อมูลให้เป็นลอจิก "0" ทุกบิต
CALL IPW ; เขียนข้อมูลลงบนชิป
JB ERRFAG,$ ; ตรวจสอบความถูกต้องของการเขียนข้อมูล
CALL DELAY ; เรียกโปรแกรมย่อยหน่วงเวลา
SJMP MAIN ; ทำซ้ำ

DELAY: MOV R1,#0FH ; โปรแกรมย่อยหน่วงเวลา
DE1: MOV R2,#0FFH
DE2: MOV R3,#0FFH
DJNZ R3,$
DJNZ R2,DE2
DJNZ R1,DE1
RET

$INCLUDE "IO8574H.ASM" ; เรียกโปรแกรมร่วมในการติดต่อกับชิป
END

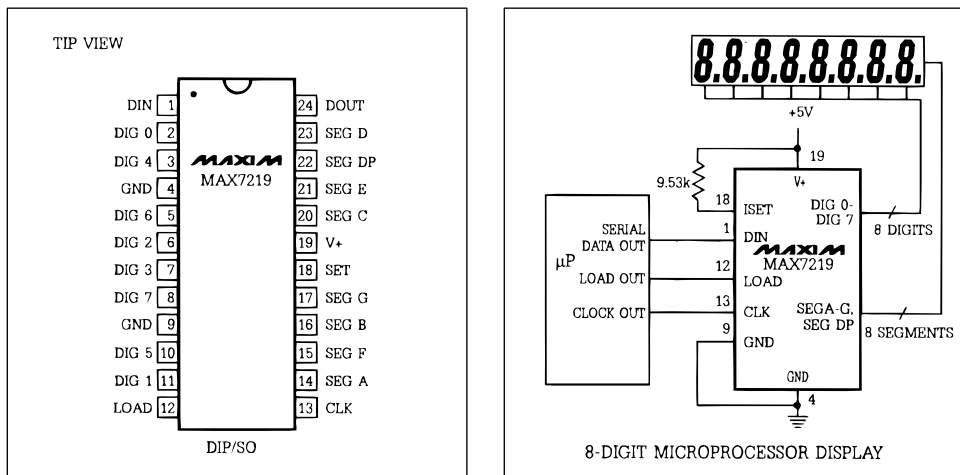
```



15.6 การสแกนหน่วยแสดงผลด้วยชิป MAX7219

การแสดงผลทาง LED 7 ส่วนหลาย ๆ หลักที่ผ่านมาจะใช้วิธีการนำ LED 7 ส่วนมาต่อเรียงกัน และใช้เทคนิคการเขียนโปรแกรมแบบการสแกน (scan) โดยให้ LED สว่างครั้งละหลัก จากนั้นให้หลักต่อไปสว่าง โดยจะทำเช่นนี้ครบทุกหลัก จากนั้นจึงเริ่มต้นสแกนใหม่ การทำงานลักษณะนี้ไมโครคอนโทรลเลอร์จะต้องแกนด้วยความเร็วสูง ซึ่งจะทำให้ไมโครคอนโทรลเลอร์ต้องเสียเวลาส่วนหนึ่งไป ในหัวข้อนี้จะแนะนำไอซีเบอร์ MAX7219 ซึ่งเป็นไอซีที่สามารถใช้ในการสแกนหน่วยแสดงผลได้ โดยถ้าหากส่งข้อมูลให้กับชิปตัวนี้แล้ว วงจรภายในตัวมันจะทำการสแกนหน่วยแสดงผลให้โดยอัตโนมัติ ทำให้ไมโครคอนโทรลเลอร์สามารถใช้เวลาไปทำงานอื่นได้ ไม่ต้องเสียเวลาสแกนหน่วยแสดงผลอีกต่อไป

ชิปไอซี MAX7219 เป็นไอซีที่ถูกออกแบบมาสำหรับรับข้อมูลจากไมโครคอนโทรลเลอร์แบบอนุกรม และมีวงจรขับทางเอาต์พุตให้กับหลอด LED ที่ต่อแบบคาโทดรวม โดยสามารถต่อกับ LED 7 ส่วนได้ 8 หลัก หรือต่อกับ LED แบบเมทริกซ์ได้ 64 ดวง โดยภายในชิปจะมีวงจรสแกน วงจรถอดรหัส BCD สำหรับเปลี่ยนเป็นรูปแบบข้อมูลที่จะแสดงผลทาง LED 7 ส่วน และวงจรขับในแต่ละหลัก การใช้งานไอซีเบอร์นี้ให้แสดงผลเป็นรูปแบบใดก็เพียงแต่ส่งข้อมูลทางอนุกรมให้กับตัวไอซีเท่านั้น โดยภายในตัวไอซีจะมีหน่วยความจำ RAM ขนาด 8×8 สำหรับเก็บข้อมูลที่ต้องการแสดงผลและส่งให้วงจรขับ สำหรับการเชื่อมต่อกับไมโครคอนโทรลเลอร์จะใช้สายสัญญาณเพียง 3 เส้น โดยมีขารับข้อมูลแบบอนุกรม ขารับสัญญาณนาฬิกา และขาโหลดข้อมูล สำหรับรายละเอียดของไอซีเบอร์นี้สามารถศึกษาได้จากคู่มือของมันได้โดยตรง ลักษณะของชิป MAX7219 แสดงได้ดังรูปที่ 15.10



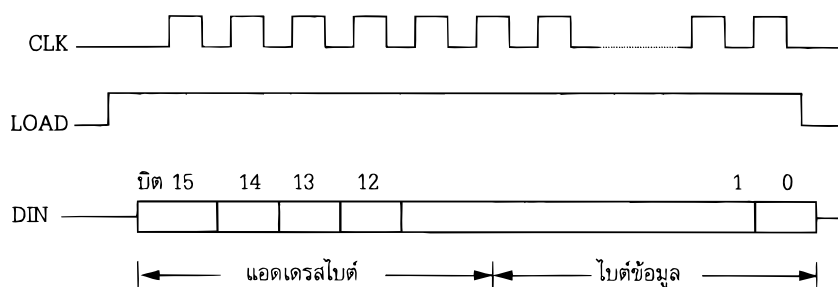
รูปที่ 15.10 ไอซี MAX7219 และตัวอย่างการเชื่อมต่อกับไมโครคอนโทรลเลอร์



ในการใช้งานไอซี MAX7219 นั้น เริ่มต้นจะต้องส่งข้อมูลค่าเริ่มต้นให้กับรีจิสเตอร์ต่าง ๆ ภายในตัวมันก่อน เพื่อกำหนดรูปแบบการทำงาน โดยรีจิสเตอร์ต่าง ๆ จะมีทั้งหมด 14 ตัว เป็นรีจิสเตอร์สำหรับเก็บข้อมูลที่จะให้แสดงผลแต่ละหลักจำนวน 8 ตัว ส่วนที่เหลือเป็นรีจิสเตอร์ควบคุม เช่น กำหนดความสว่างของ LED กำหนดว่าการสแกนนั้นจะให้สแกนถึงหลักใด กำหนดว่าจะให้วงจรถอดรหัสภายในทำการถอดรหัสสำหรับ LED 7 ส่วนหรือไม่ รีจิสเตอร์แต่ละตัวจะมีแอดเดรสประจำตัวมัน โดยหน้าที่และแอดเดรสของรีจิสเตอร์ใน MAX7219 แด ดงได้ดังตารางที่ 15.1 โดยการเขียนข้อมูลรีจิสเตอร์แต่ละตัวลงไปจะต้องเขียนครั้งละ 16 บิต โดยค่า 8 บิตบนจะเป็นค่าแอดเดรส สำหรับค่า 8 บิตต่ำจะเป็นค่าที่ส่งให้กับรีจิสเตอร์ตัวนั้น ๆ ดังแสดงในรูปที่ 15.11

ตารางที่ 15.1 รีจิสเตอร์และแอดเดรสใน MAX7219

รีจิสเตอร์	แอดเดรส	การทำงาน
NO-OP	00	ส่งข้อมูลให้ไอซีที่ต่อกันหลายตัว
DIGIT 0	01	ส่งข้อมูลให้หลักที่ 0
DIGIT 1	02	ส่งข้อมูลให้หลักที่ 1
DIGIT 2	03	ส่งข้อมูลให้หลักที่ 2
DIGIT 3	04	ส่งข้อมูลให้หลักที่ 3
DIGIT 4	05	ส่งข้อมูลให้หลักที่ 4
DIGIT 5	06	ส่งข้อมูลให้หลักที่ 5
DIGIT 6	07	ส่งข้อมูลให้หลักที่ 6
DIGIT 7	08	ส่งข้อมูลให้หลักที่ 7
DECODE MODE	09	กำหนดว่าจะให้มีการถอดรหัสข้อมูลหรือไม่
INTENSITY	0A	กำหนดความสว่างของหลอด LED
SCAN LIMIT	0B	กำหนดจำนวนหลักที่ต้องการให้การสแกนไปถึง
SHUTDOWN	0C	กำหนดการ Shutdown
DISPLAY TEST	0F	ทดสอบการแสดงผล



รูปที่ 15.11 ไตอะแกรมเวลาการส่งข้อมูลให้ไอซี MAX7219



ถ้าหากนำ MAX7219 ไปต่อกับ LED 7 ส่วน การให้ LED แต่ละหลักแสดงเป็นตัวเลขต่าง ๆ นั้น จะต้องส่งไบต์ข้อมูลให้กับหน่วยความจำภายในของ MAX7219 ดังตารางที่ 15.2

ตารางที่ 15.2 ข้อมูลที่ต้องส่งให้กับรีจิสเตอร์ภายใน โดยแสดงถึงการแสดงผลในแต่ละเซกเมนต์

รูปตัวเลข	A	B	C	D	E	F	G	ค่าไบต์ข้อมูล
0	1	1	1	1	1	1	0	7EH
1	0	1	1	0	0	0	0	30H
2	1	1	0	1	1	0	1	6DH
3	1	1	1	1	0	0	1	79H
4	0	1	1	0	0	1	1	33H
5	1	0	1	1	0	1	1	5BH
6	1	0	1	1	1	1	1	5FH
7	1	1	1	0	0	0	0	70H
8	1	1	1	1	1	1	1	7FH
9	1	1	1	1	0	1	1	7BH
-	0	0	0	0	0	0	1	01H
E	1	0	0	1	1	1	1	4FH
H	0	1	1	0	1	1	1	37H
L	0	0	0	1	1	1	0	0EH
P	1	1	0	0	1	1	1	67H
Blank	0	0	0	0	0	0	0	00H

ในการนำไอซี MAX7219 มาใช้งานนั้นควรจะออกแบบหน่วยความจำ RAM สำหรับเก็บข้อมูลที่ต้องการให้ MAX7219 สแกน (ในที่นี้จะใช้ชื่อว่า DISBUF) โดยให้หนึ่งไบต์แทนหนึ่งหลักของการแสดงผล เมื่อต้องการแสดงผลแต่ละหลักเป็นค่าใดก็ให้นำรูปแบบการแสดงผล (จากตารางที่ 15.2) ไปเก็บในหน่วยความจำ DISBUF นี้ก่อน จากนั้นจึงโหลดข้อมูลใน DISBUF ลงสู่ชิปไอซีอีกทีหนึ่ง สำหรับการทดลองต่อไปจะสร้างโปรแกรมร่วมชื่อ MAX7219H.ASM สำหรับใช้ในการติดต่อกับชิป MAX7219 โดยจะประกอบด้วยโปรแกรมน้อย DISMOV สำหรับเลื่อนข้อมูลจากหลักต่ำสุดไปหลักสูงสุด โปรแกรมย่อย MXBYTE สำหรับโหลดข้อมูลหนึ่งไบต์ลงชิป โดยให้รีจิสเตอร์ R0 เก็บแอดเดรสภายในชิป รีจิสเตอร์ R1 สำหรับเก็บข้อมูลที่จะเขียนลงชิป โดยการทำงานของโปรแกรมน้อยนี้สัญญาณต่าง ๆ จะเป็นไปตามรูปที่ 15.11 โปรแกรมย่อย MXLOAD สำหรับโหลดข้อมูลที่จะแสดงผลทั้งหมดจาก DISBUF ลงสู่ชิป โดยโปรแกรมน้อย MAX7219H.ASM สามารถเขียนได้ดังนี้



; ***** MOVE DISBUF *****

; INPUT = DISBUF

; OUTPUT = DISBUF

```
DISMOV:  MOV    A,DISBUF+0      ; ย้ายข้อมูลจากหลักที่ 8 มาใส่ในรีจิสเตอร์ A
          MOV    DISBUF+0,DISBUF+1 ; ย้ายข้อมูลจากหลักที่ 7 มาใส่หลักที่ 8
          MOV    DISBUF+1,DISBUF+2 ; ย้ายข้อมูลจากหลักที่ 6 มาใส่หลักที่ 7
          MOV    DISBUF+2,DISBUF+3
          MOV    DISBUF+3,DISBUF+4
          MOV    DISBUF+4,DISBUF+5
          MOV    DISBUF+5,DISBUF+6
          MOV    DISBUF+6,DISBUF+7
          MOV    DISBUF+7,A      ; นำข้อมูลในรีจิสเตอร์ A มาใส่ในหลักที่ 1
          RET
```

; ***** MXBYTE SUB *****

; SEND ADDRESS,DATA TO MAX7219

; IN = R0 ADDRESS (B0-B3)

; = R1 DATA

; REG = A,R0,R1,R2

;โปรแกรมส่งข้อมูลให้ชิป MAX7219 โดยแอดเดรสใส่ใน R0 ข้อมูลที่จะเขียนใส่ใน R1

```
MXBYTE:  MOV    R2,#8          ; ส่งค่าแอดเดรส โดยแอดเดรสมีขนาด 8 บิต
          MOV    A,R0          ; นำค่าแอดเดรสที่เก็บใน R0 มาใส่ในรีจิสเตอร์ A
MXBYTE1: RLC    A              ; เลื่อนค่าบิตสูงมาไว้ในบิตตัวทศ
          MOV    MXDAT,C       ; นำค่าบิตตัวทศส่งออกไปทางบิต MXDAT
          SETB   MXCLK         ; ส่งสัญญาณกระตุ้นหนึ่งลูก
          CLR    MXCLK
          DJNZ   R2,MXBYTE1    ; ส่งออกไปจนครบ 8 บิต
          MOV    R2,#8          ; ส่งค่าข้อมูลโดยข้อมูลมีขนาด 8 บิต
          MOV    A,R1          ; นำค่าข้อมูลในรีจิสเตอร์ R1 มาเก็บในรีจิสเตอร์ A
MXBYTE2: RLC    A              ; เลื่อนค่าบิตสูงสุดมาใส่บิตตัวทศ
          MOV    MXDAT,C       ; ส่งค่าบิตออกไปทางขา MXDAT
```



```
SETB    MXCLK                ; ส่งสัญญาณกระตุ้นหนึ่งลูก
CLR      MXCLK
DJNZ     R2,MXBYTE2          ; ส่งข้อมูลออกไปจนครบ 8 บิต
SETB     MXLDB                ; LOAD BIT
CLR      MXLDB
RET
```

```
; ***** MXLOAD SUB *****
```

```
; LOAD DISBUF TO DISPLAY (MAX7219)
```

```
; IN = DISBUF
```

```
; REG = A,R0,R1,R2
```

```
;โปรแกรมย่อหน้าข้อมูลจำนวน 8 ไบต์ที่เก็บในหน่วยความจำ DISBUF มาเขียนลง MAX7219
```

```
MXLOAD:
```

```
;DISPLAY TEST - NORMAL MODE (RESET)
```

```
MOV      R0,#0FH
```

```
MOV      R1,#00H
```

```
CALL     MXBYTE
```

```
MOV      R0,#1                ; ส่งข้อมูลตำแหน่งที่ 1 ลงรีจิสเตอร์แอดเดรส 1
```

```
MOV      R1,DISBUF+0          ; อ่านค่าข้อมูลจาก DISBUF
```

```
CALL     MXBYTE                ; ส่งไบต์ข้อมูลไปยังแอดเดรสที่ต้องการ
```

```
MOV      R0,#2                ; ส่งข้อมูลตำแหน่งที่ 2
```

```
MOV      R1,DISBUF+1
```

```
CALL     MXBYTE
```

```
MOV      R0,#3                ; ส่งข้อมูลตำแหน่งที่ 3
```

```
MOV      R1,DISBUF+2
```

```
CALL     MXBYTE
```

```
MOV      R0,#4                ; ส่งข้อมูลตำแหน่งที่ 4
```

```
MOV      R1,DISBUF+3
```

```
CALL     MXBYTE
```

```
MOV      R0,#5                ; ส่งข้อมูลตำแหน่งที่ 5
```

```
MOV      R1,DISBUF+4
```

```
CALL     MXBYTE
```

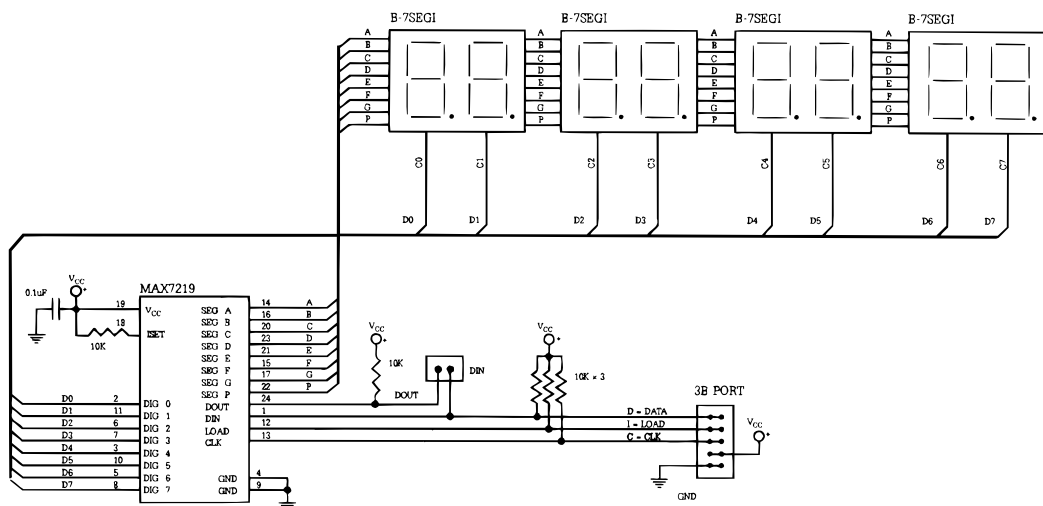


```

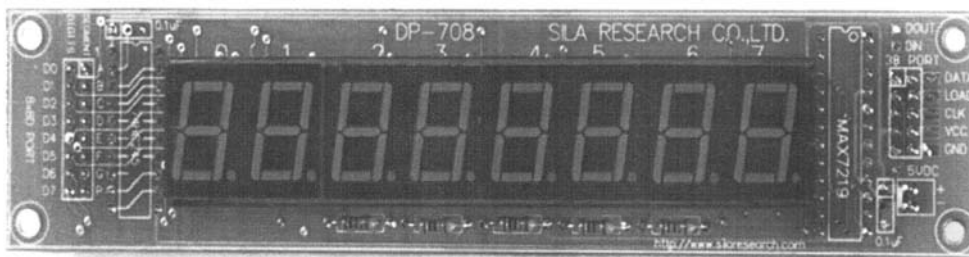
MOV    R0,#6                ; ส่งข้อมูลตำแหน่งที่ 6
MOV    R1,DISBUF+5
CALL   MXBYTE
MOV    R0,#7                ; ส่งข้อมูลตำแหน่งที่ 7
MOV    R1,DISBUF+6
CALL   MXBYTE
MOV    R0,#8                ; ส่งข้อมูลตำแหน่งที่ 8
MOV    R1,DISBUF+7
CALL   MXBYTE
RET

```

ถ้าหากนำไอซี MAX7219 มาใช้ในการสแกนหน่วยแสดงผลแบบ LED 7 ส่วน โดยทำการต่อวงจร ดังรูปที่ 15.12 ซึ่งจะเห็นว่าขาสัญญาณนาฬิกาจะต่อกับบิต P3.2 ของไมโครคอนโทรลเลอร์ ขาโหลดข้อมูล ต่อกับบิต P3.3 ส่วนขาข้อมูลต่อกับบิต P3.5



รูปที่ 15.12 การต่อชิป MAX7219 กับหน่วยแสดงผล LED 7 ส่วน และต่อกับไมโครคอนโทรลเลอร์จำนวน 3 บิต



รูปที่ 15.13 วงจรจริงที่สร้างขึ้น

จาก LED 7 ส่วนจำนวน 7 หลักที่ต่อดังวงจรที่ 15.12 ถ้าหากต้องการให้แสดงเลข 0 ถึง 7 ทาง LED สามารถทำได้โดยส่งค่าควบคุมการทำงานของ MAX7219 เข้าไปก่อนเพื่อควบคุมว่าจะให้ทำการสแกนที่หลักมีความสว่างของหลอดเท่าใดเป็นต้น จากนั้นจะส่งรหัสที่เป็นค่าที่ทำให้แต่ละเซกเมนต์แสดงเป็นตัวเลขดังกล่าวเข้าไปเก็บในรีจิสเตอร์ของหน่วยแสดงผลแต่ละหลัก ดังตัวอย่างโปรแกรมต่อไปนี้

```
; FILENAME      MX7512_1.ASM
; DESCRIPTION   PROGRAM USE MAX7219

MXCLK EQU P3.2      ; ขาสัญญาณนาฬิกาต่อกับบิต P3.2
MXLDB EQU P3.3
MXDAT EQU P3.5

DISBUF EQU 30H      ; หน่วยความจำแสดงผลขนาด 8 ไบต์

; ***** MAIN *****
ORG 0000H
MAIN: CLR MXCLK      ; ส่งสัญญาณกระตุ้นเพื่อให้ MAX7219 เริ่มทำงาน
      CLR MXLDB
      CALL MXSET      ; เรียกโปรแกรมย่อยกำหนดค่าเริ่มต้นสำหรับการ
                        ; แ ด ง ผล
MAIN1: MOV R2,#0AH    ; ตั้งค่าการหน่วงเวลา
      CALL DTSEC      ; เรียกโปรแกรมย่อยหน่วงเวลาตามค่าที่กำหนดใน R2
      CALL SHOW       ; เรียกโปรแกรมย่อยแสดงผล
      SJMP MAIN1      ; ทำซ้ำ
```



```

; ***** DISPLAY SHOW2 *****

; INPUT = DISBUF
; OUTPUT =

SHOW:  MOV    R7,#9
        MOV    DISBUF+0,#7EH      ; ใส่รูปแบบการแสดงผลของเลข 0
        MOV    DISBUF+1,#30H      ; ใส่รูปแบบการแสดงผลของเลข 1
        MOV    DISBUF+2,#6DH      ; ใส่รูปแบบการแสดงผลของเลข 2
        MOV    DISBUF+3,#79H      ; ใส่รูปแบบการแสดงผลของเลข 3
        MOV    DISBUF+4,#33H      ; ใส่รูปแบบการแสดงผลของเลข 4
        MOV    DISBUF+5,#5BH      ; ใส่รูปแบบการแสดงผลของเลข 5
        MOV    DISBUF+6,#5FH      ; ใส่รูปแบบการแสดงผลของเลข 6
        MOV    DISBUF+7,#70H      ; ใส่รูปแบบการแสดงผลของเลข 7

LOOP1:  CALL    MXLOAD             ; โหลดค่าในรีจิสเตอร์ต่าง ๆ ลงในตัวไอซี
        MOV    R2,#0AH
        CALL    DTSEC              ; หน่วยเวลาตามค่าที่กำหนดใน R2
        CALL    DISMOV             ; เลื่อนตัวเลข
        DJNZ   R7,LOOP1            ; เลื่อนตัวเลขจนครบ 8 ตัว
        RET

; ***** MXSET SUB *****

; FIRST PARAMETER SET (MAX7219)
; REG = A,R0,R1,R2
; โปรแกรมย่อยกำหนดค่าเริ่มต้นให้กับ MAX7219
MXSET:
; FALLING CLOCK EDGE
        CLR    MXCLK
        CLR    MXLDB

; DISPLAY TEST - NORMAL (XXXXXXX0)
        MOV    R0,#0FH             ; ทดสอบการแสดงผล
        MOV    R1,#00H
        CALL    MXBYTE             ; เขียนไบต์ข้อมูลลงไป

```



```
; SHUTDOWN - NORNAL (XXXXXXX1)

    MOV    R0,#0CH                ; กำหนดการ Shutdown
    MOV    R1,#01H
    CALL   MXBYTE                ; เขียนไบต์ข้อมูลลงไป
; DECODE MODE - NO DECODE (00000000)

    MOV    R0,#09H                ; กำหนดว่าจะให้มีการถอดรหัสหรือไม่
    MOV    R1,#00H
    CALL   MXBYTE                ; เขียนไบต์ข้อมูลลงไป
; INTENSITY - MIN-MAX (00-0FH)

    MOV    R0,#0AH                ; กำหนดค่าความสว่างให้กับ LED
    MOV    R1,#0FH                ; กำหนดค่าให้สว่างที่สุด
    CALL   MXBYTE                ; เขียนไบต์ข้อมูลลงไป
; SCAN LIMIT - 8 DIGIT (03H)

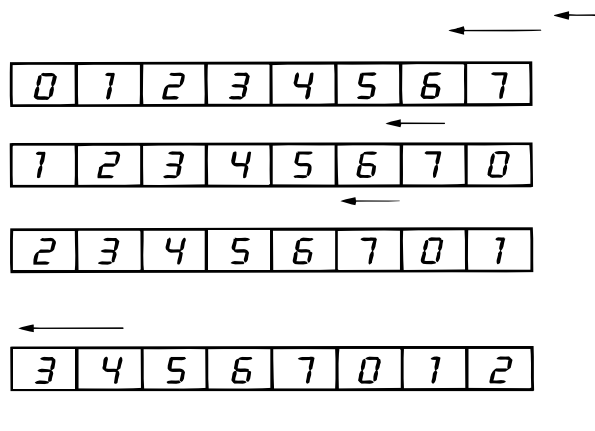
    MOV    R0,#0BH                ; กำหนดจำนวนหลักที่ต้องการสแกน
    MOV    R1,#07H                ; สแกนไปถึงหลักที่ 7 หรือครบทุกหลัก
    CALL   MXBYTE                ; เขียนไบต์ข้อมูลลงไป
    RET

; ***** DTSEC SUB *****
; DELAY 1/5 SECOND
; IN = R2
; REG = R2,R3,R4
; โปรแกรมย่อยหน่วงเวลา 1/5 วินาที
DTSEC:  MOV    R3,#179
DTSEC1: MOV    R4,#0
        DJNZ   R4,$
        NOP
        NOP
        DJNZ   R3,DTSEC1
        DJNZ   R2,DTSEC
        RET

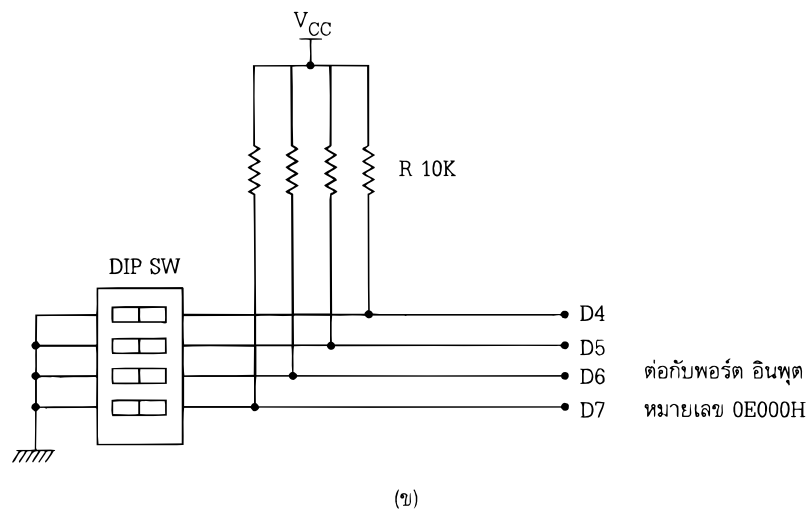
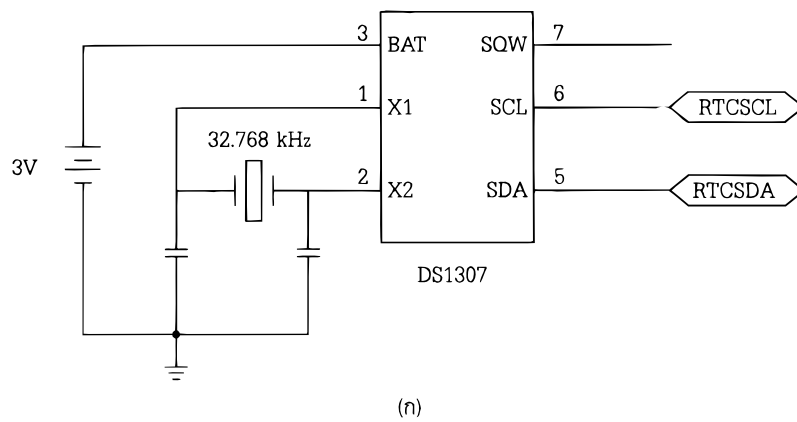
$INCLUDE "MAX7219H.ASM" ; เรียกโปรแกรมร่วม MAX7219.ASM
END
```



เมื่อโปรแกรมทำงานผลลัพธ์จะเป็นดังนี้



ถ้าหากมีไอซีฐานเวลาเบอร์ DS1307 ต่ออยู่กับไมโครคอนโทรลเลอร์ดังรูปที่ 15.14 และต้องการ
แ ดงค่าเวลาและวันที่ออกทาง LED 7 ส่วนจำนวน 8 หลักที่ใช้ไอซี MAX7219 ทำการสแกนก็สามารถทำได้
เช่นกัน โดยให้อ่านค่าเวลาและวันที่จากไอซี DS1307 มาเก็บในหน่วยความจำ จากนั้นนำค่าที่อ่านมาได้ส่งให้
กับหน่วยความจำแสดงผล (DISBUF) ของระบบที่มี MAX7219 ต่ออยู่ จากรูปที่ 15.14 ซึ่งจะเห็นว่ามีส่วนสวิตช์ต่อ
อยู่ทางพอร์ตอินพุต โดยสวิตช์นี้ถ้าหากมีการกดจะทำให้บิตอินพุตบิตนั้นเป็นลอจิก “0” ถ้าหากต้องการเขียน
โปรแกรมสำหรับแสดงผลวันและเวลาออกทางหน่วยแสดงผล โดยถ้าหากไม่มีการกดสวิตช์ตัวใดให้ระบบ
แ ดงผลเป็นวันที่ แต่ถ้ามีการกดสวิตช์ตัวใดตัวหนึ่งให้ระบบแสดงผลเป็นเวลา สามารถออกแบบโปรแกรม
โดยอ่านค่าเวลาจากชิป DS1307 ออกมาทั้งหมดมาเก็บไว้ในหน่วยความจำ ถ้าหากอ่านค่าจากสวิตช์เข้ามา
เป็นลอจิก “1” ทั้งหมดให้นำข้อมูลที่เป็นส่วนของวันที่นำมาแสดงผล ถ้าอ่านค่าจากสวิตช์ไม่เป็นลอจิก “1”
ทุกบิตให้นำค่าข้อมูลเวลามาแสดงผล แต่เนื่องจากการเก็บข้อมูลของ DS1307 นั้นจะอยู่ในรูปแบบของรหัส
BCD ดังนั้นจะต้องทำการแปลงรหัส BCD ให้เป็นรูปแบบที่แสดงได้โดย MAX7219 เสียก่อน



รูปที่ 15.14 การต่อไอซีฐานเวลา และการต่อสวิตช์อินพุตเข้ากับพอร์ตของ MCS-51

ตัวอย่างต่อไปจะเป็นการนำค่าวันที่และเวลามาแสดงทางหน่วยแสดงผลที่ออกแบบไว้ดังวงจรในรูปที่ 15.12 ซึ่งเป็นหน่วยแสดงผลแบบ 8 หลัก โดยจะให้การแสดงวันที่ และเวลามีรูปแบบดังนี้



สำหรับโปรแกรมการแสดงผลวันที่และเวลาแสดงได้ตั้งโปรแกรมต่อไปนี้ โดยจะใช้โปรแกรมร่วมชื่อ MAX7219H.ASM เพื่อใช้ในการแสดงผล และใช้โปรแกรมร่วมชื่อ RTC_IOH.ASM ในการติดต่อกับชิป DS1307 โดยในโปรแกรมจะสร้างโปรแกรมย่อยชื่อ BCD2DEC สำหรับอ่านข้อมูลเวลาที่เก็บไว้หน่วยความจำ HEXBUF มาถอดรหัสเป็นตัวเลข และนำตัวเลขที่ได้ไปใส่ในหน่วยความจำแสดงผล สำหรับโปรแกรมย่อยชื่อ BCD2DEC_2 จะใช้สำหรับอ่านข้อมูลวันที่จากหน่วยความจำ DATBUF มาถอดรหัสและนำตัวเลขที่ได้ไปใส่ในหน่วยความจำแสดงผล

```
;FILENAME      MX7512_3.ASM
;DESCRIPTION    PROGRAM USE MAX7219
;USES RTCDS1307

INKEY    EQU    0E000H      ; พอร์ตอินพุตที่ต่อกับดิฟลวิตซ์
MXCLK    EQU    P3.2        ; บิตที่ต่อกับขา CLK ของชิป MAX7219
MXLDB    EQU    P3.3        ; บิตที่ต่อกับขาโหลดข้อมูล
MXDAT    EQU    P3.5        ; บิตที่ต่อกับขาข้อมูล

RTCSCL   EQU    P1.3        ; บิตที่ต่อกับสัญญาณนาฬิกาของ DS1307
RTCSDA   EQU    P1.4        ; บิตที่ต่อกับขาข้อมูลของ DS1307

DISBUF   EQU    30H         ; USE 8 BYTE
HEXBUF   EQU    20H         ; หน่วยความจำเก็บเลข BCD ของเวลา
DATBUF   EQU    25H         ; หน่วยความจำเก็บเลข BCD ของวันที่
RTCERR   EQU    79H         ; บิตตรวจสอบความถูกต้องของข้อมูล

; ***** MAIN *****
                ORG    0000H
MAIN:         CLR    MXCLK      ; START MAX7219
                CLR    MXLDB
                CALL   MXSET     ; เรียกโปรแกรมย่อยกำหนดค่าเริ่มต้นการแสดงผล
MAIN1:
                CALL   SHOW1     ; เรียกโปรแกรมย่อย SHOW1 เพื่อแสดงผลเป็นขีด -
                MOV    DPTR,#TIMEBUF ; ชี้ไปที่หน่วยความจำบัฟเฟอร์เก็บค่าวันเวลาในแรม
```



```
MOV    A,#00H           ; กำหนดค่าวินาทีเป็น 00
MOVX   @DPTR,A          ; เขียนลงหน่วยความจำบัฟเฟอร์เวลา
INC    DPTR              ; ซ้ำไปที่หน่วยความจำของนาฬิกา
MOV    A,#58H
MOVX   @DPTR,A          ; กำหนดเป็น 58 นาที
INC    DPTR              ; ซ้ำไปที่หน่วยความจำของชั่วโมง
MOV    A,#23H           ; กำหนดเป็นเวลา 23 นาฬิกา
MOVX   @DPTR,A          ; ***** END TIME *****
INC    DPTR              ; ซ้ำไปที่หน่วยความจำวัน
MOV    A,#07H           ; กำหนดเป็นวันลำดับที่ 7 ของสัปดาห์
MOVX   @DPTR,A
INC    DPTR              ; ซ้ำไปที่หน่วยความจำวัน
MOV    A,#31H
MOVX   @DPTR,A          ; กำหนดเป็นวันที่ 31
INC    DPTR
MOV    A,#01H
MOVX   @DPTR,A          ; กำหนดเป็นเดือนที่ 1
INC    DPTR
MOV    A,#03H           ; กำหนดเป็นปี 03
MOVX   @DPTR,A          ; ***** END DATE *****
CALL   RTWR              ; เรียกโปรแกรมย่อยเขียนค่าวันเวลาลง DS1307
DIS:   CALL   RTRD        ; อ่านข้อมูลวันเวลาออกจากชิปไปเก็บในบัฟเฟอร์
MOV    DPTR,#TIMEBUF    ; ซ้ำไปที่หน่วยความจำบัฟเฟอร์เวลา
MOVX   A,@DPTR
MOV    HEXBUF+2,A        ; อ่านค่าวินาทีออกมาเก็บใน HEXBUF + 2
INC    DPTR
MOVX   A,@DPTR
MOV    HEXBUF+1,A        ; อ่านค่านาฬิกาออกมาเก็บใน HEXBUF + 1
INC    DPTR
MOVX   A,@DPTR
MOV    HEXBUF,A          ; อ่านค่าชั่วโมงออกมาเก็บใน HEXBUF
INC    DPTR
```



```
MOVX A,@DPTR
MOV DATBUF,A ; อ่านข้อมูลวันของสัปดาห์มาเก็บใน DATBUF
INC DPTR
MOVX A,@DPTR
MOV DATBUF+1,A ; อ่านข้อมูลวันที่มาเก็บใน DATBUF+1
INC DPTR
MOVX A,@DPTR
MOV DATBUF+2,A ; อ่านข้อมูลของเดือนมาเก็บใน DATBUF + 2
INC DPTR
MOVX A,@DPTR
MOV DATBUF+3,A ; อ่านข้อมูลปีมาเก็บใน DATBUF + 3
MOV DPTR,#INKEY ; ซี่ไปที่พอร์ตอินพุตที่ต่อกับคีย์บอร์ด
MOVX A,@DPTR ; อ่านค่าการกดคีย์บอร์ดเข้ามา
ANL A,#11110000B ; ตรวจสอบว่า 4 บิตบนมีการกดคีย์บอร์ดหรือไม่
CJNE A,#11110000B,GO1 ; ถ้ามีการกดให้ไป GO1
ACALL BCD2DEC_2 ; ถ้าไม่มีการกดให้เรียกโปรแกรมย่อย BCD2DEC_2
JMP GO
GO1: ACALL BCD2DEC ; เรียกโปรแกรมย่อย BCD2DEC
GO: CALL SHOW ; เรียกโปรแกรมย่อย SHOW เพื่อแสดงผล
SJMP DIS ; ทำซ้ำ

BCD2DEC: ; โปรแกรมย่อยนำค่าเวลาที่อ่านได้ไปใส่ในหน่วยความจำแสดงผล
PUSH ACC
PUSH B ; เก็บรีจิสเตอร์ ACC และ B ลงหน่วยความจำสแต็ก
MOV A,HEXBUF ; อ่านเลข BCD สองหลักของชั่วโมงออกมา
MOV B,A ; คัดลอกเก็บไว้ใน B
SWAP A ; สลับระหว่างหลักสิบกับหลักหน่วย
ANL A,#00001111B ; เลือกเฉพาะหลักสิบ นำตัวเลขมาเก็บใน A
MOV DPTR,#SEGTAB ; เปิดตาราง นำรูปแบบการแสดงผลของ
MOVC A,@A+DPTR ; ตัวเลขใน A มาเก็บในรีจิสเตอร์ A
MOV DISBUF,A ; นำรูปแบบการแสดงผลไปเก็บในหน่วยความจำ
```



```
MOV    A,B
ANL    A,#00001111B    ; หาค่าตัวเลขหลักหน่วยของชั่วโมง
MOV    DPTR,#SEG TAB
MOVC   A,@A+DPTR
MOV    DISBUF+1,A      ; เก็บรูปแบบการแสดงผลใน DISBUF + 1
MOV    A,HEXBUF+1      ; อ่านเลข BCD สองหลักของนาฬิกาออกมา
MOV    B,A
SWAP   A
ANL    A,#00001111B
MOV    DPTR,#SEG TAB
MOVC   A,@A+DPTR
MOV    DISBUF+3,A      ; รูปแบบการแสดงผลของนาฬิกาหลักสิบ
MOV    A,B
ANL    A,#00001111B
MOV    DPTR,#SEG TAB
MOVC   A,@A+DPTR
MOV    DISBUF+4,A      ; รูปแบบการแสดงผลของนาฬิกาหลักหน่วย
MOV    A,HEXBUF+2      ; อ่านค่าเลข BCD สองหลักของวินาทีออกมา
MOV    B,A
SWAP   A
ANL    A,#00001111B
MOV    DPTR,#SEG TAB
MOVC   A,@A+DPTR
MOV    DISBUF+6,A      ; รูปแบบการแสดงผลของวินาทีหลักสิบ
MOV    A,B
ANL    A,#00001111B
MOV    DPTR,#SEG TAB
MOVC   A,@A+DPTR
MOV    DISBUF+7,A      ; รูปแบบการแสดงผลของวินาทีหลักหน่วย
POP     B
POP     ACC
RET
```



```
BCD2DEC_2:      ; โปรแกรมย่อยค่าเลข BCD ของวันที่มาเปลี่ยนเป็นข้อมูลเพื่อแสดงผล
                PUSH    ACC
                PUSH    B
                MOV     A,DATBUF+1      ; อ่านเลข BCD ของวันที่ ที่เก็บใน DATBUF +
                                         1 ออกมา
                MOV     B,A            ; คัดลอกเก็บไว้ใน B
                SWAP    A              ; สลับระหว่างหลักสิบและหลักหน่วย
                ANL     A,#00001111B   ; เลือกเอาเฉพาะวันที่หลักสิบ มาเก็บใน A
                MOV     DPTR,#SEGTAB   ; ชี้ไปที่ตารางเก็บรูปแบบการแสดงผล
                MOVC    A,@A+DPTR      ; อ่านรูปแบบตัวเลขที่แสดงผลมาเก็บใน A
                MOV     DISBUF,A       ; นำรูปแบบการแสดงผลหลักสิบไปเก็บใน
DISBUF
                MOV     A,B
                ANL     A,#00001111B   ; เลือกเอาเฉพาะวันที่หลักหน่วยมาเก็บใน A
                MOV     DPTR,#SEGTAB
                MOVC    A,@A+DPTR
                MOV     DISBUF+1,A     ; นำรูปแบบการแสดงผลวันที่หลักหน่วยเก็บใน
                                         DISBUF + 1
                MOV     A,DATBUF+2     ; อ่านข้อมูลเดือน
                MOV     B,A
                SWAP    A
                ANL     A,#00001111B
                MOV     DPTR,#SEGTAB
                MOVC    A,@A+DPTR
                MOV     DISBUF+3,A     ; เก็บรูปแบบของเดือนหลักสิบ
                MOV     A,B
                ANL     A,#00001111B
                MOV     DPTR,#SEGTAB
                MOVC    A,@A+DPTR
                MOV     DISBUF+4,A     ; เก็บรูปแบบของเดือนหลักหน่วย
                MOV     A,DATBUF+3     ; อ่านข้อมูลปี
                MOV     B,A
                SWAP    A
```



```
ANL    A,#00001111B
MOV    DPTR,#SEGTAB
MOVC   A,@A+DPTR
MOV    DISBUF+6,A      ; เก็บรูปแบบการแสดงผลของปีหลักสิบ
MOV    A,B
ANL    A,#00001111B
MOV    DPTR,#SEGTAB
MOVC   A,@A+DPTR
MOV    DISBUF+7,A      ; เก็บรูปแบบการแสดงผลของปีหลักหน่วย
POP    B
POP    ACC
RET

; ***** DISPLAY SHOW2 *****
; INPUT = DISBUF
; OUTPUT =

SHOW1:      ; โปรแกรมย่อยแสดงเครื่องหมายขีดทุกหลัก
MOV    A,#01      ; รูปแบบของเครื่องหมายขีด
MOV    DISBUF+0,A  ; เก็บรูปแบบการแสดงผลหน่วยความจำแสดงผล
MOV    DISBUF+1,A  ; ทุกหลัก
MOV    DISBUF+2,A
MOV    DISBUF+3,A
MOV    DISBUF+4,A
MOV    DISBUF+5,A
MOV    DISBUF+6,A
MOV    DISBUF+7,A

LOOP1: CALL    MXLOAD      ; โหลดข้อมูลจาก DISBUF ลง MAX7219
RET          ; จบโปรแกรมย่อย SHOW1

SHOW: CALL    MXLOAD      ; โหลดข้อมูลจาก DISBUF ลง MAX7219
RET
```



```
SEG TAB: DB 7EH,30H,6DH,79H ; รูปแบบการแสดงผล 0123 (MAX7219)
          DB 33H,5BH,5FH,70H ; 4567
          DB 7FH,7BH,77H,1FH ; 89AB
          DB 4EH,3DH,4FH,47H ; CDEF
```

```
; ***** MXSET SUB *****
```

```
; FIRST PARAMETER SET (MAX7219)
```

```
; REG = A,R0,R1,R2
```

```
; โปรแกรมย่อยกำหนดค่าเริ่มต้นการทำงานของ MAX7219
```

```
MXSET:
```

```
; FALLING CLOCK EDGE
```

```
CLR MXCLK
```

```
CLR MXLDB
```

```
; DISPLAY TEST - NORMAL (XXXXXXX0)
```

```
MOV R0,#0FH
```

```
MOV R1,#00H
```

```
CALL MXBYTE
```

```
; SHUTDOWN - NORMAL (XXXXXXX1)
```

```
MOV R0,#0CH
```

```
MOV R1,#01H
```

```
CALL MXBYTE
```

```
; DECODE MODE - NO DECODE (00000000)
```

```
MOV R0,#09H
```

```
MOV R1,#00H
```

```
CALL MXBYTE
```

```
; INTENSITY - MIN-MAX (00-0FH)
```

```
MOV R0,#0AH
```

```
MOV R1,#0FH ; กำหนดค่าความสว่างของการแสดงผล
```

```
CALL MXBYTE
```

```
; SCAN LIMIT - 8 DIGIT (03H)
```

```
MOV R0,#0BH
```

```
MOV R1,#07H
```



```

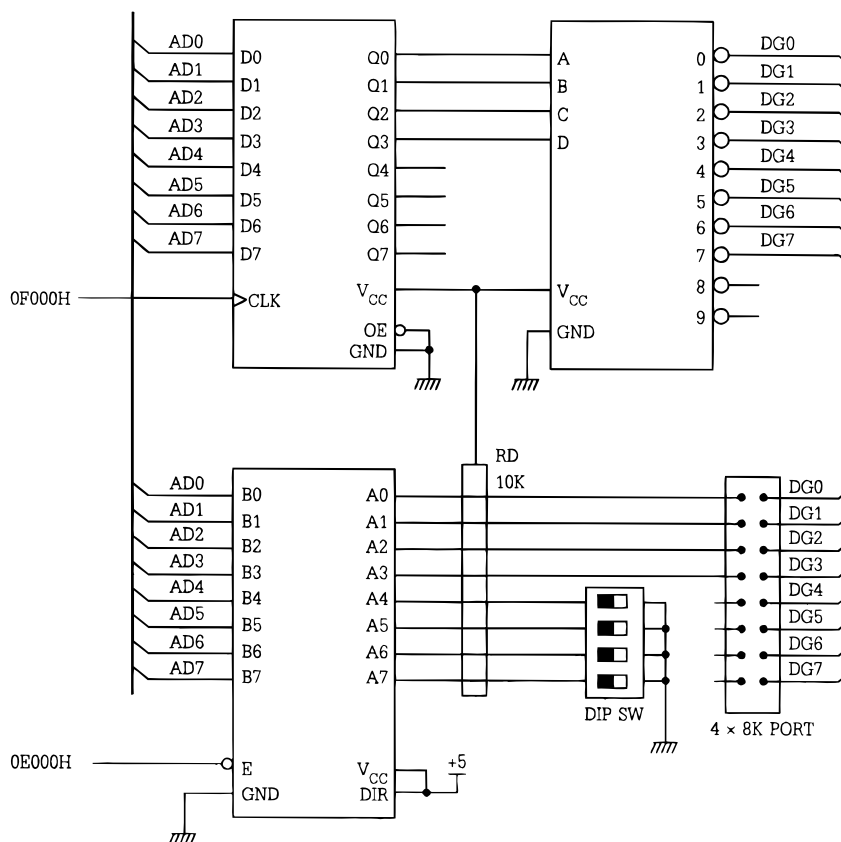
CALL    MXBYTE
RET

$INCLUDE "MAX7219H.ASM"           ; เรียกโปรแกรมร่วมติดต่อกับ MAX7219
$INCLUDE "RTC_I0H.ASM"           ; เรียกโปรแกรมร่วมติดต่อกับ DS1307
END

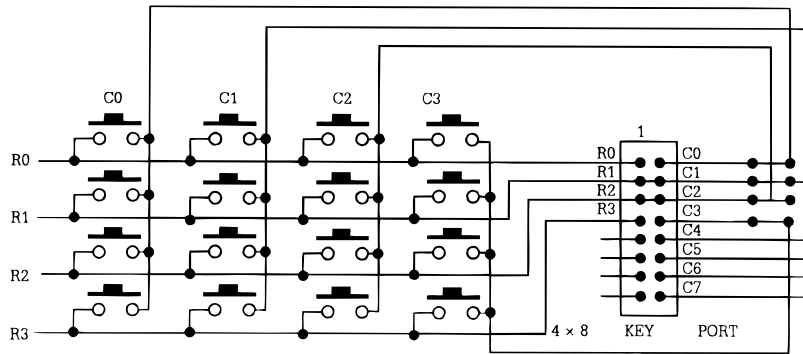
```

ถ้าหากมีการออกแบบพอร์ตอินพุตเอาต์พุตเอาไว้ดังรูปที่ 15.16 และถ้าหากต้องการต่อสวิตช์แบบเมทริกซ์ขนาด 4x4 ที่มีวงจรดังรูปที่ 15.17 เข้าไปก็สามารถทำได้ โดยการอ่านข้อมูลจากคีย์ที่กดให้ใช้เทคนิคการสแกนคีย์สวิตช์

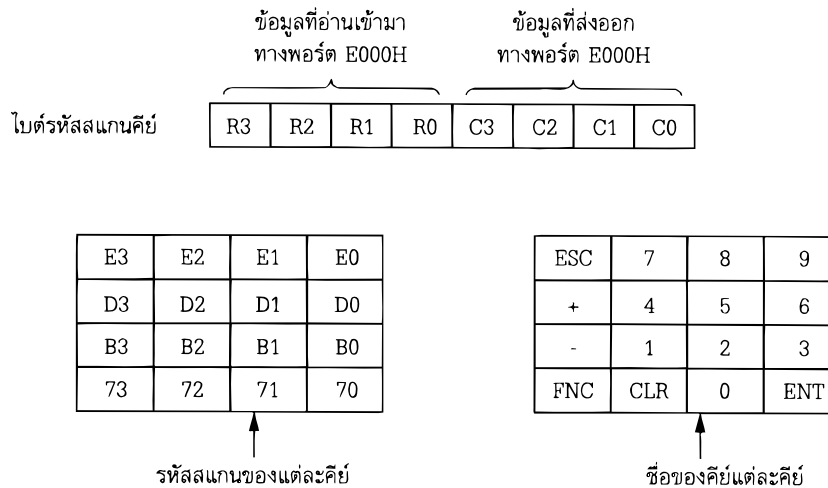
ในการสแกนคีย์บอร์ดเริ่มต้นจะส่งข้อมูลออกไปทางพอร์ต F000H ให้บิต DG0 เป็นลอจิก “0” และบิตอื่น ๆ เป็นลอจิก “1” (ดูรูปที่ 15.18 ประกอบ) และอ่านข้อมูลทางบิต R0,R1,R2 และ R3 เข้ามาทางพอร์ต E000H จากนั้นให้บิต DG1 เป็นลอจิก “0” บิตอื่น ๆ เป็นลอจิก “1” และทำการสแกนซ้ำไปเรื่อย ๆ ถ้าหากนำข้อมูลที่สแกนทางเอาต์พุตและอ่านเข้ามาทางอินพุตรวมกันเป็นไบต์ข้อมูลดังรูปที่ 15.18



รูปที่ 15.16 พอร์ตอินพุตเอาต์พุตที่ออกแบบไว้สำหรับต่อกับสวิตช์



รูปที่ 15.17 การต่อสวิตช์แบบเมทริกซ์ขนาด 4 × 4 คีย์



รูปที่ 15.18 คีย์บอร์ดและรหัสแกนที่ใช้ในการทดลองนี้

จากรูปที่ 15.18 จะเห็นว่าคีย์แต่ละคีย์จะถูกตั้งชื่อคีย์เอาไว้ ถ้าหากเขียนโปรแกรมสแกนคีย์และได้รหัสแกนเป็น 0x71 หมายความว่าคีย์หมายเลข “0” ถูกกด หรือถ้าหากอ่านรหัสแกนคีย์ได้เป็น 0xB2 หมายความว่าคีย์หมายเลข “1” ถูกกด ในการเขียนโปรแกรมเราอาจนำรหัสแกนมาใส่ไว้ในตารางในตำแหน่งที่สอดคล้องกับชื่อคีย์ ถ้าหากได้รหัสแกนคีย์ค่าใดเข้ามาก็ให้เปรียบเทียบกับค่าในตาราง ก็จะทำให้ทราบว่าคีย์ใดถูกกด สำหรับตัวอย่างโปรแกรมต่อไปจะเป็นการอ่านค่าจากคีย์สวิตช์ และนำหมายเลขคีย์ที่อ่านได้ส่งออกไปทางพอร์ต P1 ถ้าหากที่พอร์ต P1 มี LED ต่ออยู่ 8 หลอด เมื่อรันโปรแกรมและกดคีย์จะทำให้ค่าเลขฐานสองของการสว่างของหลอดมีค่าเท่ากับหมายเลขคีย์ที่กด เช่นถ้ากดคีย์ 1 จะทำให้หลอดที่สว่างที่สุด ถ้ากดคีย์ 3 จะทำให้หลอดที่สว่างที่สุดสามหลอดเป็นต้น ซึ่งตัวอย่างนี้สามารถนำไปประยุกต์กับงานต่าง ๆ ได้



```

                ORG    0000H

KEY_OUT EQU    0F000H        ; พอร์ตเอาต์พุต
KEY_IN  EQU    0E000H        ; พอร์ตอินพุต
KEYBUF  EQU    20H          ; หน่วยความจำบัฟเฟอร์คีย์บอร์ด

SCANPF  EQU    60H

;***** MAIN *****
MAIN:
DIS:      ACALL  SCAN_K        ; เรียกโปรแกรมย่อยสแกนคีย์บอร์ด
          MOV   P1,KEYBUF      ; นำค่าการกดคีย์ส่งออกไปทางพอร์ต P1
          SJMP  DIS            ; ทำซ้ำ

; โปรแกรมย่อยสแกนคีย์บอร์ด เมื่อมีการกดคีย์จะส่งค่าหมายเลขคีย์ไปเก็บใน KEYBUF
SCAN_K:   MOV   R2,#0          ; ค่าเริ่มต้นที่จะส่งออกไปทางพอร์ต
          MOV   R3,#5

LSCAN:    MOV   A,R2           ; นำค่าที่จะสแกนเก็บใน A โดยจะส่งเป็น 0 ครั้งละบิต
          MOV   DPTR,#KEY_OUT
          MOVB  @DPTR,A        ; ส่งออกไปทางเอาต์พุต
          MOV   DPTR,#KEY_IN
          MOVB  A,@DPTR        ; อ่านค่าเข้ามาทางพอร์ตอินพุต
          ANL   A,#0FH         ; เอาเฉพาะสี่บิตล่าง ถ้าไม่กดคีย์จะเป็น 1 ทั้งหมด
          CJNE  A,#0FH,SCAN1    ; ถ้าพบการกดคีย์ให้ไป SCAN1
          JMP   SCAN11         ; ถ้าไม่พบการกดคีย์ให้ไป SCAN11

SCAN1:    MOV   B,R2           ; นำค่าที่ส่งออกไปสแกนมาเก็บใน B
          SWAP  A              ; สลับค่าสี่บิตบนกับสี่บิตล่างของค่าที่อ่านเข้ามา
          ORL   A,B            ; เอามา OR กับ B จะได้รหัสสแกนตามรูปที่ 15.18
          CALL  CHK_KEY        ; เรียกโปรแกรมย่อยตรวจสอบว่าคีย์ที่กดเป็นคีย์ใด
          MOV   KEYBUF,A       ; นำค่าคีย์ที่กดมาเก็บใน KEYBUF

SCAN11:   INC   R2             ; เพิ่มค่า R2 เพื่อสแกนหลักต่อไป
          DJNZ  R3,LSCAN       ; สแกนให้ครบทุกหลัก
          RET                  ; จบโปรแกรมย่อย

CHK_KEY:                                     ; โปรแกรมย่อยตรวจสอบคีย์ที่กด
          MOV   B,A

```



```
MOV    R4,#00H          ; ให้ค่าแรกเป็น 0
MOV    R5,#16            ; คีย์ที่กำหนดไว้ในตารางมี 16 คีย์
L_CHK: MOV    DPTR,#KEYCODE ; ชี้ไปที่ตารางรหัสสแกน
MOV    A,R4
MOVC   A,@A+DPTR        ; อ่านค่าในตารางออกมาเรียงลำดับกันไป
CJNE   A,B,L_CHK1       ; ถ้าค่าที่ได้จากการสแกนไม่เท่ากับในตารางให้ไป
                        L_CHK1
MOV    A,R4              ; เอาค่าตำแหน่งคีย์เก็บในรีจิสเตอร์ A
RET
L_CHK1: INC    R4         ; เปรียบเทียบค่าถัดไปในตาราง
DJNZ   R5,L_CHK
MOV    A,#0FFH          ; ถ้าไม่เท่ากับค่าในเลย ให้ A เท่ากับ 0FFH
RET

KEYCODE: DB    71H,0B2H,0B1H,0B0H ; ตารางเก็บค่ารหัสสแกนของคีย์ 0 1 2 3
           DB    0D2H,0D1H,0D0H,0E2H ; 4 5 6 7
           DB    0E1H,0E0H,0D3H,0B3H ; 8 9 + -
           DB    73H,72H,70H,0E3H   ; FNC CLR ENT ESC
END
```

การทำงานของโปรแกรมจะประกาศหน่วยความจำชื่อ KEYBUF สำหรับเก็บหมายเลขคีย์ที่กด เมื่อไมโครคอนโทรลเลอร์ได้รับรหัสสแกนเข้ามา จะนำไปเปรียบเทียบกับค่าของรหัสสแกนที่กำหนดเอาไว้ในตาราง KEYCODE ถ้าพบว่าค่าที่อ่านเข้ามานั้นเท่ากับค่าตำแหน่งใดในตาราง โปรแกรมก็จะนำค่าตำแหน่งนั้นไปเก็บในหน่วยความจำ KEYBUF

สำหรับอุปกรณ์ที่ใช้ในการทดลองบทนี้ผู้เขียนได้ใช้แผงวงจรต่าง ๆ ของบริษัท คิลาร์เสิร์ช จำกัด สำหรับรายละเอียดของวงจร และคู่มือของไอซีเบอร์ต่าง ๆ ผู้สนใจสามารถเข้าไปชมได้ใน www.silaresearch.com ซึ่งในเว็บไซต์นี้จะมีตัวอย่างโปรแกรมอื่น ๆ ให้ผู้สนใจศึกษาอีกด้วย



ประวัติผู้เขียน

รองศาสตราจารย์ธีรวัฒน์ ประกอบพล

- การศึกษา**
- วิทยาศาสตรบัณฑิต สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ปี พ.ศ. 2534
 - วิศวกรรมศาสตรมหาบัณฑิต จุฬาลงกรณ์มหาวิทยาลัย ปี พ.ศ. 2537
- การทำงาน**
- พ.ศ. 2537-2541 อาจารย์ประจำภาควิชาฟิสิกส์ประยุกต์ คณะวิทยาศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
 - พ.ศ. 2541-ปัจจุบัน อาจารย์ประจำภาควิชาคณิตศาสตร์และวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
 - ทำงานวิจัยทางด้านคอมพิวเตอร์กราฟิก การประยุกต์ใช้คอมพิวเตอร์ในงานด้านอุตสาหกรรม นอกจากนี้ยังเป็นอาจารย์พิเศษสอนทางด้านไมโครคอมพิวเตอร์ ไมโครโพรเซสเซอร์ให้กับมหาวิทยาลัยต่าง ๆ หลายแห่ง

ทางสมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น) ขอขอบพระคุณท่านผู้อ่านที่ได้ให้ความสนใจหนังสือของทาง ส.ส.ท. มาโดยตลอด และเพื่อเป็นการแสดงความขอบคุณต่อความกรุณาของท่านทาง ส.ส.ท. ได้จัดเตรียมของสมนาคุณให้แก่ท่านผู้อ่าน 100 ท่านแรกที่ได้ตอบแบบสอบถามนี้อย่างครบถ้วนและส่งกลับไปยังสมาคมฯ ข้อเสนอแนะของท่าน เป็นกำลังใจในการพัฒนางานของสมาคมฯ ต่อไป

แบบสอบถาม

ชื่อ นามสกุล
 ที่อยู่ แขวง/ตำบล
 เขต/อำเภอ จังหวัด รหัสไปรษณีย์
 โทรศัพท์ E - mail

ตอนที่ 1 ข้อมูลส่วนตัว

ระดับการศึกษา	อาชีพ	รายได้ต่อเดือน
☐ มัธยมศึกษา	☐ นักเรียน/นักศึกษา	☐ ต่ำกว่า 10,000 บาท
☐ ปวช./ปวส.	☐ รับราชการ	☐ 10,001-20,000 บาท
☐ ปริญญาตรี	☐ รัฐวิสาหกิจ	☐ 20,001-50,000 บาท
☐ ปริญญาโท	☐ พนักงานบริษัท	☐ 50,001 บาทขึ้นไป
☐ อื่น ๆ	☐ อื่น ๆ	

ตอนที่ 2 ข้อมูลทั่วไป

1. กรุณาระบุชื่อหนังสือที่ท่านกรอกแบบสอบถาม
2. ท่านพบและซื้อหนังสือเล่มนี้จากร้านใด (กรุณาระบุชื่อร้านและสาขา)
3. ในการตัดสินใจซื้อหนังสือเล่มนี้ ท่านพิจารณาจากสิ่งใด (โปรดระบุตัวเลข 1, 2, 3,...เรียงตามลำดับ)

☐ เนื้อหา	☐ ราคา	☐ หน้าปก & รูปเล่ม
☐ ผู้เขียน	☐ สำนักพิมพ์	☐ อื่น ๆ
4. ท่านมีความคิดเห็นอย่างไรต่อหนังสือเล่มนี้ของ ส.ส.ท.

เนื้อหา	☐ ดีมาก	☐ ดี	☐ พอใช้	☐ ต้องปรับปรุง
หน้าปก	☐ ดีมาก	☐ ดี	☐ พอใช้	☐ ต้องปรับปรุง
ราคา	☐ สูงเกินไป	☐ เหมาะสม	☐ ต่ำเกินไป	
5. ท่านคิดว่าส่วนใดของหนังสือเล่มนี้ที่ควรปรับปรุง
6. ท่านมีความคิดเห็นอย่างไรเกี่ยวกับหนังสือที่ผลิตโดยสำนักพิมพ์ ส.ส.ท.
7. ท่านคิดว่า ส.ส.ท. ควรจะผลิตหนังสือในแนวใดเพิ่มเติม



สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

TECHNOLOGY PROMOTION ASSOCIATION (THAILAND-JAPAN)

5-7 ซอยสุขุมวิท 29 ถนนสุขุมวิท แขวงคลองเตยเหนือ เขตวัฒนา กรุงเทพฯ 10110 โทร. 0-2258-0320-5

5-7 Sukhumvit Road Soi 29 Klongtoey Nua Watthana Bangkok 10110 Fax : 0-2258-6440

ชื่อผู้ตอบแบบสอบถาม

ที่อยู่

.....

.....

.....รหัสไปรษณีย์

(กรุณากรอรายละเอียดให้ครบถ้วน เพื่อสะดวกในการส่งของสมนาคุณ)



ร้านหนังสือ ส.ส.ท. (TPA Books Centre)

สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

ใบอนุญาตเลขที่ ปน.(ต.)/884 ปณ.สันติสุข
ถ้าฝากส่งในประเทศไม่ต้องฉีกตราไปรษณียากร

บริการธุรกิจตอบรับ

สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

តួ ប.ណ. 3

ปณณ. สันติสุข

กรุงเทพฯ

10113